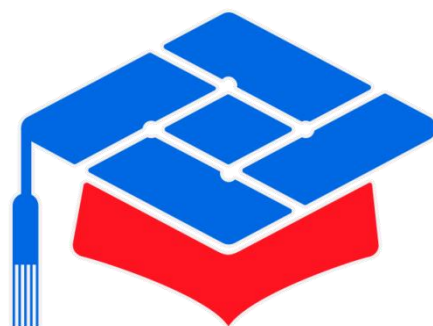




第十届

全国大学生集成电路创新创业大赛

报告类型： CPU 设计文档

参赛杯赛： 七星微杯

作品名称： 基于 RISC-V 的高性能 CPU 设计及 FPGA 验证

队伍编号： CICC1002212

团队名称： 组一辈子乐队

目录

快速预览	6
一、 作品概览与设计定位	6
二、 核心微架构创新点	6
三、 处理器配置概览	8
四、 THP 外设应用与硬件实现预览	8
五、 系统级集成与软硬件组成	8
RISC-V CPU 设计介绍	10
1 项目概述	10
1.1 设计层次与配置组织	12
1.2 硬件环境与 FPGA 目标平台	13
2 CPU 顶层接口	19
2.1 时钟及复位	19
2.2 中断与异常控制接口	20
2.3 AXI 接口	21
2.4 处理器参数化配置	24
2.5 AXI4-Lite Cache 与乒乓缓存	28
3 数据流	34
3.1 流水线级划分与共同控制	38
4 时钟、复位和初始化	43
5 子模块描述	44
5.1 IFU	44
5.2 IDU	53
5.3 Dispatch	60
5.4 EXU	82
5.5 WBU	123
5.6 GPR	131
5.7 CSR	132
5.8 中断与异常处理系统	138
5.10 外设模块	161
5.11 通用寄存器与存储单元	166
5.12 AXI4-Lite Cache 与乒乓缓存	169
6 分支预测参数化设计与对比	192
6.1 参数化范围与配置传递	192
6.2 基础轻量 BPU	194
6.3 增强 BPU 与可选 TAGE 设计	194
6.4 流水线信号传递与回写通路	197
6.5 配置选择原则与设计结论	198
7 时序收敛优化设计	198
7.1 关键路径来源分析	199
7.2 级间寄存器与组合逻辑路径拆分	199
7.3 面积优化与参数默认值策略	200

7.4 实现约束与层次边界	201
8 流水线冒险与控制设计	202
8.1 RAW/WAW/结构冒险分类	202
8.2 单项等待槽状态机	204
8.3 加载后分支瓶颈与 LSU L0 Cache	205
8.4 冲刷、异常、中断与接口兼容	205
RISC-V CPU 性能与验证报告	206
1 验证目标与范围	206
1.1 流水线配置	206
1.2 验证范围	207
1.3 验证结果判定方式	208
2 验证环境与方法	208
2.1 验证环境结构	208
2.2 Case 管理、构建与调度	210
2.3 模块级自检测测试平台	210
2.4 整核系统测试平台	214
2.5 程序完成与结果检查	214
2.6 验证策略	214
2.7 回归执行流程	214
3 指令集测试	215
3.1 测试平台与执行结果	215
3.2 指令集自检程序的组织方式	221
3.3 指令集 Feature List	222
3.4 指令集 Case 清单	223
3.5 汇编指令序列补充	224
4 riscv-dv 类别随机程序与 Spike 对照	224
4.1 riscv-dv 的使用方式	224
4.2 随机程序类别与规模	225
4.3 生成、编译与运行	226
4.4 RTL monitor 与完成信息采集	226
4.5 Spike 参考模型对照与时序一致性	227
4.6 代表性随机程序	227
5 边界条件 Case 设计	228
5.1 设计原则	228
5.2 数据冒险检测方法	228
5.3 流水线边界条件	229
5.4 访存边界条件	229
5.5 异常与恢复边界条件	230
5.6 边界条件 Case 清单	230
5.7 排序程序与编译组合	231
6 Feature List	232
6.1 CPU 前端、分发与执行 Feature List	232
6.2 中断、总线、存储器与 SoC Feature List	240
6.3 指令集 Feature List	244

7 测试用例清单与覆盖率统计	246
7.1 模块级 Case 清单	246
7.2 整核 Feature 与程序 Case 清单	247
7.3 访存参数 Case 清单	248
7.4 回归计划	249
7.5 覆盖率统计方法	249
7.6 验证结果说明	249
8 验证结果	250
8.1 回归执行状态	错误！未定义书签。
8.2 覆盖率统计状态	错误！未定义书签。
9 性能分析	250
9.1 配置定义与统计条件	253
9.2 RV32 统一时钟条件下的 CoreMark 对比	253
9.3 RV64 CoreMark 与 RV32 对比	254
9.4 Cache 与乒乓缓存性能对比	255
9.5 Cache 资源、命中率与时序分析	256
9.6 RTL structure 与时序优化辅助分析	256
9.7 性能分析结论	258
10 FPGA 实现与上板结果	258
10.1 实现条件与六种配置	258
10.2 资源占用与 CoreMark 上板截图占位	259
10.3 实现结果说明	263
RISC-V CPU 软件设计介绍	264
1 软件系统结构	264
1.1 软件组成与硬件接口	264
1.2 地址空间与构建配置	265
2 裸机 BSP 设计	265
2.1 启动与链接	266
2.2 陷入入口与上下文	267
2.3 CLINT 驱动	268
2.4 PLIC 驱动	269
2.5 APB 外设与引脚复用	270
2.6 运行时支持	271
3 编译器配置与指令调度	271
3.1 指令集与 ABI	271
3.2 Alkaid 调度选项	272
3.3 构建参数与硬件一致性	272
4 RT-Thread Nano 移植	273
4.1 libcpu 与线程上下文	273
4.2 板级初始化与中断接入	273
4.3 UART 控制台	274
4.4 链接脚本	274
5 THP 温湿度气压与 OLED 应用	275
5.1 应用结构	275

5.2 传感器探测与数据处理	275
5.3 I2C 访问	276
5.4 OLED 显示	276
5.5 MSH 命令与后台刷新	277

快速预览

一、 作品概览与设计定位

Alkaid 是一款面向 FPGA 实现和 IP 核复用场景的高度可配置 RISC-V CPU 与 SoC。设计将指令集扩展、分支预测结构、访存特性、外设实例、存储初始化、AXI4-Lite Cache 和乒乓缓存统一纳入参数化配置，使同一套 RTL 能够根据目标应用裁剪为不同面积、性能和外设功能的组合。处理器基础配置为 RV32IM + Zicsr，可通过参数配置扩展到 RV64IM、C/A、Zba/Zbb/Zbs/Zbc，并可启用非对齐访存、LSU L0 Cache、ICache、DCache、基于指令预解码或分支目标缓冲器（BTB）的分支预测、TAGE 分支预测，以及实现对外设模块的裁剪。

二、 核心微架构创新点

- 参数化三级、四级和五级流水线：

处理器主通路由取指单元（IFU）、指令译码单元（IDU）、分发单元（Dispatch）、执行单元（EXU）和写回单元（WBU）组成，依次完成取指与预测、译码、数据冒险检查与发射、指令执行和结果写回。

PIPELINE_STAGES 可选择三级、四级或五级流水线。五级配置将取指、译码、发射、执行和写回分别设置为一级；四级配置不使用 idu_id_pipe 的寄存输出，将译码与发射合并为一级；三级配置进一步取消 dispatch_pipe 的寄存功能，将译码、发射与短延迟执行合并为一级。三种配置均保留 IFU 指令响应后的 ifu_pipe 和 EXU 完成结果后的 WBU 结果寄存器，外部接口以及有效、暂停和冲刷规则保持一致。

- 轻度乱序单发射与有限乱序完成：

处理器每周期最多发射一条指令。Dispatch 从当前译码指令和单项等待槽中选择发射对象，检查读后写（RAW）冒险、写后写（WAW）冒险、结构冒险和顺序冒险，分配 commit id，预计算访存与跳转地址，并生成旁路控制信号。EXU 内部包含算术逻辑单元（ALU）、分支执行单元（BRU）、加载存储单元（LSU）、乘法单元（MUL）、除法单元（DIV）和控制状态寄存器（CSR）执行单元。在保证架构状态正确的前提下，程序顺序靠后的短延迟指令可以先于此前的长延迟指令完成；WBU 负责写回仲裁和架构状态更新。

数据冒险检查单元（HDU）内部实现 4 个 commit id 槽位。四级和五级配置保存目的寄存器、有序属性、写后写阻塞属性以及 ALU、LSU、MUL 局部旁路的可选状态，并用两份最近分配表查询 rs1 和 rs2 的依赖；三级配置使用四项紧凑比较结构，关闭 EXU 局部旁路。三级和五级 Dispatch 可保存一条暂时不能发射的非有序类指令，四级配置则保持当前输入。相较完整重排序缓冲区（ROB），该结构的状态规模较小，同时覆盖高频的短距离 RAW、允许重叠的 WAW 和长延迟指令完成冲突。

- 参数化设计：

处理器通过统一配置参数控制指令集、访存能力、分支预测结构、AXI4-Lite Cache、乒乓缓存和 SoC 外设。基础配置为 RV32IM + Zicsr，处理器数据宽度由 XLEN 选择为 32 位或 64 位，C/A、Zba/Zbb/Zbs/Zbc、非对齐访存、LSU L0 Cache、ICache、DCache、乒乓缓存、存储初始化和外设实例均由参数配置。分支预测单元（BPU）也采用条件生成实现：基础 BPU 面向面积约束，包含静态偏置、分支历史表（BHT）、可选循环预测器、可选相关方向预测、JALR 目标缓存和返回地址栈（RAS）；增强版 BPU 由 ENABLE_BPU_ENHANCED 生成，包含分支目标缓冲器（BTB）、GShare 方向预测器、增强 RAS 和预测状态字段；TAGE 作为增强 BPU 下的独立可选项，由 ENABLE_BPU_TAGE 与表项参数控制。Zbc 关闭时，译码控制位保持为 0，Zbc 多周期执行状态不生成；其它功能关闭时，相应表项、选择器、Cache 状态和外设硬件不生成。SoC 顶层端口集合保持固定，关闭外设后由系统给出相应的固定输出值。

- 短距离依赖与热点访存优化：

性能优化微架构围绕短距离 RAW 依赖、加载结果被紧随其后的条件分支读取，以及分支预测错误后的取指地址修正和错误路径指令清除展开。四级和五级配置中，ALU 与 LSU 结果可供随后的 ALU、条件分支、MUL 和存储数据使用，MUL 结果可供 ALU、MUL 和存储数据使用；三级配置关闭 EXU 局部旁路。存储器访问基址依赖由地址生成单元（AGU）保存项在 Dispatch 内解决，JALR 则等待寄存器值写回后由 BRU 计算目标。LSU L0 Cache 采用 Dispatch 影子标签加 EXU 数据体的实现方式，并按单路组相联结构组织表项；命中数据与普通 LSU 读返回数据一样写入 EXU 局部旁路和 WBU，用较小硬件开销减少条件分支等待前序加载结果的次数。

三、 处理器配置概览

处理器提供默认配置和可选功能配置。默认配置采用五级流水线、RV32IM + Zicsr，并关闭 C、A、Zba、Zbb、Zbs、Zbc、非对齐访存、LSU L0 Cache、增强 BPU、TAGE、ICache、DCache 和乒乓缓存；可选配置通过参数打开对应指令扩展、流水线结构、预测器、访存缓冲和外设。设计章节统一说明各配置的结构、接口和参数边界。

四、 THP 外设应用与硬件实现预览

本项目在 RT-Thread Nano 移植基础上实现了温湿度气压传感器与 OLED 显示应用，记为 THP 应用。该应用不是单一外设读写示例，而是将环境传感器采集、I2C 总线访问、GPIO 引脚复用、UART 控制台输出、OLED 页面刷新、RTOS 线程调度和 MSH 命令交互组合为一个完整的应用工作流程，用于展示处理器从底层硬件接口到上层应用软件的协同工作能力。THP 应用的硬件连接以 GPIO0[18] 作为 I2C SCL、GPIO0[19] 作为 I2C SDA，外接 AHT30、HDC3020、BME280、BME680 等环境传感器以及 SSD1306 OLED 显示模块。传感器侧覆盖温度、湿度、气压及气体电阻等环境量读取，OLED 侧用于显示最新采样结果。应用支持两种 I2C 后端：在软件 I2C 模式下，GPIO0[18:19] 作为普通 GPIO 通过输入/输出方向切换模拟开漏总线行为；在硬件 I2C 模式下，软件通过 GPIO0.IOFCFG 将 GPIO0[18:19] 复用为 I2C0 SCL/SDA，由挂载在 AXI2APB 后端的 APB I2C0 控制器产生 START、WRITE、READ、STOP 等总线时序。

软件层面，THP 应用由 MSH 命令层、I2C 抽象层、传感器适配层和 OLED 显示层组成。用户在 RT-Thread MSH 中输入 thp、thp read、thp auto、thp status 等命令后，系统会自动完成 I2C 设备探测、在线传感器识别、原始数据读取、物理量换算、UART 终端打印和 OLED 刷新。该设计使 THP 应用既能作为板级外设调试入口，也能作为 Alkaid SoC 运行 RTOS 应用的综合展示用例。

五、 系统级集成与软硬件组成

处理器采用 AXI4-Lite 协议主接口访问指令和数据空间。IFU 通过 M0 只读主接口访问 ICache，ICache 未命中请求经 M0 乒乓缓存和 AXI 互连访问 IMEM；LSU 通过 M1 读写主接口访问 DCache，DCache 未命中、写通请求及不可缓存访问经 M1 乒乓缓存访问 DMEM、核心本地中断控制器（CLINT）、平台级中断控制器（PLIC）和 APB 外设。SoC 层面集成

CLINT、PLIC、AXI 互连、AXI2APB 桥、UART/GPIO/Timer/SPI/I2C 外设，以及 IMEM 和 DMEM 初始化参数。Vivado IP GUI 中的指令集、BPU、LSU、Cache、乒乓缓存、外设和存储初始化参数需要与工程配置保持一致。

软件侧包含裸机板级支持包（BSP）、CLINT/PLIC/UART/GPIO/SPI/I2C/Timer 驱动、GPIO0.IOFCFG 引脚复用驱动、RT-Thread Nano BSP 和 Alkaid GCC -mtune 模型。BSP 通过启动程序、链接脚本和存储器编址输入输出（MMIO）驱动完成从复位到 main 或 RT-Thread 内核入口的启动流程；RT-Thread Nano 通过 CLINT 定时器节拍、软件中断、PLIC 外部中断和 UART 控制台完成基本实时系统运行。

RISC-V CPU 设计介绍

1 项目概述

本项目设计并实现了一个基础配置为 RV32IM + Zicsr、可参数化扩展到 RV64/C/A/Zba/Zbb/Zbs/Zbc 的 Alkaid RISC-V CPU 与 SoC。设计的核心目的不是单纯堆叠单点功能，而是形成可作为 FPGA IP 核交付的统一配置模型：指令集架构（ISA）扩展、分支预测结构、LSU 特性、外设实例、SoC 顶层端口、存储初始化和 Vivado 图形化配置界面（GUI）设置均通过参数传递和条件生成结构保持一致。CPU 在功能正确的基础上重点解决三类工程问题：经 AXI4-Lite 访问的同步存储器带来的取指/访存延迟、有限硬件开销下的流水线冒险解除、以及 FPGA 实现时的面积与时序收敛。指令集扩展、分支预测、LSU L0 Cache、非对齐访存、外设和存储初始化均由统一配置通路控制。

1. 作品设计特色如下：

2. 指令集支持：基础配置覆盖 RV32I 基础整数、RV32M 乘除扩展和 Zicsr；通过参数支持 RV64、C、A、Zba、Zbb、Zbs 和 Zbc 等扩展。Zbc 在现有乘法执行单元内实现 `clmul`、`clmulh` 和 `clmulr`，默认将操作数按 16 位片段逐次处理。扩展关闭时，相应译码、流水数据字段和执行单元控制不生成。

3. 流水线结构：PIPELINE_STAGES 支持三级、四级和五级三种配置，通过条件生成选择译码与发射级的寄存器边界，不改变处理器核外部接口。Dispatch 承担操作数选择、执行单元分发、冒险检测和部分地址预计算。

4. 有限乱序完成与状态追踪：长延迟乘除、访存和普通 ALU 指令可按完成时机进入 WBU。四级和五级 HDU 通过最近分配表、commit id 槽位和执行类别跟踪 RAW 与 WAW，WBU 通过每个通用寄存器组（GPR）的最新 commit id 决定是否写入；三级 HDU 对全部同 rd 写入实施阻塞。

5. 数据旁路优化：ALU、LSU、MUL 结果通过 EXU 局部旁路暂存区、Dispatch AGU 前递保持和 WBU 提交前递分层传递。地址类 RAW 在 Dispatch 侧解决，执行级操作数尽量在 EXU 内部局部选择，减少跨模块组合长路径。

6. 总线与外设：取指和数据访存通过 AXI 互连连接 IMEM、DMEM、APB、CLINT、PLIC 及 UART/GPIO/Timer/SPI/I2C 等可选外设，并使用未完成事务跟踪与先进先出队列（FIFO）缓冲提高吞吐。

7. 分支预测：基础 BPU 使用小型 BHT、静态偏置、循环预测器、相关方向预测、JALR 目标缓存和 RAS；增强 BPU 使用 BTB、GShare 和增强 RAS，并可选 TAGE。增强结构通过条件生成实现，关闭时不生成相关表项和控制逻辑。

8. 配置交付：通过统一的参数表、SoC 顶层接口和 Vivado IP GUI 组织 CPU、存储器、Cache、乒乓缓存及外设配置，保证不同配置具有一致的端口定义和复位行为。

本设计的 SoC 架构如图 1 所示。

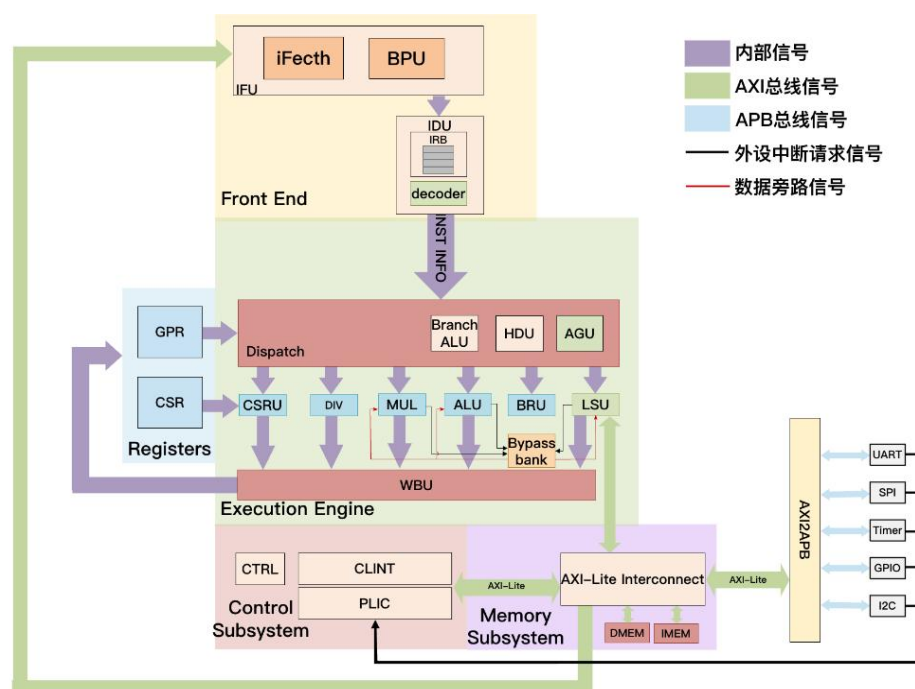


图 1 SoC 架构示意图

本 CPU 设计实现参数化分支预测单元（BPU），执行单元集成分支运算单元（BRU）、ALU 数据通路、乘法器、试商法除法器、CSR 单元和加载存储单元（LSU）。运行中的 RAW、WAW、结构、控制和访存冒险由 Dispatch、HDU、三级与五级等待项、四级与五级 EXU 局部旁路、WBU 最新 commit id 过滤和 Ctrl 协同处理。BRU 负责普通控制流校验，CLINT 负责异常、中断入口和 mret 返回；两者的地址输入保持独立，IFU 以中断地址优先。

CPU 通过 AXI4-Lite 协议接口访问 SoC 内的 IMEM 指令存储器从设备和 DMEM 数据存储器从设备。从接口类别看，二者是 SoC 存储器地址空间内的存储类从设备，与 CLINT、PLIC、AXI2APB 后端外设一样挂接在 AXI 互连目标端口上，由互连完成 M0/M1 主机仲裁、地址解码、未完成事务跟踪以及 CPU 与各目标设备之间的连接。

Cache 采用固定的单字缓存行设计，ICache 以一路 2048 行组织，DCache 以两路共 1024 行组织，读未完成请求和响应位置的深度均为 2。ICache 只处理读请求，DCache 采用读分配和写通策略；Cache 之外的乒乓缓存只保存 AXI4-Lite 通道字段，不参与命中判断、请求编号或返回顺序。该划分使 Cache 控制器负责本地存储一致性和响应顺序，AXI 互连负责目标选择与从设备事务管理。

1.1 设计层次与配置组织

Alkaid 按照 SoC 系统层、CPU 流水线层、存储访问层、中断控制层和外设层组织硬件。各层只通过已定义的模块接口交换请求、响应和控制状态，参数由系统层逐级传入需要生成相应结构的模块。主要层次如下表所示。

设计层次	主要组成	功能说明
SoC 系统层	CPU、AXI 互连、AXI2APB、IMEM、DMEM、CLINT、PLIC 和外设顶层	完成模块连接、地址分配、外设裁剪、存储初始化以及系统级中断接入。
CPU 前端	IFU、基础 BPU、增强 BPU、IDU 和指令保留栈	完成取指、指令流整理、分支预测、指令缓冲和译码。
分发与控制	Dispatch、HDU 和 Ctrl	完成操作数读取、数据冒险检查、commit id 分配、执行请求生成、暂停和冲刷控制。
执行与写回	EXU、ALU、BRU、LSU、MUL、DIV、CSR 执行单元、WBU、GPR 和 CSR	完成算术、控制转移、访存、乘除、系统寄存器访问、完成结果仲裁和架构状态写入。
存储访问层	ICache、DCache、AXI4-Lite 乒乓缓存、IMEM 和 DMEM	完成取指与数据访问缓存、未完成请求管理、通道寄存以及片内存储器访问。
中断与外设层	CLINT、PLIC、AXI2APB、Timer、SPI、I2C、UART 和 GPIO	完成异常与中断处理、外部中断仲裁、总线转换和片外设备控制。

表 1 Alkaid 硬件设计层次

配置值先在 SoC 系统层确定，再传入 CPU、Cache、乒乓缓存、存储器和外设顶层。CPU 继续把流水线级数、指令集扩展、LSU L0 Cache 和分支预测参数传入相应子模块。ENABLE_BPU_ENHANCED 和 ENABLE_BPU_LEGACY 共同选择增强 BPU、基础 BPU 或无预测器结构，ENABLE_BPU_TAGE 只在增强 BPU 内控制 TAGE 结构。ENABLE_LSU_HOT_CACHE、ENABLE_MISALIGNED_DATA、ENABLE_C、ENABLE_A、ENABLE_ZBA、ENABL

E_ZBB、ENABLE_ZBS 和 ENABLE_ZBC 分别控制对应硬件结构；参数关闭时不生成相应状态、数据选择和控制逻辑。

Cache 的推荐系统配置由 SoC 层统一传递：ICache 使用 2048 行、1 路和深度为 2 的读请求控制，DCache 使用 1024 行、2 路和深度为 2 的读请求控制；M0 读地址通道使用一级直通乒乓缓存，M0 读数据通道保持直连。M1 的推荐级数为 1，AW、W、B、AR 和 R 五个通道均使用一级非直通乒乓缓存。Cache 的几何参数不随 PIPELINE_STAGES 改变，流水线级数只影响 CPU 内部的请求产生和结果接收边界。

1.2 硬件环境与 FPGA 目标平台

本设计面向 Xilinx Artix-7 xc7a200tfbg484-3 器件。FPGA 工程根据流水线级数和功能配置设置时钟约束，三级、四级和五级配置采用各自独立的目标频率，不以单一频率限定全部配置。Vivado 工程通过 Block Design 直接例化

Alkaid IP 核，顶层端口由 Block Design 顶层封装和 IP 核参数共同决定。IP GUI 配置界面如下图所示。

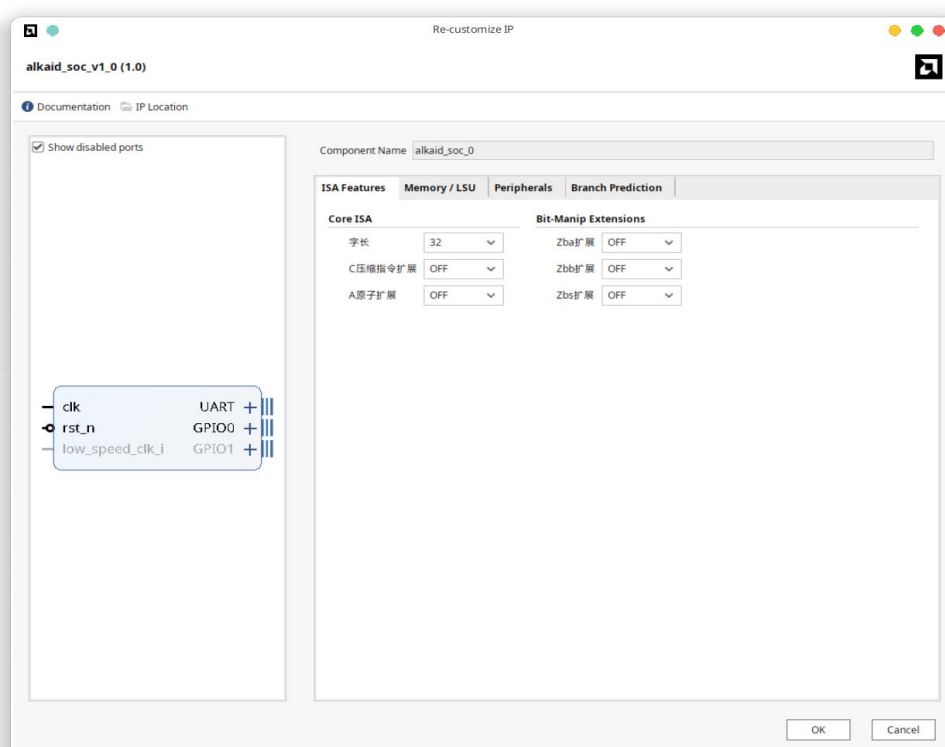


图 2 ISA Features 配置图

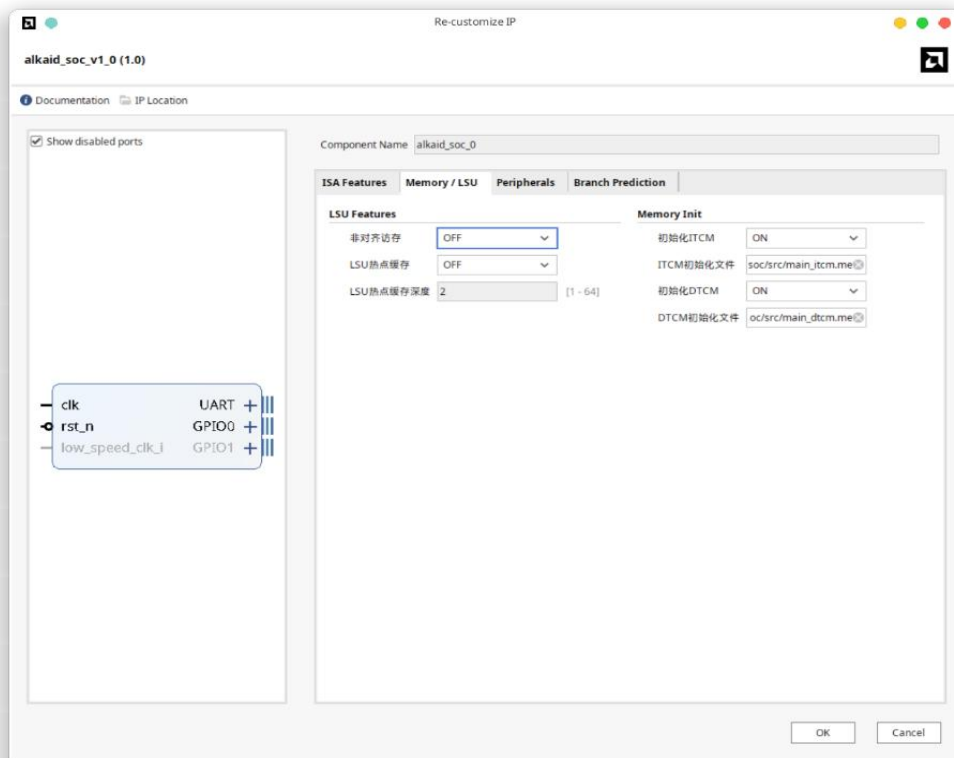


图 3 Memory 和 LSU L0 Cache 配置图

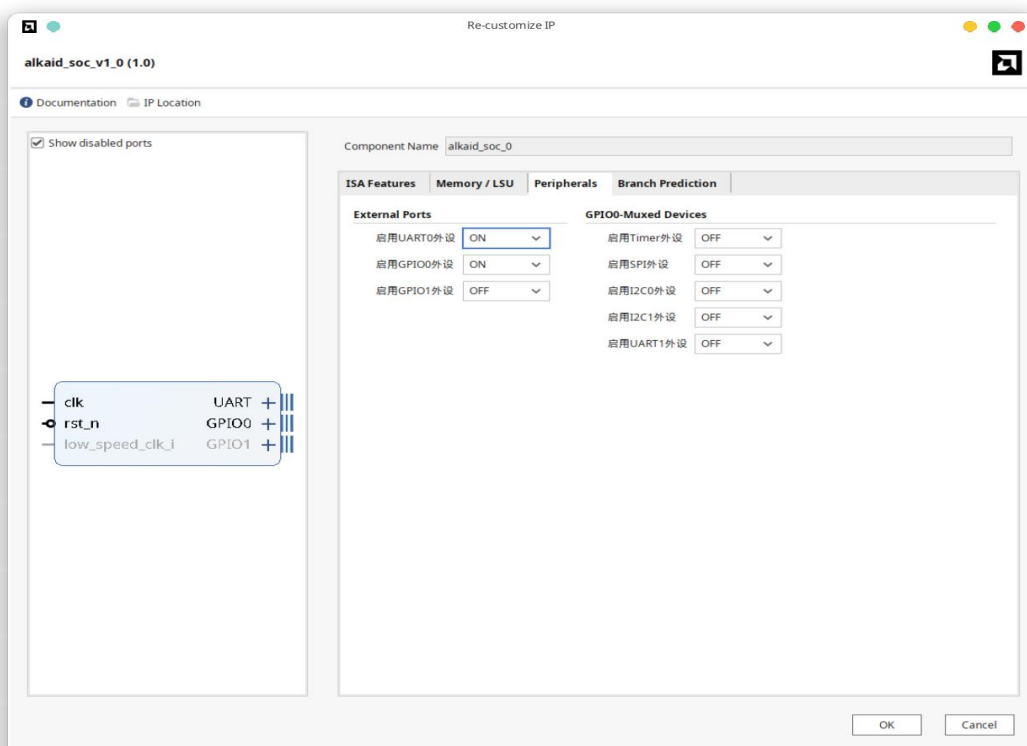


图 4 外设配置图

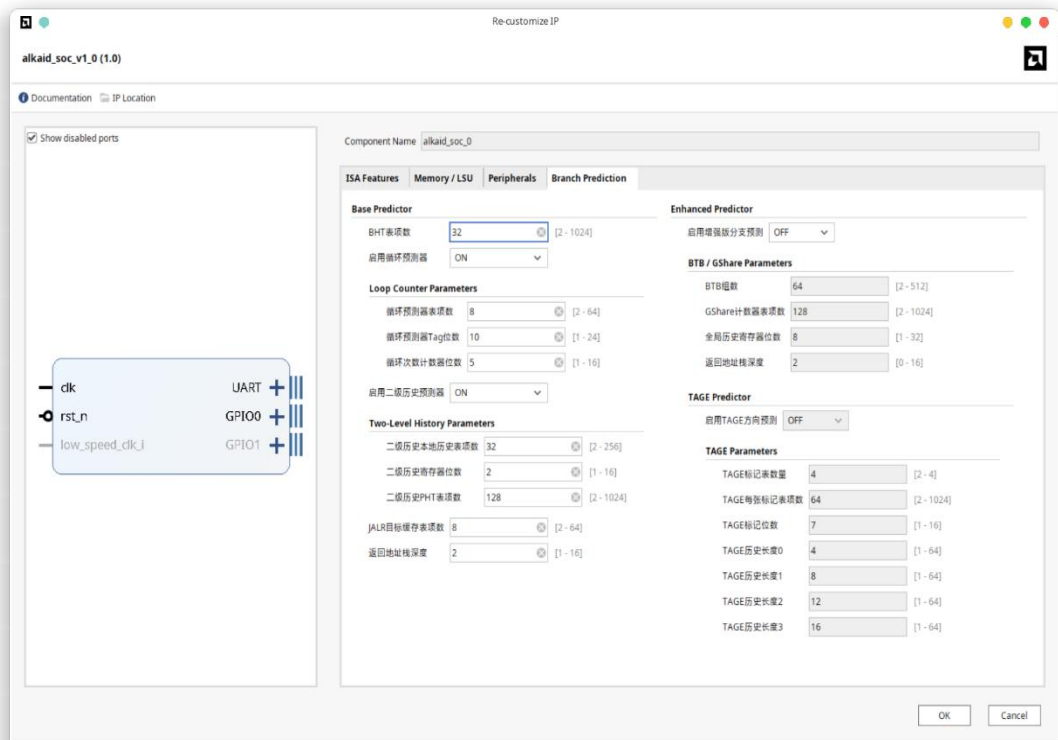


图 5 分支预测配置图

软件构建生成指令/数据存储初始化文件，Vivado 工程纳入 RTL、头文件目录、BD 顶层封装、约束文件和存储初始化文件。下表列出会影响 RTL 结构、SoC 端口或 Vivado IP GUI 的主要配置项；默认值对应当前工程默认配置。

配置项	默认值	说明
XLEN	32	选择 RV32 或 RV64 数据通路 with 软件 ABI。
PIPELINE_STAGES	5	选择三级、四级或五级主流水线，并确定译码后和发射后的寄存器边界。
ENABLE_C	0	控制 RVC 压缩指令取指、解压和指令长度信息。
ENABLE_A	0	控制 LR/SC/AMO 原子访存控制和 LSU 原子序列。
ENABLE_ZBA	0	控制地址生成类位操作扩展译码与 ALU 数据通路。
ENABLE_ZBB	0	控制基础位操作扩展译码与 ALU 数据通路。
ENABLE_ZBS	0	控制单比特置位、清零、取反等位操作扩展。

配置项	默认值	说明
ENABLE_ZBC	0	控制 clmul、clmulh 和 clmulr 无进位乘法扩展。
ENABLE_MISALIGNED_DATA	0	控制 LSU 非对齐拆分访问路径。
ENABLE_LSU_HOT_CACHE	0	控制可选 LSU L0 Cache 结构。
LSU_HOT_CACHE_DEPTH	2	控制 LSU L0 Cache 每路深度。
LSU_HOT_CACHE_WAYS	1	控制 LSU L0 Cache 路数。
BPU_BHT_ENTRIES	32	控制基础 BPU 方向计数器表项数。
ENABLE_BPU_LOOP_PRED	1	控制基础/增强 BPU 循环预测器是否生成。
BPU_LOOP_ENTRIES	8	控制循环预测器表项数；硬件将小于 2 的取值调整为 2，并将其余取值调整为不小于请求值的 2 的幂。
BPU_LOOP_TAG_WIDTH	10	控制循环预测器 PC 标签位宽。
BPU_LOOP_COUNT_WIDTH	5	控制循环迭代计数字段位宽。
ENABLE_BPU_CORR_PRED	1	控制基础 BPU 相关方向预测器是否生成。
BPU_CORR_HISTORY_ENTRIES	32	控制相关方向预测器局部历史表项数。
BPU_CORR_HISTORY_BITS	2	控制相关方向预测器局部历史长度。
BPU_CORR_PHT_ENTRIES	128	控制相关方向预测器模式历史表项数。
BPU_JALR_ENTRIES	8	控制基础 BPU JALR 目标缓存表项数。
BPU_RAS_DEPTH	2	控制基础 BPU 返回地址栈深度。
ENABLE_BPU_ENHANCED	0	为 1 时生成增强 BPU；为 0 时是否生成基础 BPU 由 ENABLE_BPU_LEGACY 决定，SoC 默认启用基础 BPU。
ENABLE_BPU_TAGE	0	控制增强 BPU 内的 TAGE 表项，只有增强 BPU 打开时有意义。

配置项	默认值	说明
BPU_BTBTB_SETS	128	控制增强 BPU BTB 组数。
BPU_GSHARE_ENTRIES	256	控制增强 BPU GShare PHT 表项数。
BPU_GHR_BITS	2	控制增强 BPU 全局历史长度和流水携带的 GHR 快照位宽。
BPU_ENH_RAS_DEPTH	2	控制增强 BPU 返回地址栈深度。
BPU_TAGE_TABLES	4	控制 TAGE 表数量；硬件采用的有效表数限制为 2 到 4。
BPU_TAGE_ENTRIES	128	控制每张 TAGE 表项数。
BPU_TAGE_TAG_WIDTH	7	控制 TAGE 标签位宽。
BPU_TAGE_HIST0	4	控制第 0 张 TAGE 表历史长度。
BPU_TAGE_HIST1	8	控制第 1 张 TAGE 表历史长度。
BPU_TAGE_HIST2	12	控制第 2 张 TAGE 表历史长度。
BPU_TAGE_HIST3	16	控制第 3 张 TAGE 表历史长度。
INIT_ITCM	0	控制 IMEM 是否使用初始化文件装载初值。
ITCM_INIT_FILE	main_itcm.mem	指定 IMEM 初始化文件。
INIT_DTCM	0	控制 DMEM 是否使用初始化文件装载初值。
DTCM_INIT_FILE	main_dtcmmem	指定 DMEM 初始化文件。
DTCM_REGISTER_RDATA	0	控制 DMEM 读数据是否增加一级输出寄存器；启用后关闭 DTCM 加载提前完成。
M0_AXI_PINGPONG_STAGES	0	设置 M0 读地址通道的乒乓缓存级数，合法值为 0、1 或 2；M0 读数据通道保持零级。
M0_AXI_PINGPONG_FALL_THROUGH	0	控制 M0 读地址通道在缓存为空时是否允许直接完成传输。

配置项	默认值	说明
M1_AXI_PINGPONG_STAGE S	0	设置 M1 AW、W、B、AR 和 R 五个通道共同采用的乒乓缓存级数，合法值为 0、1 或 2。
ENABLE_ICACHE	0	控制 M0 侧 ICache 是否生成。
ICACHE_ENTRIES	64	设置 ICache 全部路合计的缓存行数。
ICACHE_WAYS	1	设置 ICache 路数，合法值为 1 或 2。
ICACHE_OUTSTANDING_DEPTH	2	设置 ICache 未命中请求 FIFO 和响应 FIFO 深度。
ICACHE_PREFETCH	1	保留 ICache 预取配置接口；当前控制器不发起预取请求。
ENABLE_DCACHE	0	控制 M1 侧 DCache 是否生成。
DCACHE_ENTRIES	64	设置 DCache 全部路合计的缓存行数。
DCACHE_WAYS	2	设置 DCache 路数，合法值为 1 或 2。
DCACHE_OUTSTANDING_DEPTH	2	设置 DCache 读输入 FIFO、未命中请求 FIFO 和响应 FIFO 深度。
ENABLE_PERIP_TIMER	0	控制 Timer/PWM APB 外设和相关端口。
ENABLE_PERIP_SPI	0	控制 SPI0 APB 外设和相关端口。
ENABLE_PERIP_I2C0	0	控制 I2C0 APB 外设和相关端口。
ENABLE_PERIP_I2C1	0	控制 I2C1 APB 外设和相关端口。
ENABLE_PERIP_UART0	1	控制 UART0 调试串口外设和独立顶层端口。
ENABLE_PERIP_UART1	0	控制 UART1 外设；顶层引脚复用依赖 GPIO0.IOFCFG。
ENABLE_PERIP_GPIO0	1	控制 GPIO0 外设和 IOF 引脚复用寄存器。
ENABLE_PERIP_GPIO1	0	控制 GPIO1 外设和独立 GPIO1 端口。

表 2 参数配置表

表 2 中的默认值表示通用 SoC 配置的编译期默认值，不等同于 Cache 推荐配置。Cache 推荐配置的差异如下表所示。

配置项	推荐值	设计含义
ENABLE_ICACHE	1	在 M0 读通道生成 ICache。
ICACHE_ENTRIES	2048	ICache 的全部路合计保存 2048 条缓存行。
ICACHE_WAYS	1	采用一路组织，不生成多路命中选择和有效的路替换状态。
ICACHE_OUTSTANDING_DEPTH	2	允许两项读未命中请求和两项响应位置同时处于有效状态。
ICACHE_PREFETCH	0	保留参数接口，但不生成下一字预取请求。
ENABLE_DCACHE	1	在 M1 读写通道生成 DCache。
DCACHE_ENTRIES	1024	DCache 的全部路合计保存 1024 条缓存行。
DCACHE_WAYS	2	采用两路组相联和每组一位的真 LRU 状态。
DCACHE_OUTSTANDING_DEPTH	2	允许两项读未命中请求和两项响应位置同时处于有效状态。
M0_AXI_PINGPONG_STAGES	1	在 M0 ICache 后插入一级 AR 通道乒乓缓存。
M0_AXI_PINGPONG_FALL_THROUGH	1	空缓存且下游就绪时允许 AR 字段直接传递。
M1_AXI_PINGPONG_STAGES	1	M1 的 AW、W、B、AR 和 R 五个通道共同使用一级乒乓缓存。

表 2-1 Cache 与乒乓缓存推荐系统配置

2 CPU 顶层接口

2.1 时钟及复位

用于为 CPU 与其所有外设/总线接口提供统一的时序基准、复位条件。

信号名	方向	位宽	信号描述
clk	输入	1	CPU 核心系统时钟。
rst_n	输入	1	低电平有效异步复位，清除流水线有效状态和控制状态。

表 3 时钟及复位信号接口

2.2 中断与异常控制接口

CLINT 在 SoC 层实例化，core_top 通过专用端口向 CLINT 提供当前指令、控制转移、异常和访存状态，再接收 CLINT 给出的陷入有效信号与目标地址。外设中断先由 11 路 PLIC 完成使能和优先级仲裁，PLIC 的单比特 irq_valid 输出接入 CLINT 的 ext_int_req_i。该路径是当前 SoC 的有效外部中断路径。

信号名	方向	位宽	信号描述
clint_int_addr_i	输入	INST_ADDR_WIDTH	CLINT 给出的陷入入口或 mret 返回地址。
clint_int_jump_i	输入	1	CLINT 给出的取指跳转有效信号。
clint_int_assert_i	输入	1	CLINT 给出的陷入处理有效信号。
clint_inst_addr_o	输出	INST_ADDR_WIDTH	送往 CLINT 的当前指令地址。
clint_inst_data_o	输出	INST_DATA_WIDTH	送往 CLINT 的当前指令内容。
clint_inst_valid_o	输出	1	Dispatch 当前指令有效且该指令不是已预测条件分支时有效。
clint_inst_len_2_o	输出	1	当前指令长度为 2 字节时有效，供 CLINT 计算异常返回地址。
clint_jump_flag_o	输出	1	当前 Dispatch 指令为 JAL，或 BRU 已确认控制转移时有效。
clint_jump_addr_o	输出	INST_ADDR_WIDTH	当前 BRU 实际下一取指地址。
clint_stall_flag_o	输出	CU_BUS_WIDTH	当前流水线暂停和冲刷状态。
clint_atom_opt_busy_o	输出	1	Dispatch 原子长指令锁定或 EXU 存储事务未完成。
clint_exu_stall_o	输出	1	EXU 暂停状态。
clint_sys_op_ecall_o	输出	1	当前指令为 ecall。
clint_sys_op_ebreak_o	输出	1	当前指令为 ebreak。
clint_sys_op_mret_o	输出	1	当前指令为 mret。

信号名	方向	位宽	信号描述
clint_illegal_inst_o	输出	1	当前指令非法。
clint_misaligned_load_o	输出	1	加载地址不对齐异常。
clint_misaligned_store_o	输出	1	存储地址不对齐异常。
clint_misaligned_fetch_o	输出	1	取指目标地址不对齐异常。
clint_we_i	输入	1	CLINT 请求写 CSR。
clint_waddr_i	输入	BUS_ADDR_WIDTH	CLINT 写 CSR 地址。
clint_raddr_i	输入	BUS_ADDR_WIDTH	CLINT 读 CSR 地址。
clint_data_i	输入	REG_DATA_WIDTH	CLINT 写 CSR 数据。
clint_data_o	输出	REG_DATA_WIDTH	CLINT 读 CSR 数据。
clint_csr_mtvec_o	输出	REG_DATA_WIDTH	mtvec 当前值。
clint_csr_mepc_o	输出	REG_DATA_WIDTH	mepc 当前值。
clint_csr_mstatus_o	输出	REG_DATA_WIDTH	mstatus 当前值。
clint_csr_mie_o	输出	REG_DATA_WIDTH	mie 当前值。
irq_sources	输入	8	为 core_top 接口兼容保留，当前核内逻辑不读取该端口。SoC 顶层仍连接 irq_vec[7:0]，实际外部中断由完整的 11 位 irq_vec 直接送入 PLIC。

表 4 中断与异常控制接口

2.3 AXI 接口

AXI 接口用于 core_top 与系统互连及存储模块之间传输指令和数据。core_top 对外提供两个 AXI4-Lite 主接口：M0_AXI 是 IFU 的只读接口，只包含 AR 和 R 通道；M1_AXI 是 LSU 的读写访存与外设访问接口，包含 AW、W、B、AR 和 R 五个通道。设计不支持 Burst、ID、USER、LAST 等 AXI4 Full 扩展信号，访问并发性由未完成请求跟踪、Cache 响应 FIFO 和乒乓缓存共同提供。

参数名	默认值	参数描述
BUS_ADDR_WIDTH H	32	AXI4-Lite 地址通道位宽。
BUS_DATA_WIDTH H	XLEN	AXI4-Lite 数据通道位宽；写数据字节使能位宽为 BUS_DATA_WIDTH/8。

表 5 AXI 接口参数

2.3.1 接口 M0_AXI

M0_AXI 是 CPU 的取指主接口（IFU -> AXI 互连 -> IMEM），实现 AXI4-Lite 只读协议子集，仅包含读地址 AR 通道和读数据 R 通道。

信号名	方向	位宽	信号描述
M0_AXI_ARADDR	输出	INST_ADDR_WIDTH	IFU 发出的读地址，默认位宽为 32 位。
M0_AXI_ARPROT	输出	3	读访问属性，当前置为 3'b000。
M0_AXI_ARVALID	输出	1	读地址有效，保持到与 M0_AXI_ARREADY 完成握手。
M0_AXI_ARREADY	输入	1	下游可以接收读地址时有效。
M0_AXI_RDATA	输入	BUS_DATA_WIDTH	下游返回的指令数据。
M0_AXI_RRESP	输入	2	读响应状态，错误响应由 IFU 保存。
M0_AXI_RVALID	输入	1	下游读数据有效。
M0_AXI_RREADY	输出	1	IFU 可以接收读数据时有效。

表 6 接口 M0_AXI 定义

2.3.2 接口 M1_AXI

M1_AXI 是 CPU 的数据访存与系统访问主接口（LSU/EXU -> AXI 互连），实现 AXI4-Lite 读写协议，包含 AW、W、B、AR、R 五个通道，可访问 IMEM、DMEM、APB 外设、CLINT 和 PLIC 等目标。

信号名	方向	位宽	信号描述
M1_AXI_AWADDR	输出	BUS_ADDR_WIDTH	写地址，由 LSU 访存地址产生。

信号名	方向	位宽	信号描述
M1_AXI_AWPROT	输出	3	写访问属性，当前置为 3'b000。
M1_AXI_AWVALID	输出	1	写地址有效，保持到与 M1_AXI_AWREADY 完成握手。
M1_AXI_AWREADY	输入	1	下游可以接收写地址时有效。
M1_AXI_WDATA	输出	BUS_DATA_WIDTH	写数据。
M1_AXI_WSTRB	输出	BUS_DATA_WIDTH/ 8	逐字节写使能。
M1_AXI_WVALID	输出	1	写数据有效，保持到与 M1_AXI_WREADY 完成握手。
M1_AXI_WREADY	输入	1	下游可以接收写数据时有效。
M1_AXI_BRESP	输入	2	下游写响应状态。
M1_AXI_BVALID	输入	1	下游写响应有效。
M1_AXI_BREADY	输出	1	LSU 可以接收写响应时有效。
M1_AXI_ARADDR	输出	BUS_ADDR_WIDTH	读地址，由 LSU 访存地址产生。
M1_AXI_ARPROT	输出	3	读访问属性，当前置为 3'b000。
M1_AXI_ARVALID	输出	1	读地址有效，保持到与 M1_AXI_ARREADY 完成握手。
M1_AXI_ARREADY	输入	1	下游可以接收读地址时有效。
M1_AXI_RDATA	输入	BUS_DATA_WIDTH	下游返回的读数据。
M1_AXI_RRESP	输入	2	下游读响应状态。
M1_AXI_RVALID	输入	1	下游读数据有效。
M1_AXI_RREADY	输出	1	LSU 可以接收读数据时有效。

表 7 接口 M1_AXI 定义

2.3.3 互连内部 CLINT 目标端口

CLINT 不是 core_top 对外暴露的第三个主机接口，而是 alkaid_soc_top 内部 AXI 互连的一个目标端口。LSU 通过 M1_AXI 发起对地址 0x0200_0000 - 0x0200_FFFF 的访问，互连在地址阶段命中 CLINT 区域后，将 AW/W/B/AR/R 五个 AXI4-Lite 通道转发给 clint_swi 寄存器接口。软件通过该访问路径读写 msip、mtimecmp 和 mtime，硬件陷入逻辑则在核心内部直接读取 CSR 状态并输出中断跳转控制。

目标端口	地址范围	访问来源	主要寄存器	说明
CLINT AXI4-Lite slave	0x0200_0000 - 0x0200_FFFF	M1_AXI/LSU	msip、mtimecmp、mtime	提供机器模式软件中断和定时器中断 MMIO 访问

表 8 CLINT 内部目标端口定义

2.3.4 互连内部 PLIC 目标端口

PLIC 同样位于 SoC 内部，作为 AXI 互连的目标端口连接到 M1_AXI。LSU 对 0x0C00_0000 - 0x0C00_FFFF 区域的读写会被路由到 plic_top，软件通过寄存器配置中断使能、优先级、向量表和参数表；PLIC 仲裁出的外部中断请求通过核心内部 ext_int_req_i 送入 CLINT 陷入逻辑。

目标端口	地址范围	访问来源	主要寄存器	说明
PLIC AXI4-Lite slave	0x0C00_0000 - 0x0C00_FFFF	M1_AXI/LSU	INT_EN、INT_PRI、INT_VECTABLE、INT_OBJS、INT_MVEC、INT_MARG	管理外部中断源、优先级和中断向量分发

表 9 PLIC 内部目标端口定义

2.4 处理器参数化配置

处理器核心通过参数选择流水级数、指令集行为、非对齐数据访问和分支预测结构。寄存器与数据总线宽度由编译期 XLEN 决定，ENABLE_RV64 选择 RV64 指令行为，两者必须保持一致。参数由 SoC 顶层传入 CPU 及相关子模块，各项配置不会改变第 2.1 至 2.3 节所列端口的数量和方向。

PIPELINE_STAGES 只允许取 3、4 或 5；其它使能参数使用 0 表示关闭，使用 1 表示启用。

参数名	CPU 默认值	SoC 默认配置	功能说明
ENABLE_RV64	0	由 XLEN==64 决定	选择 RV64 指令行为；寄存器和总线位宽由 XLEN 决定，二者必须一致。
PIPELINE_STAGES	5	5	选择三级、四级或五级主流水线。
ENABLE_ZBA	0	0	启用 Zba 地址生成指令。
ENABLE_ZBB	0	0	启用 Zbb 基本位操作指令。
ENABLE_ZBS	0	0	启用 Zbs 单比特操作指令。
ENABLE_ZBC	0	0	启用 Zbc 无进位乘法指令。
ENABLE_C	0	0	启用 C 压缩指令扩展。
ENABLE_A	0	0	启用 A 原子指令扩展。
ENABLE_MISALIGNED_DATA	0	0	启用非对齐加载与存储的拆分访问。
ENABLE_DTCM_LOAD_LOOKAHEAD	0	由 SoC 生成	仅在 DCache 关闭、DMEM 读数据输出寄存器关闭且 M1 乒乓缓存为零级时启用 DTCM 加载提前完成。
ENABLE_BPU_LEGACY	ENABLE_LSU_HOT_CACHE	ENABLE_LSU_HOT_CACHE !ENABLE_BPU_ENHANCED	CPU 核参数用于生成基础 BPU；SoC 内部按左述表达式派生，增强 BPU 优先，因此 SoC 配置不提供无 BPU 组合。
ENABLE_CSR_READ_PREDECODE	全局配置，默认 0	0	启用控制状态寄存器读选择预译码。

表 10 处理器基础参数

分支预测参数分别控制基础预测器的分支历史表、循环预测、相关历史预测、

间接跳转目标和返回地址栈，以及增强预测器的分支目标缓冲、gshare、返回地址栈和 TAGE 表。各容量参数只改变内部表项数量和索引宽度，不改变取指 AXI4-Lite 接口。

参数名	默认值	功能说明
BPU_BHT_ENTRIES	32	基础分支历史表项数。
ENABLE_BPU_LOOP_PRED	1	启用基础循环预测。
BPU_LOOP_ENTRIES	8	循环预测表项数。
BPU_LOOP_TAG_WIDTH	10	循环预测标签位宽。
BPU_LOOP_COUNT_WIDTH	5	循环次数位宽。
ENABLE_BPU_CORR_PRED	1	启用相关历史预测。
BPU_CORR_HISTORY_ENTRIES	32	相关历史表项数。
BPU_CORR_HISTORY_BITS	2	每项相关历史位数。
BPU_CORR_PHT_ENTRIES	128	相关模式历史表项数。
BPU_JALR_ENTRIES	8	间接跳转目标表项数。
BPU_RAS_DEPTH	2	基础返回地址栈深度。
ENABLE_BPU_ENHANCED	0	启用增强分支预测器。
ENABLE_BPU_TAGE	0	在增强分支预测器中启用 TAGE。
BPU_BTB_SETS	128	增强分支目标缓冲组数。
BPU_GSHARE_ENTRIES	256	gshare 预测表项数。
BPU_GHR_BITS	2	全局分支历史位数。
BPU_ENH_RAS_DEPTH	2	增强返回地址栈深度。
BPU_TAGE_TABLES	4	TAGE 历史表数量。
BPU_TAGE_ENTRIES	128	每张 TAGE 历史表的表项数。
BPU_TAGE_TAG_WIDTH	7	TAGE 标签位宽。
BPU_TAGE_HIST0	4	第 0 张 TAGE 表使用的历史长度。
BPU_TAGE_HIST1	8	第 1 张 TAGE 表使用的历史长度。
BPU_TAGE_HIST2	12	第 2 张 TAGE 表使用的历史长度。
BPU_TAGE_HIST3	16	第 3 张 TAGE 表使用的历史长度。

表 11 分支预测参数

下表列出 Cache 与乒乓缓存的配置参数，具体功能见第 2.5 节。

名称	配置类型	默认值	有效取值	功能说明
----	------	-----	------	------

名称	配置类型	默认值	有效取值	功能说明
ENABLE_ICACHE	全局配置	0	0、1	生成 M0 侧 ICache；关闭时 M0 读通道直通。
ICACHE_ENTRIES	全局配置	64	2 到 2048 的二次幂	ICache 的总缓存行数，必须能被 ICACHE_WAYS 整除。
ICACHE_WAYS	全局配置	1	1、2	ICache 的路数；推荐配置使用 1 路。
ICACHE_OUTSTANDING_DEPTH	全局配置	2	1 到 4	ICache 未命中请求 FIFO 与响应 FIFO 深度。
ICACHE_PREFETCH	全局配置	1	0、1	保留的 ICache 预取配置；当前控制器不发起预取请求，推荐配置取 0。
ENABLE_DCACHE	全局配置	0	0、1	生成 M1 侧 DCache；关闭时 M1 读写通道直通。
DCACHE_ENTRIES	全局配置	64	2 到 2048 的二次幂	DCache 的总缓存行数，必须能被 DCACHE_WAYS 整除。
DCACHE_WAYS	全局配置	2	1、2	DCache 的路数；推荐配置使用 2 路。
DCACHE_OUTSTANDING_DEPTH	全局配置	2	1 到 4	DCache 读未命中请求 FIFO 与响应 FIFO 深度。
M0_AXI_PINGPONG_STAGES	全局配置	0	0、1、2	M0 AXI4-Lite 读地址通道的乒乓缓存级数；读数据通道固定为零级。

名称	配置类型	默认值	有效取值	功能说明
M1_AXI_PINGPONG_STAGES	全局配置	0	0、1、2	M1 AXI4-Lite AW、W、B、AR 和 R 五个通道共同采用的乒乓缓存级数。
M0_AXI_PINGPONG_FALL_THROUGH	全局配置	0	0、1	控制 M0 读地址乒乓缓存空时是否直通；推荐配置取 1。

表 12 Cache 与乒乓缓存配置参数

推荐配置中，M0 AR 为一级并允许空缓存直通，M0 R 为零级；M1 的 AW、W、B、AR 和 R 五个通道共同使用一级缓存，且 M1 不设置独立的空缓存直通参数。五个通道分别保存本通道字段并独立完成握手。

PIPELINE_STAGES 仅允许取 3、4 或 5。不合法的取值会在 core_top 中触发构建期错误。

2.5 AXI4-Lite Cache 与乒乓缓存

AXI4-Lite Cache 位于 CPU 主接口与 AXI 互连之间。M0 侧使用只读 ICACHE，M1 侧使用读写 DCACHE；Cache 命中时直接向 CPU 返回，未命中、写通请求和不可缓存访问经过乒乓缓存进入 AXI 互连。Cache 关闭时，Cache 顶层只连接同名 AXI4-Lite 通道，不生成标签、数据、替换状态和 FIFO；其后的乒乓缓存仍由 M0、M1 级数参数独立决定是否生成。

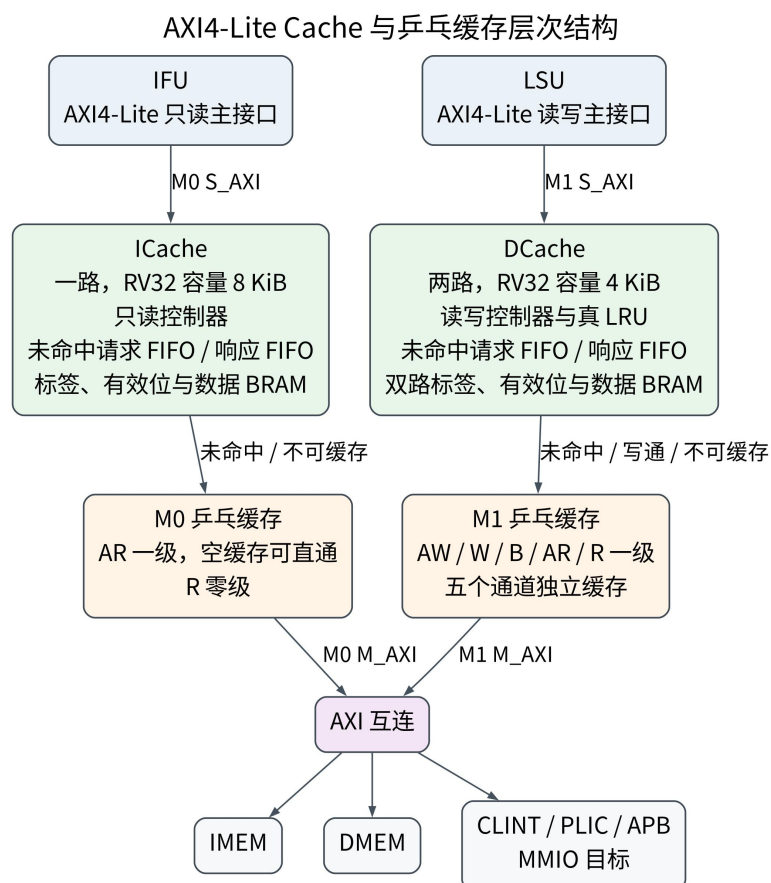


图 6 AXI4-Lite Cache 与乒乓缓存层次结构

2.5.1 乒乓缓存

乒乓缓存位于 Cache 与 AXI 互连之间，用于切分 AXI4-Lite 的长距离控制和数据通道。每一级包含两个存储位置、写指针、读指针和占用计数；当缓存已满且同周期完成一项输出时，可以在同一周期接收新的输入。空的直通级在下游就绪时不增加周期，下游暂停时才由寄存器保存完整通道字段。乒乓缓存不负责 Cache 命中判断、未命中数量管理或读响应排序，这些功能由 Cache 控制器完成。

推荐配置采用表 13 所列乒乓缓存配置：M0 ICache 后的读地址通道为一级直通乒乓缓存，M0 读数据通道不增加缓存级；M1 的 AW、W、B、AR 和 R 五个通道均使用一级非直通缓存。M1 的各通道分别保存本通道字段并独立完成握手，写地址与写数据不要求在同一周期出现，写响应和读响应也不会与请求通道共用控制状态。

通道	级数	直通条件	设计作用
M0 AR	1	空缓存且下游就绪时直通	切分 ICache 未命中读地址路径。

通道	级数	直通条件	设计作用
M0 R	0	直接连接	保持 ICache 队首未命中响应路径的低延迟。
M1 AW	1	非直通	保存 DCache 写地址，吸收短时暂停。
M1 W	1	非直通	保存 DCache 写数据和字节使能。
M1 B	1	非直通	保存 DCache 写响应，隔离下游暂停。
M1 AR	1	非直通	保存 DCache 未命中和不可缓存读地址，并在下游暂停时吸收停顿。
M1 R	1	非直通	保存下游读数据和响应状态。

表 13 M0/M1 乒乓缓存配置

2.5.2 ICache 与 DCache

AXI4-Lite Cache 按读写属性分为 ICache 和 DCache。ICache 只缓存读请求，写通道完整直通；DCache 同时处理读和写，采用写通策略，不保存脏位。缓存行数据宽度等于 C_AXI_DATA_WIDTH，缓存行字节数为 C_AXI_DATA_WIDTH/8；RV32 配置每行为 4 B，RV64 配置每行为 8 B。单次 AXI4-Lite 访问对应一条缓存行，不需要在 Cache 内部拆分缓存行。

推荐配置采用一路 2048 行 ICache 和两路共 1024 行 DCache。RV32 下容量分别为 8 KiB 和 4 KiB，RV64 下容量分别为 16 KiB 和 8 KiB。ICache 使用一路以省去路选择和 LRU 存储；DCache 使用两路组相联和真 LRU，以减少数据地址冲突。两类 Cache 的队首命中响应均为地址握手后的下一周期。推荐配置的固定下一字预取关闭，控制器只处理已经被上游接受的读请求。

属性	ICache	DCache
总缓存行数	2048	1024
路数	1	2
RV32 缓存容量	8 KiB	4 KiB
RV64 缓存容量	16 KiB	8 KiB
缓存行	RV32 为 4 B，RV64 为 8 B	RV32 为 4 B，RV64 为 8 B
读未完成请求深度	2	2
读策略	只读，未命中读取下游	读分配
写策略	写通道直通	写通，无脏位
替换方式	一路不需要替换选择	无效路优先，两路均有效时真 LRU

属性	ICache	DCache
下一字预取	保留参数取 0，当前不生成预取请求	不适用

表 14 推荐配置的 ICache 与 DCache 参数

推荐配置中，ICache 的有效地址宽度为 17 位，DCache 的有效地址宽度为 16 位。RV32 ICache 地址字段为字节位 [1:0]、组索引 [12:2] 和标签 [16:13]，DCache 地址字段为字节位 [1:0]、组索引 [10:2] 和标签 [15:11]。RV64 ICache 地址字段为字节位 [2:0]、组索引 [13:3] 和标签 [16:14]，DCache 地址字段为字节位 [2:0]、组索引 [11:3] 和标签 [15:12]。地址不在 Cache 有效区域时仍可以访问下游，但返回数据不写入 Cache。

ICache 和 DCache 均采用同步 BRAM。下降沿读取标签和数据，上升沿登记标签比较结果、有效位和数据，随后在上游响应通道完成一周期中返回。错误读响应不会更新标签、有效位或数据。DCache 整字写未命中仅在下游写响应为 OKAY 时分配，部分字节写未命中不分配；写命中按 WSTRB 更新对应字节。

ICache 失效由 M1 对 IMEM 或 ITCM 地址范围完成写地址握手时触发。控制器从第 0 组开始逐组清除有效位，清除耗时等于组数，并在最后一组清除完成后的一个周期内继续屏蔽本地命中。初始化或失效期间，读请求仍可访问下游，但本地命中、填充和成功写响应后的本地数据修改均被禁止；已经发出的旧读请求仍可返回 IFU，其返回数据不会写入 ICache。DCache 保留 `invalidate_i` 输入，推荐配置将其置为 0；若系统存在绕过 DCache 的直接写入主设备，需要由系统产生单周期失效脉冲，或使用不可缓存区域。

2.5.3 请求、响应与顺序控制

每个读 Cache 使用独立的未命中请求 FIFO 和响应 FIFO。上游 AR 握手后，请求先进入 ICache 的一项暂存寄存器或 DCache 的输入 FIFO；当该请求进入标签检查时，控制器才在响应 FIFO 中分配一个位置。命中数据、下游 R 数据和响应状态分别写入对应位置，并由响应 FIFO 的读指针按请求顺序向上游返回。后完成的请求不能越过较早请求，即使后面的请求已经命中也必须等待队首位置就绪。

未命中 FIFO 分别维护写指针、发出指针、返回指针、条目计数和已发出计数。FIFO 中存在尚未发出的旧请求时，发出指针选择该请求；没有尚未发出的旧请求时，本周期新入队字段可以直接驱动下游 AR，即使 FIFO 中仍有已经发出但等待 R 响应的请求。已有请求完成 AR 握手后，发出指针立即选择下一项。只要下游 ARREADY 持续有效，连续未命中可以在相邻周期完成 AR 握手。响应 FIFO 队首未命中且下游 R 已返回时，数据可以直接送向上游；上游 RREAD

Y 无效时，数据写入已经分配的位置。

命中和未命中在同一周期完成时，下游未命中数据优先写入响应 FIFO，命中数据暂存到延迟寄存器，下一周期再写入其分配位置。该优先级不改变请求顺序，也避免两个完成事件同时写同一响应存储端口。若队首未命中已经从下游返回且上游 RREADY 有效，控制器直接使用下游 RDATA 和 RRESP 完成返回；只有上游暂停或该请求不是响应 FIFO 队首时，数据才写入预留响应位置。

2.5.4 Cache 参数

参数	默认值	合法范围	作用
ENABLE_CACHE	0	0、1	为 0 时 Cache 顶层只做 AXI4-Lite 直通，为 1 时根据 READ_ONLY 选择 ICache 或 DCache。
READ_ONLY	0	0、1	为 1 时生成只读 ICache，为 0 时生成读写 DCache。
CACHE_WAYS	2	1、2	Cache 路数；总行数必须能被路数整除。
CACHE_ENTRIES	64	2 到 2048 的二次幂	Cache 总缓存行数，不是每路行数。
READ_OUTSTANDING_DEPTH	2	1 到 4	未命中 FIFO 和响应 FIFO 的深度。
ENABLE_READ_PREFETCH	0	0、1	保留的只读 Cache 预取参数；当前控制器不使用该参数发出预取请求。
C_AXI_DATA_WIDTH	32	与系统数据总线一致	AXI4-Lite 数据位宽，当前为 32 位。
C_AXI_ADDR_WIDTH	32	与系统地址总线一致	AXI4-Lite 地址位宽。
CACHE_ADDR_WIDTH	18	小于 AXI 地址位宽	参与标签、组索引和字节位划分的有效地址位数，且必须保证标签宽度至少为 1 位。
CACHE_BASE_ADDR	0	地址总线可表示范围内	Cache 有效地址区域的基地址。

Cache 几何配置由参数范围约束：路数只能为 1 或 2，总行数必须为二次幂且不小于两倍路数，最大不超过 2048，未完成请求深度必须为 1 到 4，数据宽度必须是不小于 8 的二次幂。组数为 $SET_COUNT = CACHE_ENTRIES / CACHE_WAYS$ ，组索引宽度为 $INDEX_WIDTH = \log_2(SET_COUNT)$ ，标签宽度为 $TAG_WIDTH = CACHE_ADDR_WIDTH - \log_2(C_AXI_DATA_WIDTH/8) - INDEX_WIDTH$ 。可缓存区域大小为 $2^{CACHE_ADDR_WIDTH}$ 字节，地址高位与 $CACHE_BASE_ADDR$ 对应高位相等时才允许本地填充。ICache 和 DCache 通过统一参数接口选择，只读模式不生成写控制状态，读写模式不生成 ICache 的写控制结构。`invalidate_i` 是输入端口，不属于配置参数；其单周期高脉冲启动逐组清除，持续置高会使清除索引反复回到第 0 组。

2.5.5 Cache 时序与控制规则

Cache 采用同步 BRAM 保存标签、有效位和数据。下降沿读取标签和数据，上升沿登记逐路比较结果、有效位和数据，队首命中在地址握手后的下一周期返回。未命中请求进入请求 FIFO，并在响应 FIFO 中预留位置；命中和下游返回可以先完成，但只允许响应 FIFO 队首向上游返回。下游读响应属于队首且上游已准备接收时，控制器可以直接完成返回；上游暂停时，结果写入预留位置。

控制事件	时序位置	处理规则
标签与数据读取	时钟下降沿	按队首请求的组索引并行读取各路 BRAM。
命中结果登记	时钟上升沿	保存标签比较结果、有效位、选路信息和读数据。
命中响应	地址握手后的下一周期	仅当该请求位于响应 FIFO 队首时向上游返回。
未命中发出	请求 FIFO 队首有效且下游就绪	按请求顺序完成 AR 握手，连续请求可在相邻周期发出。
未命中返回	下游 R 通道握手	队首请求可直接返回，其余请求写入预留的响应位置。
失效清除	每个时钟上升沿处理一组	逐组清除有效位，清除期间禁止命中和填充。

表 15 Cache 主要控制事件与时序位置

ICache 只处理读通道，DCache 处理五个 AXI4-Lite 通道并采用写通策略。失效从第 0 组开始逐组清除有效位，失效期间禁止本地命中和填充，已经发出的下游读请求仍可返回但不得写入 Cache。推荐配置的 ICache 为 2048 行、1 路，DCache 为 1024 行、2 路；缓存行字节数为 $C_AXI_DATA_WIDTH/8$ ，RV32 为 4 B，RV64 为 8 B；未完成请求深度为 2，预取关闭。

3 数据流

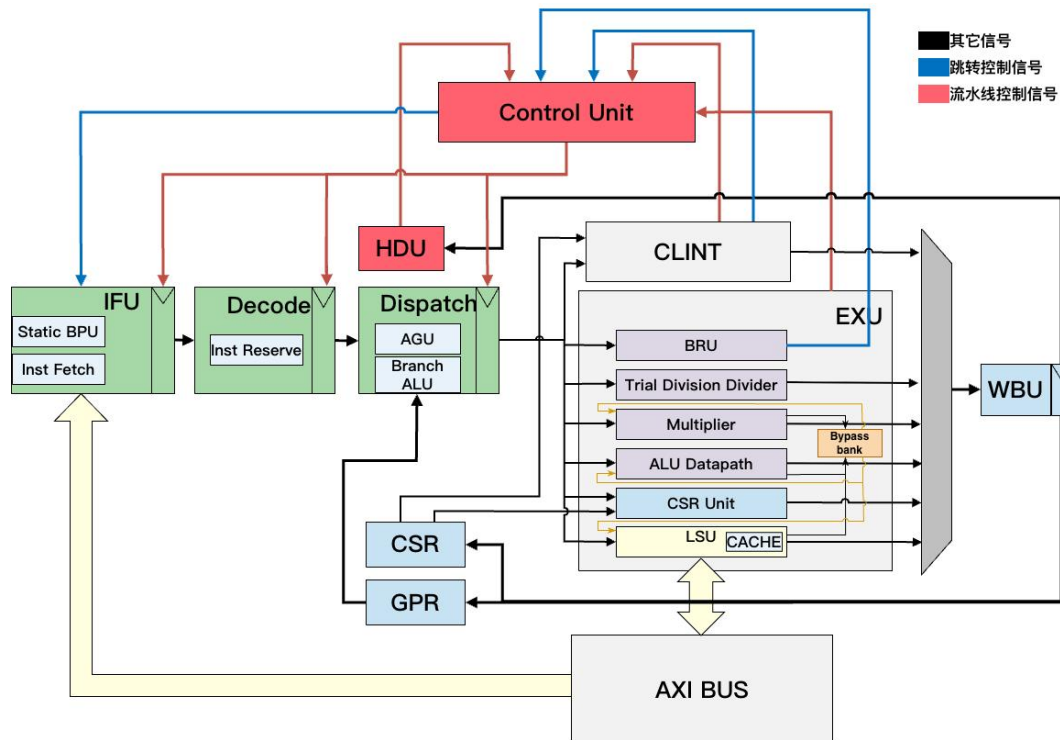


图7 CPU 信号流示意图

CPU 的架构和信号流如图 7 所示，本设计可按“前端取指预测、译码保留、分发与冒险解除、执行与访存、写回与状态更新、异常中断控制”六类功能理解。

1. 前端取指预测：IFU 通过 M0_AXI 只读接口访问 ICache；命中请求由 ICache 在地址握手后的下一周期返回，未命中和不可缓存请求经 M0 读地址乒乓缓存、AXI 互连访问 IMEM。ICache 在时钟下降沿读取标签和数据 BRAM，在随后上升沿登记标签比较结果、有效位与数据；后续响应选择只使用已登记的字段。ifu 根据 ENABLE_BPU_ENHANCED 和 ENABLE_BPU_LEGACY 选择增强 enhanced_bpu、基础 sbpu 或无预测器结构。基础 BPU 在取回指令后结合静态偏置、小型表项和可选预测器生成方向与目标；增强 BPU 在程序计数器有效时查询 BTB、GShare、RAS 和可选 TAGE，并输出 pred_taken、pred_target 和 pred_ghr 等预测状态字段；两类预测器均关闭时，全部控制转移由 BRU 在执行阶段确认。IFU 的程序计数器更新优先级为中断入口、条件分支方向恢复、BRU 通用地址修正、预测地址、暂停保持和顺序地址。

2. 译码与保留：IDU 通过指令保留栈吸收后级暂停。当 Dispatch 或 EXU 暂停时，IFU 仍可在 FIFO 未滿时继续送入指令，IDU 在 FIFO 输出和直通输入之间选择一条功能有效的指令进行译码。idu_decode 将操作码、功能码和寄存器字段转换成 dec_info_bus、立即数、源寄存器读使能、目标写回信息和

异常标志；idu_id_pipe 在暂停时保持，在冲刷时清除有效位和控制状态。

3. 分发与冒险解除：Dispatch 读取 GPR 输出和译码信息，HDU 使用 4 个 commit id 槽判断 RAW、WAW、结构和顺序冒险。三级和五级可把源操作数未就绪的非有序类指令保存到一项等待结构，保存依赖 commit id 和已经取得的操作数，但不保存预测状态；四级遇到阻塞时保持当前输入。四级和五级在 ALU、LSU 或 MUL 结果已经进入 EXU 局部暂存后生成旁路选择，三级等待 WBU 完成。

4. 执行与访存：dispatch_logic 生成各功能单元请求，并提前确定 MEM 地址和分支跳转目标。增强 BPU 配置直接使用 Dispatch 已分别计算的当前指令或等待槽地址；增强 BPU 关闭时，五级配置使用两套地址加法器，三级和四级配置共享一套地址加法器。EXU 内部的 ALU、BRU、LSU、MUL、DIV、CSR 独立产生结果；EXU 局部旁路暂存区只在执行级局部选择 ALU、LSU 和 MUL 数据，避免把宽数据组合拉回 Dispatch。LSU 的 M1_AXI 请求先经过 DCache，读命中在本地返回，读未命中、写通请求和不可缓存访问经 M1 乒乓缓存送往 AXI 互连。AXI 互连对 M1 的读地址、写地址和响应分别保存目标从设备的单热选择状态；同类事务尚未完成时，只允许继续访问已保存的目标，从而保证 AW、W、B 与 AR、R 各自的事务归属稳定。三级 RV32 的普通 MUL 使用一拍结果寄存的完整乘法器；四级和五级在 LSU L0 Cache 关闭时使用面积优先的操作数分段迭代结构，LSU L0 Cache 打开时使用寄存分段部分积结构；DIV 在各流水线配置中共享逐位试商状态和减法资源。

5. 写回与状态更新：WBU 接收 ALU、LSU、MUL、DIV 和 CSR 的完成请求，按流水线配置规定的优先级与小型暂存结构仲裁 GPR 写端口。四级和五级通过每个 GPR 的最新 commit id 防止较早结果覆盖较晚指令，三级由 HDU 预先阻塞全部同 rd 写入。GPR 与 CSR 写端口相互独立，commit_valid_o 和 commit_id_o 反馈给 HDU 与 Dispatch，用于释放槽位和唤醒等待项。

6. 控制与中断：普通分支、JAL、JALR 和预测回退由 BRU 计算真实方向与目标；异常、中断、mret 返回由 CLINT 根据 CSR 状态机产生 int_assert_o、int_jump_o 和 int_addr_o。EXU 只导出 BRU 的地址修正请求，Ctrl 在 EXU 未暂停时将该请求送往 IFU；CLINT 的冲刷事件独立输入 Ctrl，中断地址直接输入 IFU。Ctrl 是纯组合控制模块，根据 EXU、HDU、指令保留栈和 CLINT 的状态生成 4 位 stall_flag_o，控制 IFU、IDU 和 Dispatch 的保持或冲刷。

stall_flag_o 各位由全局流水线控制定义统一规定：

位	宏定义	含义
0	CU_FLUSH	冲刷错误路径或异常路径上的功能有效位

位	宏定义	含义
1	CU_STALL_IF	暂停 IFU PC/取指前进
2	CU_STALL_ID	暂停 IDU 到 Dispatch 的更新
3	CU_STALL_DISPATCH	暂停 Dispatch 到 EXU 的发射流水寄存器

表 16 控制总线位定义

控制信号来源及触发条件如表 17 所示。

信号来源	触发条件	对应输出信号
BRU	分支失配、JAL/JALR 真实跳转	jump_flag_o/jump_addr_o 与 stall_flag_o[CU_FLUSH]
CLINT	异常入口、中断、mret 返回	独立中断地址输入与 stall_flag_o[CU_FLUSH]
EXU	乘除法未完成、LSU 未完成事务/FIFO 阻塞、CSR 或写回握手延迟	同时置位 stall_flag_o[CU_STALL_ID] 和 stall_flag_o[CU_STALL_DISPATCH]，并抑制本拍 BRU 地址修正请求
HDU/Dispatch	RAW 未就绪、WAW 冲突、commit id 用尽、顺序约束未满足、等待槽满	置位 stall_flag_o[CU_STALL_ID]；不直接抑制已经由 BRU 确认且 EXU 未暂停的地址修正请求
IDU IRS	指令保留栈 FIFO 满，不能继续吸收 IFU 指令	stall_flag_o[CU_STALL_IF]
原子与存储事务保护	Dispatch 原子长指令锁定或 EXU 存储事务未完成	core_top 将 dispatch_long_inst_atom_lock_o \ exu_mem_store_busy_o 送入 CLINT 的 atom_opt_busy_i，延迟陷入状态机推进

表 17 控制信号来源及触发条件

Ctrl 只在 stall_flag_ex_i 有效时抑制 BRU 地址修正请求；HDU 暂停只阻止译码结果继续进入 Dispatch，不取消已经在 EXU 中确认的控制转移。CU_STALL_IF 仅由指令保留栈满产生，CU_STALL_ID 是 EXU 与 HDU 暂停的逻辑或，CU_STALL_DISPATCH 仅由 EXU 暂停产生，CU_FLUSH 是有效 BRU 地址修正请求与 CLINT 冲刷请求的逻辑或。已经确认的异常或中断请求通过冲刷清除错误路径；CLINT 在原子长指令锁定或存储事务未完成时延迟状态

机推进，以保持 CSR 与访存状态一致。

CLINT 模块处理 `ecall`、`ebreak`、非法指令、取指/访存非对齐、外部中断、定时器中断、软件中断和 `mret`。进入异常或中断时，CLINT 状态机依次写入 `mepc`、`mstatus`、可选的 `mtval` 和 `mcause`；执行 `mret` 时只写入恢复后的 `mstatus`。状态机在空闲状态采样异常原因、当前指令地址、顺序地址和已确认控制转移地址，随后保持这些字段直到 CSR 写序列完成。`int_assert_o` 在待处理状态有效，用于请求流水线清除；`int_addr_o` 在待处理与 CSR 写入期间保持异常处理地址或 `mret` 返回地址，避免地址来源随外部输入变化。

三级配置使用核心顶层的专用异常保持状态：同步异常、中断或 `mret` 出现后，前端响应和新的发射保持无效，异常进入时等待 `mcause` 写入完成，`mret` 时等待 `mstatus` 写入完成。四级和五级配置由 `core_top` 在 `int_assert_o` 出现后立即阻止 Dispatch 接收新发射，并保存待处理的冲刷请求；四级配置在 `mcause` 或 `mstatus` 写入周期产生跳转脉冲，五级配置在状态机完成前保持已保存的异常目标和发射禁止条件。该分级控制避免异常处理程序读取到尚未写入的异常原因。

表 18 列出了中断处理相关的关键信号及其连接关系。

信号名称	来源	去向	描述
<code>sys_op_ecall_i</code>	EXU	CLINT	<code>ecall</code> 指令标志
<code>sys_op_ebreak_i</code>	EXU	CLINT	<code>ebreak</code> 指令标志
<code>illegal_inst_i</code>	IDU/EXU	CLINT	非法指令标志
<code>misaligned_load_i</code>	AGU/LSU	CLINT	加载地址非对齐标志
<code>misaligned_store_i</code>	AGU/LSU	CLINT	存储地址非对齐标志
<code>ext_int_req_i</code>	PLIC	CLINT	外部中断请求
<code>timer_irq</code>	CLINT(SWI)	CLINT	定时器中断请求
<code>soft_irq</code>	CLINT(SWI)	CLINT	软件中断请求
<code>sys_op_mret_i</code>	EXU	CLINT	<code>mret</code> 指令标志
<code>atom_opt_busy_i</code>	Dispatch/EXU	CLINT	Dispatch 原子长指令锁定或 EXU 存储事务未完成标志
<code>int_assert_o</code>	CLINT	Ctrl	中断断言信号，触发流水线冲刷
<code>int_jump_o</code>	CLINT	Ctrl	中断跳转请求
<code>int_addr_o</code>	CLINT	IFU	中断处理程序地址或返回地址
<code>flush_flag_clint_i</code>	<code>core_top</code>	Ctrl	<code>core_top</code> 将 <code>int_assert_o</code> 接入 Ctrl 的 CLINT 冲刷输入

表 18 中断处理信号表

3.1 流水线级划分与共同控制

本报告按照普通单周期整数指令从取指到写回所固定经过的处理区间定义主流水级数。每一级位于两个相邻寄存器边界之间，前一边界在时钟沿保存输入，组合逻辑在本周期内完成处理，后一边界在下一个时钟沿保存结果。指令保留队列、写回暂存队列、乘法器、除法器、加载存储单元内部状态机和总线事务队列只在等待或多周期操作期间保存状态，不是普通指令固定经过的新一级，因此不计入主流水级数。

三种配置共用 IFU、IDU、Dispatch、EXU 和 WBU 模块层次，区别在于译码后和发射后两个寄存器位置是否参与有效数据路径。ifu_pipe 在全部配置中保存 IFU 返回的指令、程序计数器、有效位和预测信息；idu_id_pipe 仅在五级配置的有效数据路径中保存译码结果；dispatch_pipe 在四级和五级配置中保存已选择的操作数、执行控制和 commit id，在三级配置中直接传递这些信号；WBU 内部结果寄存位置在全部配置中保存被接受的执行结果。条件生成逻辑在综合前确定各寄存器是否参与有效数据路径。

3.1.1 五级流水线

五级流水线是默认配置。五个功能区间依次为取指（IF）、译码（ID）、发射（Dispatch）、执行（EX）和写回（WB）；相邻区间之间均存在明确的寄存器边界。

阶段	主要功能	寄存器边界
取指	IFU 选择程序计数器、查询分支预测单元、发出指令读请求并整理返回指令	级末由 ifu_pipe 保存指令、程序计数器、有效位和预测信息。
译码	IDU 识别指令类别，形成立即数、源寄存器地址、目的寄存器地址和执行控制	级末由 idu_id_pipe 保存译码结果。
发射	Dispatch 读取通用寄存器，调用 HDU 检查数据冒险，选择操作数和执行单元并分配 commit id	级末由 dispatch_pipe 保存执行请求。
执行	EXU 完成算术逻辑、分支、加载存储、乘法、除法和控制状态寄存器操作	完成结果被 WBU 接受后写入 WBU 内部结果寄存器。
写回	WBU 输出已选结果，写入通用寄存器或控制状态寄存器，并释放 commit id	架构状态在本级末端完成写入，不再设置后续主流水边界。

表 19 五级流水线功能划分

3.1.2 四级与三级流水线

四级流水线将译码和发射合并为第二级，并保留发射到执行之间的 `dispatch_pipe` 寄存器。普通指令离开 `ifu_pipe` 后，IDU 译码、通用寄存器读取、HDU 数据冒险检查和 Dispatch 操作数选择均在同一周期完成；该周期末由 `dispatch_pipe` 保存执行请求，EXU 在下一周期使用这些信号。

`idu_id_pipe` 在四级配置中不位于有效数据路径，译码结果直接进入 Dispatch；`dispatch_pipe` 仍是 ID + Dispatch 与 EX 之间的主流水边界。HDU 使用上一周期已经接收的目的寄存器状态检查当前源寄存器，当前指令的目的寄存器不参与自身的源寄存器检查。算术逻辑单元、加载存储单元和乘法器的局部前递保持有效，地址生成前递只用于地址计算。条件分支复用算术逻辑单元的相等、有符号小于和无符号小于结果，不另设一组宽比较器。

分支或异常修正取指地址后，四级流水线可能已经保存一条错误路径上的发射指令。控制模块必须清除该指令的有效位和相关状态。

三级流水线将译码、发射和短延迟执行合并为第二级。普通指令离开 `ifu_pipe` 后，IDU 译码、通用寄存器读取、HDU 数据冒险检查、Dispatch 操作数选择以及 EXU 的短延迟运算在同一周期完成；周期末，WBU 内部结果寄存位置保存被接受的执行结果。乘法、除法和总线访存等多周期操作在该合并级启动，完成前由各执行单元内部状态保存，不增加普通单周期整数指令的固定级数。

`idu` 的三级分支直接输出指令保留队列首项的译码结果，不使用 `idu_id_pipe` 的寄存输出；`dispatch_pipe` 的三级分支直接把选定的操作数、执行控制和 `commit_id` 送至 EXU，不在发射后再保存一拍。三级配置的指令保留队列深度为一项，四级和五级配置的队列深度为三项。队列为空时可直接传递指令；发生数据冒险或执行单元占用时才保存指令。三级 HDU 不将本周周期执行结果组合返回至同周期的译码和发射逻辑，依赖指令等待写回数据或明确的完成通知后再发射。

条件分支在合并级中完成比较。时钟沿到来后，控制模块清除错误路径取指信息和指令保留队列内容。三级流水线不存在已经保存的、在程序顺序上较晚发射的指令，因此地址修正时不需要撤销发射寄存器中的写寄存器状态。同步异常使用专门的保持状态，等待必要的控制状态寄存器操作完成后再允许异常处理程序进入主流程。

3.1.3 共同控制规则

控制类别	处理规则
------	------

控制类别	处理规则
读后写数据冒险	HDU 根据未完成写寄存器状态、局部前递有效位和写回结果决定发射、等待或暂停。
写后写相关	对相同目的寄存器，只有最后一次有效写回可以写入通用寄存器；较早完成的结果仍释放其 commit id。
结构冲突	加载存储单元、乘法器、除法器、控制状态寄存器单元和写回端口的忙状态通过就绪信号和内部队列处理。
控制转移	分支执行单元和中断控制单元修正取指地址时，错误路径指令的有效位被清除。中断入口地址优先于分支恢复地址。
异常与中断保持	三级由专用异常保持状态等待关键 CSR 写入；四级和五级在 CLINT 待处理期间阻止新的 Dispatch 发射，并保持已保存的异常目标地址。
总线返回	IFU、LSU、ICache 和 DCache 以请求顺序和内部队列信息匹配返回数据。错误路径取指返回不得进入译码级，Cache 只允许响应 FIFO 队首向上游返回。
Cache 失效	M1 对 IMEM 或 ITCM 区域完成写地址握手后逐组清除 ICache 有效位；失效期间禁止命中和填充，已经发出的读请求仍可返回。

表 20 各流水线配置的共同控制规则

3.1.4 参数传递与模块边界

PIPELINE_STAGES 由 alkaid_soc_top 传入 core_top，再传入取指、译码、发射、HDU、执行、写回和中断处理模块。core_top 对参数取值进行检查，只接受 3、4 和 5。三种配置共用同一组 core_top 端口，不因流水线级数改变时钟、复位、中断或 AXI4-Lite 主接口信号。

模块或功能	五级配置	四级配置	三级配置
ifu_pipe	保留取指到译码的寄存边界	同五级配置	作为取指到执行组合级的寄存边界
idu_id_pipe	保存译码结果和预测字段	不位于有效数据路径	不位于有效数据路径
dispatch_pipe	保留发射到执行的寄存边界	保留发射到执行的寄存边界	直接输出本拍执行请求
指令保留队列	三项	三项	一项

模块或功能	五级配置	四级配置	三级配置
hdu_select	在 ID 级推进时保存与 IDU 输出对齐的依赖 commit id，并使用 EXU 局部旁路选择状态	使用当前组合译码的源寄存器地址直接查询最近分配表，并使用 EXU 局部旁路选择状态	选择 hdu_stage3_compact 四项紧凑比较结构，关闭 EXU 局部旁路选择
wbu	使用多执行单元写回仲裁与小型队列	同五级配置	使用固定优先级与一项算术逻辑结果暂存器
普通 MUL	RV32 在 LSU L0 Cache 关闭时使用将操作数划分为固定片段后逐次计算的结构，打开时使用寄存分段部分积结构；RV64 使用将操作数划分为 16 位片段后逐次计算的结构	同五级配置	RV32 使用一拍结果寄存的完整乘法器；RV64 仍使用分段迭代
DIV	保存 rd 与 commit id，使用逐位试商和共享减法资源	同五级配置	不重复保存 rd，由 WBU 按 commit id 取得目的寄存器；计算资源不变
ICache 与 DCache	位于 CPU 主接口之外，配置不随流水线级数改变	同五级配置	同五级配置
同步异常控制	CLINT 待处理后阻止新发射，并在 CSR 写入完成前保持冲刷与异常目标状态	保存异常目标地址，在 mcause 或 mstatus 写入周期产生跳转脉冲	核心顶层保持异常状态，直至 mcause 或 mstatus 完成必要写入

表 21 流水线级数对各模块有效结构的影响

流水线级数切换保持模块之间的信号定义不变。数据字段只在对应的指令有效位为高时使用；暂停时保持当前寄存内容；取指地址修正、异常或中断发生时，通过清除有效位取消错误路径上的指令。这一规则使得数据字段不必在每次取指地址修正时全部清零，可以减少控制信号的扇出。

3.1.5 多周期执行单元的分级配置

流水线级数只改变请求和结果的寄存边界，不改变乘除法、访存和原子操作的架构接口。执行单元内部仍可保存多周期状态；HDU（数据冒险检查单元）使用 commit id 和结果有效状态判断依赖是否解除。当前实现对不同级数采用

以下专门配置。

功能单元	三级配置	四级和五级配置	对流水线控制的影响
RV32 MUL	SINGLE_CYCLE_RV32=1, 使用一套完整乘法资源, 在 EXU 接收请求后的时钟沿登记结果; 不生成 MUL 迭代状态。	CPU 内部令 AREA_OPT=!ENABLE_LSU_HOT_CACHE : LSU L0 Cache 关闭时复用一套 17 位乘法资源, 将操作数划分为四个片段并依次形成部分积; 打开时保留寄存分段部分积结构和一项请求暂存。	三级不使用 EXU 局部旁路, 结果由 WBU 和 HDU 完成状态解除依赖; 四级、五级按结果有效和旁路状态允许 ALU、MUL 或存储数据使用。
RV64 MUL	不采用单周期结构, 仍使用按 16 位操作数片段逐次计算的迭代结构, 逐项累加十六个部分积; MULW 单独处理低 32 位并符号扩展。	与三级相同, RV64 不生成单周期完整乘法器; 结果保持到 WBU 接收。	迭代期间输出忙状态, 完成时携带目的寄存器和 commit id; 局部旁路只在四级、五级参与操作数选择。
DIV	RV32 与四级、五级共用同一套逐位试商状态机和 33 位减法资源; 三级只省略结果中的重复目的寄存器地址。	RV32 复用同一条 33 位减法器完成取绝对值、逐位试减和符号恢复; RV64 每个商位拆为低半部分和高半部分两个状态, 缩短单拍进位传播范围。	DIV 结果保持有效直到 WBU 接收; 未完成时设置长指令锁定, 阻止会破坏写回顺序的发射。
LSU 普通访问	输入请求队列、读返回跟踪、写数据保存和非对齐拆分状态保持不变; 三级不保存重复的目的寄存器地址。	与三级使用相同的 AXI4-Lite 五通道控制, DCache、乒乓缓存或片上存储器的可变返回延迟均由实际 R/B 握手解除。	mem_busy 或原子锁定有效时, Ctrl 和 CLINT 延后异常、中断和新请求的推进。

功能单元	三级配置	四级和五级配置	对流水线控制的影响
LSU AMO	A 扩展打开时使用六状态原子控制器，原子请求在普通读写队列、拆分状态和写响应计数均为空时进入；状态结束后由 WBU 查询目的寄存器。	状态和握手规则相同；AW 与 W 分别记录完成状态，不能假定同拍握手。	原子控制器独占 AXI4-Lite 五通道，直到 AMO_DONE 才恢复普通访问；LR/SC 保留地址和 AMO 旧值在状态寄存器中保持。

RV32 单周期 MUL 只把完整乘积的低半或高半结果登记到结果寄存器，仍经过 WBU 仲裁，不绕过 commit id 管理。面积优先的 RV32 MUL 将四个 16×16 部分积分成四拍，乘法资源在每拍只承担一个操作数片段的计算；低位常见操作可以由独立快速计算级完成，但快速结果仍需等待 WBU 接收。RV64 MUL 的状态机依次执行空闲、迭代和结果保持，结果保持状态负责吸收写回暂停。

DIV 的共享资源设计把取绝对值、试商、余数更新和符号恢复安排到同一条减法资源上。RV32 每一商位占用一拍；RV64 每一商位先登记低 32 位借位，再在下一拍完成高 32 位试减和商位更新；字操作只迭代 32 个商位。除数为零时，商返回全一，余数返回原被除数；有符号最小值除以负一按标准结果规则输出，不增加专用组合除法器。

LSU 的 AMO 控制器只有在普通输入队列、读返回跟踪、写数据队列、未返回写响应计数和非对齐第二次访问均为空时接受新原子操作。AMO_IDLE 接收 LR、SC 或 AMO；LR 和普通 AMO 进入 AMO_AR、AMO_R，SC 根据保留地址比较结果直接进入 AMO_AW 或 AMO_DONE。AMO_AW 独立等待 AW 与 W 握手，二者都完成后进入 AMO_B；B 握手后进入 AMO_DONE，结果在该状态保持一个写回周期。原子状态非空时，普通访存暂停，atom_opt_busy 送入 CLINT，避免异常或中断打断读改写过程。

4 时钟、复位和初始化

CPU 核心、AXI4-Lite 互连和 APB 接口使用系统主时钟。Timer 内部可根据寄存器配置使用门控后的系统主时钟计数；low_speed_clk_i 先在 Timer 的工作时钟域内完成同步，再作为低速计数事件使用，不直接驱动 CPU 流水线寄存器。

rst_n 和 APB 侧的 PRESETn 均为低电平有效，但具体采用同步复位还是异步复位由模块局部时序逻辑决定。CPU 流水控制、CLINT 陷入状态、PLIC 核心、AXI 互连选择状态和主要执行单元通常采用时钟上升沿配合复位下降沿的异步复位；CLINT 软件中断与定时器寄存器接口以及 PLIC 的 AXI 包装状

态采用时钟上升沿内判断复位的同步复位。复位后，IFU 的 PC 置为 0x8000_0000；AXI 请求计数、批次标识和指令 FIFO 计数，IDU 指令保留结构，Dispatch 等待项，HDU commit id 槽位，EXU 长延迟单元忙状态，WBU FIFO 计数、Cache 请求 FIFO、Cache 响应 FIFO、乒乓缓存的读写指针与占用计数，以及 AXI/APB 状态机均回到空闲状态。Cache 标签有效位不使用全阵列复位，而是在复位释放后从第 0 组开始逐组清除；替换状态不要求复位，在相应组尚无有效行时不会参与有效数据选择。流水线宽数据字段、通用寄存器数据体、预测表、Cache 数据 BRAM、乒乓缓存数据存储和部分存储阵列不要求逐项清零；对有效位、计数器或初始化状态保证这些数据在重新写入前不被采用。

ICache 和 DCache 不在复位时对全部标签有效位产生大范围清零网络。复位首先清空请求状态和响应状态，控制器随后按组清除标签有效位；清除过程未完成时，Cache 不允许本地命中和填充。ICache 接收到运行期失效请求后采用相同的逐组清除过程，并在最后一组清除后保留一个保护周期，防止同步标签 BRAM 的旧读出值被采用。DCache 的系统配置将失效输入固定为低电平；若系统增加绕过 DCache 直接写 DMEM 的其他主设备，该主设备必须在写入后发出单周期失效请求，或使用不可缓存地址区域。Cache 数据 BRAM 不复位，只有与有效标签同时命中时数据才可以返回。

初始化分为硬件复位和软件启动两层。硬件复位只保证流水线、总线和控制寄存器处于可启动状态；.bss 清零、gp/sp 设置、陷入入口写入、UART、C LINT、PLIC 和 GPIO 初始化由启动程序与 BSP 完成。存储器初始化由 INIT_ITCM、INIT_DTCM、ITCM_INIT_FILE 和 DTCM_INIT_FILE 等参数决定，属于 SoC 存储器从设备的初始化内容，不等同于 CPU 内部复位逻辑。Cache 不从存储初始化文件建立初值，复位后首次访问必须经过有效位检查。

5 子模块描述

CPU 包含的主要模块有 IFU、IDU、Dispatch、EXU、WBU、GPR、CSR，接下来将对这些模块的设计进行介绍。

5.1 IFU

取指单元（IFU）负责维护程序计数器（PC）、发起取指请求、整理返回指令、执行分支预测，并把指令及预测状态送入 IDU。IFU 由 PC 控制、AXI4-Lite 只读主接口、可选 C 扩展半字流整理、基础或增强 BPU、预测状态保存结构和 ifu_pipe 组成。地址 FIFO、指令 FIFO 和预测器内部状态数组分别保存各自所需的信息，并通过同一组请求和出队事件保持指令、PC 与预测状态对齐。

其输入输出端口如下表所示。

端口名	方向	位宽	功能描述
clk	输入	1	系统主时钟。
rst_n	输入	1	低电平有效的异步复位。
jump_flag_i	输入	1	清除旧批次取指请求和 C 扩展半字状态；不直接提供下一取指地址。
jump_addr_i	输入	INST_ADDR_WIDTH	兼容保留的地址输入；当前功能逻辑不读取该端口。
resolved_jump_flag_i	输入	1	BRU 已确认存在控制转移事件。
bru_pc_redirect_i	输入	1	BRU 实际地址允许修正 PC。
resolved_jump_addr_i	输入	INST_ADDR_WIDTH	BRU 已确认的实际下一取指地址。
cond_taken_redirect_i	输入	1	在 ENABLE_BPU_ENHANCED=1 且 ENABLE_C=0 时表示预测不跳转而实际跳转。
cond_rollback_redirect_i	输入	1	在 ENABLE_BPU_ENHANCED=1 且 ENABLE_C=0 时表示预测跳转而实际不跳转。
interrupt_jump_flag_i	输入	1	中断、异常或 mret 地址有效，PC 选择优先级最高。
interrupt_jump_addr_i	输入	INST_ADDR_WIDTH	中断、异常入口或 mret 返回地址。
jump_sequential_addr_i	输入	INST_ADDR_WIDTH	为模块接口兼容保留；当前固定宽度增强 BPU 不使用该输入。

端口名	方向	位宽	功能描述
cond_recovery_valid_i	输入	1	条件分支恢复信息有效；当前只用于接口一致性检查。
cond_recovery_pc_i	输入	INST_ADDR_WIDTH	Dispatch 保存的条件分支实际下一取指地址。
cond_recovery_ghr_i	输入	BPU_GHR_BITS	Dispatch 保存的恢复后全局历史值。
stall_flag_i	输入	CU_BUS_WIDTH	流水线冲刷和暂停控制总线。
bp_update_valid_i	输入	1	基础 BPU 条件分支训练有效。
bp_update_pc_i	输入	INST_ADDR_WIDTH	基础 BPU 条件分支训练 PC。
bp_update_taken_i	输入	1	基础 BPU 条件分支实际方向。
bp_update_static_taken_i	输入	1	基础 BPU 静态方向结果。
bp_update_backward_i	输入	1	为基础 BPU 保留的条件分支方向属性输入；当前基础 BPU 不读取该端口。
bp_enh_update_valid_i	输入	1	增强 BPU 训练有效。
bp_enh_update_pc_i	输入	INST_ADDR_WIDTH	增强 BPU 训练 PC。
bp_enh_update_target_i	输入	INST_ADDR_WIDTH	增强 BPU 实际目标地址。
bp_enh_update_next_i	输入	INST_ADDR_WIDTH	增强 BPU 实际顺序地址。
bp_enh_update_kind_i	输入	BP_KIND_WIDTH	增强 BPU 控制转移类别。
bp_enh_update_taken_i	输入	1	增强 BPU 实际方向。
bp_enh_update_ghr_i	输入	BPU_GHR_BITS	预测时的全局历史快照。
jalr_update_valid_i	输入	1	基础 BPU 的 JALR 目标训练有效。
jalr_update_pc_i	输入	INST_ADDR_WIDTH	基础 BPU 的 JALR 指令 PC。

端口名	方向	位宽	功能描述
jalr_update_target_i	输入	INST_ADDR_WIDTH	基础 BPU 的 JALR 实际目标。
inst_o	输出	INST_DATA_WIDTH	送往 IDU 的统一 32 位指令。
inst_addr_o	输出	INST_ADDR_WIDTH	与输出指令对齐的 PC。
inst_len_2_o	输出	1	输出指令长度为 16 位。
inst_valid_o	输出	1	输出指令有效。
read_resp_error_o	输出	1	当前请求批次出现 AXI4-Lite 读错误后置位并保持至复位的状态。错误响应本身不形成有效指令。
pred_taken_o	输出	1	与输出指令对齐的预测方向；无预测器时为零。
is_pred_branch_o	输出	1	与输出指令对齐、且预测结果为跳转的条件分支标志。
is_pred_jalr_o	输出	1	与输出指令对齐、且 RAS 或目标表命中并形成预测跳转的 JALR 标志。
pred_target_o	输出	INST_ADDR_WIDTH	与输出指令对齐的预测目标。
pred_ghr_o	输出	BPU_GHR_BITS	与输出指令对齐的全局历史快照；基础 BPU 输出零。
M_AXI_ARADDR	输出	INST_ADDR_WIDTH	AXI4-Lite 读地址。
M_AXI_ARPROT	输出	3	AXI4-Lite 读访问属性，固定为零。
M_AXI_ARVALID	输出	1	AXI4-Lite 读地址有效。
M_AXI_ARREADY	输入	1	AXI4-Lite 读地址就绪。
M_AXI_RDATA	输入	BUS_DATA_WIDTH	AXI4-Lite 读数据。
M_AXI_RRESP	输入	2	AXI4-Lite 读响应。
M_AXI_RVALID	输入	1	AXI4-Lite 读数据有效。

端口名	方向	位宽	功能描述
M_AXI_RREADY	输出	1	AXI4-Lite 读数据接收就绪。

表 22 IFU 输入输出端口

IFU 的主要参数为 PIPELINE_STAGES、ENABLE_C、ENABLE_BPU_LEGACY、ENABLE_BPU_ENHANCED 和 BPU_GHR_BITS。三种流水线配置均保留 ifu_pipe；PIPELINE_STAGES 在 IFU 内只用于选择五级、基础 BPU、非 C 扩展组合下的并行 PC 寄存结构。两个 BPU 开关选择增强 BPU、基础 BPU 或无预测器三种结构，ENABLE_C 选择半字对齐和解压路径，BPU_GHR_BITS 决定增强预测历史字段宽度。

IFU 包含 ifu_ifetch、ifu_axi_master、ifu_c_stream、sbpu 或 enhanced_bpu 以及 ifu_pipe。PC 选择与请求发射在前端完成；ifu_axi_master 只对齐请求 PC、返回指令和直接跳转目标；增强 BPU 以及固定宽度基础 BPU 的预测状态由预测器自身的两项状态数组保存，并使用 AXI 地址 FIFO 和指令 FIFO 的写入、读出事件同步推进；C 扩展半字拼接由 ifu_c_stream 完成，最后统一经 ifu_pipe 输出给 IDU。

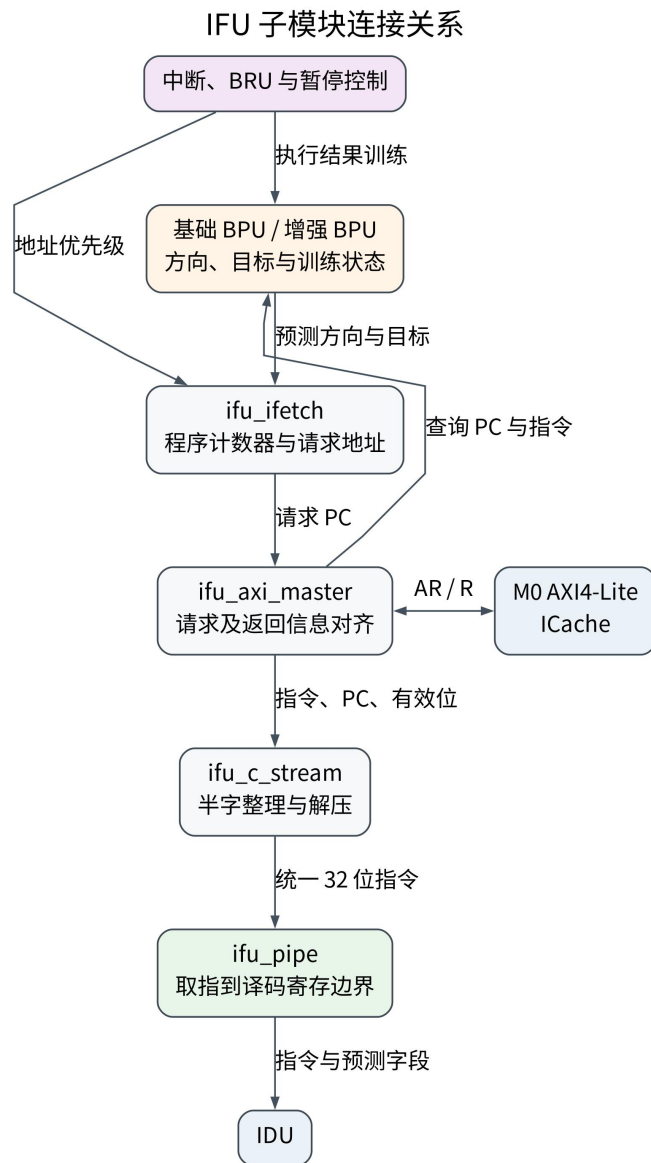


图 8 IFU 子模块连接关系

5.1.1 取指地址控制

ifu_ifetch 维护取指请求 PC，并根据已确认的地址修正、预测结果和暂停状态决定下一拍取指地址。复位后 PC 指向 IMEM 基地址 0x8000_0000。未开启 C 扩展时，PC 按 4 字节前进；开启 C 扩展时，PC 保持半字对齐，地址 bit1 指示从返回字的低半字或高半字开始整理指令，顺序请求推进至下一个 4 字节边界。中断地址、条件分支方向恢复地址或 BRU 实际地址有效时，该地址优先于预测和顺序地址。若 IFU 因 AXI 请求未被接受、内部 FIFO 无空间或 C 指令流整理暂停而不能前进，则 PC 保持不变。

PC 非对齐检测也在取指地址生成阶段完成。未开启 C 扩展时，取指地址必须 4 字节对齐；开启 C 扩展后允许半字对齐，但仍禁止非法对齐。非对齐信号不会直接修改架构状态，而是作为异常条件进入 CLINT 陷入处理路径，避免 IFU 自行处理异常导致控制流多源化。

PC 更新优先级依次为中断入口、固定宽度增强 BPU 条件分支方向恢复、BRU 已确认地址、预测地址、暂停保持和顺序地址。中断优先于两类条件分支方向恢复事件。固定宽度基础 BPU 在五级流水线下使用并行 PC 寄存区和窄选择状态，其余配置使用单一 PC 寄存器；两种实现遵循相同的请求接受和地址优先级规则。

5.1.2 AXI4-Lite 取指主接口

ifu_axi_master 是 IFU 的 AXI4-Lite 只读主接口模块。它在 AR 通道握手成功时保存请求 PC 和请求批次，在 R 通道返回时按请求顺序取出地址信息。地址 FIFO 复用固定宽度取指地址的 bit0 保存基础 BPU 的 JALR 请求命中标志；指令 FIFO 保存返回指令、PC 和由指令编码计算的直接跳转目标。预测方向、预测目标、GHR 和分支类别不保存在本模块 FIFO 中。

模块内部使用多类 FIFO/计数状态：

结构	进入条件	退出条件	保存内容	跳转地址修正时的处理
地址 FIFO	ARVALID && ARREADY	匹配批次的 RVALID && RREADY	请求 PC；固定宽度模式下复用地址 bit0 保存基础 BPU 的 JALR 请求命中标志	清空可见计数，丢弃旧取指地址
请求批次 FIFO	ARVALID && ARREADY	任意 R 通道响应被接收	本地批次标识 arid_reg 的入队值	arid_reg 递增，旧批次响应会被接收但不会成为有效指令
指令 FIFO	有效 R 响应无法直通，或前面已有缓存指令	下游未暂停且 FIFO 非空	指令数据、PC 和直接跳转目标	清空缓存的旧指令

表 23 内部 FIFO 详情表

地址 FIFO、指令 FIFO 和请求批次 FIFO 的深度均为 2。响应有效需要同时满足三个条件：R 通道握手完成、RRESP 无错误、请求批次 FIFO 队首编号等于当前批次号。因此，地址修正前已经发出的旧请求即使随后返回，也

只会弹出批次跟踪项，不会置位 `inst_valid_o`。当前批次的错误响应置位 `read_resp_error_o` 并保持到复位，但不会进入指令 FIFO。

`pc_stall_o` 的生成遵循“请求不可丢、返回不可错配”的原则。当 AR 通道有效但目标端尚未就绪、地址 FIFO 满、请求批次 FIFO 满，或指令 FIFO 无法在接收 R 响应时腾出空间，IFU 暂停 PC 前进。`M_AXI_RREADY` 只由指令 FIFO 空间决定，不把下游暂停和冲刷组合接入 R 通道就绪路径；若下游暂停而 R 响应已经到达，模块把指令和上下文写入指令 FIFO。该写法把 `AXI RREADY` 从全局控制中分离出来，是 IFU 时序优化的重要实现方法。

跳转地址修正发生时，模块递增批次号并清空地址 FIFO 与指令 FIFO 的可见内容。请求批次 FIFO 不会被立即清空，因为旧请求仍可能在 AXI R 通道返回；这些旧响应必须继续弹出对应的批次项，才能保持 AXI 读请求和读响应的顺序一致。

5.1.3 基础 BPU 与增强 BPU 条件生成

ifu 使用条件生成块实现三种互斥配置：`ENABLE_BPU_ENHANCED=1` 时生成增强 `enhanced_bpu`；增强 BPU 关闭且 `ENABLE_BPU_LEGACY=1` 时生成基础 `sbpu`；两者均关闭时不生成预测表，预测输出固定为 0，由 BRU 对条件分支、JAL、JALR 和取指屏障给出实际下一取指地址。`alkaid_soc_top` 在增强 BPU 关闭时把 `ENABLE_BPU_LEGACY` 置为 1，因此 SoC 默认配置使用基础 BPU。

基础 `sbpu` 面向面积和时序约束，输入为取回后的指令、程序计数器、有效位和暂停信息。静态方向对后向条件分支给出跳转偏置，并把前向 BNE 和 BGEU 作为倾向跳转类型；BHT 和相关方向预测器学习是否同意静态结果。BHT 和相关方向计数器的初始状态为弱同意静态方向，而不是统一预测跳转；相关方向选择计数器初始优先采用基础 BHT。循环表与 JALR 目标表初始无效。循环预测器修正稳定循环最后一次退出，JALR 目标缓存和 RAS 处理间接跳转与函数返回目标。

增强 BPU `enhanced_bpu` 面向需要提前形成控制流目标的配置，输入为 PC 和 `pc_valid`，输出 `branch_taken`、`branch_addr`、`pred_taken`、`pred_target` 和 `pred_ghr`。它包含 BTB、GShare、循环预测器、增强 RAS，并可通过 `ENABLE_BPU_TAGE` 生成 2 到 4 张类 TAGE 表。增强 BPU 的训练信息来自 EXU/BRU：BRU 在真实方向和目标确定后输出更新 PC、目标地址、下一 PC、分支类型、真实方向和 GHR 快照，BPU 使用这些信息更新 BTB、

BHT、GHR、RAS 和 TAGE。TAGE 表项资源较大，因此默认关闭，只有在需要更长历史方向信息时才打开。

5.1.4 C 指令流整理与取指流水寄存器

开启 ENABLE_C 后，IFU 不能简单地按字输出 32 位指令。ifu_c_stream 负责在取回的 32 位数据中选择低/高半字，保存待处理的高半字，必要时跨字拼接 32 位非压缩指令，并输出 inst_len_2。它内部用 upper_valid/upper_half/upper_pc 保存尚未输出的高半字：若高半字本身是压缩指令，则先输出该 16 位指令并对取指侧施加 fetch_stall_o；若高半字是 32 位指令的低半部分，则允许下一拍取指字到达后用后一字的低半字拼接完整 32 位指令。冲刷会清除 upper_valid，防止错误路径残留半字在取指地址修正完成后仍然输出。

ifu_pipe 在 C 扩展打开时还调用纯组合的 rvc_decompressor。该模块以 ENABLE_RV64 选择 RV32 或 RV64 解压规则，输入端口 inst_i 接收 16 位压缩指令，输出端口 inst_o 给出 32 位展开指令。当 inst_len_2_r=1 时，流水寄存器保存的低 16 位指令会在 IFU/IDU 级间寄存后解压成统一 32 位指令；当 inst_len_2_r=0 时，原 32 位指令直接下传。RV32 与 RV64 都支持整数压缩算术、访存、条件分支、直接跳转、寄存器间接跳转和栈指针访存；共用编码 01/001 在 RV32 中展开为 C.JAL，在 RV64 中展开为 C.ADDIW，而 C.LD、C.SD、C.LDSP、C.SDSP、C.SUBW 和 C.ADDW 只在 RV64 配置下有效。未支持的功能类别输出 INST_NONE 并由 IDU 作为非法指令处理；部分提示或保留编码仍先形成普通 32 位指令，再由 IDU 判断是否合法。解压模块不自行判断指令长度，inst_len_2 由前级半字流整理模块产生。设计把半字流整理、流水寄存和后级统一译码分开，避免 C 解压选择、PC 选择和 AXI 返回路径串接成过长的组合路径。

ifu_pipe 是 IFU 与 IDU 之间的级间寄存器，用于保存指令、PC、有效位、预测标志、预测目标地址以及 GHR 快照等信息。流水线暂停时，寄存器保持当前内容；发生冲刷时，flush_pending 屏蔽并清除有效位，宽数据字段可以保持原值。后级只在有效位为高时采用这些字段，从而减少冲刷对宽寄存器的控制扇出。

IFU 的关键约束是：任何送到 IDU 的指令都必须携带同一次取指请求对应的 PC 和预测信息；任何冲刷后返回的旧请求都不能重新变成有效指令；错误响应只置位状态输出，不随某条有效指令下传；增强 BPU 预测状态必须跟随同一条指令通过预测器内部数组、指令保留栈和流水寄存器。

5.2 IDU

指令译码单元（IDU）位于 IFU 与 Dispatch 之间，负责缓冲前端指令，并把统一的 32 位指令解析为操作类型、源寄存器与目的寄存器地址、立即数和扩展控制字段。IDU 的参数包括 PIPELINE_STAGES、ENABLE_RV64、ENABLE_ZBA、ENABLE_ZBB、ENABLE_ZBS、ENABLE_ZBC、ENABLE_C、ENABLE_A、ENABLE_BPU_ENHANCED 和 BPU_GHR_BITS。流水线级数决定指令保留结构和 idu_id_pipe 是否位于有效数据路径，其余参数控制相应译码与预测字段是否生成。

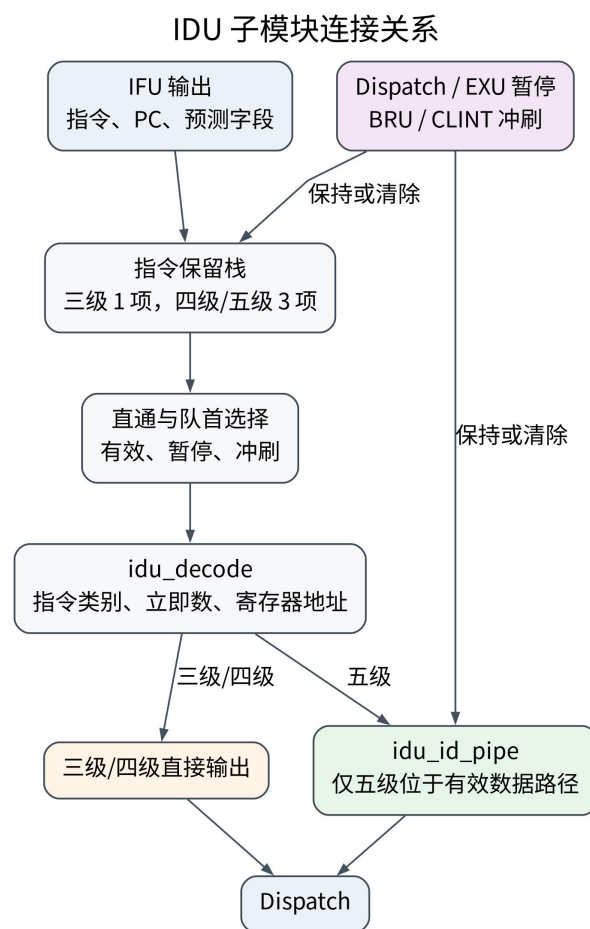


图 9 IDU 子模块连接关系

IDU 的输入输出端口如下表所示。

端口名	方向	位宽	功能描述
clk	输入	1	系统主时钟。
rst_n	输入	1	低电平有效复位。
inst_i	输入	INST_DATA_WIDTH	IFU 输出的统一 32 位指令。
inst_addr_i	输入	INST_ADDR_WIDTH	与输入指令对齐的 PC。

端口名	方向	位宽	功能描述
inst_valid_i	输入	1	输入指令有效。
inst_len_2_i	输入	1	当前指令长度为 16 位；仅 C 扩展配置使用。
is_pred_branch_i	输入	1	BPU 给出的预测跳转条件分支标志。
is_pred_jalr_i	输入	1	BPU 给出的已形成预测跳转的 JALR 标志。
pred_taken_i	输入	1	预测方向。
pred_target_i	输入	INST_ADDR_WIDTH	预测目标地址。
pred_ghr_i	输入	BPU_GHR_BITS	增强 BPU 的全局历史快照。
stall_flag_i	输入	CU_BUS_WIDTH	流水线暂停与冲刷控制总线。
inst_o	输出	INST_DATA_WIDTH	送往 Dispatch 的指令。
inst_addr_o	输出	INST_ADDR_WIDTH	与输出指令对齐的 PC。
inst_valid_o	输出	1	输出指令有效。
inst_len_2_o	输出	1	与输出指令对齐的 16 位指令标志。
reg_we_o	输出	1	通用寄存器写使能。
reg_waddr_o	输出	REG_ADDR_WIDTH	与输出指令对齐的目的寄存器地址。
reg1_raddr_o	输出	REG_ADDR_WIDTH	与输出指令对齐的 rs1 地址。
reg2_raddr_o	输出	REG_ADDR_WIDTH	与输出指令对齐的 rs2 地址。
next_reg_waddr_o	输出	REG_ADDR_WIDTH	当前组合译码得到的目的寄存器地址，供 Dispatch 提前比较等待项。
next_reg1_raddr_o	输出	REG_ADDR_WIDTH	当前组合译码的 rs1 地址，供四级 HDU 直接查询或五级 HDU 提前保存依赖。
next_reg2_raddr_o	输出	REG_ADDR_WIDTH	当前组合译码的 rs2 地址，供四级 HDU 直接查询或五级 HDU 提前保存依赖。
dec_imm_o	输出	REG_DATA_WIDTH	符号扩展后的立即数。
dec_info_bus_o	输出	DECINFO_WIDTH	分组译码控制总线。
is_pred_branch_o	输出	1	与输出指令对齐的预测跳转条件分支标志。
is_pred_jalr_o	输出	1	与输出指令对齐的已形成预测跳转的 JALR 标志。
pred_taken_o	输出	1	与输出指令对齐的预测方向。
pred_target_o	输出	INST_ADDR_WIDTH	与输出指令对齐的预测目标。
pred_ghr_o	输出	BPU_GHR_BITS	与输出指令对齐的全局历史快照。

端口名	方向	位宽	功能描述
illegal_inst_o	输出	1	当前参数配置下无法识别的指令标志。
rs1_re_o	输出	1	rs1 实际读取使能。
rs2_re_o	输出	1	rs2 实际读取使能。
ex_info_bus_o	输出	EX_INFO_BUS_WIDTH	ALU、MUL、DIV、CSR、BJP、LOAD 或 OTHER 执行类别。
irs_stall_flag_o	输出	1	指令保留结构无空间时送往 Ctrl 的暂停请求。

表 24 IDU 输入输出端口

5.2.1 指令保留栈

IDU 使用指令保留结构（IRS）吸收后级暂停期间已经返回的取指结果。当 Dispatch 或 EXU 暂停时，IDU 可把 IFU 当前返回的指令、PC、有效位以及按配置生成的指令长度和预测字段写入内部暂存结构；空间耗尽时，irs_stall_flag_o 经 Ctrl 暂停 IFU。

IRS 的状态可以抽象为三类：

状态	进入条件	退出条件	关键输出	异常/冲刷处理
直通	下游未暂停，FIFO 空	下游暂停或 FIFO 非空	IFU 当前指令直接进入译码	冲刷时当前有效位清零
缓冲	下游暂停且 IFU 有有效指令	FIFO 非空且下游恢复	FIFO 头项进入译码	冲刷清空 FIFO，避免错误路径残留
反压	FIFO 满	FIFO 弹出后释放空间	irs_stall_flag_o=1	取指地址修正后清空旧批次指令

表 25 IRS 的三种状态

三级配置选择一项直通暂存结构；四级和五级配置选择三项环形 FIFO。两种结构在为空时均可把 IFU 输入直接送往译码器，只有下游暂停时才保存指令，因此正常流动不增加固定延迟。三级结构冲刷时清除单项有效位；三项结构先登记冲刷状态，下一周期从功能上呈现为空并修正计数，不要求统一清零读写指针或逐项清除存储阵列。

三项结构将指令本体与 PC、预测类别、预测目标、C 扩展长度和增强预测字段分开保存。is_pred_branch、is_pred_jalr 和 pred_target 在基础 BPU 配置下也随指令保存；inst_len_2 仅在 C 扩展打开时生成；pred_taken 和

pred_ghr 仅在增强 BPU 配置下生成。所有字段使用相同的写入、读出和冲刷条件，防止暂停恢复后出现指令与预测状态错位。

5.2.2 指令译码

idu_decode 是纯组合译码模块，通过并行解析操作码、funct3、funct7 和寄存器字段，识别当前参数打开的 RV32/RV64 整数、M、Zicsr、Zba、Zbb、Zbs、Zbc 与 A 扩展指令。模块按 R、I、S、B、U 和 J 型格式拼接并符号扩展立即数至 REG_DATA_WIDTH，产生源寄存器读使能、寄存器地址、目的寄存器写使能和分组控制字段。目的寄存器为 x0 时不产生通用寄存器写使能。

本设计的核心特点是统一的译码信息总线 dec_info_bus_o。该总线采用分组编码方式，将不同类型的指令信息打包在同一条总线上。总线的低位 DECINFO_GRP_BUS 用于指示指令所属功能组（ALU、BJP、MULDIV、CSR、MEM、SYS），高位则根据功能组携带该组特定控制信号。例如 ALU 组指示具体运算和操作数来源；BJP 组指示 JAL/JALR/条件分支以及比较类型；MEM 组指示加载/存储、宽度、符号扩展、A 扩展 AMO/LR/SC 控制；SYS 组指示 ecall、ebreak、mret、fence 等顺序执行规则。

译码阶段只产生结构化控制，不直接解决 RAW。rs1_re_o 和 rs2_re_o 指明源寄存器是否实际使用，reg_we_o 和 reg_waddr_o 指明是否需要分配 commit id。ex_info_bus_o 的三位编码表示 ALU、MUL、DIV、CSR、BJP、LOAD 或 OTHER，其中加载和原子指令归入 LOAD，存储和系统类通常归入 OTHER。若操作码、功能码或扩展开关不匹配，illegal_inst_o 置位并送入 CLINT。

针对不同指令类型，为 EXU 中各执行单元定义的译码信息总线字段如表 26 至表 31 所示。

总线中对应的位	宏定义名称	含义
[2:0]	DECINFO_GRP_BUS	指令组标识，固定为 ALU
3	DECINFO_ALU_LUI	LUI 指令
4	DECINFO_ALU_AUIPC	AUIPC 指令
5	DECINFO_ALU_ADD	ADD/ADDI 指令
6	DECINFO_ALU_SUB	SUB 指令
7	DECINFO_ALU_SLL	SLL/SLLI 指令
8	DECINFO_ALU_SLT	SLT/SLTI 指令
9	DECINFO_ALU_SLTU	SLTU/SLTIU 指令

总线中对应的位	宏定义名称	含义
10	DECINFO_ALU_XOR	XOR/XORI 指令
11	DECINFO_ALU_SRL	SRL/SRLI 指令
12	DECINFO_ALU_SRA	SRA/SRAI 指令
13	DECINFO_ALU_OR	OR/ORI 指令
14	DECINFO_ALU_AND	AND/ANDI 指令
15	DECINFO_ALU_OP2IMM	第二操作数为立即数
16	DECINFO_ALU_OP1PC	第一操作数为 PC
17	DECINFO_ALU_WORD	RV64 的 32 位字操作
18	DECINFO_ALU_ZEXT32	将低 32 位零扩展
19	DECINFO_ALU_SH1ADD	左移 1 位后相加
20	DECINFO_ALU_SH2ADD	左移 2 位后相加
21	DECINFO_ALU_SH3ADD	左移 3 位后相加
22	DECINFO_ALU_ANDN	与非操作
23	DECINFO_ALU_MIN	有符号最小值
24	DECINFO_ALU_MAX	有符号最大值
25	DECINFO_ALU_MINU	无符号最小值
26	DECINFO_ALU_MAXU	无符号最大值
27	DECINFO_ALU_SEXT_H	半字符号扩展
28	DECINFO_ALU_ZEXT_H	半字零扩展
29	DECINFO_ALU_BSET	单比特置位
30	DECINFO_ALU_BEXT	单比特提取
31	DECINFO_ALU_BCLR	单比特清零
32	DECINFO_ALU_BINV	单比特取反
33	DECINFO_ALU_ORN	或非操作
34	DECINFO_ALU_XNOR	同或操作
35	DECINFO_ALU_CLZ	前导零计数
36	DECINFO_ALU_CTZ	末尾零计数
37	DECINFO_ALU_CPOP	置位比特计数
38	DECINFO_ALU_SEXT_B	字节符号扩展
39	DECINFO_ALU_ORC_B	按字节归并置位
40	DECINFO_ALU_REV8	字节次序反转
41	DECINFO_ALU_ROL	循环左移
42	DECINFO_ALU_ROR	循环右移

表 26 基础运算类指令组译码信息总线 - ALU

总线中对应的位	宏定义名称	含义
[2:0]	DECINFO_GRP_BUS	指令组标识, 固定为 BJP
3	DECINFO_BJP_JUMP	JAL/JALR 指令
4	DECINFO_BJP_BEQ	BEQ 指令
5	DECINFO_BJP_BNE	BNE 指令

总线中对应的位	宏定义名称	含义
6	DECINFO_BJP_BLT	BLT 指令
7	DECINFO_BJP_BGE	BGE 指令
8	DECINFO_BJP_BLTU	BLTU 指令
9	DECINFO_BJP_BGEU	BGEU 指令
10	DECINFO_BJP_OP1RS1	第一操作数为 RS1

表 27 分支跳转类指令组译码信息总线 - BJP

总线中对应的位	宏定义名称	含义
[2:0]	DECINFO_GRP_BUS	指令组标识, 固定为 MULDIV
3	DECINFO_MULDIV_MUL	MUL 与 MULW 指令; MULW 同时置位 DECINFO_MULDIV_MULH。
4	DECINFO_MULDIV_MULH	MULH 指令; 与 DECINFO_MULDIV_MUL 同时有效 时表示 MULW。
5	DECINFO_MULDIV_MULHSU	MULHSU 指令
6	DECINFO_MULDIV_MULHU	MULHU 指令
7	DECINFO_MULDIV_DIV	DIV 与 DIVW 指令。
8	DECINFO_MULDIV_DIVU	DIVU 与 DIVUW 指令。
9	DECINFO_MULDIV_REM	REM 与 REMW 指令。
10	DECINFO_MULDIV_REMU	REMU 与 REMUW 指令。
11	DECINFO_MULDIV_OP_MUL	乘法操作
12	DECINFO_MULDIV_OP_DIV	除法操作
13	DECINFO_MULDIV_WORD	RV64 的 32 位字操作
14	DECINFO_MULDIV_CLMUL	无进位乘积低位选择
15	DECINFO_MULDIV_CLMULH	无进位乘积高位选择
16	DECINFO_MULDIV_CLMULR	无进位乘积右对齐选择

表 28 乘除法类指令组译码信息总线 - MULDIV

总线中对应的位	宏定义名称	含义
[2:0]	DECINFO_GRP_BUS	指令组标识, 固定为 CSR
3	DECINFO_CSR_CSRRW	CSRRW/CSRRWI 指令
4	DECINFO_CSR_CSRRS	CSRRS/CSRRSI 指令
5	DECINFO_CSR_CSRRC	CSRRC/CSRRCI 指令
6	DECINFO_CSR_RS1IMM	RS1 为立即数
[18:7]	DECINFO_CSR_CSRADDR	CSR 地址 (12 位)

表 29 CSR 操作类指令组译码信息总线 - CSR

总线中对应的位	宏定义名称	含义
[2:0]	DECINFO_GRP_BUS	指令组标识, 固定为 MEM
3	DECINFO_MEM_LB	LB 指令
4	DECINFO_MEM_LH	LH 指令

总线中对应的位	宏定义名称	含义
5	DECINFO_MEM_LW	LW 指令
6	DECINFO_MEM_LBU	LBU 指令
7	DECINFO_MEM_LHU	LHU 指令
8	DECINFO_MEM_LWU	RV64 的 LWU 指令
9	DECINFO_MEM_LD	RV64 的 LD 指令
10	DECINFO_MEM_SB	SB 指令
11	DECINFO_MEM_SH	SH 指令
12	DECINFO_MEM_SW	SW 指令
13	DECINFO_MEM_SD	RV64 的 SD 指令
14	DECINFO_MEM_OP_LOAD	加载操作
15	DECINFO_MEM_OP_STORE	存储操作
16	DECINFO_MEM_WORD64	64 位访存宽度选择
17	DECINFO_MEM_AMO	原子访存操作
[21:18]	DECINFO_MEM_AMO_OP	原子操作种类
22	DECINFO_MEM_AMO_D	64 位原子访存选择

表 30 访存类指令组译码信息总线 - MEM

总线中对应的位	宏定义名称	含义
[2:0]	DECINFO_GRP_BUS	指令组标识, 固定为 SYS
3	DECINFO_SYS_ECALL	ECALL 指令
4	DECINFO_SYS_EBREAK	EBREAK 指令
5	DECINFO_SYS_NOP	NOP 指令
6	DECINFO_SYS_MRET	MRET 指令
7	DECINFO_SYS_FENCE	FENCE/FENCE.I 指令
8	DECINFO_SYS_DRET	DRET 指令

表 31 系统调用类指令组译码信息总线 - SYS

idu_decode 还输出 rs1_re_o 和 rs2_re_o, 供 HDU 进行读后写冒险检查。若操作码、funct3、funct7 或移位量等字段不符合当前指令集配置, 译码器置位 illegal_inst_o。idu_decode 可对任意输入码产生组合字段, 其 inst_valid_i 和 rst_n 端口当前不参与组合译码; 三级与四级由 idu 的输出门控有效位和写使能, 五级由 idu_id_pipe 的可见有效位门控, 后级不得在 inst_valid_o=0 时采用译码结果。

5.2.3 译码流水寄存器

idu_id_pipe 是五级配置中 IDU 与 Dispatch 之间的级间寄存器, 用于保存译码信息总线、立即数、源寄存器与目的寄存器地址、执行类别、非法指令标

志、压缩指令标志以及分支预测状态。三级配置虽然生成该模块，但不采用其输出；四级配置不生成该模块；只有五级配置把它作为主流水边界。

暂停时，PC、寄存器地址和译码信息等数据字段保持不变；冲刷状态由 `flush_pending` 屏蔽有效位与写使能等控制字段。五级配置的立即数使用两组交替寄存器，选择位只在有效推进时翻转，从而避免为立即数的每一位增加保持选择。C 扩展关闭时，`inst_len_2` 通过条件生成输出常量；增强 BPU 关闭时，`pred_taken` 和 `pred_ghr` 输出常量，预测快照寄存器不进入硬件。

IDU 的处理过程为：来自 IFU 的指令先经过 IRS 的直通或缓冲选择，再进入 `idu_decode` 组合译码。五级配置经 `idu_id_pipe` 保存后送往 Dispatch，三级和四级配置直接输出组合译码结果。IDU 发出的寄存器读地址连接 GPR，GPR 读数据在 Dispatch 与译码信息汇合，再由 HDU 决定是否发射、等待或选择局部旁路。

5.3 Dispatch

Dispatch 位于 IDU 和 EXU 之间，根据译码结果选择执行单元，并在进入 EXU 前完成发射可行性判断。模块参数包括 `PIPELINE_STAGES`、`ENABLE_RV64`、`ENABLE_ZBA`、`ENABLE_ZBB`、`ENABLE_ZBS`、`ENABLE_ZBC`、`ENABLE_C`、`ENABLE_A`、`ENABLE_MISALIGNED_DATA`、`ENABLE_BPU_ENHANCED` 和 `BPU_GHR_BITS`。流水线级数决定等待项、HDU 和 `dispatch_pipe` 的具体结构，其余参数控制相应执行请求、地址检查和预测字段是否生成。

Dispatch 的输入来自三类来源：第一类是 IDU 输出的指令、PC、立即数、译码总线、源/目的寄存器信息和预测状态字段；第二类是 GPR 读出的 `rs1/rs2` 数据；第三类是 EXU/WBU 返回的旁路、完成和提交信息。Dispatch 的输出也分成三类：发往 EXU 的已选择流水线数据，发往 WBU/HDU 的 `commit id` 分配信息，以及发往 Ctrl 的冒险暂停请求。

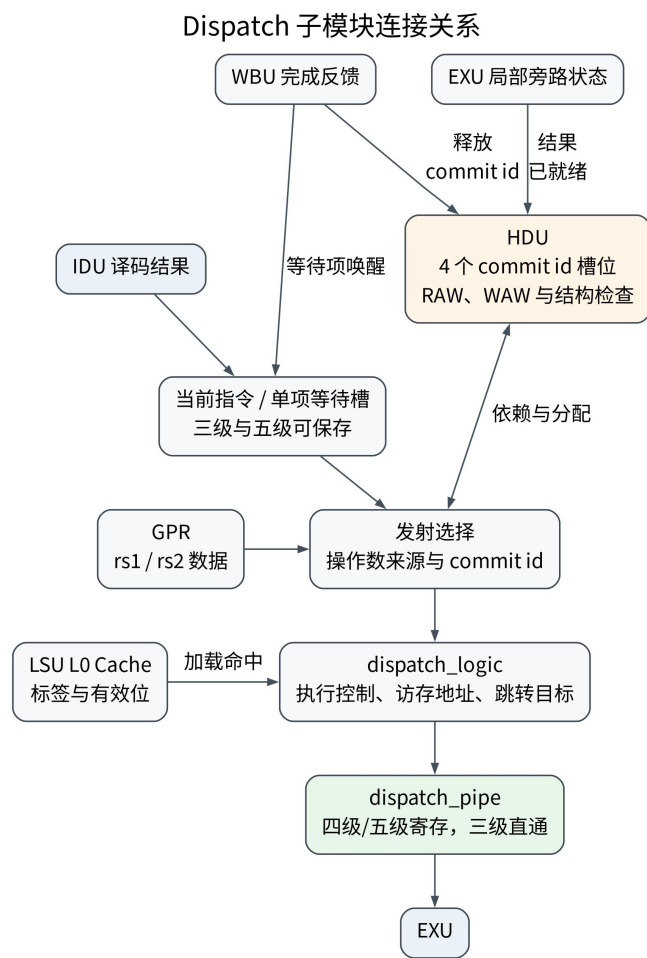


图 10 Dispatch 子模块连接关系

Dispatch 的端口定义如下表所示。输入译码字段只在 `inst_valid_i` 有效且未被暂停或冲刷时参与发射；执行结果反馈仅在各自有效信号置位时参与依赖解除和操作数选择。

端口名	方向	位宽	有效条件与功能
<code>clk</code>	输入	1	Dispatch、HDU、等待项和发射流水寄存器的系统时钟。
<code>rst_n</code>	输入	1	低电平有效异步复位，清除有效状态、占用状态和等待项。
<code>stall_flag_i</code>	输入	<code>CU_BUS_WIDTH</code>	输入各级暂停与冲刷控制，决定输入保持、等待项处理和发射流水寄存器写入。

端口名	方向	位宽	有效条件与功能
inst_valid_i	输入	1	IDU 输出指令有效； 为低时不得分配 commit id 或发出执行 请求。
illegal_inst_i	输入	1	当前指令非法，随指 令进入异常处理通 路。
dec_info_bus_i	输入	DECINFO_WIDTH	IDU 产生的分组译码 控制字段。
dec_imm_i	输入	REG_DATA_WIDTH	符号扩展后的立即 数，用于执行操作数 和地址计算。
dec_pc_i	输入	INST_ADDR_WIDTH	当前指令 PC。
inst_i	输入	INST_DATA_WIDTH	当前 32 位指令编 码。
inst_len_2_i	输入	1	当前指令长度为 16 位，供顺序 PC 和控 制转移计算使用。
rs1_rdata_i	输入	REG_DATA_WIDTH	GPR 处理 x0 和同周 期写回后的 rs1 数 据。
rs1_raw_rdata_i	输入	REG_DATA_WIDTH	GPR 原始 rs1 读数 据，供 LSU L0 Cache 地址候选比较使用。
rs2_rdata_i	输入	REG_DATA_WIDTH	GPR 处理 x0 和同周 期写回后的 rs2 数 据。
pred_taken_i	输入	1	与当前指令对齐的预 测方向。
is_pred_branch_i	输入	1	当前条件分支已经形 成预测跳转。
is_pred_jalr_i	输入	1	当前 JALR 已经形成 预测跳转。
pred_target_i	输入	INST_ADDR_WIDTH	与当前指令对齐的预 测目标地址。
pred_ghr_i	输入	BPU_GHR_BITS	增强 BPU 预测时保 存的全局历史值。
reg_waddr_i	输入	REG_ADDR_WIDTH	当前 IDU 输出的目 的寄存器地址。

端口名	方向	位宽	有效条件与功能
reg1_raddr_i	输入	REG_ADDR_WIDTH	当前 IDU 输出的 rs1 地址。
reg2_raddr_i	输入	REG_ADDR_WIDTH	当前 IDU 输出的 rs2 地址。
next_reg_waddr_i	输入	REG_ADDR_WIDTH	组合译码阶段得到的目的寄存器地址，供提前比较使用。
next_reg1_raddr_i	输入	REG_ADDR_WIDTH	组合译码阶段得到的 rs1 地址，供 HDU 提前查询。
next_reg2_raddr_i	输入	REG_ADDR_WIDTH	组合译码阶段得到的 rs2 地址，供 HDU 提前查询。
reg_we_i	输入	1	当前指令需要写 GPR；目的寄存器为 x0 时保持为低。
rs1_re_i	输入	1	当前指令实际读取 rs1。
rs2_re_i	输入	1	当前指令实际读取 rs2。
csr_we_i	输入	1	当前指令可能写 CSR。
ex_info_bus_i	输入	EX_INFO_BUS_WIDTH	当前指令的执行类别，用于结构冒险和顺序约束。
int_assert_i	输入	1	CLINT 请求清除当前流水状态，并撤销最近一次尚未生效的分配。
retire_issue_hold_i	输入	1	原始陷入事件已经出现时禁止继续发射新指令。
retire_raw_event_i	输入	1	五级配置的原始陷入事件，用于抑制同周期延迟分配写入。
stage3_sync_trap_i	输入	1	三级配置的同步异常取消信号，禁止为本拍指令分配有效写回标签。

端口名	方向	位宽	有效条件与功能
commit_valid_i	输入	1	WBU 完成一项结果处理，允许 HDU 释放对应槽位。
commit_id_i	输入	COMMIT_ID_WIDTH	WBU 本拍完成项的 commit id。
commit_reg_we_i	输入	1	WBU 本拍实际写 GPR。
commit_reg_waddr_i	输入	REG_ADDR_WIDTH	WBU 本拍实际写入的 GPR 地址。
commit_reg_wdata_i	输入	REG_DATA_WIDTH	WBU 已选择的写回数据。
commit_reg_wdata_src_sel_i	输入	3	WBU 已登记的写回来源选择值。
commit_reg_wdata_candidates_i	输入	8*REG_DATA_WIDTH	WBU 各执行来源的候选数据，LSU L0 Cache 地址判断后按来源选择。
stage3_tag_query_valid_i	输入	1	三级 WBU 请求查询 commit id 对应的目的寄存器。
stage3_tag_query_id_i	输入	COMMIT_ID_WIDTH	三级 WBU 当前查询的 commit id。
exu_alu_bypass_valid_i	输入	1	ALU 结果已写入 EXU 局部旁路暂存区。
exu_alu_bypass_commit_id_i	输入	COMMIT_ID_WIDTH	ALU 局部旁路结果对应的 commit id。
exu_alu_bypass_bank_i	输入	1	ALU 结果写入的两项暂存区编号。
exu_alu_bypass_data_i	输入	REG_DATA_WIDTH	ALU 完成数据，供 AGU 保存项写入。
exu_alu_agu_forward_valid_i	输入	1	ALU 结果可作为存储器访问基址保存。
exu_alu_agu_forward_data_i	输入	BUS_ADDR_WIDTH	ALU 提供的存储器访问基址数据。
exu_lsu_bypass_valid_i	输入	1	普通 LSU 结果已进入局部旁路暂存区。
exu_lsu_bypass_hot_valid_i	输入	1	LSU L0 Cache 命中结果已完成。

端口名	方向	位宽	有效条件与功能
exu_lsu_dtcn_bypass_lookahead_valid_i	输入	1	固定一周期 DTCM 配置下，读地址握手预告下一周期可用。Cache 或乒乓缓存引入可变延迟时固定为低。
exu_lsu_dtcn_bypass_lookahead_commit_id_i	输入	COMMIT_ID_WIDTH	DTCM 提前完成提示对应的 commit id。
exu_lsu_bypass_commit_id_i	输入	COMMIT_ID_WIDTH	LSU 旁路结果对应的 commit id。
exu_lsu_bypass_data_i	输入	BUS_ADDR_WIDTH	LSU 完成数据，供等待项的 AGU 基址保存。
exu_lsu_bypass_bank_data_i	输入	REG_DATA_WIDTH	LSU 局部旁路暂存区当前保存的数据。
lsu_hot_cache_ready_i	输入	1	LSU L0 Cache 可以接受本拍命中访问，且不存在冲突事务。
lsu_hot_cache_meta_valid_i	输入	1	LSU L0 Cache 标签或有效位修改信息有效。
lsu_hot_cache_meta_addr_i	输入	BUS_ADDR_WIDTH	LSU L0 Cache 修改信息对应的地址。
lsu_hot_cache_meta_way_i	输入	1	LSU L0 Cache 修改信息对应的路。
lsu_hot_cache_meta_clear_i	输入	1	清除指定 LSU L0 Cache 项；为低时写入或刷新标签。
lsu_hot_cache_meta_live_store_valid_i	输入	1	当前正在发送的存储字地址有效，供命中冲突检查。
lsu_hot_cache_meta_live_store_beat_i	输入	DTCM_ADDR_WIDTH - $\log_2(\text{BUS_DATA_WIDTH}/8)$	当前正在发送的存储自然字地址。
lsu_hot_cache_meta_queued_store_valid_i	输入	1	LSU 请求 FIFO 队首存储字地址有效。

端口名	方向	位宽	有效条件与功能
lsu_hot_cache_meta_queued_store_beat_i	输入	DTCM_ADDR_WIDTH H- log2(BUS_DATA_WIDTH/8)	LSU 请求 FIFO 队首存储自然字地址。
lsu_hot_cache_meta_split_store_valid_i	输入	1	非对齐拆分的第二次存储字地址有效。
lsu_hot_cache_meta_split_store_beat_i	输入	DTCM_ADDR_WIDTH H- log2(BUS_DATA_WIDTH/8)	非对齐拆分第二次存储的自然字地址。
lsu_hot_cache_meta_fill_valid_i	输入	1	LSU 正在写入 L0 Cache 数据体的自然字地址有效。
lsu_hot_cache_meta_fill_beat_i	输入	DTCM_ADDR_WIDTH H- log2(BUS_DATA_WIDTH/8)	LSU L0 Cache 本拍写入的自然字地址。
exu_mul_bypass_valid_i	输入	1	MUL 结果已进入 EXU 局部旁路暂存区。
exu_mul_bypass_commit_id_i	输入	COMMIT_ID_WIDTH	MUL 局部旁路结果对应的 commit id。
hazard_stall_o	输出	1	当前指令因 RAW、WAW、结构、顺序或槽位不足而不能发射。
long_inst_atom_lock_o	输出	1	存在必须保持完整执行次序的长指令或原子操作，CLINT 据此延后陷入处理。
commit_id_o	输出	COMMIT_ID_WIDTH	本拍选中指令分配的 commit id。
wb_alloc_valid_o	输出	1	本拍发射一条需要写 GPR 的指令，WBU 应登记最新 commit id。
wb_alloc_waddr_o	输出	REG_ADDR_WIDTH	本拍 WBU 分配对应的目的寄存器地址。
wb_alloc_commit_id_o	输出	COMMIT_ID_WIDTH	本拍 WBU 分配对应的 commit id。

端口名	方向	位宽	有效条件与功能
wb_alloc_rollback_o	输出	1	地址修正或冲刷撤销最近一次尚未生效的 WBU 分配。
wb_alloc_rollback_commit_id_o	输出	COMMIT_ID_WIDTH	需要撤销的 commit id。
wb_alloc_prewrite_inhibit_o	输出	1	原始陷入事件与延迟分配同拍出现时，禁止该分配写入 WBU 查询表。
wb_alloc_prewrite_inhibit_commit_id_o	输出	COMMIT_ID_WIDTH	需要禁止写入的 commit id。
stage3_arch_live_o	输出	$2^{\text{COMMIT_ID_WIDTH}}$	三级配置各 commit id 当前仍具有架构写入资格的独热状态。
stage3_tag_valid_o	输出	1	三级 WBU 查询的 commit id 仍有效。
stage3_tag_waddr_o	输出	REG_ADDR_WIDTH	三级 WBU 查询得到的目的寄存器地址。
pipe_inst_addr_o	输出	INST_ADDR_WIDTH	发往 EXU 和 CLINT 的指令 PC。
pipe_inst_o	输出	INST_DATA_WIDTH	发往 EXU 和 CLINT 的指令编码。
pipe_inst_len_2_o	输出	1	发往 EXU 的 16 位指令长度标志。
pipe_inst_valid_o	输出	1	发射流水输出有效；暂停时保持，冲刷时清零。
pipe_stage3_ticket_valid_o	输出	1	三级配置当前执行指令已取得有效写回标签。
pipe_reg_we_o	输出	1	当前发射指令需要写 GPR。
pipe_reg_waddr_o	输出	REG_ADDR_WIDTH	当前发射指令的目的寄存器地址。
pipe_csr_we_o	输出	1	当前发射指令需要写 CSR。
pipe_rs1_rdata_o	输出	REG_DATA_WIDTH	当前发射指令的 rs1 数据，供 BRU 和系统控制使用。

端口名	方向	位宽	有效条件与功能
req_alu_o	输出	1	当前发射指令请求 ALU。
alu_op1_o	输出	REG_DATA_WIDTH	ALU 第一操作数。
alu_op2_o	输出	REG_DATA_WIDTH	ALU 第二操作数。
alu_op_info_o	输出	ALU_OP_WIDTH	ALU 子操作控制字段。
alu_result_sel_o	输出	ALU_RESULT_SEL_WIDTH	ALU 结果选择控制字段。
req_bjp_o	输出	1	当前发射指令请求 BRU。
bjp_op_jal_o	输出	1	当前 BRU 请求为 JAL。
bjp_op_beq_o	输出	1	当前 BRU 请求为 BEQ。
bjp_op_bne_o	输出	1	当前 BRU 请求为 BNE。
bjp_op_bltn_o	输出	1	当前 BRU 请求为 BLT。
bjp_op_bltu_o	输出	1	当前 BRU 请求为 BLTU。
bjp_op_bge_o	输出	1	当前 BRU 请求为 BGE。
bjp_op_bgeu_o	输出	1	当前 BRU 请求为 BGEU。
bjp_op_jalr_o	输出	1	当前 BRU 请求为 JALR。
bjp_adder_result_o	输出	INST_ADDR_WIDTH	Dispatch 计算的 JAL 或条件分支目标地址。
bpu_cond_recovery_pc_o	输出	INST_ADDR_WIDTH	条件分支方向恢复时应采用的实际下一 PC。
bpu_cond_recovery_ghr_o	输出	BPU_GHR_BITS	条件分支恢复后的全局历史值。
req_mul_o	输出	1	当前发射指令请求 MUL。
mul_op_mul_o	输出	1	选择 MUL。
mul_op_mulh_o	输出	1	选择 MULH。
mul_op_mulhsu_o	输出	1	选择 MULHSU。
mul_op_mulhu_o	输出	1	选择 MULHU。

端口名	方向	位宽	有效条件与功能
mul_op_clmul_o	输出	1	选择 CLMUL。
mul_op_clmulh_o	输出	1	选择 CLMULH。
mul_op_clmulr_o	输出	1	选择 CLMULR。
mul_op1_o	输出	REG_DATA_WIDTH	MUL 第一操作数。
mul_op2_o	输出	REG_DATA_WIDTH	MUL 第二操作数。
mul_commit_id_o	输出	COMMIT_ID_WIDTH	MUL 请求对应的 commit id。
req_div_o	输出	1	当前发射指令请求 DIV。
div_op_div_o	输出	1	选择有符号商。
div_op_divu_o	输出	1	选择无符号商。
div_op_rem_o	输出	1	选择有符号余数。
div_op_remu_o	输出	1	选择无符号余数。
div_op_word_o	输出	1	RV64 配置下选择 32 位字除法或余数操作。
div_op1_o	输出	REG_DATA_WIDTH	被除数。
div_op2_o	输出	REG_DATA_WIDTH	除数。
div_commit_id_o	输出	COMMIT_ID_WIDTH	DIV 请求对应的 commit id。
req_csr_o	输出	1	当前发射指令请求 CSR 执行单元。
csr_addr_o	输出	BUS_ADDR_WIDTH	CSR 地址。
csr_read_sel_o	输出	CSR_READ_SEL_WIDTH	CSR 读选择预译码值。
csr_op1_o	输出	REG_DATA_WIDTH	CSR 源操作数或零扩展立即数。
csr_csrrw_o	输出	1	选择 CSRRW 类操作。
csr_csrrs_o	输出	1	选择 CSRRS 类操作。
csr_csrrc_o	输出	1	选择 CSRRC 类操作。
req_mem_o	输出	1	当前发射指令请求 LSU。
mem_commit_id_o	输出	COMMIT_ID_WIDTH	LSU 请求对应的 commit id。
mem_op_lb_o	输出	1	选择有符号字节加载。

端口名	方向	位宽	有效条件与功能
mem_op_lh_o	输出	1	选择有符号半字加载。
mem_op_lw_o	输出	1	选择有符号字加载。
mem_op_lbu_o	输出	1	选择无符号字节加载。
mem_op_lhu_o	输出	1	选择无符号半字加载。
mem_op_lwu_o	输出	1	RV64 配置下选择无符号字加载。
mem_op_ld_o	输出	1	RV64 配置下选择双字加载。
mem_op_load_o	输出	1	当前访存请求为加载。
mem_op_store_o	输出	1	当前访存请求为存储。
mem_op_sb_o	输出	1	选择字节存储。
mem_op_sh_o	输出	1	选择半字存储。
mem_op_sw_o	输出	1	选择字存储。
mem_op_sd_o	输出	1	RV64 配置下选择双字存储。
mem_op_amo_o	输出	1	当前请求为 LR、SC 或 AMO。
mem_amo_op_o	输出	AMO_OP_WIDTH	原子操作类别。
mem_op_amo_d_o	输出	1	RV64 配置下选择双字原子操作。
mem_addr_o	输出	BUS_ADDR_WIDTH	Dispatch 已计算的有效访存地址。
mem_wdata_o	输出	REG_DATA_WIDTH	存储或原子操作写数据。
mem_wmask_o	输出	BUS_DATA_WIDTH/ 8	与地址低位对齐的逐字节写使能。
mem_misaligned_load_o	输出	1	加载地址不满足当前配置的对齐规则。
mem_misaligned_store_o	输出	1	存储地址不满足当前配置的对齐规则。
lsu_hot_cache_hit_o	输出	1	LSU L0 Cache 标签命中且本拍允许使用本地数据。
lsu_hot_cache_way_o	输出	1	LSU L0 Cache 命中的路。

端口名	方向	位宽	有效条件与功能
sys_op_nop_o	输出	1	当前系统类操作为空操作。
sys_op_mret_o	输出	1	当前指令为 MRET。
sys_op_ecall_o	输出	1	当前指令为 ECALL。
sys_op_ebreak_o	输出	1	当前指令为 EBREAK。
sys_op_fence_o	输出	1	当前指令为 FENCE 或 FENCE.I。
sys_op_dret_o	输出	1	当前指令为调试返回操作。
pred_taken_o	输出	1	与发射指令对齐的预测方向。
is_pred_branch_o	输出	1	与发射指令对齐的条件分支预测标志。
is_pred_jalr_o	输出	1	与发射指令对齐的 JALR 预测标志。
pred_target_o	输出	INST_ADDR_WIDTH	与发射指令对齐的预测目标。
pred_ghr_o	输出	BPU_GHR_BITS	与发射指令对齐的全局历史值。
illegal_inst_o	输出	1	与发射指令对齐的非法指令标志。
bypass_src_op1_o	输出	BYPASS_SRC_WIDTH	EXU 第一操作数的局部旁路来源编码。
bypass_src_op2_o	输出	BYPASS_SRC_WIDTH	EXU 第二操作数的局部旁路来源编码。
bypass_bank_op1_o	输出	1	第一操作数选择 ALU 暂存区 0 或 1。
bypass_bank_op2_o	输出	1	第二操作数选择 ALU 暂存区 0 或 1。
alu_result_bank_o	输出	1	本拍 ALU 结果应写入的局部暂存区编号。

表 32 Dispatch 输入输出端口

5.3.1 发射选择与单项等待槽

Dispatch 每周期最多发射一条指令。三级和五级配置可以保存一条因源操作数未就绪而等待的非有序类指令；四级配置不启用等待项，发生阻塞时直接

保持当前 Dispatch 输入。等待项保存译码控制、立即数、PC、原始指令、通用寄存器数据、目的寄存器、执行类别、源操作数依赖信息和压缩指令长度，但不保存分支预测状态；从等待项发射时，预测相关输出固定为零。

从微架构角度看，该机制允许不依赖等待项的当前指令优先进入执行单元，因此具备受限的乱序发射能力。但它只包含一项等待结构，不包含重排序缓冲区（ROB）和多项发射队列，也不改变顺序取指与顺序译码。处理器状态由 commit id、WBU 最新 commit id 过滤以及流水线冲刷共同保证。因此，Alkaid 属于顺序前端、轻度乱序单发射与有限乱序完成的微架构。发射选择遵守以下优先级：

- 若等待槽有效、操作数就绪且不存在结构冒险，则发射等待槽指令。
- 否则，若当前指令有效且允许发射，则发射当前指令。
- 否则，三级或五级配置仅在等待槽可用、当前指令不是分支、跳转、CSR、系统、存储或非法指令、没有 WAW 与结构冒险，且存在未解除的 RAW 冒险时，捕获当前指令到等待槽。若等待槽正在本拍发射，当前指令读取该等待项的目的寄存器时也可捕获，并在时钟沿写入新的等待项。
- 其余情况暂停 Dispatch/ID。

等待槽是否就绪并不只看寄存器值是否已经写回。它还会检查本拍 EXU ALU/LSU/MUL 旁路是否命中等待槽依赖，或 AGU 前递保持是否已经保存了 MEM 地址所需基值。若操作数可在 EXU 局部旁路中使用，等待槽不把宽数据强行写回 Dispatch，而是生成 bypass_src_op 和 bypass_bank_op 信号来控制。若操作数用于 MEM 地址，等待槽必须等到 AGU 前递数据可用，因为 MEM 地址要在 Dispatch 阶段形成并随流水寄存器一同锁存。

当前指令能否越过未就绪等待项取决于流水线配置、执行类别、数据冒险和写后写冲突。五级配置允许与等待项不存在 RAW 和 WAW 冲突的前向条件分支先发射，其余分支、跳转、CSR、系统、存储和非法指令仍保持顺序；三级配置不允许条件分支越过等待项；四级配置不生成等待项。ALU、加载、乘法和除法等普通指令在不破坏寄存器相关关系时可以先发射。

5.3.2 HDU 与 commit id 记分牌

HDU（数据冒险检查单元）使用 2 位 commit id 标识 4 项未完成操作。每次有效分配从空闲项中选择编号，WBU 完成结果时携带相同编号释放对应项。HDU 只保存目的寄存器、执行类别、commit id 和结果可用状态，不保存

运算结果数据。三级配置与四级、五级配置采用不同的状态结构，具体参数如下表所示。

参数	默认值	有效取值	功能说明
PIPELINE_STAGES	5	3、4、5	选择三级、四级或五级流水线对应的 HDU 结构。
ENABLE_BPU_ENHANCED	0	0、1	在五级流水线中生成与增强 BPU 控制时序配合的依赖完成保持寄存器。
LSU_BYPASS_COMMITS_NEXT_CYCLE	0	0、1	指定 LSU 旁路结果是否在下一周期由 WBU 完成接口再次提供；集成 CPU 固定取 1。

表 33 HDU 参数

HDU 的端口定义如下表所示。表中的三级查询端口只在三级流水线中参与功能，局部旁路和地址前递端口只在四级和五级流水线中参与功能。

端口名	方向	位宽	有效条件与功能
clk	输入	1	HDU 状态时钟。
rst_n	输入	1	低电平有效异步复位，清除占用、旁路可用和依赖有效状态。
rd_we_valid	输入	1	本周周期接受一次 HDU 分配。
alloc_cancel_i	输入	1	取消本周周期分配，禁止 HDU 状态改变。
id_payload_advance_i	输入	1	ID 级内容向前推进；五级配置据此保存下一条指令的源寄存器依赖。
next_rs1_addr_i	输入	REG_ADDR_WIDTH	下一条 ID 级指令的 rs1 地址。
next_rs2_addr_i	输入	REG_ADDR_WIDTH	下一条 ID 级指令的 rs2 地址。
held_rs1_alloc_match_i	输入	1	ID 级保持期间，已保存的 rs1 与本周周期实际分配的目的寄存器相同。

端口名	方向	位宽	有效条件与功能
held_rs2_alloc_match_i	输入	1	ID 级保持期间，已保存的 rs2 与本周周期实际分配的寄存器地址相同。
rd_addr	输入	REG_ADDR_WIDTH	当前待发射指令的目的寄存器地址。
rs1_addr_i	输入	REG_ADDR_WIDTH	当前待发射指令的 rs1 地址。
rs2_addr_i	输入	REG_ADDR_WIDTH	当前待发射指令的 rs2 地址。
rd_we	输入	1	当前待发射指令需要写 GPR。
rs1_re	输入	1	当前待发射指令实际读取 rs1。
rs2_re	输入	1	当前待发射指令实际读取 rs2。
ex_info_bus	输入	EX_INFO_BUS_WIDTH	当前待发射指令的执行类别。
inst_valid_i	输入	1	当前待发射指令有效。
is_mem_inst_i	输入	1	当前指令属于加载或存储。
is_mem_store_i	输入	1	当前指令属于存储。
bjp_cond_compare_i	输入	1	当前指令属于需要寄存器比较的条件分支。
full_barrier_i	输入	1	当前指令必须等待全部未完成项结束。
agu_forward_valid_i	输入	$2^{\text{COMMIT_ID_WIDTH}}$	按 commit id 指示地址计算所需结果已经进入 AGU 保存项。
lsu_load_bypass_i	输入	1	LSU 加载结果可供本周周期操作数选择。
lsu_load_bypass_commit_now_i	输入	1	LSU 结果与本周周期 WBU 完成事件对应。
lsu_load_bypass_id_i	输入	COMMIT_ID_WIDTH	LSU 旁路结果的 commit id。
mul_bypass_i	输入	1	MUL 结果可供本周周期操作数选择。

端口名	方向	位宽	有效条件与功能
mul_bypass_id_i	输入	COMMIT_ID_WIDTH	MUL 旁路结果的 commit id。
alloc_rd_addr_i	输入	REG_ADDR_WIDTH	实际发射指令的目的寄存器地址。
alloc_rd_we_i	输入	1	实际发射指令需要写 GPR。
alloc_ex_info_bus_i	输入	EX_INFO_BUS_WIDTH	实际发射指令的执行类别。
commit_valid_i	输入	1	WBU 完成事件有效。
commit_id_i	输入	COMMIT_ID_WIDTH	WBU 完成事件对应的 commit id。
commit_reg_we_i	输入	1	WBU 完成事件实际写 GPR。
commit_reg_waddr_i	输入	REG_ADDR_WIDTH	WBU 完成事件的目的寄存器地址。
flush_pop_i	输入	1	撤销错误路径对应的最近分配或架构写入资格。
stage3_tag_query_valid_i	输入	1	三级 WBU 发起 commit id 至目的寄存器地址查询。
stage3_tag_query_id_i	输入	COMMIT_ID_WIDTH	三级 WBU 查询的 commit id。
commit_id_o	输出	COMMIT_ID_WIDTH	下一项可分配的 commit id。
long_inst_atom_lock_o	输出	1	至少一个 commit id 项仍被占用，供陷入和原子操作控制使用。
non_alu_busy_o	输出	1	存在必须保持顺序的未完成项。
bypass_src_op1_o	输出	BYPASS_SRC_WIDTH	第一操作数旁路来源，编码依次表示不使用旁路、ALU、LSU 和 MUL。
bypass_src_op2_o	输出	BYPASS_SRC_WIDTH	第二操作数旁路来源，编码与第一操作数相同。

端口名	方向	位宽	有效条件与功能
bypass_bank_op1_o	输出	1	第一操作数选择两项 ALU 暂存数据中的哪一项；选择 LSU 或 MUL 时固定为 0。
bypass_bank_op2_o	输出	1	第二操作数选择两项 ALU 暂存数据中的哪一项；选择 LSU 或 MUL 时固定为 0。
alu_result_bank_o	输出	1	新 ALU 结果写入两项 ALU 暂存数据中的哪一项。
waw_hazard_o	输出	1	当前目的寄存器与必须阻塞写后写冒险的未完成项相同。
waw_blocked_o	输出	1	有效指令因写后写数据冒险被阻塞。
fifo_full_o	输出	1	4 项 commit id 状态均被占用。
struct_hazard_o	输出	1	当前写 GPR 指令因无空闲 commit id 而发生结构冒险。
barrier_hazard_o	输出	1	当前指令因顺序约束被阻塞。
raw_hazard_o	输出	1	当前指令至少一个源操作数尚未就绪。
hazard_blocked_o	输出	1	RAW、WAW、结构冒险或顺序约束中至少一项成立。
rs1_dep_blocked_o	输出	1	rs1 存在未就绪依赖。
rs2_dep_blocked_o	输出	1	rs2 存在未就绪依赖。
agu_forward_op1_o	输出	1	存储器访问基址可从按 commit id 保存的地址数据取得。
agu_forward_candidate_op1_o	输出	1	存储器访问基址依赖 ALU 结果，但对应地址数据尚未确认有效。

端口名	方向	位宽	有效条件与功能
rs1_dep_valid_o	输出	1	rs1 存在有效未完成依赖。
rs1_dep_id_o	输出	COMMIT_ID_WIDTH	rs1 所依赖结果的 commit id。
rs1_dep_has_older_o	输出	1	保留的多项依赖状态输出，当前固定为 0。
rs2_dep_valid_o	输出	1	rs2 存在有效未完成依赖。
rs2_dep_id_o	输出	COMMIT_ID_WIDTH	rs2 所依赖结果的 commit id。
rs2_dep_has_older_o	输出	1	保留的多项依赖状态输出，当前固定为 0。
stage3_arch_live_o	输出	$2^{\text{COMMIT_ID_WIDTH}}$	三级配置中各 commit id 是否仍具有 GPR 写入资格。
stage3_tag_valid_o	输出	1	三级目的寄存器查询命中有效 commit id 项。
stage3_tag_waddr_o	输出	REG_ADDR_WIDTH	三级目的寄存器查询返回的目的寄存器地址。

表 34 HDU 输入输出端口

HDU（数据冒险检查单元）结构与配置差异

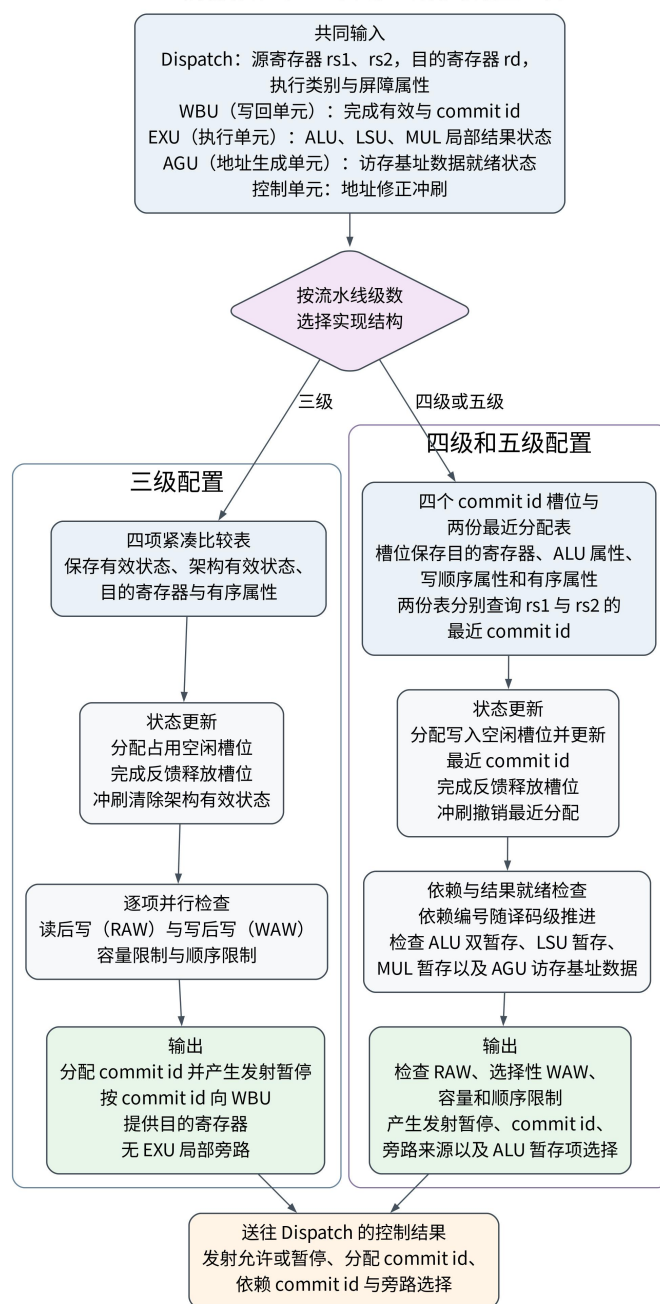


图 11 HDU 结构与控制关系

四级和五级配置的每个槽位保存 `fifo_valid`、`fifo_rd_addr`、`fifo_is_alu`、`fifo_waw_block` 和 `fifo_ordered`。两份 32 项分布式 RAM 分别按 `rs1` 和 `rs2` 地址查询该寄存器最近一次分配的 `commit id`；刚完成分配但尚未写入 RAM 的结果由分配寄存器直接前递。四级配置使用当前组合译码的源寄存器地址查询；五级配置在 ID 级推进时保存两个源操作数的依赖有效位和 `commit id`。五级增强 BPU 配置还保存与下一次 ID 推进对齐的完成标志，避免执行完成事件直接进入依赖有效路径。

EXU 局部旁路的数据保存在两项 ALU、一项 LSU 和一项 MUL 暂存区，HDU 只保存按 commit id 展开的独热可选状态。条件分支比较可以选择 ALU 或 LSU 结果，不能直接选择 MUL 结果；ALU 和 MUL 指令以及存储数据可以选择 ALU、LSU 或 MUL 结果。存储器访问基址只有在前序 ALU 结果已经进入 AGU 保存项时才能直接放行；前序 LSU 结果需要先进入等待项，再由 Dispatch 保存其地址数据后唤醒。JALR 不使用 AGU 保存值，必须等待所需寄存器值经 WBU 和 GPR 可见。

四级和五级的写后写处理按前序执行类别区分。较早的 DIV、CSR、分支或跳转以及 OTHER 类写指令设置 fifo_waw_block，相同 rd 的新指令必须等待；较早的 ALU、MUL 和加载写回允许同 rd 指令继续分配，最终由 WBU 的每寄存器最新 commit id 表决定是否真正写入 GPR。flush_pop_i 只撤销最近一次已经登记的分配，并同步清除该 commit id 对应的旁路选择状态；正常完成按 commit id 释放槽位。

三级配置由 hdu_select 选择 hdu_stage3_compact。该结构使用四项内容寻址比较表保存有效状态、架构写入有效状态、目的寄存器和有序属性，对任意尚未完成的同 rd 指令阻塞写后写冒险，且所有 EXU 局部旁路与 AGU 保存选择固定为零。依赖指令必须等待 WBU 的 commit_valid_i、commit_id_i 和 commit_reg_wdata_i。冲刷只取消槽位的架构写入资格，槽位仍保留到相应结果到达 WBU 后释放，以防 commit id 过早复用；stage3_arch_live_o 和 stage3_tag_* 端口供 WBU 查询结果是否仍可写入以及对应 rd。

5.3.3 分发控制与地址生成

dispatch_logic 是组合分发逻辑，输入已选择的当前指令或等待槽流水线数据。它根据 dec_info_bus 生成 ALU、BJP、MUL、DIV、CSR、MEM、SYS 请求，并完成部分地址与掩码预计算。

地址生成包含两类计算：存储器访问地址为 $rs1 + imm$ ，JAL 和条件分支目标为 $PC + imm$ 。JALR 不使用 Dispatch 的存储器访问地址，而是在 BRU 中计算 $rs1 + I$ 型立即数 并清零最低位。

dispatch_logic 按配置选择三种实现。ENABLE_BPU_ENHANCED=1 时，模块使用 Dispatch 为当前指令和等待项分别预计算后再按发射来源选择的存储器访问地址与直接控制转移目标；增强 BPU 关闭且 PIPELINE_STAGES=5 时，两类地址使用独立加法器；增强 BPU 关闭且流水线为三级或四级时，两

类地址共享一套加法器。三种实现都在 Dispatch 阶段确定存储器访问地址以及 JAL 和条件分支目标，JALR 目标仍由 BRU 计算。

AGU 是纯组合地址辅助模块。dispatch_logic 先形成 $rs1 + imm$ 的有效地址，AGU 再根据访存译码字段输出加载类型、加载或存储属性、写字节使能和地址非对齐标志。op_mem 只门控有效地址和写字节使能，传入 AGU 的访存信息已由 dispatch_logic 按指令类别屏蔽。AMO、LR 和 SC 的操作控制由 dispatch_logic 根据译码信息直接生成，不属于 AGU 的地址和字节使能计算。RV32 配置打开 ENABLE_MISALIGNED_DATA 时，AGU 抑制可由 LSU 拆分处理的地址非对齐异常；RV64 尚未实现非对齐拆分，因此半字、字和双字非对齐访问仍形成异常。RV32 配置下 LWU、LD 和 SD 固定无效。A 扩展关闭时，AMO、LR 和 SC 控制通过条件生成固定为无效，不进入普通 RV32IM 流水数据。

5.3.4 AGU 前递保持

地址类 RAW 是 Dispatch 中最特殊的一类。例如：

```
lw x5, 0(x1)
lw x6, 8(x5)
```

第二条加载指令的地址必须由 $x5 + 8$ 形成。由于地址会随 dispatch_pipe.mem_addr 进入 EXU，EXU 局部旁路无法事后修正地址。Dispatch 维护 agu_forward_valid[id] 和 agu_forward_data[id]。前序 ALU 结果可直接写入对应保存项；前序 LSU 结果先使依赖指令进入等待项，结果到达后由 Dispatch 保存所需基址并唤醒等待项。HDU 发现存储器访问 rs1 依赖已经就绪的保存项时输出 agu_forward_op1_o，Dispatch 用该数据参与地址加法。

这个结构把地址计算所需数据与执行级局部旁路分开：AGU 保存项只服务存储器访问基址，JALR 不使用该数据；执行级旁路在 EXU 内选择 ALU、BRU、MUL 和存储数据操作数。三级配置关闭两类选择，相关指令等待 WBU 写回后再读取 GPR。

5.3.5 LSU L0 Cache 标签管理机制

当 ENABLE_LSU_HOT_CACHE=1 时，Dispatch 生成轻量标签/有效位影子表，用于判断 DMEM 热点加载/存储是否命中 EXU LSU 内部的数据体。Dispatch 只保存标签和有效位，不保存宽数据。命中判定需要满足：访问地址落在 DMEM 地址区域、加载/存储非 AMO、非不支持的非对齐访问、基址已经

就绪，且 EXU LSU 返回 `lsu_hot_cache_ready_i` 表示内部没有未完成事务、拆分访问、原子操作或写后读风险。

命中后，Dispatch 输出 `lsu_hot_cache_hit_o` 和 `lsu_hot_cache_way_o` 给 EXU。EXU LSU 本地使用字节数据 RAM 产生 `lsu_hot_cache_rdata`，并让该加载指令像普通读返回一样进入加载扩展、写回和旁路通路。EXU LSU 在填充、整字存储分配、部分存储清除、原子操作清除时通过 `lsu_hot_cache_meta_valid/addr/way/clear` 更新或清空 Dispatch 影子表。该实现规定：标签命中判断可以前移到 Dispatch，数据体留在 EXU；不能把缓存数据送回 Dispatch 参与分支比较。

5.3.6 发射流水寄存器

`dispatch_pipe` 在四级和五级配置中是 Dispatch 与 EXU 之间的级间寄存器，保存指令有效位、PC、原始指令、`inst_len_2`、`commit id`、`rd`、写使能、CSR 地址、`rs1/rs2` 数据、ALU 操作数、BJP 控制、MEM 访问包、MUL/DIV/CSR/SYS 控制和旁路选择。三级配置使用组合直通，不设置 Dispatch 后寄存边界。

控制规则如下：

条件	有效位处理	流水线数据处理	目的
复位	有效位与会产生状态修改的控制字段清零	宽数据字段不要求逐项复位	初始化流水线
Dispatch 暂停	保持	保持	该条件优先，EXU 忙时不丢请求
Dispatch 未暂停，且 ID 暂停或冲刷	有效位与会产生状态修改的控制字段清零	宽数据字段可以保持	清除错误路径或未确认发射
接收新指令	写入新有效位	写入新流水线数据	发射一条确定指令

表 35 `dispatch_pipe` 控制规则表

这种设计将发射选择、等待队列管理以及 HDU 冒险检测集中在 Dispatch 阶段完成，使 EXU 接收到的均为已经完成发射决策的流水线信息，旁路控制也已编码为操作数来源和旁路编号，避免执行阶段再次进行复杂判断。

Dispatch 与 EXU 之间的寄存边界保存已经选定的操作数、执行控制和 `commit id`，使旁路选择、算术逻辑或分支比较以及取指地址修正分别位于明确的处理区间。四级和五级配置使用该寄存边界限制控制信号跨级传播；三级配置虽然直接传递执行请求，但关闭 EXU 局部旁路，并由紧凑型 HDU 控制源寄存器等待，以控制合并执行级的逻辑深度。

5.4 EXU

EXU 是执行级封装模块，内部例化 ALU、BRU、LSU、MUL、DIV 和 CSR 执行单元。四级和五级配置还例化 EXU 局部旁路暂存区，三级配置不生成该结构。Dispatch 已经完成发射选择、存储器访问地址和直接控制转移目标预计算以及旁路控制编码，EXU 接收这些控制与操作数，完成具体计算、访存、控制流校验、异常输出和写回请求。EXU 总暂停信号是 ALU、MUL、DIV、CSR 和 LSU 暂停信号的逻辑或。

EXU 的重要约束是：四级和五级的执行操作数可以使用本地旁路，Dispatch 已经形成的存储器访问地址不能被迟到数据事后修改。ALU、BRU、MUL 和存储数据可以从旁路暂存区选择；JAL 和条件分支使用 Dispatch 提供的 `bjp_adder_result_i`，JALR 则在 BRU 内根据 `rs1` 和 I 型立即数计算目标。

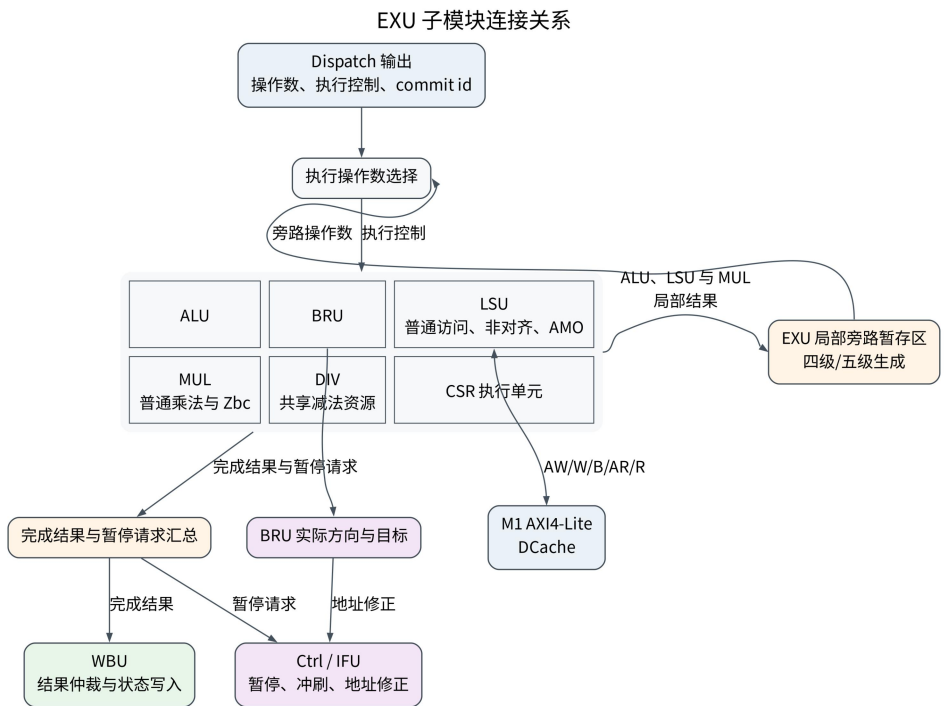


图 12 EXU 子模块连接关系

EXU 的端口定义如下表所示。Dispatch 发来的控制字段只在相应请求信号有效时使用；各执行单元的完成信号、数据和 `commit id` 必须保持一致，直至 WBU 对该执行来源给出接收条件。

端口名	方向	位宽	有效条件与功能
clk	输入	1	EXU 多周期单元、局部旁路暂存区和输出状态的系统时钟。

端口名	方向	位宽	有效条件与功能
rst_n	输入	1	低电平有效异步复位，清除各执行单元有效状态和占用状态。
inst_addr_i	输入	INST_ADDR_WIDTH	当前执行指令的 PC。
inst_len_2_i	输入	1	当前执行指令长度为 16 位。
reg_we_i	输入	1	当前指令需要写 GPR。
reg_waddr_i	输入	REG_ADDR_WIDTH	当前指令的目的寄存器地址。
csr_we_i	输入	1	当前指令需要写 CSR。
csr_waddr_i	输入	BUS_ADDR_WIDTH	当前 CSR 地址。
csr_rdata_i	输入	REG_DATA_WIDTH	CSR 模块返回的旧值。
int_assert_i	输入	1	CLINT 已请求清除流水状态；未开始的普通执行请求不得产生架构写入。
int_jump_i	输入	1	CLINT 的陷入或返回地址已经有效，合入 EXU 总跳转控制。
dec_imm_i	输入	REG_DATA_WIDTH	当前指令立即数，JALR 和 CSR 等操作使用。
commit_id_i	输入	COMMIT_ID_WIDTH	当前执行指令的 commit id。
stage3_issue_ticket_val_id_i	输入	1	三级配置当前执行指令已取得有效写回标签。
alu_wb_ready_i	输入	1	WBU 可以接收 ALU 结果。
mul_wb_ready_i	输入	1	WBU 可以接收 MUL 结果。
div_wb_ready_i	输入	1	WBU 可以接收 DIV 结果。
csr_wb_ready_i	输入	1	WBU 可以接收 CSR 执行结果。

端口名	方向	位宽	有效条件与功能
is_pred_branch_i	输入	1	当前条件分支已经形成预测跳转。
is_pred_jalr_i	输入	1	当前 JALR 已经形成预测跳转。
pred_target_i	输入	INST_ADDR_WIDTH	当前指令的预测目标地址。
pred_taken_i	输入	1	当前指令的预测方向。
pred_ghr_i	输入	BPU_GHR_BITS	增强 BPU 预测时保存的全局历史值。
inst_i	输入	INST_DATA_WIDTH	当前执行指令编码。
reg1_rdata_i	输入	REG_DATA_WIDTH	当前执行指令的 rs1 数据，主要供 BRU 使用。
hazard_stall_i	输入	1	HDU 阻止当前指令推进。
req_alu_i	输入	1	当前指令请求 ALU。
alu_op1_i	输入	REG_DATA_WIDTH	ALU 第一操作数。
alu_op2_i	输入	REG_DATA_WIDTH	ALU 第二操作数。
alu_op_info_i	输入	ALU_OP_WIDTH	ALU 子操作控制字段。
alu_result_sel_i	输入	ALU_RESULT_SEL_WIDTH	ALU 写回结果选择。
bypass_src_op1_i	输入	BYPASS_SRC_WIDTH	第一操作数局部旁路来源编码。
bypass_src_op2_i	输入	BYPASS_SRC_WIDTH	第二操作数局部旁路来源编码。
bypass_bank_op1_i	输入	1	第一操作数选择 ALU 暂存区 0 或 1。
bypass_bank_op2_i	输入	1	第二操作数选择 ALU 暂存区 0 或 1。
alu_result_bank_i	输入	1	当前 ALU 结果写入的暂存区编号。
req_bjp_i	输入	1	当前指令请求 BRU。
bjp_op_jal_i	输入	1	当前 BRU 请求为 JAL。
bjp_op_beq_i	输入	1	当前 BRU 请求为 BEQ。
bjp_op_bne_i	输入	1	当前 BRU 请求为 BNE。

端口名	方向	位宽	有效条件与功能
bjp_op_blt_i	输入	1	当前 BRU 请求为 BLT。
bjp_op_bltu_i	输入	1	当前 BRU 请求为 BLTU。
bjp_op_bge_i	输入	1	当前 BRU 请求为 BGE。
bjp_op_bgeu_i	输入	1	当前 BRU 请求为 BGEU。
bjp_op_jalr_i	输入	1	当前 BRU 请求为 JALR。
bjp_adder_result_i	输入	INST_ADDR_WIDTH	Dispatch 已计算的 JAL 或条件分支目标。
mul_op1_i	输入	REG_DATA_WIDTH	MUL 第一操作数。
mul_op2_i	输入	REG_DATA_WIDTH	MUL 第二操作数。
mul_op_mul_i	输入	1	选择 MUL。
mul_op_mulh_i	输入	1	选择 MULH。
mul_op_mulhsu_i	输入	1	选择 MULHSU。
mul_op_mulhu_i	输入	1	选择 MULHU。
mul_op_clmul_i	输入	1	选择 CLMUL。
mul_op_clmulh_i	输入	1	选择 CLMULH。
mul_op_clmulr_i	输入	1	选择 CLMULR。
req_mul_i	输入	1	当前指令请求 MUL。
mul_commit_id_i	输入	COMMIT_ID_WIDTH	MUL 请求对应的 commit id。
div_op1_i	输入	REG_DATA_WIDTH	DIV 被除数。
div_op2_i	输入	REG_DATA_WIDTH	DIV 除数。
div_op_div_i	输入	1	选择有符号商。
div_op_divu_i	输入	1	选择无符号商。
div_op_rem_i	输入	1	选择有符号余数。
div_op_remu_i	输入	1	选择无符号余数。
div_op_word_i	输入	1	RV64 配置下选择 32 位字操作。
req_div_i	输入	1	当前指令请求 DIV。
div_commit_id_i	输入	COMMIT_ID_WIDTH	DIV 请求对应的 commit id。
req_csr_i	输入	1	当前指令请求 CSR 执行单元。

端口名	方向	位宽	有效条件与功能
csr_opl_i	输入	REG_DATA_WIDTH	CSR 源操作数或零扩展立即数。
csr_csrrw_i	输入	1	选择 CSRRW 类操作。
csr_csrrs_i	输入	1	选择 CSRRS 类操作。
csr_csrrc_i	输入	1	选择 CSRRC 类操作。
req_mem_i	输入	1	当前指令请求 LSU。
mem_commit_id_i	输入	COMMIT_ID_WIDTH	LSU 请求对应的 commit id。
mem_op_lb_i	输入	1	选择有符号字节加载。
mem_op_lh_i	输入	1	选择有符号半字加载。
mem_op_lw_i	输入	1	选择有符号字加载。
mem_op_lbu_i	输入	1	选择无符号字节加载。
mem_op_lhu_i	输入	1	选择无符号半字加载。
mem_op_lwu_i	输入	1	RV64 配置下选择无符号字加载。
mem_op_ld_i	输入	1	RV64 配置下选择双字加载。
mem_op_load_i	输入	1	当前访存请求为加载。
mem_op_store_i	输入	1	当前访存请求为存储。
mem_op_sb_i	输入	1	选择字节存储。
mem_op_sh_i	输入	1	选择半字存储。
mem_op_sw_i	输入	1	选择字存储。
mem_op_sd_i	输入	1	RV64 配置下选择双字存储。
mem_op_amo_i	输入	1	当前访存请求为 LR、SC 或 AMO。
mem_amo_op_i	输入	AMO_OP_WIDTH	原子操作类别。
mem_op_amo_d_i	输入	1	RV64 配置下选择双字原子操作。
mem_addr_i	输入	BUS_ADDR_WIDTH	Dispatch 已计算的访存地址。

端口名	方向	位宽	有效条件与功能
mem_wdata_i	输入	REG_DATA_WIDTH	存储或原子操作写数据。
mem_wmask_i	输入	BUS_DATA_WIDTH/ 8	与地址低位对齐的逐字节写使能。
mem_misaligned_load_i	输入	1	当前加载存在不能由 LSU 处理的地址不对齐。
mem_misaligned_store_i	输入	1	当前存储存在不能由 LSU 处理的地址不对齐。
lsu_hot_cache_hit_i	输入	1	Dispatch 已确认 LSU L0 Cache 命中。
lsu_hot_cache_way_i	输入	1	LSU L0 Cache 命中的路。
sys_op_nop_i	输入	1	当前系统类操作为空操作。
sys_op_mret_i	输入	1	当前指令为 MRET。
sys_op_ecall_i	输入	1	当前指令为 ECALL。
sys_op_ebreak_i	输入	1	当前指令为 EBREAK。
sys_op_fence_i	输入	1	当前指令为 FENCE 或 FENCE.I。
sys_op_dret_i	输入	1	当前指令为调试返回操作。
mem_stall_o	输出	1	LSU 当前不能接受或继续处理发射请求。
alu_reg_wdata_o	输出	REG_DATA_WIDTH	ALU 写回数据。
alu_reg_we_o	输出	1	ALU 写回结果有效。
alu_reg_waddr_o	输出	REG_ADDR_WIDTH	ALU 结果目的寄存器。
alu_commit_id_o	输出	COMMIT_ID_WIDTH	ALU 结果对应的 commit id。
alu_ticket_valid_o	输出	1	三级 ALU 结果具有有效写回标签。
alu_agu_forward_valid_o	输出	1	ALU 结果可供 Dispatch 保存为访存基址。
alu_agu_forward_data_o	输出	BUS_ADDR_WIDTH	ALU 提供的访存基址数据。

端口名	方向	位宽	有效条件与功能
alu_bypass_valid_o	输出	1	ALU 结果已经写入局部旁路暂存区。
alu_agu_forward_arch_valid_o	输出	1	ALU 访存基址结果仍属于有效指令。
mul_reg_wdata_o	输出	REG_DATA_WIDTH	MUL 写回数据。
mul_reg_we_o	输出	1	MUL 写回结果有效。
mul_reg_waddr_o	输出	REG_ADDR_WIDTH	MUL 结果目的寄存器。
mul_commit_id_o	输出	COMMIT_ID_WIDTH	MUL 结果对应的 commit id。
mul_ticket_valid_o	输出	1	三级 MUL 结果具有有效写回标签。
div_reg_wdata_o	输出	REG_DATA_WIDTH	DIV 写回数据。
div_reg_we_o	输出	1	DIV 写回结果有效。
div_reg_waddr_o	输出	REG_ADDR_WIDTH	DIV 结果目的寄存器。
div_commit_id_o	输出	COMMIT_ID_WIDTH	DIV 结果对应的 commit id。
div_ticket_valid_o	输出	1	三级 DIV 结果具有有效写回标签。
lsu_reg_wdata_o	输出	REG_DATA_WIDTH	LSU 加载或原子操作写回数据。
lsu_reg_we_o	输出	1	LSU 本拍存在有效 GPR 写回结果。
lsu_bypass_hot_valid_o	输出	1	LSU L0 Cache 命中结果本拍有效。
lsu_bypass_ticket_valid_o	输出	1	当前 LSU 完成结果具有有效写回标签。
lsu_pending_reg_waddr_o	输出	REG_ADDR_WIDTH	普通总线返回或原子操作结果的目的寄存器。
lsu_pending_commit_id_o	输出	COMMIT_ID_WIDTH	普通总线返回或原子操作结果的 commit id。
lsu_pending_ticket_valid_o	输出	1	普通总线返回或原子操作结果具有有效写回标签。
lsu_hot_reg_waddr_o	输出	REG_ADDR_WIDTH	LSU L0 Cache 命中结果的目的寄存器。

端口名	方向	位宽	有效条件与功能
lsu_hot_commit_id_o	输出	COMMIT_ID_WIDTH	LSU L0 Cache 命中结果的 commit id。
lsu_hot_ticket_valid_o	输出	1	LSU L0 Cache 命中结果具有有效写回标签。
lsu_dtcn_bypass_lookahead_valid_o	输出	1	固定一周期 DTCN 配置下预告下一周期读结果可用。
lsu_dtcn_bypass_lookahead_commit_id_o	输出	COMMIT_ID_WIDTH	DTCN 提前完成提示对应的 commit id。
lsu_reg_waddr_o	输出	REG_ADDR_WIDTH	当前 LSU 完成结果最终选择的寄存器。
lsu_commit_id_o	输出	COMMIT_ID_WIDTH	当前 LSU 完成结果最终选择的 commit id。
lsu_bypass_bank_data_o	输出	REG_DATA_WIDTH	LSU 局部旁路暂存区保存的数据。
alu_bypass_bank_o	输出	1	本拍 ALU 结果写入的局部暂存区编号。
mul_bypass_valid_o	输出	1	MUL 结果已经写入局部旁路暂存区。
mul_bypass_commit_id_o	输出	COMMIT_ID_WIDTH	MUL 局部旁路结果对应的 commit id。
csr_reg_wdata_o	输出	REG_DATA_WIDTH	CSR 旧值写入 GPR 时的数据。
csr_reg_waddr_o	输出	REG_ADDR_WIDTH	CSR 指令的 GPR 目的寄存器。
csr_commit_id_o	输出	COMMIT_ID_WIDTH	CSR 指令对应的 commit id。
csr_reg_we_o	输出	1	CSR 旧值需要写入 GPR。
csr_ticket_valid_o	输出	1	三级 CSR 的 GPR 结果具有有效写回标签。
csr_wdata_o	输出	REG_DATA_WIDTH	CSR 新值。
csr_we_o	输出	1	当前 CSR 新值有效。
csr_waddr_o	输出	BUS_ADDR_WIDTH	CSR 写地址。

端口名	方向	位宽	有效条件与功能
stall_flag_o	输出	1	任一执行单元不能继续接收当前请求。
jump_flag_o	输出	1	BRU 地址修正或 CLINT 跳转事件有效。
flush_flag_o	输出	1	必须清除错误路径或陷入前的流水状态。
bru_jump_flag_o	输出	1	BRU 已确认实际控制转移地址有效。
bru_jump_addr_o	输出	INST_ADDR_WIDTH	BRU 已确认的实际下一 PC。
bru_ifu_generic_redirect_o	输出	1	IFU 应采用 BRU 通用地址修正。
bru_cond_taken_redirect_o	输出	1	定长指令增强 BPU 配置下，条件分支由不跳转改为跳转。
bru_cond_rollback_redirect_o	输出	1	定长指令增强 BPU 配置下，条件分支由跳转改为不跳转。
jump_sequential_addr_o	输出	INST_ADDR_WIDTH	当前控制转移指令的顺序下一 PC。
bp_update_valid_o	输出	1	基础 BPU 条件分支训练信息有效。
bp_update_pc_o	输出	INST_ADDR_WIDTH	基础 BPU 训练对应的分支 PC。
bp_update_taken_o	输出	1	基础 BPU 训练对应的实际方向。
bp_update_static_taken_o	输出	1	基础 BPU 静态方向结果。
bp_update_backward_o	输出	1	基础 BPU 训练对应的后向分支属性。
jalr_update_valid_o	输出	1	基础 BPU 的 JALR 目标训练有效。
jalr_update_pc_o	输出	INST_ADDR_WIDTH	JALR 训练对应的指令 PC。
jalr_update_target_o	输出	INST_ADDR_WIDTH	JALR 的实际目标地址。
bp_enh_update_valid_o	输出	1	增强 BPU 训练信息有效。

端口名	方向	位宽	有效条件与功能
bp_enh_update_pc_o	输出	INST_ADDR_WIDTH	增强 BPU 训练对应的指令 PC。
bp_enh_update_target_o	输出	INST_ADDR_WIDTH	增强 BPU 训练对应的实际目标。
bp_enh_update_next_o	输出	INST_ADDR_WIDTH	增强 BPU 训练对应的实际下一 PC。
bp_enh_update_kind_o	输出	BP_KIND_WIDTH	增强 BPU 训练对应的控制转移类别。
bp_enh_update_taken_o	输出	1	增强 BPU 训练对应的实际方向。
bp_enh_update_ghr_o	输出	BPU_GHR_BITS	增强 BPU 训练对应的预测历史值。
mem_store_busy_o	输出	1	LSU 存在尚未完成的存储、拆分访问或原子操作。
lsu_hot_cache_ready_o	输出	1	LSU L0 Cache 可以接受新的命中访问。
lsu_hot_cache_meta_valid_o	输出	1	LSU L0 Cache 标签或有效位修改信息有效。
lsu_hot_cache_meta_addr_o	输出	BUS_ADDR_WIDTH	LSU L0 Cache 修改信息对应的地址。
lsu_hot_cache_meta_way_o	输出	1	LSU L0 Cache 修改信息对应的路。
lsu_hot_cache_meta_clear_o	输出	1	清除指定 LSU L0 Cache 项。
lsu_hot_cache_meta_live_store_valid_o	输出	1	当前正在发送的存储自然字地址有效。
lsu_hot_cache_meta_live_store_beat_o	输出	DTCM_ADDR_WIDTH - log2(BUS_DATA_WIDTH/8)	当前正在发送的存储自然字地址。
lsu_hot_cache_meta_queued_store_valid_o	输出	1	LSU 请求 FIFO 队首存储自然字地址有效。
lsu_hot_cache_meta_queued_store_beat_o	输出	DTCM_ADDR_WIDTH - log2(BUS_DATA_WIDTH/8)	LSU 请求 FIFO 队首存储自然字地址。

端口名	方向	位宽	有效条件与功能
lsu_hot_cache_meta_split_store_valid_o	输出	1	非对齐拆分的第二次存储自然字地址有效。
lsu_hot_cache_meta_spllit_store_beat_o	输出	DTCM_ADDR_WIDTH H- $\log_2(\text{BUS_DATA_WIDTH}/8)$	非对齐拆分第二次存储的自然字地址。
lsu_hot_cache_meta_fill_valid_o	输出	1	LSU L0 Cache 本拍写入的自然字地址有效。
lsu_hot_cache_meta_fill_beat_o	输出	DTCM_ADDR_WIDTH H- $\log_2(\text{BUS_DATA_WIDTH}/8)$	LSU L0 Cache 本拍写入的自然字地址。
exu_op_ecall_o	输出	1	当前执行指令触发 ECALL 异常。
exu_op_ebreak_o	输出	1	当前执行指令触发 EBREAK 异常。
exu_op_mret_o	输出	1	当前执行指令请求 MRET。
misaligned_load_o	输出	1	当前加载地址不对齐异常有效。
misaligned_store_o	输出	1	当前存储地址不对齐异常有效。
misaligned_fetch_o	输出	1	BRU 实际目标地址不满足取指对齐规则。
M_AXI_AWADDR	输出	BUS_ADDR_WIDTH	LSU AXI4-Lite 写地址，VALID 有效后保持到 READY。
M_AXI_AWPROT	输出	3	LSU AXI4-Lite 写访问属性。
M_AXI_AWVALID	输出	1	LSU AXI4-Lite 写地址有效。
M_AXI_AWREADY	输入	1	下游可以接收写地址。
M_AXI_WDATA	输出	BUS_DATA_WIDTH	LSU AXI4-Lite 写数据。
M_AXI_WSTRB	输出	$\text{BUS_DATA_WIDTH}/8$	LSU AXI4-Lite 逐字节写使能。

端口名	方向	位宽	有效条件与功能
M_AXI_WVALID	输出	1	LSU AXI4-Lite 写数据有效。
M_AXI_WREADY	输入	1	下游可以接收写数据。
M_AXI_BRESP	输入	2	下游写响应状态；LSU 当前只使用握手完成信息。
M_AXI_BVALID	输入	1	下游写响应有效。
M_AXI_BREADY	输出	1	LSU 可以接收写响应。
M_AXI_ARADDR	输出	BUS_ADDR_WIDTH	LSU AXI4-Lite 读地址，VALID 有效后保持到 READY。
M_AXI_ARPROT	输出	3	LSU AXI4-Lite 读访问属性。
M_AXI_ARVALID	输出	1	LSU AXI4-Lite 读地址有效。
M_AXI_ARREADY	输入	1	下游可以接收读地址。
M_AXI_RDATA	输入	BUS_DATA_WIDTH	下游读数据。
M_AXI_RRESP	输入	2	下游读响应状态；LSU 当前只使用握手完成信息。
M_AXI_RVALID	输入	1	下游读数据有效。
M_AXI_RREADY	输出	1	LSU 可以接收读数据。

表 36 EXU 输入输出端口

5.4.1 EXU 局部旁路暂存区

四级和五级配置中的 `exu_bypass_bank` 接收 ALU、LSU 和 MUL 的结果以及 Dispatch 传来的 `bypass_src_op1/op2`、`bypass_bank_op1/op2`。两项 ALU 结果寄存器覆盖相邻 ALU RAW 和 WBU 接收前的短窗口，LSU 与 MUL 各使用一项数据寄存器。数据由本模块保存，有效状态及其与 `commit id` 的对应关系由 HDU 中按 `commit id` 展开的独热状态维护。三级配置不例化 `exu_bypass_bank`，全部局部旁路选择固定为零。

旁路数据路径如下：

- ◆ HDU 判断 RAW 依赖已就绪。

- ◆ `dispatch_pipe` 携带旁路来源和暂存区编号。
- ◆ `exu_bypass_bank` 选择 ALU/LSU/MUL 暂存区数据。
- ◆ ALU/BRU/MUL/存储数据本地操作数选择器使用对应旁路数据。

这一路径要求旁路可用状态只能在对应结果已经写入暂存区后置位，不能用 LSU 地址握手或尚未稳定的存储器数据提前解除依赖。若 WBU 未准备好接收结果，EXU 暂停保持暂存数据和上游流水状态，使已经允许选择的数据继续稳定。

5.4.2 ALU

ALU 是组合运算单元，执行整数算术、逻辑、移位、比较、LUI/AUIPC/JAL 链接地址以及 Zba/Zbb/Zbs 中已经打开的标量位操作。输入操作数来自发射流水寄存器；四级和五级配置下，旁路选择有效时在 EXU 内替换为局部暂存数据。ALU 输出写回数据、写使能、目的寄存器地址、commit id 和比较结果，局部 ALU 暂存区的写入编号由 HDU 生成并经 Dispatch 送入 EXU。需要写回而 WBU 未准备好时，ALU 拉高暂停信号。三级紧凑配置将 ALU 输出的目的寄存器地址固定为 x0，WBU 按 commit id 向 HDU 查询实际目的寄存器。

参数	默认值	有效取值	功能说明
ENABLE_RV64	0	0、1	选择 RV32 或 RV64 数据宽度相关操作，并控制 RV64 字操作逻辑。
ENABLE_ZBA	0	0、1	生成 Zba 移位加法和相关零扩展操作。
ENABLE_ZBB	0	0、1	生成 Zbb 逻辑取反、计数、最值、符号扩展、字节处理和旋转操作。
ENABLE_ZBS	0	0、1	生成 Zbs 单比特设置、提取、清除和取反操作。
COMPACT_STAGE3_WADDR	0	0、1	三级配置取 1，ALU 不随结果传递目的寄存器地址，由 WBU 按 commit id 查询。

表 37 ALU 参数

端口名	方向	位宽	有效条件与功能
rst_n	输入	1	低电平有效复位输入；ALU 数据路径不保存时序状态。
req_alu_i	输入	1	当前 ALU 请求有效。
hazard_stall_i	输入	1	HDU 暂停输入；当前由 EXU 上游保持请求，ALU 内部不使用该信号改变结果。
alu_op1_i	输入	REG_DATA_WIDTH	第一原始操作数。
alu_op2_i	输入	REG_DATA_WIDTH	第二原始操作数。
alu_op_info_i	输入	ALU_OP_WIDTH	ALU 操作类型独热控制。
alu_result_sel_i	输入	ALU_RESULT_SEL_WIDTH	ALU 结果类别选择码。
alu_rd_i	输入	REG_ADDR_WIDTH	目的寄存器地址。
commit_id_i	输入	COMMIT_ID_WIDTH	当前指令的 commit id。
alu_pass_op1_i	输入	1	第一操作数选择旁路数据。
alu_pass_op2_i	输入	1	第二操作数选择旁路数据。
alu_result_bypass_op1_i	输入	REG_DATA_WIDTH	第一操作数的旁路数据。
alu_result_bypass_op2_i	输入	REG_DATA_WIDTH	第二操作数的旁路数据。
wb_ready_i	输入	1	WBU 可以接收 ALU 结果。
reg_we_i	输入	1	当前 ALU 指令需要写 GPR。
arch_result_ticket_valid_i	输入	1	当前指令已经取得有效 commit id，并具有架构结果写入资格。
int_assert_i	输入	1	中断或异常取消当前 ALU 结果。
misaligned_fetch_i	输入	1	JAL 或 JALR 目标地址不对齐，禁止链接地址写入 GPR。

端口名	方向	位宽	有效条件与功能
alu_stall_o	输出	1	ALU 存在有效写回请求而 WBU 不能接收。
result_o	输出	REG_DATA_WIDTH	按结果选择码生成的 ALU 写回数据。
agu_forward_result_o	输出	REG_DATA_WIDTH	可供 Dispatch 地址计算保存的 ALU 结果。
agu_forward_valid_o	输出	1	当前操作属于允许地址前递的 ALU 操作。
bypass_valid_o	输出	1	ALU 结果具有有效 commit id，可以进入 EXU 局部旁路状态。
agu_forward_arch_valid_o	输出	1	地址前递结果同时具有有效 commit id。
compare_eq_o	输出	1	两个实际操作数相等。
compare_lt_signed_o	输出	1	第一实际操作数按有符号数比较小于第二实际操作数。
compare_lt_unsigned_o	输出	1	第一实际操作数按无符号数比较小于第二实际操作数。
reg_we_o	输出	1	ALU 结果写回请求有效。
reg_waddr_o	输出	REG_ADDR_WIDTH	ALU 结果的目的寄存器地址；三级紧凑配置固定为 x0。
commit_id_o	输出	COMMIT_ID_WIDTH	ALU 结果的 commit id。

表 38 ALU 输入输出端口

ALU 先根据旁路选择确定两个实际操作数，所有后续算术、移位、逻辑和比较操作均使用选择后的数据。加法、减法、AUIPC、JAL 链接地址和 Zba 移位加法复用主加法器；RV32 的有符号与无符号比较复用减法结果。可变移位和旋转采用分段选择结构，RV32 处理 5 位移位量，RV64 处理 6 位移位量。前导零计数、尾随零计数和置位计数采用平衡分组结构，避免形成逐位串行优先选择。

结果选择先区分加法、比较、位计数和其它操作，再在各类别内部选择具体结果。RV32 且未启用 Zba、Zbb 和 Zbs 时，ALU 只生成基础整数指令需要的精简结果选择结构。地址前递仅对 ADD、SUB、LUI、AUIPC 和 Zba 移位加法有效；逻辑、普通移位、比较和位计数结果不进入 AGU 保存项。

ALU 暂停只由有效写回请求和 wb_ready_i 共同产生，hazard_stall_i 当前不参与 ALU 内部逻辑。中断抑制 GPR 写回、局部旁路和地址前递的有效状态；JAL 或 JALR 目标地址不对齐时，链接地址不写入 GPR。三级配置将 reg_waddr_o 固定为 x0，WBU 根据 commit id 查询实际目的寄存器；四级和五级配置直接随结果传递目的寄存器地址。

条件分支比较结构随流水线级数变化。三级和四级配置复用 ALU 产生的相等、有符号小于和无符号小于结果；五级配置在旁路暂存区附近设置独立比较器，以缩短旁路数据到分支决策的路径。该选择在较短流水线的面积与五级配置的控制转移时序之间取得平衡。

5.4.3 BRU

BRU 处理条件分支、JAL、JALR、FENCE 相关控制流以及 BPU 训练信息。四级和五级配置的条件分支操作数可以选择 EXU 局部 ALU 或 LSU 结果，不能直接选择 MUL 结果；三级配置必须等待寄存器值写回。JAL 使用 Dispatch 计算的 $PC + imm$ ，JALR 在 BRU 内计算 $rs1 + I$ 型立即数并清零最低位。BRU 得到实际方向和目标后，与 IFU 下传的预测状态比较。

参数	默认值	有效取值	功能说明
ENABLE_C	0	0、1	选择 C 扩展对应的指令长度和目标地址对齐检查。
ENABLE_BPU_LEGACY	1	0、1	生成基础 BPU 的预测检查和训练输出。
ENABLE_BPU_ENHANCED	0	0、1	生成增强 BPU 的预测检查、控制转移类别识别和训练输出。
BPU_GHR_BITS	2	正整数	指定全局历史字段宽度。
PIPELINE_STAGES	5	3、4、5	保留流水线级数配置接口；BRU 内部组合功能不直接依赖该参数，级数差异由 EXU 比较结构处理。

表 39 BRU 参数

端口名	方向	位宽	有效条件与功能
rst_n	输入	1	低电平有效复位输入；BRU 数据路径不保存时序状态。
req_bjp_i	输入	1	当前控制转移请求有效。
bjp_op_jal_i	输入	1	当前指令为 JAL。
bjp_op_beq_i	输入	1	当前指令为 BEQ。
bjp_op_bne_i	输入	1	当前指令为 BNE。
bjp_op_blt_i	输入	1	当前指令为 BLT。
bjp_op_bltu_i	输入	1	当前指令为 BLTU。
bjp_op_bge_i	输入	1	当前指令为 BGE。
bjp_op_bgeu_i	输入	1	当前指令为 BGEU。
bjp_op_jalr_i	输入	1	当前指令为 JALR。
pred_taken_i	输入	1	BPU 给出的预测跳转方向。
is_pred_branch_i	输入	1	当前指令带有条件分支预测状态。
is_pred_jalr_i	输入	1	当前指令带有 JALR 目标预测状态。
pred_target_i	输入	INST_ADDR_WIDTH	BPU 给出的预测目标地址。
pred_ghr_i	输入	BPU_GHR_BITS	查询当前指令时保存的全局历史值。
bjp_adder_result_i	输入	INST_ADDR_WIDTH	Dispatch 预先计算的 JAL 或条件分支目标地址。
inst_i	输入	INST_DATA_WIDTH	原始指令，用于识别调用、返回和间接调用。
inst_len_2_i	输入	1	当前指令长度为 2 字节。
dec_imm_i	输入	REG_DATA_WIDTH	分支或 JALR 立即数。
bp_update_pc_i	输入	INST_ADDR_WIDTH	当前控制转移指令的 PC。
reg1_rdata_i	输入	REG_DATA_WIDTH	JALR 基址寄存器数据。
bjp_pass_op1_i	输入	1	JALR 基址选择旁路数据。

端口名	方向	位宽	有效条件与功能
bjp_bypass_data_op1_i	输入	REG_DATA_WIDTH	JALR 基址旁路数据。
cond_branch_taken_i	输入	1	条件分支实际方向。
sys_op_fence_i	输入	1	当前指令要求 FENCE 取指地址刷新。
int_assert_i	输入	1	中断或异常取消当前 BRU 控制及训练输出。
jump_flag_o	输出	1	需要地址修正且目标地址对齐。
flush_flag_o	输出	1	必须清除错误路径指令；目标未对齐时仍可有效。
jump_addr_o	输出	INST_ADDR_WIDTH	通用地址修正采用的目标地址或顺序地址。
jump_sequential_addr_o	输出	INST_ADDR_WIDTH	当前指令之后的顺序取指地址。
ifu_generic_redirect_o	输出	1	IFU 采用 jump_addr_o 执行通用地址修正。
cond_taken_redirect_o	输出	1	增强 BPU 且关闭 C 扩展时，条件分支预测不跳转而实际跳转。
cond_rollback_redirect_o	输出	1	增强 BPU 且关闭 C 扩展时，条件分支预测跳转而实际不跳转。
bp_update_valid_o	输出	1	基础 BPU 条件分支训练信息有效。
bp_update_pc_o	输出	INST_ADDR_WIDTH	条件分支指令 PC。
bp_update_taken_o	输出	1	条件分支实际方向。
bp_update_static_taken_o	输出	1	静态预测方向。
bp_update_backward_o	输出	1	条件分支立即数为负，表示后向分支。

端口名	方向	位宽	有效条件与功能
bp_enh_update_valid_o	输出	1	增强 BPU 训练或错误表项清除信息有效。
bp_enh_update_pc_o	输出	INST_ADDR_WIDTH	被训练指令的 PC。
bp_enh_update_target_o	输出	INST_ADDR_WIDTH	实际控制转移目标地址。
bp_enh_update_next_o	输出	INST_ADDR_WIDTH	当前指令的顺序下一地址。
bp_enh_update_kind_o	输出	BP_KIND_WIDTH	条件分支、JAL、调用、返回、间接调用或普通间接跳转类别。
bp_enh_update_taken_o	输出	1	控制转移实际方向。
bp_enh_update_ghr_o	输出	BPU_GHR_BITS	查询时保存的全局历史值。
jalr_update_valid_o	输出	1	JALR 目标训练信息有效。
jalr_update_pc_o	输出	INST_ADDR_WIDTH	JALR 指令 PC。
jalr_update_target_o	输出	INST_ADDR_WIDTH	JALR 实际目标地址。
misaligned_fetch_o	输出	1	实际采用的 JAL、JALR 或条件分支目标地址不对齐。
link_misaligned_o	输出	1	JAL 或 JALR 目标地址不对齐，用于禁止链接地址写回。

表 40 BRU 输入输出端口

BRU 的预测错误判断可以概括为：

```

actual_taken = condition_true || jal || jalr
actual_target = jalr ? ((rs1_plus_imm) & ~1) : bjp_adder_result
taken_miss = predicted_taken != actual_taken
target_miss = predicted_taken && actual_taken && (pred_target != actual_target)

```

当预测方向或目标地址与实际结果不一致时，BRU 产生清除与地址修正控制。flush_flag_o 表示必须清除错误路径指令，jump_flag_o 表示目标地址合法并允许 IFU 改写 PC；目标未对齐时仍可清除错误路径并报告取指地址不对齐异常，但不采用该目标地址。开启 C 扩展时检查目标地址 bit0，关闭 C 扩展时还要检查 bit1。增强 BPU 且仅支持定长 32 位指令时，条件分支方向错误

使用专用方向恢复输出；目标错误、JAL、JALR 和 FENCE 使用通用地址修正输出。基础与增强 BPU 的训练信息均在 EXU 中寄存一拍后输出。

未启用 BPU 时，BRU 对所有实际发生的条件分支、JAL、JALR 和 FENCE 产生地址修正。启用基础 BPU 且关闭 C 扩展时，BRU 检查条件分支方向、JALR 目标和 FENCE，不单独比较条件分支预测目标；启用基础 BPU 和 C 扩展时，已预测跳转且实际跳转的条件分支还要检查目标地址。启用增强 BPU 且关闭 C 扩展时，条件分支方向错误使用两个专用控制信号，条件分支目标错误以及非条件控制转移使用通用地址修正信号；启用增强 BPU 和 C 扩展时，各类预测错误统一使用通用地址修正信号。

中断地址通过 IFU 的独立输入提供，EXU 不在 BRU 目标与中断目标之间选择地址；但 EXU 顶层会把 CLINT 的中断跳转有效并入对外的 `jump_flag_o` 和 `flush_flag_o`，使总控制能够清除错误路径并停止旧批次取指。

5.4.4 LSU

LSU 是 AXI4-Lite 数据主接口模块，负责加载、存储、可选非对齐拆分访问和可选 A 扩展原子操作。Dispatch 已经提供 `mem_addr_i`、`mem_wmask_i`、存储数据、加载或存储类型、`commit id` 和异常标志，LSU 不再计算普通访问地址。三级紧凑配置仍保留完整队列和并行事务处理，仅不重复保存 `rd`，加载完成后由 WBU 按 `commit id` 查询目的寄存器。

LSU 的结构参数如下表所示。AXI4-Lite 不使用 ID 和 USER 字段，因此相应兼容参数不改变对外端口；其余参数直接决定数据宽度、指令功能、队列数据字段和局部 Cache 结构。

参数	默认值	有效取值	功能
C_M_AXI_ID_WIDTH	2	正整数	AXI 兼容参数；AXI4-Lite 端口不包含 ID 字段。
C_M_AXI_ADDR_WIDTH	32	与系统地址宽度一致	决定 LSU 和 M1 AXI4-Lite 地址宽度。
C_M_AXI_DATA_WIDTH	32	32 或 64	决定总线数据宽度、地址低位宽度和写使能位数。
C_M_AXI_AWUSER_WIDTH	1	正整数	AXI 兼容参数；当前写地址端口不包含 USER 字段。

参数	默认值	有效取值	功能
C_M_AXI_ARUSER_WIDTH	1	正整数	AXI 兼容参数；当前读地址端口不包含 USER 字段。
C_M_AXI_WUSER_WIDTH	1	正整数	AXI 兼容参数；当前写数据端口不包含 USER 字段。
C_M_AXI_RUSER_WIDTH	1	正整数	AXI 兼容参数；当前读数据端口不包含 USER 字段。
C_M_AXI_BUSER_WIDTH	1	正整数	AXI 兼容参数；当前写响应端口不包含 USER 字段。
ENABLE_RV64	0	0、1	生成 64 位加载、存储以及 RV64 原子操作数据选择。
ENABLE_A	0	0、1	生成 LR、SC 和 AMO 六状态控制器。
ENABLE_MISALIGNED_DATA	0	0、1	在 RV32 中生成跨自然字加载和存储的两次访问控制。
ENABLE_LSU_HOT_CACHE	0	0、1	生成 LSU L0 Cache 数据体和相关状态修改接口。
LSU_HOT_CACHE_DEPTH	2	1、2、4、8、16、32、64、128	设置 LSU L0 Cache 总项数。
LSU_HOT_CACHE_WAYS	1	1、2	设置 LSU L0 Cache 路数；深度为 1 时固定为一路。
COMPACT_STAGE3_WADDR	0	0、1	三级配置不在 LSU 队列中重复保存 rd，由 HDU 按 commit id 提供写回地址。
PIPELINE_STAGES	5	3、4、5	选择与核心流水线一致的完成有效控制。
ENABLE_BPU_ENHANCED	0	0、1	选择增强 BPU 配置下的中断清除时序条件。

参数	默认值	有效取值	功能
ENABLE_DTCM_LO AD_LOOKAHEAD	0	0、1	仅在无 DCache、无 M1 乒乓缓存且 DTCM 读延迟固定时，允许 AR 握手预告下一周期完成。

表 41 LSU 参数

LSU 的端口定义如下表所示。访存请求输入只在 req_mem_i 有效且未发生中断清除时接受；AXI4-Lite 各通道按照 VALID/READY 独立握手，暂停期间地址、数据和控制字段保持稳定。

端口名	方向	位宽	有效条件与功能
clk	输入	1	LSU 队列、事务跟踪、非对齐控制、原子状态机和 LSU L0 Cache 的系统时钟。
rst_n	输入	1	低电平有效异步复位，清除有效状态、指针、计数和原子保留状态。
req_mem_i	输入	1	当前执行请求为访存操作。
mem_op_lb_i	输入	1	选择有符号字节加载。
mem_op_lh_i	输入	1	选择有符号半字加载。
mem_op_lw_i	输入	1	选择有符号字加载。
mem_op_lbu_i	输入	1	选择无符号字节加载。
mem_op_lhu_i	输入	1	选择无符号半字加载。
mem_op_lwu_i	输入	1	RV64 配置下选择无符号字加载。
mem_op_ld_i	输入	1	RV64 配置下选择双字加载。
mem_op_load_i	输入	1	当前请求为加载。
mem_op_store_i	输入	1	当前请求为存储。
mem_op_sb_i	输入	1	选择字节存储。
mem_op_sh_i	输入	1	选择半字存储。
mem_op_sw_i	输入	1	选择字存储。

端口名	方向	位宽	有效条件与功能
mem_op_sd_i	输入	1	RV64 配置下选择双字存储。
mem_op_amo_i	输入	1	当前请求为 LR、SC 或 AMO。
mem_amo_op_i	输入	AMO_OP_WIDTH	原子操作类别。
mem_op_amo_d_i	输入	1	RV64 配置下选择双字原子操作。
rd_addr_i	输入	REG_ADDR_WIDTH	加载或原子操作的目的寄存器地址；三级紧凑配置不保存该值。
mem_addr_i	输入	C_M_AXI_ADDR_WIDTH	Dispatch 已计算的有效访存地址。
mem_wdata_i	输入	REG_DATA_WIDTH	存储、SC 或 AMO 的写数据。
mem_wmask_i	输入	C_M_AXI_DATA_WIDTH/8	与地址低位对齐的逐字节写使能。
commit_id_i	输入	COMMIT_ID_WIDTH	当前访存请求的 commit id。
stage3_ticket_valid_i	输入	1	三级配置当前访存请求具有有效写回标签。
int_assert_i	输入	1	中断或异常清除请求；阻止新请求，并在原子空闲时清除 LR 保留。
mem_stall_o	输出	1	输入 FIFO 无空间、原子操作占用或当前请求不能前进时置位。
mem_busy_o	输出	1	存在未完成读写、拆分访问、待写回结果或原子操作。
lsu_hot_cache_hit_i	输入	1	Dispatch 已确认 LSU L0 Cache 命中。
lsu_hot_cache_way_i	输入	1	LSU L0 Cache 命中的路。

端口名	方向	位宽	有效条件与功能
lsu_hot_cache_ready_o	输出	1	当前不存在会使本地命中结果失效的读写、拆分或原子状态。
lsu_hot_cache_meta_valid_o	输出	1	LSU L0 Cache 标签或有效位修改信息有效。
lsu_hot_cache_meta_addr_o	输出	C_M_AXI_ADDR_WIDTH	LSU L0 Cache 修改信息对应的地址。
lsu_hot_cache_meta_way_o	输出	1	LSU L0 Cache 修改信息对应的路。
lsu_hot_cache_meta_clear_o	输出	1	清除指定标签；为低时登记新标签。
lsu_hot_cache_meta_live_store_valid_o	输出	1	当前正在发送的存储自然字地址有效。
lsu_hot_cache_meta_live_store_beat_o	输出	DTCM_ADDR_WIDTH - $\log_2(\text{C_M_AXI_DATA_WIDTH}/8)$	当前正在发送的存储自然字地址。
lsu_hot_cache_meta_queued_store_valid_o	输出	1	输入 FIFO 队首存储自然字地址有效。
lsu_hot_cache_meta_queued_store_beat_o	输出	DTCM_ADDR_WIDTH - $\log_2(\text{C_M_AXI_DATA_WIDTH}/8)$	输入 FIFO 队首存储自然字地址。
lsu_hot_cache_meta_split_store_valid_o	输出	1	非对齐拆分第二次存储自然字地址有效。
lsu_hot_cache_meta_split_store_beat_o	输出	DTCM_ADDR_WIDTH - $\log_2(\text{C_M_AXI_DATA_WIDTH}/8)$	非对齐拆分第二次存储自然字地址。
lsu_hot_cache_meta_fill_valid_o	输出	1	LSU L0 Cache 数据体本拍写入地址有效。
lsu_hot_cache_meta_fill_beat_o	输出	DTCM_ADDR_WIDTH - $\log_2(\text{C_M_AXI_DATA_WIDTH}/8)$	LSU L0 Cache 数据体本拍写入的自然字地址。
reg_wdata_o	输出	REG_DATA_WIDTH	加载、LR、SC 或 AMO 的 GPR 写回数据。

端口名	方向	位宽	有效条件与功能
reg_we_o	输出	1	普通返回、LSU L0 Cache 命中或原子操作结果有效。
reg_hot_we_o	输出	1	本拍写回数据来自 LSU L0 Cache 命中。
reg_pending_waddr_o	输出	REG_ADDR_WIDTH	普通总线返回或原子操作结果的目的寄存器。
pending_commit_id_o	输出	COMMIT_ID_WIDTH	普通总线返回或原子操作结果的 commit id。
pending_ticket_valid_o	输出	1	普通总线返回或原子操作结果具有有效写回标签。
reg_hot_waddr_o	输出	REG_ADDR_WIDTH	LSU L0 Cache 命中结果的目的寄存器。
hot_commit_id_o	输出	COMMIT_ID_WIDTH	LSU L0 Cache 命中结果的 commit id。
hot_ticket_valid_o	输出	1	LSU L0 Cache 命中结果具有有效写回标签。
reg_dtcn_bypass_lookahead_valid_o	输出	1	固定一周期 DTCN 配置下，AR 握手预告下一周期读结果可用。
reg_dtcn_bypass_lookahead_commit_id_o	输出	COMMIT_ID_WIDTH	DTCN 提前完成提示对应的 commit id。
reg_waddr_o	输出	REG_ADDR_WIDTH	本拍最终选择的 LSU 结果目的寄存器。
commit_id_o	输出	COMMIT_ID_WIDTH	本拍最终选择的 LSU 结果 commit id。
M_AXI_AWADDR	输出	C_M_AXI_ADDR_WIDTH	写地址，M_AXI_AWVALID 有效后保持到握手。
M_AXI_AWPROT	输出	3	写访问属性。
M_AXI_AWVALID	输出	1	写地址有效。
M_AXI_AWREADY	输入	1	下游可以接收写地址。
M_AXI_WDATA	输出	C_M_AXI_DATA_WIDTH	与写地址对应的写数据。

端口名	方向	位宽	有效条件与功能
M_AXI_WSTRB	输出	C_M_AXI_DATA_WIDTH/8	与写数据对应的逐字节写使能。
M_AXI_WVALID	输出	1	写数据有效。
M_AXI_WREADY	输入	1	下游可以接收写数据。
M_AXI_BRESP	输入	2	下游写响应状态；当前 LSU 只使用握手完成信息。
M_AXI_BVALID	输入	1	下游写响应有效。
M_AXI_BREADY	输出	1	LSU 可以接收写响应。
M_AXI_ARADDR	输出	C_M_AXI_ADDR_WIDTH	读地址，M_AXI_ARVALID 有效后保持到握手。
M_AXI_ARPROT	输出	3	读访问属性。
M_AXI_ARVALID	输出	1	读地址有效。
M_AXI_ARREADY	输入	1	下游可以接收读地址。
M_AXI_RDATA	输入	C_M_AXI_DATA_WIDTH	下游读数据。
M_AXI_RRESP	输入	2	下游读响应状态；当前 LSU 只使用握手完成信息。
M_AXI_RVALID	输入	1	下游读数据有效。
M_AXI_RREADY	输出	1	LSU 可以接收读数据。

表 42 LSU 输入输出端口

LSU（加载存储单元）访存总控制关系

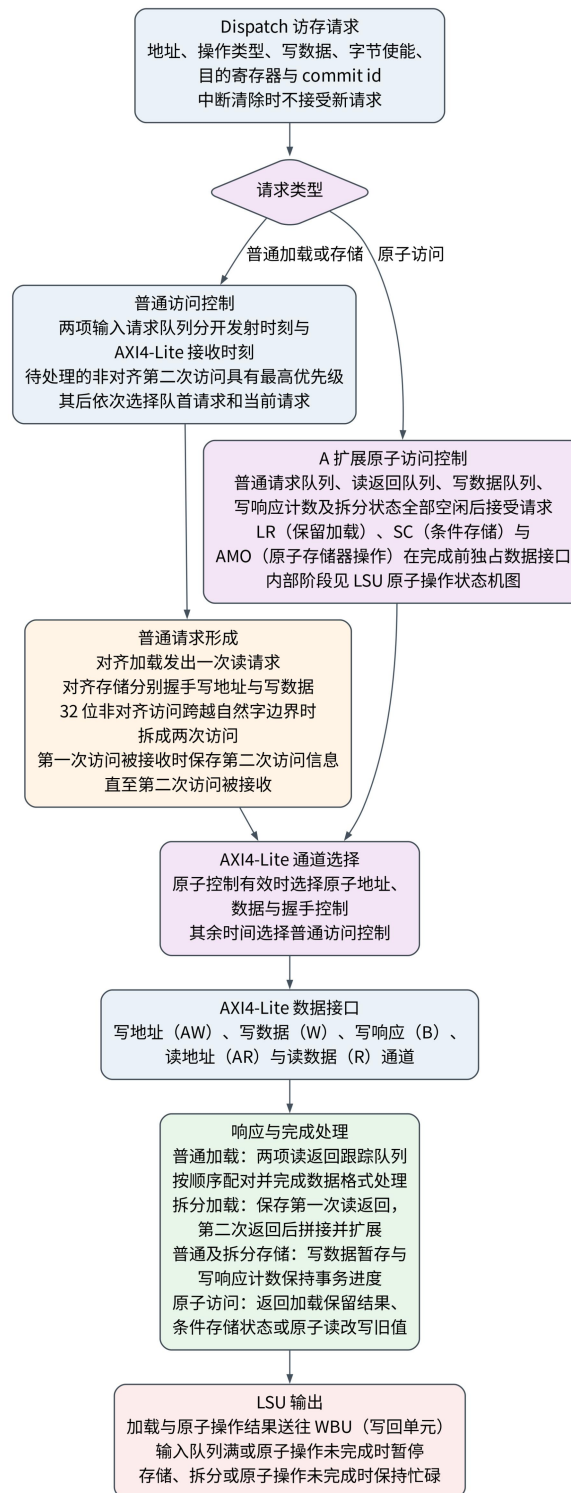


图 13 LSU 访存总控制关系

LSU 内部包含两项请求 FIFO、两项读事务跟踪 FIFO、两项写数据 FIFO，以及最大值为 4 的未返回写响应计数。加载请求经 AR 通道接受后，其 commit id、访问类型和地址低位进入读事务跟踪 FIFO，后续只在 R 通道握手时形成加载结果。存储请求经 AW 通道接受后即可从请求 FIFO 移除；W 数据可以

同拍发送，也可以进入写数据 FIFO，B 响应只负责减少未返回写响应计数。普通读取必须等待写数据 FIFO 为空且全部写响应返回，以维持先写后读的顺序。LSU 不检查 RRESP 和 BRESP，因此总线错误不会由 LSU 转换为处理器异常。

固定一周期 DTCM 读取是普通加载完成规则的特例。仅当系统关闭 DCache 和 M1 乒乓缓存，且 DTCM 从 AR 握手到读数据有效的延迟固定为一个周期时，ENABLE_DTCM_LOAD_LOOKAHEAD 才能置位。LSU 在 AR 握手时登记 commit id，Dispatch 保存该提前完成状态，并在下一周期与 DTCM 返回数据对齐。只要读取路径中存在 Cache、乒乓缓存或其它可变等待状态，该功能必须关闭，加载依赖只能由实际 R 通道返回解除。

可抽象为以下状态流：

事务	进入条件	等待条件	完成条件	输出
加载	有效加载且输入 FIFO 可接收	AR 未握手或读返回未到	R 通道握手或 LSU L0 Cache 命中	加载扩展后写回 rd/commit id
存储	有效存储且请求 FIFO 与写事务状态可接收	AW、W 或 B 通道仍有未完成部分	B 通道握手后减少未返回写响应计数	不分配通用寄存器写回 commit id, mem_busy 保持到写数据和写响应均完成
非对齐拆分	RV32 打开非对齐支持，且半字或字访问跨越总线数据宽度边界	第一拍完成后保存部分数据或地址	第二拍完成并合并	加载合并结果，存储分两次写；RV64 拆分尚未实现
Atomic	A 扩展打开且 AMO/LR/SC 有效	read-modify-write 序列未完成	写回/状态更新完成	保持原子操作占用，延迟异常/中断
LSU L0 Cache 命中	Dispatch 影子表命中且 LSU 就绪	无 AXI 读等待	当前拍使用 LSU L0 Cache 数据体	走普通加载写回与旁路通路

表 43 LSU 内部状态流表

原子操作使用独立的六状态控制器，并且只在普通请求 FIFO、读事务 FIFO、写数据 FIFO、写响应计数、非对齐拆分状态和待写回加载结果全部为空时接受新请求。接受条件还要求当前没有已经确认的中断。原子操作开始后，LSU 暂停普通读写请求，AXI4-Lite 五个通道由原子状态控制器驱动；直到 AMO_DONE 输出结果后，普通访问才可以继续。

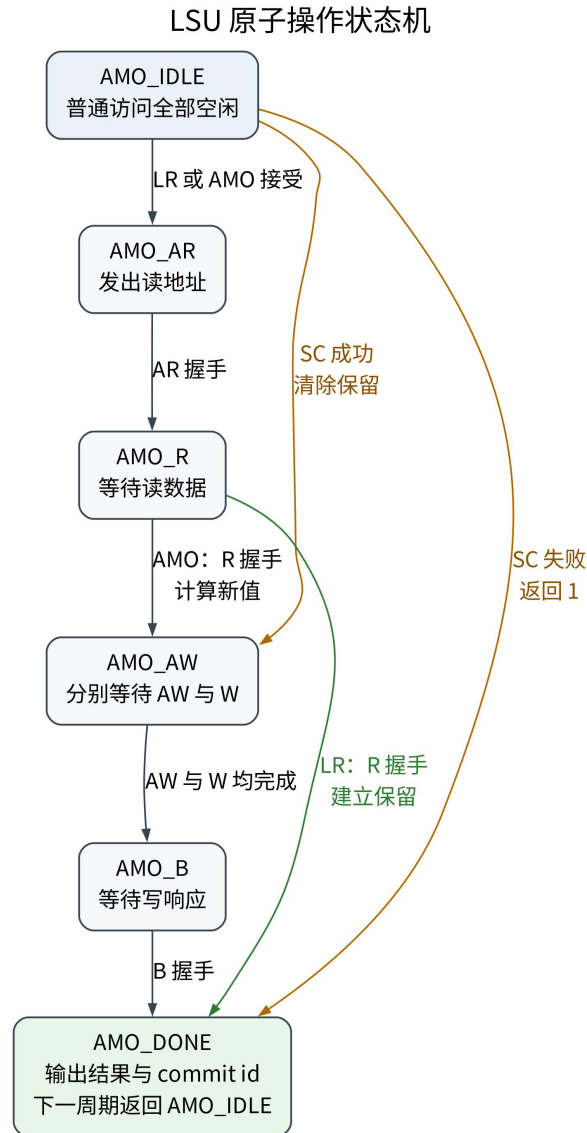


图 14 LSU 原子操作状态机

LR、SC 和普通 AMO 的状态转移如下表所示。AW 与 W 通道相互独立，AMO_AW 分别保存两者的完成状态，只有两个通道均完成握手后才进入 AMO_B。状态机不要求 AW 与 W 在同一周期完成。

当前状态	条件	下一状态	时序行为
AMO_IDLE	LR 或普通 AMO 被接受	AMO_AR	保存操作类型、地址、rs2 数据、字节使能、rd 和 commit id。
AMO_IDLE	SC 被接受且保留地址相等	AMO_AW	清除保留标志，准备写入 rs2 数据，结果预置为 0。

当前状态	条件	下一状态	时序行为
AMO_IDLE	SC 被接受且保留地址不等	AMO_DONE	不发出 AXI 写请求，结果置为 1。
AMO_AR	AR 通道握手	AMO_R	保持地址和操作属性，等待读数据。
AMO_R	LR 的 R 通道握手	AMO_DONE	保存旧值作为写回结果，并保存保留地址。
AMO_R	普通 AMO 的 R 通道握手	AMO_AW	保存旧值作为写回结果，根据操作类型计算新值并清除保留标志。
AMO_AW	仅 AW 或 W 完成握手	AMO_AW	保存已完成通道状态，继续保持另一通道的有效信号。
AMO_AW	AW 与 W 均已完成	AMO_B	清除 AW/W 完成状态，等待写响应。
AMO_B	B 通道握手	AMO_DONE	完成存储条件部分，不改变已经保存的写回旧值。
AMO_DONE	无附加条件	AMO_IDLE	输出写回数据、rd 和 commit id，一个周期后回到空闲。

表 44 LSU 原子操作状态转移

中断只在 AMO_IDLE 清除 LR 保留标志，不会在读改写进行到一半时取消总线事务。atomic_busy 在原子请求出现或状态不为空闲时有效，并经核心控制送入 CLINT，使中断状态机等待原子操作结束。LR 建立保留后，SC 无论成功或失败都会清除保留；普通 AMO 在读响应到达时清除保留。RV64 的字操作根据地址 bit2 选择 64 位返回数据的低 32 位或高 32 位，写回旧值时符号扩展到 64 位；双字 AMO 使用完整 64 位数据。

LSU L0 Cache 打开时，EXU 侧只保存字节数据 RAM 和替换路信息；Dispatch 侧保存标签/有效位影子表。命中时 axi_read_data 从 LSU L0 Cache 数据体选择，reg_write_valid_set 像普通读返回一样置位。填充、整字存储分配、部分存储清除和原子操作清除通过状态修改信息修改 Dispatch 影子表。这个路径避免了 LSU 数据反向进入 Dispatch/BRU 组合控制。

5.4.5 MUL 与 Zbc 无进位乘法扩展

乘法单元按处理器数据宽度、流水线级数和面积配置选择不同结构。三级

RV32 配置使用一拍结果寄存的完整乘法器，不生成 EXU 局部旁路。RV32 四级和五级的面积优先配置复用一套 17 位分段乘法资源执行四个片段步骤，并设置独立的 17 位低位快速计算级；非面积优先配置使用寄存的分段部分积结构。RV64 普通全宽乘法和高半乘法固定采用按 16 位操作数片段逐次计算的迭代结构，共处理十六个部分积，不启用分段并行结构；MULW 单独计算操作数低 32 位，并将低 32 位结果符号扩展为 64 位。只有非面积优先且非三级配置生成深度为一项的普通乘法请求缓冲。结果有效状态保持到 WBW 接收。

MUL 完成时输出 mul_reg_wdata_o、mul_reg_we_o、mul_reg_waddr_o 和 mul_commit_id，并向 HDU 给出 mul_bypass_valid_o 与 mul_bypass_commit_id_o。四级和五级 HDU 据此置位单组 MUL 旁路选择状态，后续 ALU、MUL 或存储数据可在 EXU 内选择相应结果；条件分支不能直接选择 MUL 结果。三级配置关闭该局部旁路，并由 WBW 按 commit id 查询 rd。

ENABLE_ZBC 置位时，译码模块识别 clmul、clmulh 和 clmulr，并将相应控制字段送至乘法执行单元。该参数为零时，三条指令的译码控制位固定为 0，指令不被识别，乘法执行单元中的 Zbc 多周期状态不生成。

参数	默认值	有效取值	功能说明
AREA_OPT	0	0、1	为 1 时，RV32 普通乘法使用面积优先的按 16 位操作数片段逐次计算结构，操作数片段按符号属性扩展为 17 位；CPU 集成时取 !ENABLE_LSU_HOT_CACHE。
SINGLE_CYCLE_RV32	0	0、1	为 1 且处理器数据宽度为 32 位时，使用一拍结果寄存的完整乘法结构；CPU 仅在三级 RV32 配置中置位。
COMPACT_STAGE3_WADDR	0	0、1	为 1 时不在 MUL 内重复保存目的寄存器地址，由 WBW 根据 commit id 查询；CPU 仅在三级配置中置位。

参数	默认值	有效取值	功能说明
ENABLE_ZBC	0	0、1	控制 clmul、clmulh 和 clmulr 无进位乘法运算结构。
ZBC_CHUNK_BITS	16	4、8、16	设置 Zbc 每次迭代处理的乘数片段位数。

表 45 MUL 执行控制参数

乘法运算部分还具有 USE_SEGMENTED_RV64 参数，默认值为 0。该参数为 0 时，RV64 普通乘法采用按 16 位操作数片段逐次计算的结构；为 1 时采用寄存部分积和分行求和结构。CPU 的 MUL 执行控制部分未向该参数传入其它值，因此 CPU 中的 RV64 普通乘法固定采用前述片段迭代结构。

端口名	方向	位宽	有效条件与功能
clk	输入	1	请求暂存、计算状态和结果寄存器的时钟。
rst_n	输入	1	低电平有效异步复位，清除请求有效、计算忙和结果有效状态。
wb_ready	输入	1	WBU 可以接收当前 MUL 结果；为 0 时已完成结果保持不变。
reg_waddr_i	输入	REG_ADDR_WIDTH	当前乘法指令的 GPR 目的寄存器地址。
reg1_rdata_i	输入	REG_DATA_WIDTH	第一乘法操作数。
reg2_rdata_i	输入	REG_DATA_WIDTH	第二乘法操作数。
alu_result_bypass_op1_i	输入	REG_DATA_WIDTH	第一操作数选择 ALU 局部旁路时使用的数据。
alu_result_bypass_op2_i	输入	REG_DATA_WIDTH	第二操作数选择 ALU 局部旁路时使用的数据。
mul_pass_op1_i	输入	1	第一操作数选择 alu_result_bypass_op1_i。
mul_pass_op2_i	输入	1	第二操作数选择 alu_result_bypass_op2_i。

端口名	方向	位宽	有效条件与功能
commit_id_i	输入	COMMIT_ID_WIDTH	当前乘法请求的 commit id。
stage3_ticket_valid_i	输入	1	三级配置中，当前请求已取得有效 commit id。
req_mul_i	输入	1	当前执行请求属于 MUL。
mul_op_mul_i	输入	1	选择 MUL，在 RV64 中还参与 MULW 控制组合。
mul_op_mulh_i	输入	1	选择有符号高半乘积 MULH。
mul_op_mulhsu_i	输入	1	选择有符号数与无符号数高半乘积 MULHSU。
mul_op_mulhu_i	输入	1	选择无符号高半乘积 MULHU。
mul_op_clmul_i	输入	1	选择无进位乘积低半部分 CLMUL。
mul_op_clmulh_i	输入	1	选择无进位乘积高半部分 CLMULH。
mul_op_clmulr_i	输入	1	选择右对齐无进位乘积 CLMULR。
int_assert_i	输入	1	中断或异常清除有效；阻止尚未接受的 MUL 请求启动。
mul_stall_flag_o	输出	1	当前 MUL 请求尚未被内部计算结构或请求暂存位置接受。
reg_wdata_o	输出	REG_DATA_WIDTH	当前已完成普通乘法或 Zbc 指令的写回数据。
reg_we_o	输出	1	当前 MUL 结果需要写入 GPR。
reg_waddr_o	输出	REG_ADDR_WIDTH	当前 MUL 结果的目的地寄存器地址；三级配置输出 x0，由 WBU 查询实际地址。

端口名	方向	位宽	有效条件与功能
commit_id_o	输出	COMMIT_ID_WIDTH	当前 MUL 结果的 commit id。
stage3_ticket_valid_o	输出	1	三级配置中，当前 MUL 结果带有有效分配状态。
normal_mul_bypass_valid_o	输出	1	普通整数乘法结果已经完成，并可进入局部旁路状态。
normal_mul_bypass_commit_id_o	输出	COMMIT_ID_WIDTH	普通整数乘法局部旁路结果的 commit id。
zbc_bypass_valid_o	输出	1	Zbc 结果已经进入结果保持状态，并可进入局部旁路状态。
zbc_bypass_commit_id_o	输出	COMMIT_ID_WIDTH	Zbc 局部旁路结果的 commit id。

表 46 MUL 执行控制端口

端口名	方向	位宽	有效条件与功能
clk	输入	1	乘法状态、部分积和结果寄存器的时钟。
rst_n	输入	1	低电平有效异步复位，清除状态和结果有效位。
multiplicand_i	输入	REG_DATA_WIDTH	被乘数。
multiplier_i	输入	REG_DATA_WIDTH	乘数。
operand_capture_i	输入	1	允许低位快速计算级捕获操作数，不单独表示完整请求已被接受。
valid_in	输入	1	乘法请求有效。
ctrl_ready_i	输入	1	外部可以接收或推进当前结果。
op_i	输入	4	普通整数乘法子操作编码。
reg_waddr_i	输入	REG_ADDR_WIDTH	随请求传入的目的寄存器地址。
commit_id_i	输入	COMMIT_ID_WIDTH	随请求传入的 commit id。
result_o	输出	REG_DATA_WIDTH	按乘法子操作选出的结果。

端口名	方向	位宽	有效条件与功能
valid_o	输出	1	结果有效；外部尚未接收时保持有效。
reg_waddr_o	输出	REG_ADDR_WIDTH	与结果对应的目的寄存器地址。
commit_id_o	输出	COMMIT_ID_WIDTH	与结果对应的 commit id。
ready_o	输出	1	当前类型的乘法请求可以被接受。
fast_ready_o	输出	1	低位快速计算级可以接受请求。
slow_ready_o	输出	1	全宽或高半乘法计算部分可以接受请求。

表 47 乘法运算部分端口

普通整数乘法按配置生成以下结构。

配置条件	计算结构	请求与结果处理
三级 RV32	33 位带符号扩展操作数形成完整乘积，结果在时钟沿寄存	不使用独立多周期状态；结果位置可推进时可以每拍接受一项请求。
四级或五级 RV32， AREA_OPT=1	一套 17 位分段迭代资源计算 4 个部分积，另设一套 17 位低位快速计算级	全宽或高半结果依次计算低片段乘低片段、低片段乘高片段、高片段乘低片段和高片段乘高片段。
四级或五级 RV32， AREA_OPT=0	4 个寄存部分积形成 64 位乘积，低位小操作数使用独立快速计算级	慢速部分依次经过部分积寄存和结果选择，执行控制部分提供一项待处理请求暂存。
任意流水线级数的 RV64	一套 16 位乘法资源依次计算 16 个部分积	MULW 直接计算低 32 位并符号扩展，其它操作完成 16 次部分积累加后选择低半或高半结果。

表 48 普通整数乘法结构选择

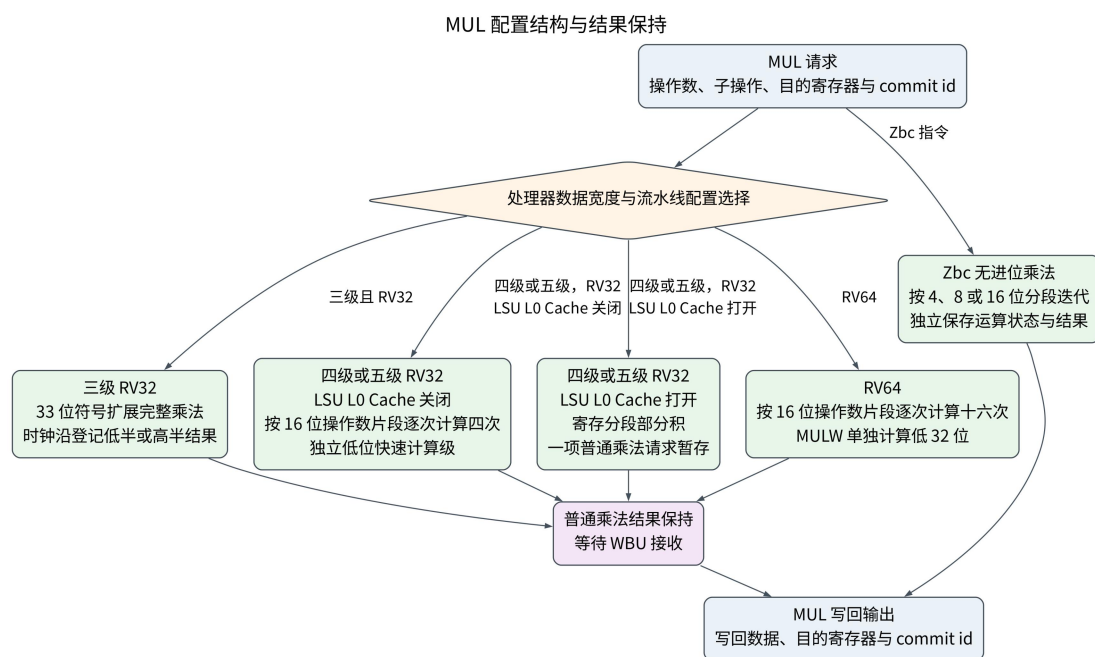


图 14-1 MUL 配置结构与结果保持

RV32 面积优先结构使用空闲、等待、4 个部分积计算和结果保持状态。空闲状态可以接受低位快速请求，或保存普通请求并进入第一个部分积计算状态；若共享结果位置仍被较早的快速结果占用，则先进入等待状态。4 个部分积状态每周期复用同一乘法资源，最后进入结果保持状态，直至 WBU 接收结果。RV64 普通乘法使用空闲、迭代和结果三个状态，迭代状态以行列计数选择两个 16 位操作数片段，每周期计算并累加一个部分积，16 个部分积全部完成后恢复符号并选择低 64 位或高 64 位结果。

中断或异常清除只阻止尚未接受的新 MUL 请求，不清除已经开始的普通乘法或 Zbc 运算。已经开始的运算继续形成结果，错误路径结果由 HDU 的架构有效状态和 WBU 的 commit id 检查阻止写入 GPR。普通乘法和 Zbc 分别输出局部旁路有效状态，EXU 再将两类状态合并送往 HDU；两类结果均保持到 WBU 接收。

设 P 为两个处理器数据宽度（XLEN）位操作数在二进制有限域上的无进位乘积，宽度为两倍处理器数据宽度。三条指令的写回结果如下。

指令	写回结果
clmul rd, rs1, rs2	$P[XLEN-1:0]$ 。
clmulh rd, rs1, rs2	$P[2 \times XLEN-1:XLEN]$ 。
clmulr rd, rs1, rs2	$P[2 \times XLEN-2:XLEN-1]$ 。

表 49 Zbc 无进位乘法结果选择

无进位乘法使用乘法执行单元内部的多周期状态，不在一个组合周期形成完整乘积。操作数、目的寄存器、commit id、子操作类型和结果保存在本地寄存器

中。默认每次处理 16 位操作数片段时，RV32 和 RV64 分别执行两次和四次片段迭代。启动无进位乘法前，普通乘法的慢速执行通道和等待队列必须为空；结果产生后，仍按普通乘法结果接受 WBU 的就绪控制。因此，无进位乘法不绕过写回仲裁和 commit id 管理；EXU 局部旁路只在四级和五级配置下使用。

5.4.6 DIV

除法单元采用试商迭代状态机，保存被除数、除数、符号、余数、商和计数器；外层 exu_div 保存目的寄存器地址与 commit id。op_i[4] 表示 RV64 字操作，op_i[3:0] 依次表示 REMU、REM、DIVU 和 DIV。RV32 使用一套 33 位减法资源，依次完成被除数取绝对值、除数取绝对值、逐位计算和符号恢复。RV64 全宽运算执行 64 个商位，每个商位把试减分成低 32 位和高位两个周期；字操作只执行 32 个商位。结果根据运算类型和符号状态选择，除数为零时商返回全 1、余数返回原被除数，有符号最小值除以负一的结果由迭代数据通路自然形成。RV64 还实现 DIVW、DIVUW、REMW 和 REMUW，这四条指令对低 32 位操作数执行相应运算，并将 32 位结果符号扩展为 64 位。

DIV 不接入 EXU 局部旁路。结果有效后保持到 WBU 接收，上一结果被接收的同一周期可以接收下一请求。三级配置不保存目的寄存器号，由 WBU 按 commit id 向 HDU 查询 rd；四级和五级配置随请求保存 rd。该结构以多周期延迟换取较小硬件面积，并避免把 DIV 数据加入短延迟操作数选择器。

参数	默认值	有效取值	功能说明
COMPACT_STAGE3_WADDR	0	0、1	为 1 时不保存目的寄存器地址，由 WBU 根据 commit id 查询；CPU 仅在三级配置中置位。

表 50 DIV 参数

端口名	方向	位宽	有效条件与功能
clk	输入	1	请求信息、迭代状态和结果寄存器的时钟。
rst_n	输入	1	低电平有效异步复位，清除忙状态和结果有效状态。
wb_ready	输入	1	WBU 可以接收当前 DIV 结果。
reg_waddr_i	输入	REG_ADDR_WIDTH	当前除法指令的 GPR 目的寄存器地址。
regl_rdata_i	输入	REG_DATA_WIDTH	被除数。

端口名	方向	位宽	有效条件与功能
reg2_rdata_i	输入	REG_DATA_WIDTH	除数。
commit_id_i	输入	COMMIT_ID_WIDTH	当前除法请求的 commit id。
stage3_ticket_valid_i	输入	1	三级配置中，当前请求已取得有效 commit id。
req_div_i	输入	1	当前执行请求属于 DIV。
div_op_div_i	输入	1	选择有符号除法 DIV。
div_op_divu_i	输入	1	选择无符号除法 DIVU。
div_op_rem_i	输入	1	选择有符号余数 REM。
div_op_remu_i	输入	1	选择无符号余数 REMU。
div_op_word_i	输入	1	RV64 中选择 32 位 字操作 DIVW、 DIVUW、REMW 或 REMUW。
int_assert_i	输入	1	中断或异常清除有效；阻止尚未启动的 DIV 请求。
div_stall_flag_o	输出	1	当前 DIV 请求未能 启动，需要保持上游 执行请求。
reg_wdata_o	输出	REG_DATA_WIDTH	商或余数写回数据。
reg_we_o	输出	1	当前 DIV 结果需要 写入 GPR。
reg_waddr_o	输出	REG_ADDR_WIDTH	当前 DIV 结果的目 的寄存器地址；三级 配置输出 x0。
commit_id_o	输出	COMMIT_ID_WIDTH	当前 DIV 结果的 commit id。
stage3_ticket_valid_o	输出	1	三级配置中，当前 DIV 结果带有有效分 配状态。

表 51 DIV 执行控制端口

逐位试商运算部分无独立参数，其端口定义如下表所示。

端口名	方向	位宽	有效条件与功能
clk	输入	1	除法状态和数据寄存器的时钟。
rst_n	输入	1	低电平有效异步复位。
dividend_i	输入	REG_DATA_WIDTH	被除数。
divisor_i	输入	REG_DATA_WIDTH	除数。
start_i	输入	1	启动一次新的除法运算。
ctrl_ready_i	输入	1	当前结果可以被外部接收，或结果位置为空。
op_i	输入	5	由字操作、REMU、REM、DIVU 和 DIV 组成的子操作编码。
result_o	输出	REG_DATA_WIDTH	商或余数结果。
busy_o	输出	1	逐位试商仍在进行。
valid_o	输出	1	结果有效；外部未接收时保持有效。

表 52 逐位试商运算部分端口

RV32 复用一套 33 位减法资源，状态和主要操作如下表所示。

状态	转移条件	主要操作
空闲	接受请求后进入被除数绝对值状态	保存子操作、被除数和除数。
被除数绝对值	除数为零时直接完成，否则进入除数绝对值状态	计算被除数绝对值并保存结果符号；除数为零时，商取全 1，余数取原被除数。
除数绝对值	无附加条件进入逐位计算状态	计算除数绝对值，清零商和余数，计数器置为 32。
逐位计算	未完成最后一个商位时保持，完成后进入符号恢复状态	每周期左移被除数和部分余数，执行一次试减并产生一个商位。
符号恢复	无附加条件返回空闲状态	按有符号操作类型恢复商或余数符号，并置位结果有效。

表 53 RV32 除法状态机

RV64 将一次 65 位试减分成低 32 位和高位两个周期，以缩短单周期减法的进位传播范围。其状态和主要操作如下表所示。

状态	转移条件	主要操作
空闲	接受请求后进入启动状态	保存子操作、被除数和除数。

状态	转移条件	主要操作
启动	除数为零时直接完成，否则进入低半试减状态	计算操作数绝对值，初始化商、余数、符号状态和迭代计数。
低半试减	无附加条件进入高半试减状态	计算试减低 32 位，保存低位差和借位。
高半试减	尚有商位时返回低半试减状态，最后一个商位完成时进入结果状态	使用已保存借位完成高位试减，更新商、余数和计数器。
结果	无附加条件返回空闲状态	选择商或余数，恢复符号并置位结果有效。

表 54 RV64 除法状态机

DIV 状态机与按位宽区分的计算处理

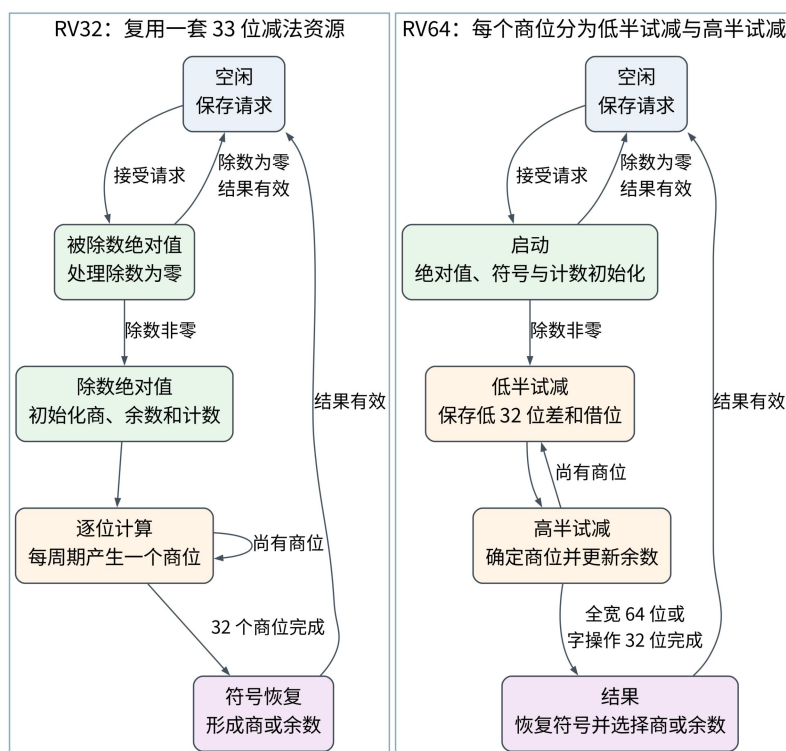


图 14-2 DIV 状态机与按位宽区分的计算处理

中断或异常清除只阻止尚未启动的新 DIV 请求，已经开始的除法继续完成。请求控制部分在启动时保存 commit id、分配有效状态以及四级和五级配置使用的目的寄存器地址；错误路径结果由 HDU 和 WBU 的有效状态检查阻止写入 GPR。三种流水线配置共用相同的逐位试商资源，流水线级数不改变除法状态数和每个商位的计算方法。

5.4.7 CSR 执行单元

CSR 执行单元是组合逻辑，处理 CSRRW、CSRRS、CSRRC 及其立即数形式。立即数操作数由此前的控制逻辑根据指令 bit14 和 inst[19:15] 选择。单元读取 CSR 旧值，产生 CSR 新值，并在需要时把旧值作为目的寄存器写回数据。CSR 属于有序类，Dispatch 不允许它进入等待项或越过需要保持顺序的前序指令。

CSR 单元同时输出 CSR 写地址、写数据与写使能，以及 GPR 写回数据、写使能和 commit id。单元内部不保存请求；WBU 未准备好时，CSR 暂停信号使 dispatch_pipe 保持同一请求。WBU 分别寄存 CSR 状态写入和可选的 GPR 旧值写回。

CSR 执行单元没有模块参数，端口定义如下表所示。

端口名	方向	位宽	有效条件与功能
clk	输入	1	保留的时钟输入，当前组合实现不使用该信号保存状态。
rst_n	输入	1	保留的复位输入，当前组合实现不使用该信号保存状态。
req_csr_i	输入	1	当前执行请求属于 CSR 指令。
csr_op1_i	输入	REG_DATA_WIDTH	rs1 数据或零扩展立即数。
csr_csrrw_i	输入	1	选择直接写入操作。
csr_csrrs_i	输入	1	选择按位置位操作。
csr_csr_rc_i	输入	1	选择按位清零操作。
csr_rdata_i	输入	REG_DATA_WIDTH	CSR 寄存器模块返回的写入前数值。
commit_id_i	输入	COMMIT_ID_WIDTH	当前 CSR 指令的 commit id。
csr_we_i	输入	1	当前指令需要修改 CSR。
csr_reg_we_i	输入	1	当前指令需要把 CSR 写入前数值写入 GPR。
csr_waddr_i	输入	BUS_ADDR_WIDTH	CSR 写地址，低 12 位有效。
reg_waddr_i	输入	REG_ADDR_WIDTH	GPR 目的寄存器地址。

端口名	方向	位宽	有效条件与功能
wb_ready_i	输入	1	WBU 可以接收 CSR 指令的 GPR 结果。
int_assert_i	输入	1	中断或异常清除有效；撤销当前 CSR 请求的两个写使能并将数据输出置零。
csr_stall_o	输出	1	CSR 指令需要写 GPR 且 WBU 尚不能接收。
csr_wdata_o	输出	REG_DATA_WIDTH	根据 CSR 子操作形成的新值。
csr_we_o	输出	1	当前 CSR 新值有效。
csr_waddr_o	输出	BUS_ADDR_WIDTH	CSR 写地址。
reg_wdata_o	输出	REG_DATA_WIDTH	写入 GPR 的 CSR 写入前数值。
reg_waddr_o	输出	REG_ADDR_WIDTH	CSR 指令的 GPR 目的寄存器地址。
commit_id_o	输出	COMMIT_ID_WIDTH	CSR 指令的 commit id。
csr_reg_we_o	输出	1	CSR 写入前数值需要写入 GPR。

表 55 CSR 执行单元端口

子操作	CSR 新值
CSRRW 或 CSRRWI	csr_op1_i
CSRRS 或 CSRRSI	csr_rdata_i csr_op1_i
CSRRC 或 CSRRCI	csr_rdata_i & ~csr_op1_i

表 56 CSR 写数据运算

暂停只在 CSR 指令需要写 GPR 且 WBU 未准备好时产生；只写 CSR 而不写 GPR 时不等待 wb_ready_i。中断或异常清除同时禁止 CSR 与 GPR 写使能。三级、四级和五级配置共用相同组合运算，差别仅在 WBU 对 CSR 状态写端口和 GPR 写端口的接收方式。

5.5 WBU

写回单元（WBU）接收 ALU、LSU、MUL、DIV 和 CSR 执行单元的完成结果，在 GPR 单写端口约束下完成仲裁、冲突结果暂存、写后写过滤和 commit id 释放。GPR 与 CSR 使用独立写端口：不写 GPR 的 CSR 指令可以与其他 GPR 结果同拍更新 CSR；需要把 CSR 旧值写入 GPR 时，CSR 状态写入与

其 GPR 结果共同等待仲裁。

参数	默认值	有效取值	功能说明
PIPELINE_STAGES	5	3、4、5	选择三级紧凑写回结构或四级、五级共用的多来源写回结构。
ENABLE_BPU_ENHANCED	0	0、1	在五级配置中允许原始陷入事件撤销尚未写入最新 commit id 表的分配信息。

表 57 WBU 参数

端口名	方向	位宽	有效条件与功能
clk	输入	1	仲裁结果、结果暂存和最新 commit id 状态的时钟。
rst_n	输入	1	低电平有效异步复位，清除有效位、队列状态和输出写使能。
alloc_valid_i	输入	1	Dispatch 已为一条写 GPR 指令分配 commit id。
alloc_waddr_i	输入	REG_ADDR_WIDTH	新分配指令的目的寄存器地址。
alloc_commit_id_i	输入	COMMIT_ID_WIDTH	新分配指令的 commit id。
alloc_rollback_i	输入	1	撤销尚未写入最新 commit id 表的分配信息。
alloc_rollback_commit_id_i	输入	COMMIT_ID_WIDTH	需要撤销的 commit id。
alloc_prewrite_inhibit_i	输入	1	五级增强 BPU 配置中，禁止尚未写表的指定分配生效。
alloc_prewrite_inhibit_commit_id_i	输入	COMMIT_ID_WIDTH	需要禁止写表的 commit id。
stage3_arch_live_i	输入	$2^{\text{COMMIT_ID_WIDTH}}$	三级配置中，每个 commit id 对应的架构有效状态。
stage3_tag_valid_i	输入	1	三级配置中，HDU 查询返回的目的寄存器地址有效。

端口名	方向	位宽	有效条件与功能
stage3_tag_waddr_i	输入	REG_ADDR_WIDTH	三级配置中，HDU 根据查询编号返回的目的寄存器地址。
stage3_tag_query_valid_o	输出	1	WBU 请求 HDU 查询当前所选结果的目的寄存器。
stage3_tag_query_id_o	输出	COMMIT_ID_WIDTH	三级配置中送往 HDU 的查询 commit id。
alu_reg_wdata_i	输入	REG_DATA_WIDTH	ALU 完成结果数据。
alu_reg_we_i	输入	1	ALU 完成结果有效。
alu_reg_waddr_i	输入	REG_ADDR_WIDTH	ALU 结果目的寄存器地址。
alu_commit_id_i	输入	COMMIT_ID_WIDTH	ALU 结果的 commit id。
alu_ticket_valid_i	输入	1	三级配置中，ALU 结果具有有效分配状态。
alu_ready_o	输出	1	WBU 可以直接接收或暂存 ALU 结果。
mul_reg_wdata_i	输入	REG_DATA_WIDTH	MUL 完成结果数据。
mul_reg_we_i	输入	1	MUL 完成结果有效。
mul_reg_waddr_i	输入	REG_ADDR_WIDTH	MUL 结果目的寄存器地址。
mul_commit_id_i	输入	COMMIT_ID_WIDTH	MUL 结果的 commit id。
mul_ticket_valid_i	输入	1	三级配置中，MUL 结果具有有效分配状态。
mul_ready_o	输出	1	WBU 可以接收 MUL 结果。
div_reg_wdata_i	输入	REG_DATA_WIDTH	DIV 完成结果数据。
div_reg_we_i	输入	1	DIV 完成结果有效。
div_reg_waddr_i	输入	REG_ADDR_WIDTH	DIV 结果目的寄存器地址。
div_commit_id_i	输入	COMMIT_ID_WIDTH	DIV 结果的 commit id。

端口名	方向	位宽	有效条件与功能
div_ticket_valid_i	输入	1	三级配置中，DIV 结果具有有效分配状态。
div_ready_o	输出	1	WBU 可以接收 DIV 结果。
csr_wdata_i	输入	REG_DATA_WIDTH	CSR 指令形成的新 CSR 数值。
csr_we_i	输入	1	CSR 状态写请求有效。
csr_waddr_i	输入	BUS_ADDR_WIDTH	CSR 写地址。
csr_commit_id_i	输入	COMMIT_ID_WIDTH	CSR 指令的 commit id。
csr_ready_o	输出	1	WBU 可以接收 CSR 指令的 GPR 结果。
csr_reg_wdata_i	输入	REG_DATA_WIDTH	CSR 指令写入 GPR 的旧 CSR 数值。
csr_reg_waddr_i	输入	REG_ADDR_WIDTH	CSR 指令的 GPR 目的寄存器地址。
csr_reg_we_i	输入	1	CSR 指令需要写 GPR。
csr_ticket_valid_i	输入	1	三级配置中，CSR 的 GPR 结果具有有效分配状态。
lsu_reg_wdata_i	输入	REG_DATA_WIDTH	LSU 完成结果数据。
lsu_reg_we_i	输入	1	LSU 完成结果有效。
lsu_reg_hot_we_i	输入	1	当前 LSU 结果来自 LSU L0 Cache 命中。
lsu_pending_reg_waddr_i	输入	REG_ADDR_WIDTH	普通总线返回或原子操作结果的目的寄存器地址。
lsu_pending_commit_id_i	输入	COMMIT_ID_WIDTH	普通总线返回或原子操作结果的 commit id。
lsu_pending_ticket_valid_i	输入	1	三级配置中，普通 LSU 结果具有有效分配状态。
lsu_hot_reg_waddr_i	输入	REG_ADDR_WIDTH	LSU L0 Cache 命中结果的目的寄存器地址。

端口名	方向	位宽	有效条件与功能
lsu_hot_commit_id_i	输入	COMMIT_ID_WIDTH	LSU L0 Cache 命中结果的 commit id。
lsu_hot_ticket_valid_i	输入	1	三级配置中，LSU L0 Cache 命中结果具有有效分配状态。
commit_valid_o	输出	1	当前仲裁结果可以释放 HDU 中的 commit id。三级配置要求执行结果具有有效分配状态且 HDU 查询有效；四级和五级配置要求目的寄存器不是 x0。
commit_id_o	输出	COMMIT_ID_WIDTH	当前完成结果的 commit id。
reg_wdata_o	输出	REG_DATA_WIDTH	写入 GPR 的最终数据。
reg_we_o	输出	1	当前结果通过架构有效检查并允许写入 GPR。
reg_waddr_o	输出	REG_ADDR_WIDTH	GPR 写地址。
reg_wdata_src_sel_o	输出	3	已寄存的 GPR 数据来源选择值。
reg_wdata_candidates_o	输出	8*REG_DATA_WIDTH	8 个已寄存候选数据，编号与 reg_wdata_src_sel_o 一致。
csr_wdata_o	输出	REG_DATA_WIDTH	写入 CSR 寄存器模块的数据。
csr_we_o	输出	1	CSR 写输出有效。
csr_waddr_o	输出	BUS_ADDR_WIDTH	CSR 写地址。

表 58 WBU 输入输出端口

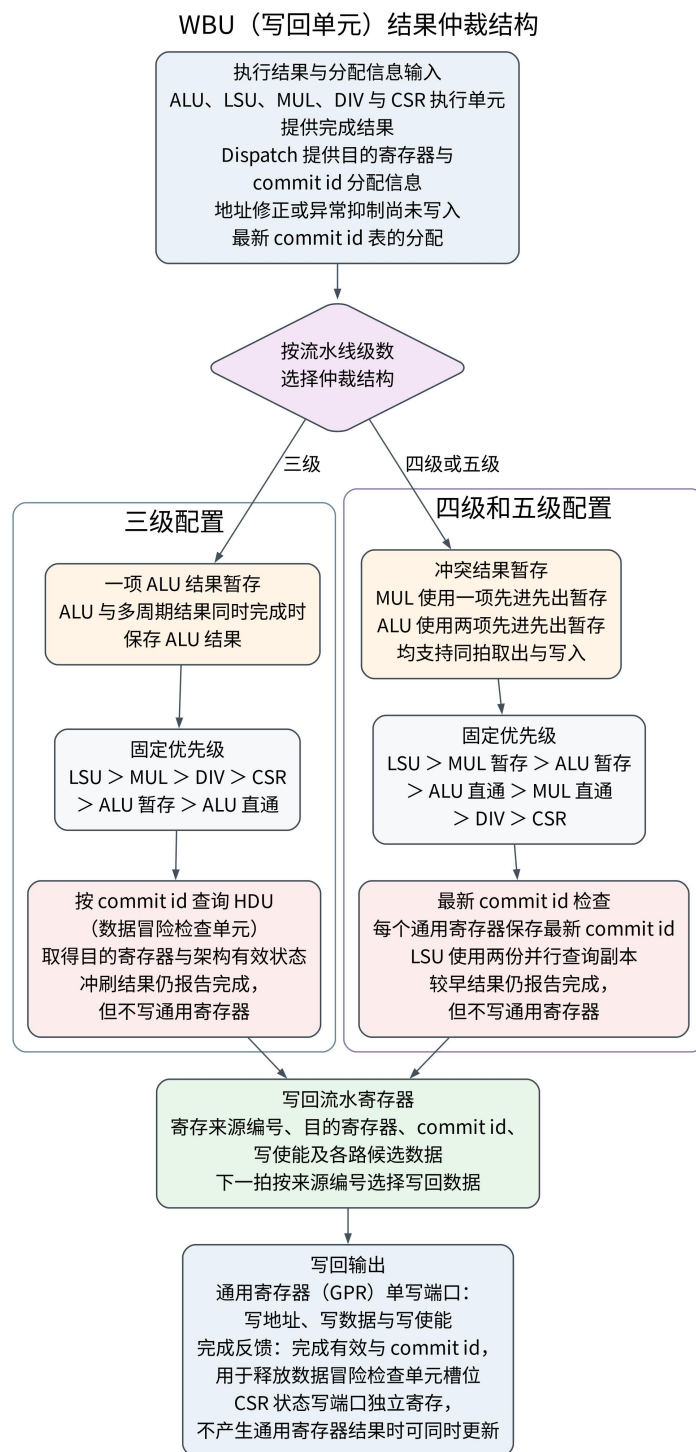


图 15 WBU 结果仲裁结构

四级和五级配置为每个通用寄存器保存三张 commit id 表，分别服务普通执行结果、普通 LSU 返回和 LSU L0 Cache 命中结果的并行查询。Dispatch 分配新的写寄存器指令时，WBU 先寄存分配有效位、目的寄存器地址和 commit id，下一周期再写入三张表；查询端同时比较尚未写表的寄存分配，以保证连续分配时仍能取得最新编号。地址修正可以撤销待写表分配，五级增强

BPU 配置还可以阻止原始陷入事件对应的待写表分配生效。执行结果只有在自身 `commit id` 等于该目的寄存器的最新编号时才允许写 GPR。已经被随后写指令替代的非零目的寄存器结果仍产生 `commit_valid_o` 和 `commit_id_o`，使 HDU 释放原槽位，但不写寄存器。ALU 和 MUL 的小 FIFO 吸收多个执行单元同拍完成造成的端口冲突。

四级和五级配置使用 `reg_wb_fifo` 保存暂时未被仲裁选中的 ALU 和 MUL 结果。MUL FIFO 深度为 1，使用一个有效位及一组数据、目的寄存器和 `commit id` 寄存器；ALU FIFO 深度为 2，使用两项分布式 RAM、读写指针和计数器，避免在每次出队时移动两组宽数据。同拍出队和入队时可以释放队首并接收新结果；复位只清除有效状态、指针和计数，数据阵列不复位。三级 WBU 使用内部的一项 ALU 暂存器，不使用上述 FIFO，也不生成未接入当前 CPU 的备用写回队列。

四级和五级的 GPR 结果选择优先级为：LSU、MUL FIFO、ALU FIFO、ALU 直通、MUL 直通、DIV、CSR 的 GPR 结果。选择编号和候选数据先进入寄存器，最终数据在该寄存边界之后选择，再由 `reg_wdata_o`、`reg_waddr_o` 和 `reg_we_o` 输出。该两段结构缩短多源完成信号到 GPR 写端口的组合路径。

三级配置使用独立的紧凑实现。所有写后写冲突已经由 `hdu_stage3_compact` 阻塞，因此 WBU 不需要每寄存器最新 `commit id` 表；各执行单元也不随结果重复保存 `rd`，WBU 使用 `commit id` 查询 `stage3_tag_waddr_i`。三级配置设置一项 ALU 结果暂存器，实际优先级为 LSU、MUL、DIV、CSR 的 GPR 结果、ALU 暂存和 ALU 直通。

例如，四级或五级中较早的 `mul x5, x1, x2` 获得 `commit id A`，较晚的 `add x5, x3, x4` 获得 `commit id B`。B 分配时，x5 的最新 `commit id` 改为 B；A 即使更晚完成，也只释放 A 对应的 HDU 槽位，不写 x5，只有 B 的结果可以写入 GPR。三级配置不会允许这两个同 `rd` 指令重叠，因而不需要执行相同过滤。

GPR 写回、CSR 写入和完成输出均经过寄存器，各执行单元的 `ready` 输出以及三级配置的 `commit id` 查询端口为组合信号。三级配置使用各来源的 `ticket_valid_i`、`stage3_arch_live_i` 和 HDU 查询结果共同确认完成状态，只有发射时取得有效 `commit id` 且仍然有效的结果才可以产生完成输出。四级和五级配置不读取各来源的 `ticket_valid_i`，完成输出只过滤目的寄存器为 `x0` 的结果；这两种配置依靠 Dispatch 分配规则、执行单元结果保持和 WBU 最新 `commit id` 表维持状态一致性。三级配置的 CSR 状态写入每周周期独立寄存，GPR 旧值写回仍参与固定优先级选择；四级和五级配置中，CSR 指令需要写

GPR 时，CSR 状态写入与旧值写回共同等待仲裁。若 WBU 未准备好，执行单元必须保持结果及 commit id，避免 HDU 槽位在结果被接收前释放。

reg_wb_fifo 的参数和端口如下表所示。深度参数只允许取 1 或 2，端口定义在两种深度下保持一致。

类别	名称	方向	默认值或位宽	功能说明
参数	FIFO_DEPTH	—	2	选择一项或两项写回 FIFO。
参数	FIFO_PTR_WIDTH	—	$\$clog2(\text{FIFO_DEPTH})$	确定指针和计数状态所需宽度。
端口	clk	输入	1	时钟。
端口	rst_n	输入	1	低电平有效异步复位，只清除有效状态、指针和计数。
端口	wdata_i	输入	REG_DATA_WIDTH	入队写回数据。
端口	waddr_i	输入	REG_ADDR_WIDTH	入队目的寄存器地址。
端口	commit_id_i	输入	COMMIT_ID_WIDTH	入队结果的 commit id。
端口	push	输入	1	入队请求。
端口	pop	输入	1	出队请求。
端口	wdata_o	输出	REG_DATA_WIDTH	队首写回数据。
端口	waddr_o	输出	REG_ADDR_WIDTH	队首目的寄存器地址。
端口	commit_id_o	输出	COMMIT_ID_WIDTH	队首结果的 commit id。
端口	full	输出	1	指示 FIFO 已满。
端口	empty	输出	1	指示 FIFO 为空。
端口	count	输出	FIFO_PTR_WIDTH+1	给出当前有效项目数。

表 59 写回 FIFO 参数和端口

配置	写后写处理	冲突暂存	GPR 结果选择优先级
----	-------	------	-------------

配置	写后写处理	冲突暂存	GPR 结果选择优先级
四级、五级	每个 GPR 保存最新 commit id, 被替代的结果只释放 HDU 槽位	ALU FIFO、MUL FIFO	LSU、MUL FIFO、ALU FIFO、ALU 直通、MUL 直通、DIV、CSR
三级	HDU 对全部同 rd 写入实施阻塞, WBU 按 commit id 查询 rd	一项 ALU 结果暂存器	LSU、MUL、DIV、CSR、ALU 暂存、ALU 直通

表 60 WBU 流水线配置差异

5.6 GPR

GPR 是整数通用寄存器组, 提供两个常规异步读端口、一个 rs1 原始数据读端口和一个同步写端口。两份分布式寄存器阵列 regs_rp1 和 regs_rp2 实现双读口; 写回时同一份 wdata_i 同步写入两份阵列, rs1 与 rs2 分别读取一份副本。

其输入输出端口如下表所示:

端口名	方向	位宽	功能描述
clk	输入	1	时钟信号
rst_n	输入	1	复位信号 (低电平有效)
we_i	输入	1	写寄存器标志
waddr_i	输入	REG_ADDR_WIDTH	写寄存器地址
wdata_i	输入	REG_DATA_WIDTH	写寄存器数据
raddr1_i	输入	REG_ADDR_WIDTH	读寄存器 1 地址
rdata1_o	输出	REG_DATA_WIDTH	读寄存器 1 数据
rdata1_raw_o	输出	REG_DATA_WIDTH	第一份寄存器阵列的原始读值, 不执行 x0 清零和同拍写入前递, 供 Dispatch 的并行地址计算使用
raddr2_i	输入	REG_ADDR_WIDTH	读寄存器 2 地址
rdata2_o	输出	REG_DATA_WIDTH	读寄存器 2 数据

表 61 GPR 模块输入输出端口

寄存器阵列没有复位端口, rst_n 只参与写使能条件。写端口仅在 rst_n=1、we_i=WriteEnable 且 waddr_i 非 x0 时生效。常规读端口在地址为 x0 时返回全零; 若读地址与本拍写地址相同且写使能有效, 则优先返回 wdata_i, 形成 WBU 到 IDU 或 Dispatch 的同拍写读前递; 否则返回对应阵列副本中的数据。rdata1_raw_o 始终输出第一份阵列的原始读值, 专供 Dispatch 的并行地址计算, 不得作为架构可见的 x0 读值。

该同拍前递只覆盖“写回阶段已经选中并输出到 GPR”的数据，不替代 HDU/EXU 局部旁路。对于尚未进入 WBU 的 ALU、LSU 或 MUL 结果，RAW 仍由 HDU 的 commit id 槽位、EXU 局部旁路暂存区和等待槽共同处理。这个分工避免把所有执行结果直接接入 GPR 读端口，也避免把 GPR 读选择器扩展成全局旁路网络。

GPR 的接口不包含有效/就绪握手。它默认每拍对读地址给出组合结果，由上游 IDU/Dispatch 的有效位和暂停控制这些读值是否被采样。WBU 写回输出已经寄存，因此 GPR 写地址、写数据和写使能在时钟边沿稳定；若 WBU 因仲裁未选择某个执行结果，该结果不会进入 GPR，相关 commit id 也不会被释放。

5.7 CSR

CSR（控制状态寄存器）模块保存机器模式运行状态、陷入处理状态和计数器状态，并分别向执行单元与核心本地中断控制器提供读写接口。模块使用独立的执行单元写端口和 CLINT 写端口，两个端口共用内部寄存器组；执行单元还可在完整地址读取与预译码读取之间选择。模块参数如下表所示。

参数名	默认值	有效取值	功能描述
ENABLE_RV64	0	0、1	选择 RV32 或 RV64 计数器组织。取值为 0 时生成高、低两个 32 位计数寄存器；取值为 1 时使用完整数据宽度计数器，高半计数地址读零。该参数必须与 XLEN 保持一致；misa.MXL 由 XLEN 决定。

表 62 CSR 模块参数

模块端口如下表所示。

端口名	方向	位宽	有效条件与功能
clk	输入	1	CSR 状态、计数器和执行单元写入历史信息时钟。

端口名	方向	位宽	有效条件与功能
rst_n	输入	1	低电平有效异步复位，清零可写 CSR、计数器和写入历史有效位。
we_i	输入	1	执行单元 CSR 写请求有效。
raddr_i	输入	BUS_ADDR_WIDTH	执行单元 CSR 读地址，低 12 位参与地址选择和写读前递比较。
rsel_i	输入	CSR_READ_SEL_WIDTH	读选择预译码开启时使用的紧凑选择值。
waddr_i	输入	BUS_ADDR_WIDTH	执行单元 CSR 写地址，低 12 位有效。
data_i	输入	REG_DATA_WIDTH	执行单元写入 CSR 的数据。
clint_we_i	输入	1	CLINT 的 CSR 写请求有效。
clint_raddr_i	输入	BUS_ADDR_WIDTH	CLINT 的 CSR 读地址，低 12 位有效。
clint_waddr_i	输入	BUS_ADDR_WIDTH	CLINT 的 CSR 写地址，低 12 位有效。
clint_data_i	输入	REG_DATA_WIDTH	CLINT 写入 CSR 的数据。
inst_valid_i	输入	1	发往执行级的有效指令指示；在 IR 位未禁止计数时使用 minstret 加一。
clint_data_o	输出	REG_DATA_WIDTH	CLINT 通用读端口返回数据。
clint_csr_mtvec	输出	REG_DATA_WIDTH	直接输出 mtvec，供陷入目标地址计算使用。
clint_csr_mepc	输出	REG_DATA_WIDTH	直接输出 mepc，供 mret 返回地址计算使用。
clint_csr_mstatus	输出	REG_DATA_WIDTH	直接输出 mstatus，供全局中断使能和陷入状态控制使用。

端口名	方向	位宽	有效条件与功能
clint_csr_mie	输出	REG_DATA_WIDTH	直接输出 mie，供外部、定时器和软件中断使能检查使用。
data_o	输出	REG_DATA_WIDTH	执行单元 CSR 读端口返回数据。

表 63 CSR 模块输入输出端口

CSR 地址及其实际功能如下表所示。表中“可写”表示执行单元端口和 CLINT 端口均可按该地址写入；两个端口在同一周期写相同寄存器时，执行单元写数据优先。

寄存器名称	地址	访问属性	RV32 与 RV64 行为	功能描述
mvendorid	0xF11	只读	相同	厂商编号，固定返回零。
marchid	0xF12	只读	相同	微架构编号，固定返回零。
mimpid	0xF13	只读	相同	实现编号，固定返回零。
mhartid	0xF14	只读	相同	硬件线程编号，固定返回零。
mstatus	0x300	可写	寄存器宽度随 REG_DATA_WIDTH 变化	保存机器模式状态；陷入进入时将 MIE 复制到 MPIE 并清零 MIE，执行 mret 时将 MPIE 恢复到 MIE 并置位 MPIE。
misa	0x301	只读	MXL 分别报告 RV32 或 RV64	I 和 M 位固定为一，A 与 C 位由相应扩展开关决定，B 位固定为零。Zba、Zbb、Zbs 和 Zbc 属于 B 扩展的部分子集，不单独使 B 位置位。

寄存器名称	地址	访问属性	RV32 与 RV64 行为	功能描述
medeleg	0x302	条件可写	相同	仅在支持 S 模式时生成可写寄存器，否则固定返回零。
mideleg	0x303	条件可写	相同	仅在支持 S 模式时生成可写寄存器，否则固定返回零。
mie	0x304	可写	寄存器宽度随 REG_DATA_WIDTH 变化	保存机器模式中断使能位；CLINT 使用 MEIE、MTIE 和 MSIE 分别检查外部、定时器和软件中断。
mtvec	0x305	可写	寄存器宽度随 REG_DATA_WIDTH 变化	保存机器模式陷入基地址及模式位。
mcouteren	0x306	条件可写	仅低 3 位有效	支持 S 模式或 U 模式时保存 CY、TM 和 IR；两种模式均未启用时固定返回零。
mcountinhibit	0x320	可写	仅低 3 位有效	保存 CY、TM 和 IR。CY 禁止 mcycle 自增，IR 禁止 minstret 自增，TM 可读写但不禁止本设计的时间计数。
mscratch	0x340	可写	寄存器宽度随 REG_DATA_WIDTH 变化	保存机器模式软件临时数据。

寄存器名称	地址	访问属性	RV32 与 RV64 行为	功能描述
mepc	0x341	可写	寄存器宽度随 REG_DATA_WIDTH 变化	保存陷入返回地址。同步异常保存异常指令地址，异步中断保存顺序地址或已解析控制转移目标。
mcause	0x342	可写	最高位位置随 REG_DATA_WIDTH 变化	最高位区分异步中断和同步异常，低位保存原因编号。
mtval	0x343	可写	寄存器宽度随 REG_DATA_WIDTH 变化	非法指令异常保存指令内容；本设计对取指、加载和存储地址不对齐异常写零。
mip	0x344	可写	寄存器宽度随 REG_DATA_WIDTH 变化	保存软件写入值，不直接组合反映外部中断、定时器中断或软件中断输入。
mcycle	0xB00	可写	RV32 为低 32 位，RV64 为完整 64 位	在 CY=0 时逐周期加一；写请求覆盖本周期自增结果。
minstret	0xB02	可写	RV32 为低 32 位，RV64 为完整 64 位	在 inst_valid_i=1 且 IR=0 时加一；该输入表示发往执行级的有效指令，不是 WBU 完成事件。
mcycleh	0xB80	RV32 可写，RV64 读零	RV32 为 mcycle 高 32 位	RV32 下保存周期计数高半部分，RV64 下不生成独立高半寄存器。

寄存器名称	地址	访问属性	RV32 与 RV64 行为	功能描述
minstreth	0xB82	RV32 可写， RV64 读零	RV32 为 minstret 高 32 位	RV32 下保存指令计数高半部分，RV64 下不生成独立高半寄存器。
cycle	0xC00	只读	RV32 为低 32 位，RV64 为完整 64 位	mcycle 的只读别名。
time	0xC01	只读	RV32 为独立时间计数低 32 位，RV64 读取 mcycle	RV32 下逐周期自增且不受 TM 影响；RV64 下与周期计数相同。
cycleh	0xC80	只读	RV32 读取 mcycleh，RV64 读零	周期计数高半只读地址。
timeh	0xC81	只读	RV32 读取独立时间计数高 32 位，RV64 读零	时间计数高半只读地址。

表 64 CSR 寄存器地址与功能

执行单元与 CLINT 可以同时访问 CSR。可写寄存器的写使能由低 12 位地址译码产生；当两个写端口在同一周期访问不同寄存器时，两项写操作可以同时生效；访问相同寄存器时，执行单元的写入数据优先。执行单元读端口的选择优先级依次为本周期执行单元写数据、本周期 CLINT 写数据、上一周期执行单元写数据和寄存器当前值。上一周期写入信息保存地址、数据和有效位，用于解决写回寄存器边界造成的相邻指令读写冲突。CLINT 通用读端口仅对本周期 CLINT 写入提供前递；mtvec、mepc、mstatus 和 mie 另有直接输出，不依赖通用读地址。

mcycle 与 minstret 均允许软件写入。未发生写入时，mcycle 根据 mcountinhibit.CY 决定是否自增，minstret 根据 inst_valid_i 和 mcountinhibit.IR 决定是否自增。RV32 通过高、低两个 32 位寄存器形成 64 位计数值，低半溢出时向高半进位；若软件写高半的同一周期恰好满足低半溢出条件，写入高半的数据会同时加一。RV64 直接使用完整数据宽度计数器。RV32 软件读取 64 位计数值时，应采用先读高半、再读低半、最后复读高半的一致性读取方法。

从流水线控制关系看，CSR 指令先读取旧值并形成新值，再由 WBU 分别写入 CSR 和可选的 GPR 目的寄存器。陷入控制通过独立写端口依次改写 mepc、mstatus、可选的 mtval 和 mcause。CSR 模块只保存状态并提供读写冲突处理，不直接产生流水线清除或取指地址控制。

5.7.1 控制状态寄存器读选择预译码

执行单元侧同时保留完整 CSR 地址和紧凑读选择值。完整地址始终用于写入、写读前递比较以及预译码关闭时的读选择；紧凑选择值仅在读选择预译码打开时选择寄存器数据。相关配置与接口如下表所示。

名称	类型或方向	位宽	功能描述
ENABLE_CSR_READ_PREDECODE	编译配置	1	取值为 0 时按 raddr_i[11:0] 读取，取值为 1 时按 rsel_i 读取；默认值为 0。
csr_read_sel_o	Dispatch 输出	CSR_READ_SEL_WIDTH	根据指令中的 12 位 CSR 地址形成紧凑读选择值。
rsel_i	CSR 输入	CSR_READ_SEL_WIDTH	预译码打开时选择执行单元读数据。
raddr_i	CSR 输入	BUS_ADDR_WIDTH	提供完整 CSR 地址，并参与本周期和上一周期写入的地址比较。
data_o	CSR 输出	REG_DATA_WIDTH	输出执行单元侧 CSR 读数据。

表 65 CSR 读选择预译码配置与接口

读选择预译码把 12 位地址译码移至发射寄存器之前，使执行级只保留紧凑选择器。打开与关闭两种配置使用相同的写读前递优先级，CLINT 读端口也始终使用完整地址，因此该配置不改变 CSR 可见行为。

5.8 中断与异常处理系统

中断与异常处理系统由核心陷入控制、CLINT（核心本地中断控制器）、PLIC（平台级中断控制器）和流水线控制共同构成。同步异常包括 ecall、ebreak、非法指令以及取指、加载和存储地址不对齐；异步中断包括机器模式外部中断、定时器中断和软件中断。陷入控制保存返回地址与原因，分拍写入 CSR，并向取指单元给出陷入目标或 mret 返回地址。

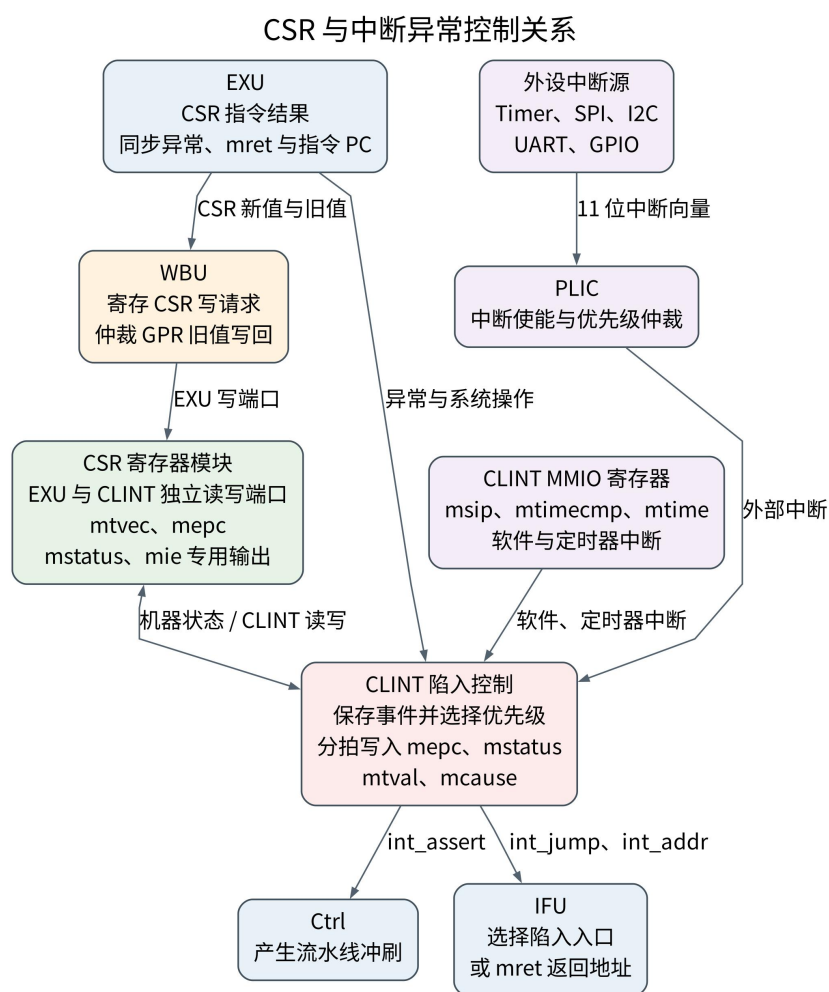


图 16 CSR 与中断异常控制关系

5.8.1 CLINT

CLINT 包含核心陷入状态机和 AXI4-Lite MMIO 定时器与软件中断寄存器组。核心陷入状态机接收与 Dispatch 输出对齐的指令地址、指令长度和有效状态，接收执行级同步异常与控制转移结果，并根据 mstatus、mie、原子操作状态和存储事务状态决定陷入时刻。MMIO 寄存器组保存 msip、mtimecmp 和 mtime，只向陷入状态机提供软件中断和定时器中断请求。

核心陷入状态机的参数如下表所示。

参数名	默认值	有效取值	功能描述
-----	-----	------	------

参数名	默认值	有效取值	功能描述
PIPELINE_STAGES	5	3、4、5	选择陷入目标跳转拍。四级流水线先保存目标地址，下一拍发出跳转有效；三级和五级流水线在待处理事件获准推进时发出跳转有效。

表 66 CLINT 参数

CLINT 端口如下表所示。

端口名	方向	位宽	有效条件与功能
clk	输入	1	核心陷入状态机及内部 MMIO 寄存器组的实际工作时钟。
rst_n	输入	1	低电平有效复位；核心陷入状态采用异步复位，MMIO 寄存器组采用同步复位。
inst_addr_i	输入	INST_ADDR_WIDTH	与当前待判断指令对齐的指令地址。
inst_data_i	输入	REG_DATA_WIDTH	当前指令内容，非法指令异常时写入 mtval。
inst_valid_i	输入	1	当前指令可参与同步异常与异步中断采样。取指地址不对齐异常不依赖该信号。
inst_len_2_i	输入	1	当前指令长度为 2 字节；异步中断未伴随已解析控制转移时，据此选择 mepc=inst_addr_i+2 或 mepc=inst_addr_i+4。
jump_flag_i	输入	1	当前指令存在已解析控制转移目标，异步中断返回地址应采用 jump_addr_i。
jump_addr_i	输入	INST_ADDR_WIDTH	已解析控制转移目标地址。

端口名	方向	位宽	有效条件与功能
atom_opt_busy_i	输入	1	原子长指令锁定或未完成存储事务存在；高电平时不得开始 CSR 陷入写入序列。
sys_op_ecall_i	输入	1	当前指令触发机器模式环境调用异常。
sys_op_ebreak_i	输入	1	当前指令触发断点异常。
sys_op_mret_i	输入	1	当前指令请求机器模式陷入返回。
illegal_inst_i	输入	1	当前指令触发非法指令异常。
misaligned_load_i	输入	1	当前指令触发加载地址不对齐异常。
misaligned_store_i	输入	1	当前指令触发存储地址不对齐异常。
misaligned_fetch_i	输入	1	取指目标地址不对齐异常。
ext_int_req_i	输入	1	PLIC 输出的机器模式外部中断请求。
stall_flag_i	输入	CU_BUS_WIDTH	兼容保留输入，本设计的陷入控制逻辑不读取该信号。
data_i	输入	REG_DATA_WIDTH	兼容保留的 CSR 读数据输入，本设计的陷入控制逻辑不读取该信号。
csr_mtvec	输入	REG_DATA_WIDTH	机器模式陷入基地址与模式。
csr_mepc	输入	REG_DATA_WIDTH	mret 使用的机器模式返回地址。
csr_mstatus	输入	REG_DATA_WIDTH	提供全局中断使能位 MIE 和陷入前中断使能位 MPIE。
csr_mie	输入	REG_DATA_WIDTH	提供机器外部、定时器和软件中断的分别使能位。
exu_stall_i	输入	1	执行级暂停；高电平时不采样新的同步异常或异步中断。

端口名	方向	位宽	有效条件与功能
we_o	输出	1	CLINT CSR 写请求有效。
waddr_o	输出	BUS_ADDR_WIDTH	CLINT CSR 写地址。
raddr_o	输出	BUS_ADDR_WIDTH	CLINT CSR 通用读地址，本设计固定输出零。
data_o	输出	REG_DATA_WIDTH	CLINT CSR 写数据。
int_addr_o	输出	INST_ADDR_WIDTH	陷入目标地址或 mret 返回地址。
int_jump_o	输出	1	int_addr_o 有效，取指单元应采用该地址。
int_assert_o	输出	1	已保存待处理事件，流水线控制应清除旧指令状态并停止继续发射。
S_AXI_ACLK	输入	1	外层兼容时钟端口；内部 MMIO 寄存器组实际使用 clk。
S_AXI_ARESETN	输入	1	外层兼容复位端口；内部 MMIO 寄存器组实际使用 rst_n。
S_AXI_AWADDR	输入	CLINT_AXI_ADDR_WIDTH	AXI4-Lite 写地址。
S_AXI_AWPROT	输入	3	AXI4-Lite 写访问属性，本设计不使用。
S_AXI_AWVALID	输入	1	AXI4-Lite 写地址有效。
S_AXI_AWREADY	输出	1	AXI4-Lite 写地址接收就绪。
S_AXI_WDATA	输入	CLINT_AXI_DATA_WIDTH	AXI4-Lite 写数据。
S_AXI_WSTRB	输入	CLINT_AXI_DATA_WIDTH/8	AXI4-Lite 写字节使能。
S_AXI_WVALID	输入	1	AXI4-Lite 写数据有效。
S_AXI_WREADY	输出	1	AXI4-Lite 写数据接收就绪。

端口名	方向	位宽	有效条件与功能
S_AXI_BRESP	输出	2	AXI4-Lite 写响应，正常完成返回 OKAY。
S_AXI_BVALID	输出	1	AXI4-Lite 写响应有效。
S_AXI_BREADY	输入	1	AXI4-Lite 写响应接收就绪。
S_AXI_ARADDR	输入	CLINT_AXI_ADDR_WIDTH	AXI4-Lite 读地址。
S_AXI_ARPROT	输入	3	AXI4-Lite 读访问属性，本设计不使用。
S_AXI_ARVALID	输入	1	AXI4-Lite 读地址有效。
S_AXI_ARREADY	输出	1	AXI4-Lite 读地址接收就绪。
S_AXI_RDATA	输出	CLINT_AXI_DATA_WIDTH	AXI4-Lite 读数据。
S_AXI_RRESP	输出	2	AXI4-Lite 读响应，正常完成返回 OKAY。
S_AXI_RVALID	输出	1	AXI4-Lite 读数据有效。
S_AXI_RREADY	输入	1	AXI4-Lite 读数据接收就绪。

表 67 CLINT 输入输出端口

陷入状态机在空闲状态保存三个地址：saved_inst_pc 保存当前指令地址，saved_next_pc 保存当前指令之后的顺序地址，saved_jump_pc 保存已解析控制转移目标。inst_len_2_i 与 inst_addr_i 同拍对齐；其值为一时，顺序地址在当前指令地址基础上加 2，否则加 4。同步异常的 mepc 使用 saved_inst_pc，使 mret 返回后可重新执行或由软件处理异常指令；异步中断的 mepc 在 jump_flag_i=1 时使用 saved_jump_pc，否则使用 saved_next_pc。因此 C 扩展打开时，inst_len_2_i 是保证 16 位指令之后中断返回地址正确的必要输入；C 扩展关闭时该信号恒为零。

陷入状态机及其 CSR 写入次序如下表所示。

状态	进入条件	主要操作	下一状态
----	------	------	------

状态	进入条件	主要操作	下一状态
S_INT_IDLE	复位后的默认状态	采样指令地址、顺序地址、控制转移目标和异常原因；发现有效事件后保存待处理状态并置位 <code>int_assert_o</code> 。	<code>atom_opt_busy_i=0</code> 时，陷入进入 S_INT_MEPC， <code>mret</code> 进入 S_INT_MSTATUS_MRET；否则保持。
S_INT_MEPC	待处理陷入获准推进	写 <code>mepc</code> 。同步异常写当前指令地址，异步中断写顺序地址或已解析控制转移目标。	S_INT_MSTATUS
S_INT_MSTATUS	<code>mepc</code> 写请求已经发出	写 <code>mstatus</code> ，将 MIE 复制到 MPIE 并清零 MIE。	需要写 <code>mtval</code> 时进入 S_INT_MTVAl，否则进入 S_INT_MCAUSE。
S_INT_MTVAl	非法指令或地址不对齐异常有效	非法指令写入指令内容，取指、加载和存储地址不对齐异常写零。	S_INT_MCAUSE
S_INT_MCAUSE	前述陷入状态写入完成	写 <code>mcause</code> ；同步异常写普通原因编号，异步中断置最高位并写入原因编号。	S_INT_IDLE
S_INT_MSTATUS_MRET	<code>mret</code> 获准推进	写 <code>mstatus</code> ，将 MPIE 恢复到 MIE 并把 MPIE 置一。	S_INT_IDLE

表 68 CLINT 陷入状态机

同步异常原因的选择顺序为环境调用、断点、取指地址不对齐、加载地址不对齐、存储地址不对齐和非法指令；异步中断的选择顺序为外部中断、定时器中断和软件中断。同步异常与异步中断同时有效时，同步异常优先。除取指地址不对齐外，其余同步异常要求 `inst_valid_i=1` 且 `exu_stall_i=0`；异步中断还要求 `mstatus.MIE=1`、`mie` 对应位为一、`atom_opt_busy_i=0`。同步异常与 `mret` 可以在原子操作或存储事务未完成时先保存，CSR 写入序列必须等待 `atom_opt_busy_i` 清零后开始。

陷入目标计算在待处理事件获准推进时进行。该周期若实时异步中断请求仍然有效且 `mtvec[0]=1`，目标地址为清除低两位后的 `mtvec` 加上当前原因编号乘 4；否则直接采用完整的 `mtvec` 值。同步异常与异步中断的原因类型已经在待处理状态中保存，但向量模式选择使用的是目标计算周期的实时异步请求

条件，因此软件应在处理期间保持中断请求稳定。mret 直接采用 mepc。三级和五级流水线在待处理事件获准推进时给出目标地址与 int_jump_o；四级流水线先保存目标地址，再在后续陷入状态或 mret 状态置位 int_jump_o。

定时器与软件中断寄存器组的有效参数如下表所示。

参数名	默认值	本设计取值	功能描述
C_S_AXI_DATA_WIDTH	32	CLINT_AXI_DATA_WIDTH	AXI4-Lite 数据位宽，与系统数据总线位宽一致。
C_S_AXI_ADDR_WIDTH	16	CLINT_AXI_ADDR_WIDTH	AXI4-Lite 地址位宽，覆盖 64 KiB 的 CLINT 地址区域。

表 69 CLINT MMIO 寄存器组参数

写通道先接收并保存 AW 地址，再开放 W 数据接收；W 数据握手后保存写数据与字节使能，并在寄存器写入完成后返回 OKAY。因此写地址必须先于写数据被接收。读通道在 AR 握手时采样寄存器值并置位 R 通道有效，直到上游接收后才重新开放 AR。mtime 复位为零并逐周期加一，mtimecmp 复位为全一，msip 复位为零。timer_irq 在 mtime 大于或等于 mtimecmp 时为一，soft_irq 直接取 msip[0]。

CLINT 基地址为 0x0200_0000，寄存器地址偏移如下表所示。

地址偏移	寄存器名称	位宽	访问属性	复位值与功能描述
0x0000	msip	1 位有效	读写	复位为零；写一产生机器模式软件中断，写零清除。仅地址最低字节的第 0 位有效。
0x4000	mtimecmp_lo	32	读写	复位为全一；保存定时比较值低 32 位。
0x4004	mtimecmp_hi	32	读写	复位为全一；保存定时比较值高 32 位。
0xBFF8	mtime_lo	32	读写	复位为零；保存自由运行时间值低 32 位。

地址偏移	寄存器名称	位宽	访问属性	复位值与功能描述
0xBFFC	mtime_hi	32	读写	复位为零；保存自由运行时间值高 32 位。

表 70 CLINT MMIO 寄存器地址分配

每个 32 位寄存器均按 WSTRB 逐字节写入。数据总线为 32 位时，高、低半分别通过相邻地址访问；数据总线不小于 64 位时，在低半地址写入可依据高、低字节使能同时改写 64 位寄存器的两个半部，也可在高半地址单独访问高 32 位。读操作仍按地址选择一个 32 位半部，并将该半部放置在对应的 32 位总线位置。

5.8.2 PLIC

平台级中断控制器（PLIC）汇集 11 个外设中断源，按使能状态和 8 位优先级选择一个中断源，并向 CLINT 输出机器模式外部中断请求。中断源编号 0 至 3 对应 Timer 的四路事件，编号 4 对应 SPI，编号 5 和 6 对应 I2C0 与 I2C1，编号 7 和 8 对应 UART0 与 UART1，编号 9 和 10 对应 GPIO0 与 GPIO1。关闭某个外设时，相应输入固定为零。

PLIC 由 AXI4-Lite 总线接口、寄存器组和流水优先级仲裁器组成。总线接口接收软件配置访问；寄存器组保存每个中断源的使能位、优先级、处理地址和处理参数；仲裁器把 11 个输入补齐到 16 项，通过四级寄存比较选择优先级最高的有效编号。相同优先级下选择编号较小的中断源。

PLIC 的外部总线参数和内部固定配置如下表所示。C_S_AXI_DATA_WIDTH 与 C_S_AXI_ADDR_WIDTH 是 PLIC 顶层参数；中断源数量由系统常量确定，处理地址表和处理参数表的基地址是 PLIC 寄存器组参数。

名称	类别	默认值	本设计取值	功能描述
----	----	-----	-------	------

名称	类别	默认值	本设计取值	功能描述
C_S_AXI_DATA_WIDTH	PLIC 顶层参数	32	BUS_DATA_WIDTH	PLIC 外部 AXI4-Lite 数据位宽。内部寄存器访问数据宽度为 32 位，系统数据总线较宽时使用低 32 位并将读数据零扩展。
C_S_AXI_ADDR_WIDTH	PLIC 顶层参数	16	16	PLIC 外部 AXI4-Lite 地址位宽，覆盖 64 KiB 地址区域。
PLIC_NUM_SOURCES	系统常量	11	11	PLIC 寄存器组的中断源数量，并作为优先级仲裁器的参数值。
PLIC_INT_VECTORABLE_ADDR	PLIC 寄存器组参数	0x2000	0x2000	每源处理地址表的基地址。
PLIC_INT_OBJECTS_ADDR	PLIC 寄存器组参数	0x3000	0x3000	每源处理参数表的基地址。

表 71 PLIC 参数

PLIC 对外端口如下表所示。

端口名	方向	位宽	有效条件与功能
S_AXI_ACLK	输入	1	AXI4-Lite 接口、寄存器组和优先级仲裁器的时钟。
S_AXI_ARESETN	输入	1	低电平有效复位；总线接口采用同步复位，寄存器组和优先级仲裁器采用异步复位。
S_AXI_AWADDR	输入	C_S_AXI_ADDR_WIDTH	AXI4-Lite 写地址。
S_AXI_AWPROT	输入	3	AXI4-Lite 写访问属性，本设计不使用。
S_AXI_AWVALID	输入	1	AXI4-Lite 写地址有效。
S_AXI_AWREADY	输出	1	AXI4-Lite 写地址接收就绪。

端口名	方向	位宽	有效条件与功能
S_AXI_WDATA	输入	C_S_AXI_DATA_WIDTH	AXI4-Lite 写数据；PLIC 寄存器组使用低 32 位。
S_AXI_WSTRB	输入	C_S_AXI_DATA_WIDTH/8	AXI4-Lite 写字节使能；PLIC 寄存器组使用低 4 位。
S_AXI_WVALID	输入	1	AXI4-Lite 写数据有效。
S_AXI_WREADY	输出	1	AXI4-Lite 写数据接收就绪。
S_AXI_BRESP	输出	2	AXI4-Lite 写响应，正常完成返回 OKAY。
S_AXI_BVALID	输出	1	AXI4-Lite 写响应有效。
S_AXI_BREADY	输入	1	AXI4-Lite 写响应接收就绪。
S_AXI_ARADDR	输入	C_S_AXI_ADDR_WIDTH	AXI4-Lite 读地址。
S_AXI_ARPROT	输入	3	AXI4-Lite 读访问属性，本设计不使用。
S_AXI_ARVALID	输入	1	AXI4-Lite 读地址有效。
S_AXI_ARREADY	输出	1	AXI4-Lite 读地址接收就绪。
S_AXI_RDATA	输出	C_S_AXI_DATA_WIDTH	AXI4-Lite 读数据；内部 32 位读数据按外部数据宽度零扩展。
S_AXI_RRESP	输出	2	AXI4-Lite 读响应，正常完成返回 OKAY。
S_AXI_RVALID	输出	1	AXI4-Lite 读数据有效。
S_AXI_RREADY	输入	1	AXI4-Lite 读数据接收就绪。
irq_sources	输入	11	外设中断源输入，位号与中断源编号一致。

端口名	方向	位宽	有效条件与功能
irq_valid	输出	1	至少一个已使能中断源有效且仲裁结果已经到达输出寄存器。

表 72 PLIC 对外输入输出端口

PLIC 寄存器组与优先级仲裁器之间的内部端口如下表所示。

端口名	方向	位宽	有效条件与功能
clk	输入	1	PLIC 寄存器组与仲裁状态时钟。
rst_n	输入	1	低电平有效异步复位。
waddr	输入	PLIC_AXI_ADDR_WIDTH	寄存器组写地址。
wdata	输入	PLIC_AXI_DATA_WIDTH	寄存器组写数据，宽度为 32 位。
wstrb	输入	4	寄存器组写字节使能。
wen	输入	1	寄存器组写请求有效。
raddr	输入	PLIC_AXI_ADDR_WIDTH	寄存器组读地址。
rdata	输出	PLIC_AXI_DATA_WIDTH	寄存器组合读数据，宽度为 32 位。
irq_sources	输入	PLIC_NUM_SOURCES	外设中断源输入。
irq_valid	输出	1	已寄存的仲裁结果有效。

表 73 PLIC 寄存器组端口

优先级仲裁器端口如下表所示。

端口名	方向	位宽	有效条件与功能
clk	输入	1	各级优先级比较寄存器的时钟。
rst_n	输入	1	低电平有效异步复位，清零各级有效状态与编号。
pri_in	输入	PLIC_NUM_SOURCES×8	每个中断源的 8 位优先级数组。
valid_in	输入	PLIC_NUM_SOURCES	已寄存中断请求与使能位相与后的有效状态。

端口名	方向	位宽	有效条件与功能
find_max_id_out	输出	$\$clog2(PLIC_NUM_SOURCES)$	经过流水比较后选出的中断源编号。
find_max_valid_out	输出	1	find_max_id_out 有效，与编号经过相同级数的寄存器。

表 74 PLIC 优先级仲裁器端口

AXI4-Lite 写接口允许 AW 与 W 同拍握手，也允许先接收 AW 后等待 W。地址来自本周期有效的 AW 地址或先前保存的 AW 地址，因此上游必须保证写地址不晚于写数据。总线接口将写地址、低 32 位写数据、低 4 位字节使能和写有效寄存一拍后送入寄存器组。读接口在 AR 握手时保存地址，随后将组合读数据保存到 R 通道数据寄存器。

每个外设中断输入先寄存一拍，再与 int_en 按位与后进入优先级仲裁器。int_pri 的取值只决定有效中断源之间的先后次序，优先级零仍可参与仲裁；禁止某个中断源必须清除对应的 int_en 位。优先级仲裁器逐级比较 8 位优先级，较大者进入下一级，优先级相同时保留编号较小者。仲裁有效状态经过相同级数的寄存器传递，保证编号与有效状态对齐。

仲裁器输出的中断编号和有效状态在 PLIC 寄存器组中再保存一拍。有效编号用于读取对应的处理地址与处理参数，所读内容先进入中间寄存器，下一拍分别写入 mvec 和 marg。没有有效中断时，irq_valid 清零，mvec 和 marg 保持上次数值；软件必须先依据外部中断状态进入处理程序，再读取二者。PLIC 不保存独立的挂起、领取或完成状态，外设中断请求应保持到设备处理程序清除其外设状态为止。

PLIC 基地址为 0x0C00_0000，寄存器地址偏移如下表所示。

寄存器名称	地址偏移	大小	访问属性	功能描述
INT_EN	0x0000	32 位，低 11 位有效	读写	每位控制相应中断源是否参与仲裁，按 WSTRB 逐字节写入。
INT_MVEC	0x0100	32 位	只读	当前已选中中断源的处理地址；没有有效中断时保持上次数值。

寄存器名称	地址偏移	大小	访问属性	功能描述
INT_MARG	0x0104	32 位	只读	当前已选中中断源的处理参数；没有有效中断时保持上一次数值。
INT_PRI	0x1000 至 0x100A	每源 8 位	读写	11 个中断源的优先级数组；一个对齐的 32 位访问最多读写连续四项，写入受 WSTRB 控制。
INT_VECTABLE	0x2000 至 0x2028	每源 32 位	读写，但第 0 项读出例外	11 个中断源的处理地址数组，条目地址为 $0x2000+4\times\text{编号}$ 。写入忽略 WSTRB 并改写完整 32 位。地址 0x2000 的读操作优先返回当前已选中的中断编号，因而不能读回第 0 项。
INT_OBJS	0x3000 至 0x3028	每源 32 位	读写	11 个中断源的处理参数数组，条目地址为 $0x3000+4\times\text{编号}$ 。写入忽略 WSTRB 并改写完整 32 位。

表 75 PLIC 寄存器地址分配

处理地址数组和处理参数数组均使用地址位 [5:2] 选择 11 个条目。地址范围比较使用完整高位，只有位于相应基地址之后的 11 个四字节条目能够访问。INT_VECTABLE 基地址同时承担当前中断编号读出功能，软件分派通常读取 INT_MVEC 和 INT_MARG，不依赖第 0 项处理地址的读回值。

5.8.3 中断处理原子性保障

核心将原子长指令锁定状态和未完成存储事务状态合并为 `atom_opt_busy_i`。异步中断只在该信号为零时采样；同步异常和 `mret` 可以先保存为待处理状态，

但 CSR 写入与取指地址改变必须等待该信号清零。等待期间 `int_assert_o` 保持有效，使流水线停止接收新的架构操作，同时保存的指令地址、指令长度、控制转移目标和异常原因保持不变。

这一控制保证 LR/SC、原子存储器操作、非对齐拆分访问和已经发出的存储事务不会在中间步骤进入陷入处理。同步异常与异步中断同时出现时，原因选择优先采用同步异常，使 `mepc`、`mcause` 和异常指令保持对应。

不同流水线配置采用不同的附加保持方法。五级流水线在增强分支预测打开时先保存 `int_assert_o`，下一拍清除流水状态并阻止继续发射，直到 `int_jump_o` 到达；三级流水线保持陷入处理状态，普通陷入等待 `mcause` 写入完成，`mret` 等待 `mstatus` 写入完成；四级流水线由 CLINT 延后一拍给出 `int_jump_o`。这些控制使处理程序开始执行时，所需 CSR 状态已经有效。### 5.9 AXI 互连模块

AXI 互连模块（`axi_interconnect`）连接两个主机端口与六个地址目标，采用 AXI4-Lite 协议完成主机仲裁、地址解码、通道选择和未完成事务跟踪。M0 是 IFU 只读取指端口，只识别 IMEM；M1 是 LSU 读写端口，识别 IMEM、DMEM、DM、APB 外设、CLINT 和 PLIC。DM 区域在当前 SoC 顶层未连接从设备，其就绪与响应有效固定为零，因此命中该区域的访问会保持等待，不能作为可正常访问的调试存储器使用。

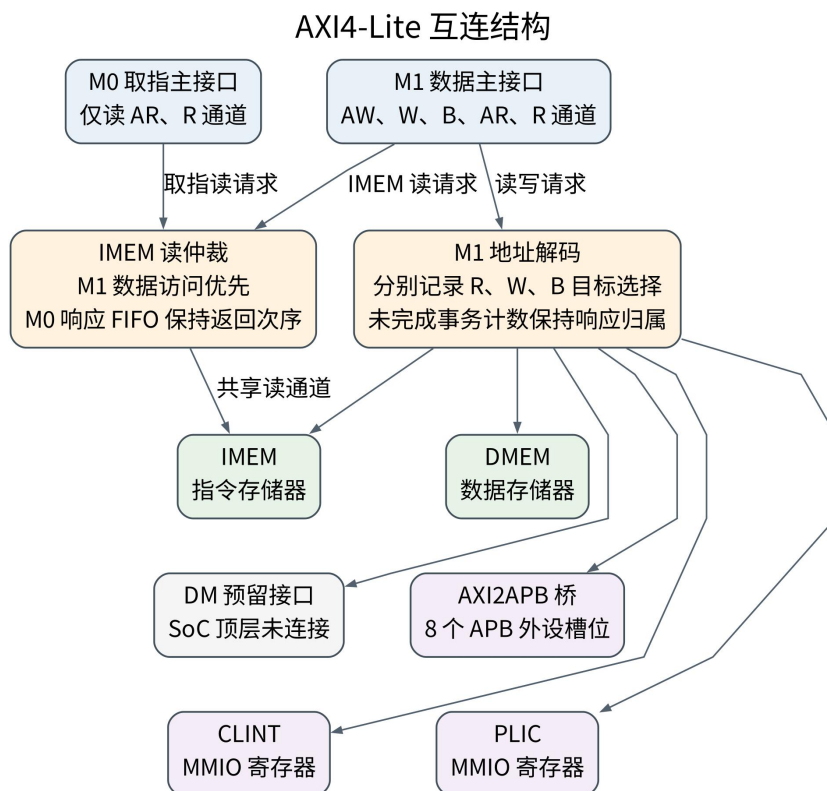


图 17 AXI4-Lite 互连结构

模块的参数定义如下表所示：

参数名称	默认值	描述
C_AXI_ID_WIDTH	2	兼容保留参数，当前 AXI4-Lite 接口不输出 ID 信号。
C_AXI_DATA_WIDTH	32	AXI4-Lite 数据通道位宽。
C_AXI_ADDR_WIDTH	32	AXI4-Lite 地址通道位宽。

表 76 AXI 互连模块参数定义表

除时钟和复位信号外，模块具有 2 个 AXI4-Lite 主接口输入，以及面向 APB 桥、CLINT、PLIC、IMEM、DMEM 和 DM 的 AXI4-Lite 目标侧输出接口；在当前 SoC 顶层中 DM 接口预留未连接。

M0 主接口专门用于指令取指，仅包含 AXI4-Lite 读地址和读数据通道；M1 主接口用于数据访问和外设控制，包含 AW/W/B/AR/R 五个通道。二者的定义分别与章节“CPU 顶层接口”中的 M0_AXI 和 M1_AXI 一致。OM0、OM1、OM2 分别连接 AXI2APB 桥、CLINT 和 PLIC。

此外，IMEM_AXI、DMEM_AXI 和 DM_AXI 分别连接指令存储器 IMEM、数据存储器 DMEM 和调试模块预留端口，接口同样采用 AXI4-Lite 读写通道。

系统在单个 axi_interconnect 模块内完成两类逻辑：一类是 M0 与 M1 对 IMEM 读通道的仲裁，另一类是 M1 到各目标的地址解码与通道选择。IMEM、DMEM、CLINT、PLIC 和 APB 在 SoC 顶层连接实际从设备；DM 仅保留地址选择和接口端口。当前互连没有例化独立的 bus_trans_cnt，而是在模块内使用 cnt_nxt() 和寄存器实现相同的计数功能；bus_trans_cnt 是供其它环境复用的单元，不属于当前 SoC 互连实例层次。

主机仲裁：M0（仅读）和 M1（读写）共享 IMEM 读通道，M1 访问 IMEM 时优先级高于 M0；当 M1 无 IMEM 读请求且无活跃 IMEM 读事务时，M0 取指才能获得授权。互连为 M0 和 M1 维护独立未完成事务计数，用于决定读响应应返回给哪个主机，并避免取指与数据访问响应错配。

目标路由：M1 根据访问地址被并行解码到 IMEM、DMEM、DM、APB、CLINT 或 PLIC。读、写数据和写响应通道分别由 m1_slave_sel_r、m1_slave_sel_w 和 m1_slave_sel_b 记录当前服务目标，并配合每目标未完成事务计数器保证响应返回到正确通道。

5.9.1 地址空间分配与解码

地址空间采用 32 位架构，按功能区域划分为六个主要段，如下表所示：

地址范围	目标设备	大小	描述
------	------	----	----

地址范围	目标设备	大小	描述
0x0000_0000 - 0x0000_0FFF	DM	4 KB	调试模块预留空间； 当前 SoC 未连接从 设备，访问会持续等 待
0x0200_0000 - 0x0200_FFFF	CLINT	64 KB	核心本地中断控制器
0x0C00_0000 - 0x0C00_FFFF	PLIC	64 KB	平台级中断控制器
0x8000_0000 - 0x8001_FFFF	IMEM	128 KB	指令存储器从设备， 地址宽度由 ITCM_ADDR_WIDTH 参 数给出
0x8010_0000 - 0x8010_FFFF	DMEM	64 KB	数据存储器从设备， 地址宽度由 DTCM_ADDR_WIDTH 参 数给出
0x8400_0000 - 0x840F_FFFF	APB 外设	1 MB	AXI2APB 桥后端外 设空间

表 77 AXI 总线地址空间分配表

地址解码使用地址高位并行比较。以 IMEM 为例，is_m_imem_r 比较 AR ADDR[C_AXI_ADDR_WIDTH-1:IMEM_ADDR_WIDTH_L] 与 IMEM_BASE_ADDR_L 的同一高位切片；DMEM、APB、CLINT、PLIC 和 DM 使用各自地址宽度做同类比较。ITCM_ADDR_WIDTH 与 DTCM_ADDR_WIDTH 是保留的参数名，用于给出 IMEM 和 DMEM 的地址宽度，并不表示 CPU 顶层还有独立紧耦合存储器端口。解码结果直接参与 ARREADY、AWREADY 授权和目标通道 VALID 生成，使地址阶段已经确定后续响应来源。

5.9.2 主机多路复用器仲裁

读地址通道仲裁：M1 访问 IMEM 时享有优先权。M1 发起 IMEM 读请求时，m1_imem_ar_grant 置起；只有在 M1 无 IMEM 读请求且无活跃 IMEM 读事务时，M0 才能通过 m0_imem_ar_grant 访问 IMEM。该策略保证数据侧对指令存储区域的访问不会被取指流长期占用，同时允许取指在空闲时持续发起 AR。

读数据通道路由：互连基于未完成事务计数和选择寄存器决定响应去向。M0 侧带有深度 4 的取指响应 FIFO，保存 IMEM_AXI_RDATA/RRESP。当 M0 已经有未返回取指响应，而 M1 又需要读取 IMEM 时，互连先把 M0 响应缓存起来；只有 m0_rdata_all_cached 成立时，M1 才允许选择 IMEM R 通

道。这样可以避免 M0 未消费的指令响应与 M1 数据响应在同一从设备 R 通道上发生归属混淆。

写通道处理：只有 M1 支持写操作。互连根据 M1_AXI_AWADDR 解码目标，并把 AW、W 和 B 分别路由到 IMEM、DMEM、DM、APB、CLINT 或 PLIC。当前写数据目标选择使用 AW 握手结果建立 m1_select__w/b，避免 AW 尚未握手而 W 先被误计数导致计数器下溢。每个目标分别维护 W/B 未完成事务计数，AW 握手递增，W 或 B 握手递减。

M1 读通道在任一目标存在尚未返回的读事务时，只继续接受指向该目标的读地址，直至该目标的读响应全部完成后才允许选择其它目标。M1 写通道同时检查已经接收但尚未配对 W 数据的写地址和尚未返回 B 响应的写事务；只要其中一类状态仍然有效，新的 AW 只能指向同一目标。读通道和写通道分别保存选择状态，因此一笔读事务与一笔写事务可以同时访问不同目标。

5.9.3 目标端口路由

地址通道路由：读地址和写地址分别根据 M1_AXI_ARADDR/M1_AXI_AWADDR 并行判断目标区域，匹配后只向对应目标端口转发 VALID/ADDR/PROT。READY 由目标端口返回并参与 M1 侧 ARREADY/AWREADY 生成。若地址没有命中任何目标，M1 侧就绪信号不会被拉高，访问会停在主机侧等待。

数据通道选择：读、写数据和写响应三类通道分别维护 6 位独热选择寄存器，第 0 位到第 5 位对应 IMEM、DMEM、DM、APB、CLINT 和 PLIC。如果某个目标已有活跃事务，选择寄存器保持该目标；如果所有目标均无活跃事务，则根据新的地址授权分配通道。

选择寄存器更新在时序逻辑中使用下一拍活跃向量。若活跃向量为单热编码，则选择对应目标；若活跃向量为 0，则清空选择；若出现多个目标同时活跃，RTL 保持上一次选择，不主动切换。该实现牺牲了多目标并发读写的调度复杂度，换取较小选择逻辑和确定的响应归属。

5.9.4 未完成事务管理

事务计数器逻辑：当前 axi_interconnect 为各主机和目标通道维护 4 位普通回绕计数。cnt_nxt() 根据递增和递减的同拍组合返回加一、减一或保持；计数器没有饱和或非法减一保护，因此协议控制必须保证完成次数不超过已经接受的事务数。地址握手时计数递增，响应或数据握手时计数递减，计数是否非零直接驱动未完成事务向量和选择寄存器。

主机端事务跟踪：M0 配备 IMEM 读事务计数器；M1 为 IMEM、DME

M、DM、APB、CLINT 和 PLIC 分别维护读、写数据和写响应计数器。这些计数器的活跃状态直接控制 M0/M1 仲裁、M1 目标选择以及响应返回路径。

目标端事务跟踪：对 M1 可访问的六个目标分别跟踪 R/W/B 三类事务，读通道从 AR 握手到 R 握手，写数据通道从 AW 握手到 W 握手，写响应通道从 AW 握手到 B 握手。计数结果组成 active 向量，驱动 m1_slave_sel_r/w/b 状态更新。

事务信号定义：地址通道的事务启动必须同时满足 VALID、READY 和地址匹配三个条件；数据或响应通道完成必须满足 VALID、READY 和当前通道选择条件。由于当前接口不支持 Burst，事务跟踪不检查 LAST 信号。

5.9.5 协议转换与外设访问

系统提供三个 AXI4-Lite 输出接口，分别用于连接 AXI2APB 桥、CLINT 和 PLIC 控制器。由于 M1 本身采用单拍 AXI4-Lite 访问，互连主要负责地址命中后的信号转发和响应回传，不需要进行突发传输拆分。

AXI2APB 桥的参数如下表所示。RV64 配置可以把 AXI4-Lite 数据端设置为 64 位，但 APB 数据端固定为 32 位。桥依据地址低位从 AXI4-Lite 写数据中选择一个 32 位数据字，并将 APB 读数据放回 AXI4-Lite 读数据的对应位置。

参数名	默认值	功能说明
C_S_AXI_DATA_WIDTH	32	AXI4-Lite 数据位宽；RV64 配置可以取 64。
C_S_AXI_ADDR_WIDTH	20	AXI4-Lite 局部地址位宽。
C_APB_ADDR_WIDTH	20	APB 地址位宽。

表 78 AXI2APB 桥参数

AXI2APB 桥端口如下表所示。

端口名	方向	位宽	功能说明
S_AXI_ACLK	输入	1	AXI4-Lite 接口和桥内状态机时钟。
S_AXI_ARESETN	输入	1	低电平有效异步复位。
S_AXI_AWADDR	输入	C_S_AXI_ADDR_WIDTH	AXI4-Lite 写地址。
S_AXI_AWPROT	输入	3	写访问属性，本设计不使用。
S_AXI_AWVALID	输入	1	写地址有效。
S_AXI_AWREADY	输出	1	写地址接收就绪。

端口名	方向	位宽	功能说明
S_AXI_WDATA	输入	C_S_AXI_DATA_WIDTH	AXI4-Lite 写数据。
S_AXI_WSTRB	输入	C_S_AXI_DATA_WIDTH/8	写字节使能，本设计不传递至 APB。
S_AXI_WVALID	输入	1	写数据有效。
S_AXI_WREADY	输出	1	写数据接收就绪；系统必须保证写地址不晚于写数据。
S_AXI_BRESP	输出	2	AXI4-Lite 写响应，由 APB 错误状态形成。
S_AXI_BVALID	输出	1	写响应有效。
S_AXI_BREADY	输入	1	上游准备接收写响应。
S_AXI_ARADDR	输入	C_S_AXI_ADDR_WIDTH	AXI4-Lite 读地址。
S_AXI_ARPROT	输入	3	读访问属性，本设计不使用。
S_AXI_ARVALID	输入	1	读地址有效。
S_AXI_ARREADY	输出	1	读地址接收就绪。
S_AXI_RDATA	输出	C_S_AXI_DATA_WIDTH	APB 读数据按地址低位放入对应的 32 位位置，其余位为零。
S_AXI_RRESP	输出	2	AXI4-Lite 读响应，由 APB 错误状态形成。
S_AXI_RVALID	输出	1	读响应有效。
S_AXI_RREADY	输入	1	上游准备接收读响应。
PCLK	输出	1	APB 时钟，直接采用 S_AXI_ACLK。
PRESETn	输出	1	APB 低电平有效复位，直接采用 S_AXI_ARESETN。
PSEL	输出	8	八个 APB 设备槽的选择信号。
PENABLE	输出	1	APB 访问阶段指示。
PADDR	输出	C_APB_ADDR_WIDTH	APB 地址。
PWRITE	输出	1	APB 写访问指示。

端口名	方向	位宽	功能说明
PWDATA	输出	32	APB 写数据。
PRDATA	输入	8×32	八个 APB 设备槽的读数据。
PREADY	输入	8	八个 APB 设备槽的完成信号。
PSLVERR	输入	8	八个 APB 设备槽的错误信号。

表 79 AXI2APB 桥端口

PCLK 直接等于 S_AXI_ACLK，PRESETn 直接等于 S_AXI_ARESETN。读写共享 IDLE、SETUP 和 ACCESS 三态 APB 状态机；同拍同时具备读写请求时写请求优先，ACCESS 保持到被选外设 PREADY=1。

写路径支持 AW 先到或 AW 与 W 同拍；当前上游必须保证 AW 不晚于 W，因为 W 先到时桥不会保存写数据。桥按 AXI 数据总线内的地址低位选取 32 位写数据字，但 APB 侧没有 PSTRB 端口，S_AXI_WSTRB 和访问属性输入也未参与功能逻辑，因此外设寄存器写入不支持字节写使能。读路径在 AR 握手后保存地址，经 SETUP 和 ACCESS 两阶段访问，在 PREADY 有效时采样 PRDATA 并通过 AXI R 通道返回。

APB 设备选择由地址位 [14:12] 解码，8 个设备槽依次对应 Timer、SPI、I2C0、I2C1、UART0、UART1、GPIO0 和 GPIO1。APB 总窗口为 1 MB，但地址 [19:15] 不参与设备选择，因此首个 32 KB 内的八槽布局会在窗口高区重复。桥使用 PSEL[APB_DEV_COUNT-1:0] 选择设备，并用被选设备的 PREADY、PSLVERR 和 PRDATA 形成 AXI 返回。当前普通外设固定 PREADY=1、PSLVERR=0，只有桥的 SETUP 和 ACCESS 状态引入访问周期；关闭外设时 perip_top 同样返回 PRDATA=0、PREADY=1 和 PSLVERR=0。

外设访问不能参与 LSU L0 Cache 和 DMEM 快速旁路。APB 外设寄存器通常具有读副作用、写清除位或等待状态，不能被当作普通可缓存内存；因此互连地址命中 APB、CLINT、PLIC 或 DM 区域时，LSU 必须走普通 AXI 4-Lite 事务完成路径。软件侧则通过 volatile 寄存器封装保证编译器不合并或重排外设访问。

5.9.6 IMEM 与 DMEM AXI4-Lite 存储器从设备

IMEM 与 DMEM 均使用 AXI4-Lite 存储器从设备包装，内部由总线控制器和伪双端口 RAM 组成。普通配置下，IMEM 的地址宽度为 17 位、容量为 128 KiB，DMEM 的地址宽度为 16 位、容量为 64 KiB；两者的数据宽度均为 XLEN。存储阵列提供一拍同步读和逐字节同步写，控制器分别保存未完成的读请求、写地址和总线响应。

控制状态采用低电平有效同步复位。存储阵列、RAM 同步读寄存器和可选的读数据寄存器不复位，只有对应的有效状态在复位时清零。读响应与写响应固定为 OKAY，S_AXI_ARPROT 和 S_AXI_AWPROT 不参与功能控制。

主要参数如下表所示。

参数名	IMEM 配置	DMEM 配置	功能说明
ADDR_WIDTH	17	16	本地字节地址宽度，分别形成 128 KiB 和 64 KiB 存储空间。
DATA_WIDTH	XLEN	XLEN	AXI4-Lite 数据位宽和存储字宽。
INIT_MEM	0	0	是否根据初始化文件建立存储初值。
INIT_FILE	main_itcm.mem	main_dtcn.mem	初始化文件名；INIT_MEM=0 时不使用。
REGISTER_RDATA	0	DTCM_REGISTER_RDATA，默认 0	是否在 RAM 同步读输出后增加一级数据寄存器。

表 80 IMEM 与 DMEM AXI4-Lite 存储器从设备参数

IMEM 与 DMEM 使用相同的端口定义，仅地址与数据位宽由参数确定。

端口名	方向	位宽	功能说明
S_AXI_ACLK	输入	1	AXI4-Lite 接口与存储控制时钟。
S_AXI_ARESETN	输入	1	低电平有效同步复位，只复位控制状态。
S_AXI_AWADDR	输入	ADDR_WIDTH	写地址。
S_AXI_AWPROT	输入	3	写访问属性，本设计不使用。
S_AXI_AWVALID	输入	1	写地址有效。
S_AXI_AWREADY	输出	1	写地址存储可以接收新地址时有效。
S_AXI_WDATA	输入	DATA_WIDTH	写数据。
S_AXI_WSTRB	输入	DATA_WIDTH/8	逐字节写使能。
S_AXI_WVALID	输入	1	写数据有效。
S_AXI_WREADY	输出	1	已有可配对写地址，或本拍同时接收写地址时有效。

端口名	方向	位宽	功能说明
S_AXI_BRESP	输出	2	写响应，固定为 OKAY。
S_AXI_BVALID	输出	1	写响应有效。
S_AXI_BREADY	输入	1	主机准备接收写响应。
S_AXI_ARADDR	输入	ADDR_WIDTH	读地址。
S_AXI_ARPROT	输入	3	读访问属性，本设计不使用。
S_AXI_ARVALID	输入	1	读地址有效。
S_AXI_ARREADY	输出	1	未完成读请求少于 4 项时有效。
S_AXI_RDATA	输出	DATA_WIDTH	按 AXI4-Lite 时序返回的读数据。
S_AXI_RRESP	输出	2	读响应，固定为 OKAY。
S_AXI_RVALID	输出	1	读数据有效。
S_AXI_RREADY	输入	1	主机准备接收读数据。
S_AXI_RDATA_RAW	输出	DATA_WIDTH	RAM 读响应流水输出；启用 REGISTER_RDATA 时为附加寄存器输出。
S_AXI_RDATA_FIFO_HEAD	输出	DATA_WIDTH	读响应 FIFO 队首数据。
S_AXI_RDATA_FIFO_NONEMPTY	输出	1	读响应 FIFO 非空。

表 81 IMEM 与 DMEM AXI4-Lite 存储器从设备端口

写控制包含深度为 4 的写地址存储和写响应存储。AW 握手后，控制器按接收顺序保存写地址；W 通道只在写地址存储中已有有效地址，或本拍 AW 与 W 同时握手时接收写数据。每次 W 握手使用最早接收的写地址，并按 W STRB 更新对应字节，因此 AW 可以早于 W，也可以与 W 同拍，W 早于 AW 时不会被接收。RAM 写入完成后产生固定为 OKAY 的 B 响应；主机暂不接收响应时，返回状态保存在写响应存储中，并按写数据接收顺序返回。

IMEM 的读数据附加寄存器固定关闭。DMEM 由 DTCM_REGISTER_RDATA 参数选择是否在 RAM 同步读输出后增加一级寄存器，该参数默认关闭。两者的读地址准入均由 4 项未完成读计数限制，主机暂停接收时，深度为 4 的读响应 FIFO 按地址接收顺序保存返回数据。IMEM 的三个辅助读数据输出供 AXI4-Lite 互连选择直接返回数据、FIFO 队首数据或 RAM 响应；DMEM

保留相同端口，但当前互连不采用这些辅助输出。

5.10 外设模块

SoC 通过参数化 APB 外设子系统提供板级输入输出能力。外设子系统顶层集成 Timer、PWM、SPI、I2C、UART 和 GPIO，并在 SoC 顶层完成引脚选择和中断连接。

5.10.1 APB 外设子系统顶层与 APB 槽位

外设子系统顶层固定提供 8 个 APB 设备槽。各外设使能参数只决定是否生成相应功能；关闭外设时，地址槽仍固定返回 PRDATA=0、PREADY=1 和 PSLVERR=0，相应事件和引脚输出保持无效，因此软件访问关闭的地址槽可以正常完成并读回零。

外设子系统参数如下表所示。

参数名	默认值	SoC 默认配置	功能说明
APB_DEV_COUNT	8	8	APB 设备槽数量；当前端口和设备选择固定按 8 个槽组织。
APB_SLAVE_ADDR_WIDTH	12	20	APB 地址端口位宽；设备选择只使用地址位 [14:12]。
BUS_DATA_WIDTH	32	XLEN	兼容保留参数；当前 APB 数据端口固定为 32 位。
ENABLE_PERIP_TIMER	0	0	生成 Timer 和 PWM。
ENABLE_PERIP_SPI	0	0	生成 SPI 主机。
ENABLE_PERIP_I2C0	0	0	生成 I2C0 主机。
ENABLE_PERIP_I2C1	0	0	生成 I2C1 主机。
ENABLE_PERIP_UART0	1	1	生成独立引脚 UART0。
ENABLE_PERIP_UART1	0	0	生成经 GPIO0 选择的 UART1。
ENABLE_PERIP_GPIO0	1	1	生成 GPIO0 及专用外设引脚选择控制。

参数名	默认值	SoC 默认配置	功能说明
ENABLE_PERIP_GPIO1	0	0	生成 GPIO1。

表 82 APB 外设子系统参数

外设子系统端口如下表所示。

端口名	方向	位宽	功能说明
HCLK	输入	1	APB 外设工作时钟。
HRESETn	输入	1	低电平有效异步复位。
PADDR	输入	APB_SLAVE_ADDR_WIDTH	APB 地址。
PWDATA	输入	32	APB 写数据。
PWRITE	输入	1	APB 写访问指示。
PSEL	输入	APB_DEV_COUNT	各设备槽选择信号。
PENABLE	输入	1	APB 访问阶段指示。
PRDATA	输出	APB_DEV_COUNT×3 2	各设备槽的 32 位读数据。
PREADY	输出	APB_DEV_COUNT	各设备槽完成信号。
PSLVERR	输出	APB_DEV_COUNT	各设备槽错误信号。
uart0_rxd_i	输入	1	UART0 串行接收数据。
uart0_txd_o	输出	1	UART0 串行发送数据。
uart0_event_o	输出	1	UART0 中断事件。
uart1_event_o	输出	1	UART1 中断事件。
spi_events_o	输出	1	SPI FIFO 阈值事件。
i2c0_interrupt_o	输出	1	I2C0 中断请求。
i2c1_interrupt_o	输出	1	I2C1 中断请求。
gpio0_in_i	输入	32	GPIO0 外部输入。
gpio0_out_o	输出	32	GPIO0 普通功能或专用外设输出。
gpio0_dir_o	输出	32	GPIO0 内部方向控制，高电平表示输出驱动。
gpio0_interrupt_o	输出	1	GPIO0 中断请求。
gpio1_in_i	输入	32	GPIO1 外部输入。
gpio1_out_o	输出	32	GPIO1 输出。
gpio1_dir_o	输出	32	GPIO1 内部方向控制，高电平表示输出驱动。

端口名	方向	位宽	功能说明
gpio1_interrupt_o	输出	1	GPIO1 中断请求。
dft_cg_enable_i	输入	1	Timer 时钟门控测试允许；SoC 默认配置固定为 0。
low_speed_clk_i	输入	1	Timer 低速计数事件输入。
timer_events_o	输出	4	四路 Timer 事件。

表 83 APB 外设子系统端口

外设访问依次经过 LSU 的 M1 AXI4-Lite 接口、AXI4-Lite 互连、AXI2APB 桥和外设子系统。AXI2APB 桥将 APB 地址译码为 PSEL，外设子系统把 PADDR、PWRITE、PDATA 和 PENABLE 送入被选设备，再返回该设备的 PRDATA、PREADY 和 PSLVERR。APB 外设访问周期与 DMEM 访问周期不同，因此 LSU 不把 APB 加载作为可提前完成的 DMEM 加载，LSU L0 Cache 和非对齐数据拆分控制也不改变 APB 地址选择。

下表给出 APB 窗口首个 32 KiB 区域内的设备地址。设备选择只检查地址位 [14:12]，因此同一分配方式以 32 KiB 为周期重复至 APB 窗口末端。

PSEL 位	外设	首个地址范围	大小	默认状态
0	Timer/PWM	0x8400_0000 - 0x8400_0FFF	4 KiB	关闭
1	SPI	0x8400_1000 - 0x8400_1FFF	4 KiB	关闭
2	I2C0	0x8400_2000 - 0x8400_2FFF	4 KiB	关闭
3	I2C1	0x8400_3000 - 0x8400_3FFF	4 KiB	关闭
4	UART0	0x8400_4000 - 0x8400_4FFF	4 KiB	启用
5	UART1	0x8400_5000 - 0x8400_5FFF	4 KiB	关闭
6	GPIO0	0x8400_6000 - 0x8400_6FFF	4 KiB	启用
7	GPIO1	0x8400_7000 - 0x8400_7FFF	4 KiB	关闭

表 84 APB 设备槽地址分配

5.10.2 Timer 与 PWM

Timer 包含 4 个 16 位计数器，每个计数器设置 4 路比较输出，共形成

16 路 PWM 信号。CH_EN 分别控制四路计数器工作时钟，dft_cg_enable_i 可在测试状态下强制允许时钟。时钟门控单元的 clk_in、test_mode 和 clock_en 均为 1 位输入，clk_out 为 1 位输出。非 FPGA 配置中，时钟门控单元在输入时钟低电平期间保存 CH_EN 与测试允许的或值，再与输入时钟相与，以避免允许信号在高电平期间变化产生窄脉冲；FPGA 配置中输出时钟直接等于输入时钟，CH_EN 不形成普通逻辑门控时钟。low_speed_clk_i 在各 Timer 工作时钟内经过三级寄存同步，只作为低速计数事件，不直接充当计数器时钟。四路 timer_events_o 可从 16 路 PWM 信号中选择，并对所选信号进行上升沿检测。PWM 通过 GPIO0[0:15] 输出。

5.10.3 SPI

SPI 主机默认发送 FIFO 和接收 FIFO 深度均为 10，支持标准串行传输、四线发送和四线接收，并提供四路片选。时钟生成、发送、接收、控制和 APB 寄存器接口分为独立子模块。事件输出在发送 FIFO 元素数低于可编程阈值，或接收 FIFO 元素数高于可编程阈值时有效。SPI 时钟、四路数据和片选信号复用 GPIO0[22:30]。

5.10.4 I2C

I2C0 和 I2C1 使用相同的 APB I2C 主机结构。寄存器偏移 0x00、0x04、0x08、0x0C、0x10 和 0x14 依次为预分频、控制、接收、状态、发送和命令寄存器。字节控制器产生 START、WRITE、READ 和 STOP 序列，位控制器完成 SCL/SDA 开漏驱动、采样与仲裁；传输完成或仲裁失败时置位中断状态，命令寄存器的 IACK 控制清除该状态。I2C0 与 I2C1 分别复用 GPIO0[18:19] 和 GPIO0[20:21]。

5.10.5 UART

UART0 和 UART1 的发送与接收 FIFO 均为 16 字节，寄存器采用 RBR/THR、IER、IIR/FCR、LCR 和 LSR 等兼容布局，DLAB 位选择分频寄存器。接收、发送和中断控制分为独立子模块。UART0 使用 SoC 独立的 uart0_rxd_i 与 uart0_txd_o 端口；UART1 经 GPIO0[16:17] 复用。两路事件分别进入 PLIC 中断源 7 和 8。

5.10.6 GPIO 与 IOF 复用

每个 GPIO 模块均包含 32 位方向、输入、输出、中断使能、两组中断类型、状态、IOFCFG 和 8 组引脚配置寄存器。输入先经过两级同步寄存器，再

由第三级寄存器保存前一拍输入，用于检测上升沿和下降沿。INTTYPE1 和 INTTYPE0 的对应位共同选择高电平、低电平、上升沿或下降沿触发。触发条件满足且相应中断使能位有效时，模块保存中断状态并拉高中断输出；软件读取 INTSTATUS 后清除状态和中断输出。

GPIO0 和 GPIO1 使用相同的寄存器结构，但只有 GPIO0 的 IOFCFG 参与 SoC 顶层专用外设引脚选择。GPIO0[0:15] 对应 PWM，GPIO0[16:17] 对应 UART1，GPIO0[18:19] 对应 I2C0，GPIO0[20:21] 对应 I2C1，GPIO0[22:30] 对应 SPI，GPIO0[31] 始终作为普通 GPIO。GPIO1 的 IOFCFG 可以读写，但其输出未参与 SoC 顶层引脚选择。GPIO0 整体关闭时，专用外设功能不能送至 SoC 顶层 GPIO0 端口。

GPIO 模块内部 PADDR 的每一位为 1 时表示输出驱动，5.10.1 节所列外设子系统方向端口沿用这一含义。SoC 顶层对方向向量逐位取反，因此 SoC 对外的 gpio0_dir_o 和 gpio1_dir_o 为 1 时表示输入或高阻，为 0 时表示输出驱动。

5.10.7 外设中断编号

SoC 顶层按固定编号构造 11 位 PLIC 输入：Timer 四路事件占 0 至 3，SPI 占 4，I2C0 和 I2C1 占 5、6，UART0 和 UART1 占 7、8，GPIO0 和 GPIO1 占 9、10。关闭某外设时对应事件固定为零，因此编号不随裁剪参数改变。完整向量进入 PLIC；CPU 的 8 位 irq_sources 仅保留接口兼容性。

5.10.8 板级应用接口

温湿度气压/OLED 应用使用 GPIO0[18] 作为 I2C SCL，GPIO0[19] 作为 I2C SDA，与外部环境传感器和 SSD1306 OLED 模块连接。该应用的软件 I2C 后端通过 GPIO 开漏方式模拟 I2C 时序，也保留了切换到 APB I2C0 控制器的硬件 I2C 后端。硬件 I2C 模式下，I2C0 挂接在 AXI2APB 后端，软件通过 GPIO0.IOFCFG 将 GPIO0[18:19] 复用为 I2C0 SCL/SDA；软件 I2C 模式下，GPIO0[18:19] 作为普通 GPIO 输入/输出方向动态切换，以实现开漏总线行为。OLED 默认使用 0x3c 地址，按照 SSD1306 128x64 页面模式刷新。传感器驱动支持 AHT30、HDC3020、BME280 和 BME680，其中 AHT30 默认地址为 0x38，HDC3020 默认地址为 0x44，BME280 支持 0x76/0x77，BME680 支持 0x76/0x77。由于这些设备均挂接在同一条 I2C 总线上，应用启动后会通过地址探测确认在线设备，并只读取已成功探测到的传感器。

该板级接口使 CPU 能够通过 AXI/APB 地址空间访问 I2C、GPIO 等低速外设，并通过 GPIO0.IOFCFG 引脚复用机制连接实际板级设备，使 SoC 顶层

端口、BSP 驱动和 RTOS 应用组成完整系统。

5.11 通用寄存器与存储单元

5.11.1 通用寄存器单元

CPU、CSR、流水线寄存器和 AXI 互连共同使用一组参数化寄存器单元。`gnrl_dfflr` 具有数据宽度参数、加载允许和低电平有效异步复位；`gnrl_dfflr_dmux` 使用显式数据选择在写入新值与保持原值之间选择，接口与复位行为相同。`gnrl_dffl` 和 `gnrl_dffl_dmux` 保留加载允许但不设置复位，用于宽数据或无需确定初值的状态；`gnrl_dff` 每拍写入输入数据，并采用低电平有效异步复位。带复位的单元只保证复位后控制状态确定，使用无复位宽数据寄存器的模块必须同时清除或屏蔽对应有效位，后续不得在有效位为零时采用数据字段。

这些寄存器单元的共同端口为时钟 `clk`、数据输入 `dnxt` 和数据输出 `qout`；带加载允许的单元增加 `lden`，带复位的单元增加 `rst_n`。数据端口宽度由 `DW` 参数确定，控制端口均为 1 位。显式数据选择版本与时钟允许版本在功能上等价，差别仅在于保持方式；模块内部延迟由寄存器边界确定。

五种通用寄存器单元的配置和时序行为如下表所示。

单元	DW 默认值	加载允许	复位	时钟行为
<code>gnrl_dfflr</code>	32	有	低电平有效异步复位	<code>lden=1</code> 时在上升沿采样 <code>dnxt</code> 。
<code>gnrl_dfflr_dmux</code>	32	有	低电平有效异步复位	每个上升沿选择 <code>dnxt</code> 或原值。
<code>gnrl_dffl</code>	32	有	无	<code>lden=1</code> 时在上升沿采样 <code>dnxt</code> 。
<code>gnrl_dffl_dmux</code>	32	有	无	每个上升沿选择 <code>dnxt</code> 或原值。
<code>gnrl_dff</code>	32	无	低电平有效异步复位	每个上升沿采样 <code>dnxt</code> 。

表 85 通用寄存器单元

通用寄存器端口如下表所示。

端口名	适用单元	方向	位宽	功能说明
<code>clk</code>	全部	输入	1	时钟。
<code>rst_n</code>	<code>gnrl_dfflr</code> 、 <code>gnrl_dfflr_dmux</code> 、 <code>gnrl_dff</code>	输入	1	低电平有效异步复位。

端口名	适用单元	方向	位宽	功能说明
lden	四种带加载允许单元	输入	1	数据加载允许。
dnxt	全部	输入	DW	下一拍输入数据。
qout	全部	输出	DW	当前寄存器数据。

表 86 通用寄存器端口

5.11.2 通用存储单元

增强 BPU 使用的 `gnrl_ram_1rlw_async` 具有一组同步写端和一组异步读端，分支目标缓冲使用该结构保存标签、类型和目标；`gnrl_ram_2rlw_async` 具有一组同步写端和两组独立异步读端，分支历史表与各 TAGE 表分别用两个读端服务预测查询和训练查询。`gnrl_ram_4rlw_async` 提供四组异步读端和一组同步写端，但当前增强 BPU 不使用该结构。

一读一写、两读一写和四读一写存储均在时钟上升沿执行同步写入，并通过组合读端直接返回当前地址的数据。三种单元均不包含复位、初始化和相同地址读写前递。四读一写单元为每个读端保存独立数据副本，并把每份副本分成两个 bank；一次写入同时修改四份副本中的相应 bank。

异步读存储参数如下表所示。

参数名	默认值	功能说明
DATA_WIDTH	32	每项数据位宽。
ADDR_WIDTH	8	读写地址位宽。
DEPTH	$1 \ll \text{ADDR_WIDTH}$	存储项数。

表 87 异步读存储参数

异步读存储端口如下表所示。

端口名	适用单元	方向	位宽	功能说明
clk	全部	输入	1	写时钟。
write_enable_i	全部	输入	1	同步写允许。
write_addr_i	全部	输入	ADDR_WIDTH	写地址。
write_data_i	全部	输入	DATA_WIDTH	写数据。
read_addr_i	一读一写	输入	ADDR_WIDTH	异步读地址。
read_data_o	一读一写	输出	DATA_WIDTH	异步读数据。
read0_addr_i	两读一写、四读一写	输入	ADDR_WIDTH	第 0 读端地址。
read0_data_o	两读一写、四读一写	输出	DATA_WIDTH	第 0 读端数据。

端口名	适用单元	方向	位宽	功能说明
read1_addr_i	两读一写、四读一写	输入	ADDR_WIDTH	第 1 读端地址。
read1_data_o	两读一写、四读一写	输出	DATA_WIDTH	第 1 读端数据。
read2_addr_i	四读一写	输入	ADDR_WIDTH	第 2 读端地址。
read2_data_o	四读一写	输出	DATA_WIDTH	第 2 读端数据。
read3_addr_i	四读一写	输入	ADDR_WIDTH	第 3 读端地址。
read3_data_o	四读一写	输出	DATA_WIDTH	第 3 读端数据。

表 88 异步读存储端口

伪双端口 RAM 作为 IMEM 与 DMEM 的存储阵列，具有单写端和单读端，支持逐字节同步写和一拍同步读。其参数如下表所示。

参数名	默认值	功能说明
ADDR_WIDTH	16	字节地址位宽。
DATA_WIDTH	32	数据字位宽。
INIT_MEM	1	为 1 时从初始化文件读取存储初值。
INIT_FILE	prog.mem	初始化文件名。

表 89 伪双端口 RAM 参数

伪双端口 RAM 端口如下表所示。

端口名	方向	位宽	功能说明
clk	输入	1	读写时钟。
rst_n	输入	1	兼容保留端口，存储阵列和读数据寄存器均不使用该信号。
we_i	输入	1	同步写允许。
we_mask_i	输入	DATA_WIDTH/8	逐字节写使能。
waddr_i	输入	ADDR_WIDTH	写字节地址。
data_i	输入	DATA_WIDTH	写数据。
raddr_i	输入	ADDR_WIDTH	读字节地址。
data_o	输出	DATA_WIDTH	延后一拍返回的同步读数据。

表 90 伪双端口 RAM 端口

伪双端口 RAM 的数据阵列与读数据寄存器均不复位；对应的 AXI4-Lite 包装和辅助读数据端口见 5.9.6 节。通用单端口 RAM 与带 AXI 突发兼容端口的 RAM 不属于当前 SoC 的 IMEM 或 DMEM 实现，其复位与握手行为不用于描述 5.9.6 节的存储器从设备。

5.12 AXI4-Lite Cache 与乒乓缓存

AXI4-Lite Cache 子系统在不改变 CPU M0/M1 接口定义的条件下，为取指和数据读取提供本地命中返回，并用乒乓缓存切分 Cache 到 AXI 互连之间的通道。ICache、DCache 和乒乓缓存均通过参数独立开关；关闭 Cache 时，上游与下游 AXI4-Lite 信号直接连接，关闭乒乓缓存时各通道直接连接。两类结构不使用 AXI4 Full 的 ID、Burst、LAST 或 USER 信号。

5.12.1 Cache 层次结构

Cache 层次结构见图 6。统一的 axi_lite_cache 根据 ENABLE_CACHE 和 READ_ONLY 选择直通、只读 ICache 或读写 DCache。ICache 和 DCache 顶层各自连接控制器、数据 BRAM、标签与有效位 BRAM；控制器内部实例化未命中请求 FIFO 和响应 FIFO。Cache 的 M_AXI 侧连接乒乓缓存，乒乓缓存的输出再进入 AXI 互连。

ICache 的控制器只处理 AR/R 通道，统一顶层将 AW/W/B 通道直接连接至下游。DCache 控制器处理全部五个通道，读请求与写请求共享数据和标签写端口，但使用独立的读填充状态和写提交状态。数据 BRAM 与标签 BRAM 不保存 AXI 通道控制，FIFO 也不执行命中比较；功能边界保持明确。

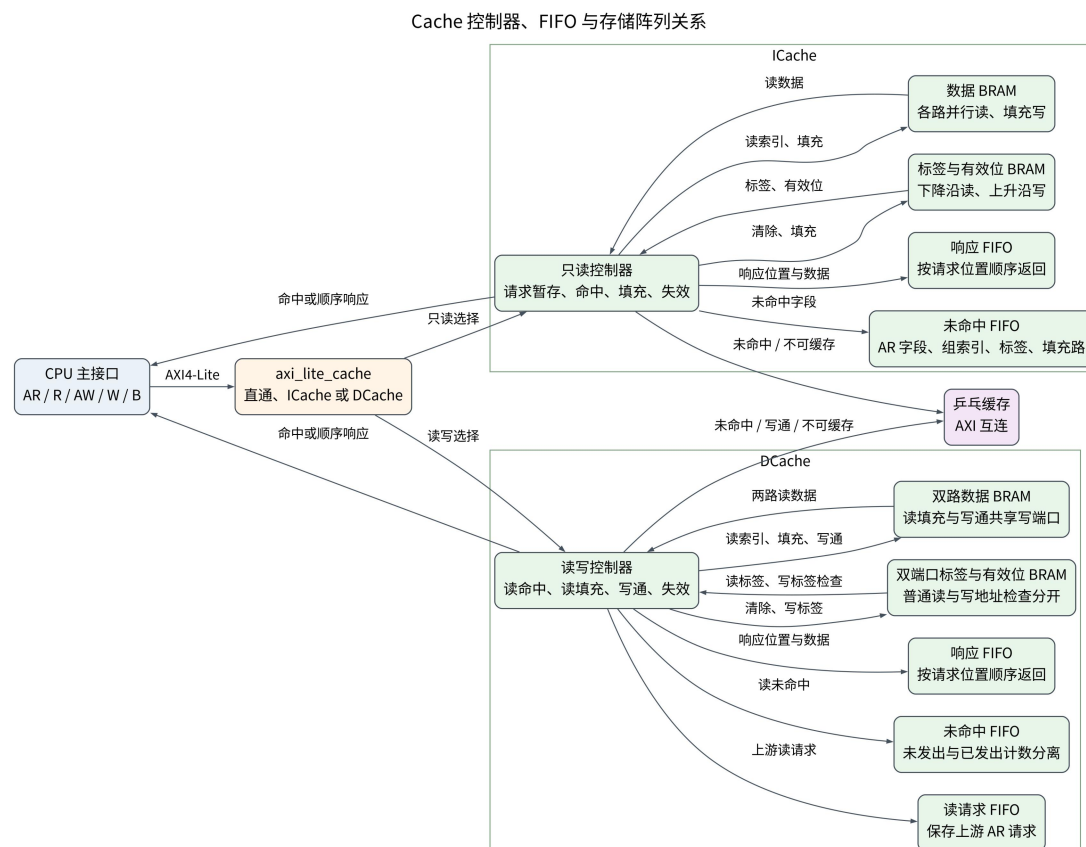


图 6-1 Cache 控制器、FIFO 与存储阵列关系

5.12.2 AXI4-Lite Cache 顶层选择

统一 Cache 顶层的参数定义见 2.5.4 节。ENABLE_CACHE=0 时不生成 Cache 状态，AW/W/B/AR/R 五个通道全部直接连接；ENABLE_CACHE=1 且 READ_ONLY=1 时生成只读 ICache，AR/R 由 ICache 控制，AW/W/B 保持直接连接；ENABLE_CACHE=1 且 READ_ONLY=0 时生成读写 DCache。invalidate_i 启动有效位逐组清除，关闭本地命中和填充，阻止 DCache 成功写响应形成本地写提交，并把未命中 FIFO 中的有效请求标记为不得填充，但不取消已经被下游接受的 AXI 请求。

统一顶层端口如下表所示。方向以 Cache 模块为基准，S_AXI 面向 CPU，M_AXI 面向乒乓缓存或 AXI 互连。

端口名	方向	位宽	有效条件与功能
clk	输入	1	Cache 控制、FIFO 和 BRAM 的系统时钟。
rst_n	输入	1	低电平有效异步复位，清空控制状态和 FIFO；有效位在复位释放后逐组清除，替换状态不要求复位。
invalidate_i	输入	1	为高时启动逐组清除；旧请求仍可返回，但不得重新填充。
S_AXI_AWADDR	输入	C_AXI_ADDR_WIDTH	上游写地址，在 S_AXI_AWVALID && S_AXI_AWREADY 时被接受。
S_AXI_AWPROT	输入	3	上游写访问属性，随 AW 地址向下游传递。
S_AXI_AWVALID	输入	1	上游写地址有效；DCache 写地址寄存器有空间时允许握手。

端口名	方向	位宽	有效条件与功能
S_AXI_AWREADY	输出	1	上游写地址就绪；暂停时保持为低，不接受新地址。
S_AXI_WDATA	输入	C_AXI_DATA_WIDTH	上游写数据，在 W 通道握手时保存或传递。
S_AXI_WSTRB	输入	C_AXI_DATA_WIDTH/8	逐字节写使能，DCache 成功写命中时按位更新。
S_AXI_WVALID	输入	1	上游写数据有效，允许与 AW 分别握手。
S_AXI_WREADY	输出	1	上游写数据就绪；写数据寄存器被占用时为低。
S_AXI_BRESP	输出	2	DCache 保存下游写响应值后返回上游；只在 OKAY 时修改本地标签或数据。
S_AXI_BVALID	输出	1	上游写响应有效，等待 S_AXI_BREADY 后释放。
S_AXI_BREADY	输入	1	上游准备接收写响应；无效时响应数据和状态保持。
S_AXI_ARADDR	输入	C_AXI_ADDR_WIDTH	上游读地址；AR 握手后先进入输入暂存结构，请求进入标签检查时才分配响应 FIFO 位置。
S_AXI_ARPROT	输入	3	上游读访问属性，未命中或不可缓存时送往下游。
S_AXI_ARVALID	输入	1	上游读地址有效。
S_AXI_ARREADY	输出	1	ICache 的输入暂存寄存器或 DCache 的输入 FIFO 可以接收新请求时有效。
S_AXI_RDATA	输出	C_AXI_DATA_WIDTH	命中数据、下游直接返回数据或响应 FIFO 队首数据。

端口名	方向	位宽	有效条件与功能
S_AXI_RRESP	输出	2	对命中返回 OKAY，未命中和不可缓存访问采用下游响应。
S_AXI_RVALID	输出	1	队首响应已经就绪时有效。
S_AXI_RREADY	输入	1	上游准备接收读响应；无效时队首响应保持。
M_AXI_AWADDR	输出	C_AXI_ADDR_WIDT H	下游写地址；只读模式和关闭模式直通，DCache 模式由写地址寄存器驱动。
M_AXI_AWPROT	输出	3	下游写访问属性。
M_AXI_AWVALID	输出	1	下游写地址有效，保持到 M_AXI_AWREADY。
M_AXI_AWREADY	输入	1	下游写地址就绪。
M_AXI_WDATA	输出	C_AXI_DATA_WIDT H	下游写数据。
M_AXI_WSTRB	输出	C_AXI_DATA_WIDT H/8	下游逐字节写使能。
M_AXI_WVALID	输出	1	下游写数据有效，保持到 M_AXI_WREADY。
M_AXI_WREADY	输入	1	下游写数据就绪。
M_AXI_BRESP	输入	2	下游写响应状态。
M_AXI_BVALID	输入	1	下游写响应有效。
M_AXI_BREADY	输出	1	Cache 可以接收写响应时有效。
M_AXI_ARADDR	输出	C_AXI_ADDR_WIDT H	未命中或不可缓存读地址，由未命中 FIFO 发出端驱动。
M_AXI_ARPROT	输出	3	与下游读地址对齐的访问属性。
M_AXI_ARVALID	输出	1	未命中 FIFO 存在待发请求时有效。
M_AXI_ARREADY	输入	1	下游读地址就绪，握手后推进发出指针。

端口名	方向	位宽	有效条件与功能
M_AXI_RDATA	输入	C_AXI_DATA_WIDT H	下游读数据，可直接返回队首或写入响应 FIFO。
M_AXI_RRESP	输入	2	下游读响应；错误时禁止 Cache 填充。
M_AXI_RVALID	输入	1	下游读数据有效。
M_AXI_RREADY	输出	1	未命中 FIFO 存在已经发出并等待返回的队首请求时有效。

表 91 AXI4-Lite Cache 顶层端口

表 91 中与输入暂存、未命中 FIFO 和写响应寄存器有关的行为只适用于已经生成的 ICache 或 DCache。Cache 关闭时，两侧同名 AXI4-Lite 信号直接连接，S_AXI_ARREADY 直接采用 M_AXI_ARREADY，M_AXI_ARVALID 直接采用 S_AXI_ARVALID；只读模式下，AW、W 和 B 通道同样直接连接。直通模式不建立请求状态，也不改变 VALID、READY、地址、数据或响应字段。

5.12.3 ICache 控制器与存储阵列

ICache 顶层由只读控制器、数据 BRAM 和标签有效位 BRAM 组成。控制器在输入侧设置一项暂存寄存器；外部 AR 握手发生而当前请求不能立即进入标签检查时，该寄存器保存地址和 ARPROT。请求进入标签检查时，控制器才为该请求在响应 FIFO 中分配位置、驱动标签和数据 BRAM 读地址，并保存地址字段用于下一上升沿的命中检查。外部 AR 握手与响应位置分配可以发生在不同周期。

ICache 顶层参数如下表所示。控制器使用除 ENABLE_PREFETCH 以外的其余参数；ENABLE_PREFETCH 保留在 ICache 顶层接口中，当前不产生预取请求。

参数	默认值	有效范围	功能
CACHE_WAYS	1	1、2	设置 ICache 路数。
CACHE_ENTRIES	64	2 至 2048 的二次幂	设置全部路合计的缓存行数。
OUTSTANDING_DEPTH	2	1 至 4	设置未命中 FIFO 和响应 FIFO 深度。
ENABLE_PREFETCH	0	0、1	保留的预取配置，当前不改变控制器行为。
C_AXI_DATA_WIDT H	32	不小于 8 的二次幂	设置 AXI4-Lite 数据位宽和缓存行宽度。

参数	默认值	有效范围	功能
C_AXI_ADDR_WIDTH	32	大于 CACHE_ADDR_WIDTH	设置 AXI4-Lite 地址位宽。
CACHE_ADDR_WIDTH	18	小于 C_AXI_ADDR_WIDTH	设置参与标签、组索引和字节位划分的有效地址宽度。
CACHE_BASE_ADDR	0	地址位宽允许范围	设置可缓存地址区域的基地址。

表 92 ICache 顶层参数

推荐配置将 CACHE_ENTRIES 设为 2048、CACHE_WAYS 设为 1、OUTSTANDING_DEPTH 设为 2。按处理器数据宽度计算，RV32 的 ICache 数据容量为 8 KiB，RV64 的 ICache 数据容量为 16 KiB；每条缓存行分别为 4 B 和 8 B。

ICache 顶层端口如下表所示。控制器也具有相同的时钟、复位、无效化和 AXI4-Lite 端口，并增加表 94 所列存储控制端口，因此表 93 与表 94 共同构成控制器的完整端口定义。

端口名	方向	位宽	有效条件与功能
clk	输入	1	ICache 控制器、FIFO 和存储阵列时钟。
rst_n	输入	1	低电平有效异步复位，清空控制状态和 FIFO。
invalidate_i	输入	1	高电平启动逐组有效位清除，并禁止本地命中和填充。
S_AXI_ARADDR	输入	C_AXI_ADDR_WIDTH	上游读地址。
S_AXI_ARPROT	输入	3	上游读访问属性。
S_AXI_ARVALID	输入	1	上游读地址有效。
S_AXI_ARREADY	输出	1	输入暂存寄存器可接收请求时有效。
S_AXI_RDATA	输出	C_AXI_DATA_WIDTH	命中数据、队首未命中直接返回数据或响应 FIFO 队首数据。
S_AXI_RRESP	输出	2	命中返回 OKAY，其余访问返回下游响应。
S_AXI_RVALID	输出	1	队首读响应已经就绪时有效。

端口名	方向	位宽	有效条件与功能
S_AXI_RREADY	输入	1	上游可以接收读响应时有效。
M_AXI_ARADDR	输出	C_AXI_ADDR_WIDTH	未命中或不可缓存读地址。
M_AXI_ARPROT	输出	3	与下游读地址对应的访问属性。
M_AXI_ARVALID	输出	1	未命中 FIFO 存在待发请求时有效。
M_AXI_ARREADY	输入	1	下游可以接收读地址时有效。
M_AXI_RDATA	输入	C_AXI_DATA_WIDTH	下游返回数据。
M_AXI_RRESP	输入	2	下游读响应，错误响应禁止填充。
M_AXI_RVALID	输入	1	下游读响应有效。
M_AXI_RREADY	输出	1	未命中 FIFO 存在已经发出并等待返回的队首请求时有效。

表 93 ICache 顶层端口

下降沿对数据 BRAM 和标签 BRAM 发出同步读，上升沿保存每路标签相等结果、有效位和数据。保存后的 probe 状态只对应一个已经接受的上游请求，不能被下一地址直接覆盖。若该请求命中且对应响应位置位于队首，控制器直接使用已经登记的数据产生 R 响应；若上游暂停，则数据写入已经分配的响应位置。未命中请求保存完整 AR 字段、可缓存标志、组索引、标签、填充路和响应位置编号后进入未命中 FIFO。

ICache 内部存储接口如下表所示。只读顶层对外的 AR/R 端口与表 91 定义一致。

信号名	方向	位宽	有效条件与功能
cache_read_enable_o	控制器输出	1	固定为高，使 BRAM 在每个下降沿读取当前选择索引；控制器只在请求进入标签检查后采用下一上升沿的读出值。
cache_read_index_o	控制器输出	INDEX_WIDTH	待读取的组索引。
cache_read_data_i	控制器输入	CACHE_WAYS*C_AXI_DATA_WIDTH	各路同步读数据，上升沿登记后参与命中选择。

信号名	方向	位宽	有效条件与功能
cache_read_tags_i	控制器输入	CACHE_WAYS*TAG_WIDTH	各路同步读标签。
cache_read_valids_i	控制器输入	CACHE_WAYS	各路同步读有效位。
tag_clear_enable_o	控制器输出	1	复位初始化或失效期间逐组清除有效位。
tag_clear_index_o	控制器输出	INDEX_WIDTH	当前需要清除的组索引。
cache_fill_enable_o	控制器输出	1	下游 R 为 OKAY、请求可缓存且未失效时写入新缓存行。
cache_fill_index_o	控制器输出	INDEX_WIDTH	填充组索引。
cache_fill_tag_o	控制器输出	TAG_WIDTH	填充标签。
cache_fill_way_o	控制器输出	WAY_INDEX_WIDTH	填充路；一路配置固定为 0。
write_data_i	数据 BRAM 输入	C_AXI_DATA_WIDTH	填充数据，来自下游 R 通道。
write_enable_i	数据 BRAM 输入	1	填充写使能。
write_index_i	数据 BRAM 输入	INDEX_WIDTH	填充数据的组索引。
write_way_i	数据 BRAM 输入	WAY_INDEX_WIDTH	填充数据的目标路。

表 94 ICACHE 控制器内部端口

ICACHE 数据存储和标签有效位存储的参数如下表所示。SET_COUNT 表示每组数，INDEX_WIDTH 表示组索引宽度，WAY_INDEX_WIDTH 在一路配置下仍保留 1 位接口。

模块	参数	默认值	功能
数据存储	CACHE_WAYS	1	设置并行数据存储路数。
数据存储	SET_COUNT	64	设置每路存储深度。
数据存储	C_AXI_DATA_WIDTH	32	设置每项数据宽度。
数据存储	INDEX_WIDTH	log2(SET_COUNT)	设置组索引宽度。
数据存储	WAY_INDEX_WIDTH	max(1,log2(CACHE_WAYS))	设置目标路编号宽度。
标签有效位存储	CACHE_WAYS	1	设置并行标签存储路数。
标签有效位存储	SET_COUNT	64	设置每路存储深度。
标签有效位存储	TAG_WIDTH	8	设置标签宽度。
标签有效位存储	INDEX_WIDTH	log2(SET_COUNT)	设置组索引宽度。

模块	参数	默认值	功能
标签有效位存储	WAY_INDEX_WIDTH	$\max(1, \log_2(\text{CACHE_WAYS}))$	设置目标路编号宽度。

表 95 ICache 存储单元参数

模块	端口名	方向	位宽	有效条件与功能
数据存储	clk	输入	1	下降沿同步读和上升沿同步写时钟。
数据存储	read_enable_i	输入	1	为高时在下降沿采样读索引。
数据存储	read_index_i	输入	INDEX_WIDTH	全部路共同采用的读组索引。
数据存储	read_data_o	输出	CACHE_WAYS* C_AXI_DATA_WIDTH	各路读数据按路编号依次拼接。
数据存储	write_enable_i	输入	1	填充写使能。
数据存储	write_index_i	输入	INDEX_WIDTH	填充目标组索引。
数据存储	write_way_i	输入	WAY_INDEX_WIDTH	填充目标路。
数据存储	write_data_i	输入	C_AXI_DATA_WIDTH	需要写入的完整缓存行数据。
标签有效位存储	clk	输入	1	下降沿同步读和上升沿同步写时钟。
标签有效位存储	read_enable_i	输入	1	为高时在下降沿采样读索引。
标签有效位存储	read_index_i	输入	INDEX_WIDTH	全部路共同采用的读组索引。
标签有效位存储	read_tags_o	输出	CACHE_WAYS* TAG_WIDTH	各路标签读值。
标签有效位存储	read_valids_o	输出	CACHE_WAYS	各路有效位读值。
标签有效位存储	clear_enable_i	输入	1	逐组清除使能，优先于普通标签写入。
标签有效位存储	clear_index_i	输入	INDEX_WIDTH	当前清除组索引。
标签有效位存储	write_enable_i	输入	1	填充标签和有效位写使能。

模块	端口名	方向	位宽	有效条件与功能
标签有效位存储	write_index_i	输入	INDEX_WIDTH	填充目标组索引。
标签有效位存储	write_tag_i	输入	TAG_WIDTH	填充标签。
标签有效位存储	write_way_i	输入	WAY_INDEX_WIDTH	填充目标路。

表 96 ICache 存储单元端口

两路配置先选择无效路，两路均有效时选择真 LRU 指定的路。推荐的一路配置不生成数据路选择和有效的 LRU 状态。读响应为 OKAY 且请求仍允许填充时，同时写入数据、标签和有效位；错误响应、不可缓存请求和失效后返回的旧请求只向 IFU 返回，不更新 ICache。

5.12.4 DCache 控制器与存储阵列

DCache 顶层由读写控制器、按 CACHE_WAYS 生成的一路或两路数据 BRAM，以及相同路数的标签有效位 BRAM 组成；Cache 性能配置使用两路。读请求先进入深度与 OUTSTANDING_DEPTH 相同的输入环形 FIFO，使控制器处理前一笔标签判断、填充或写提交时仍能接收新的 AR。队首地址在下降沿读取标签和数据，上升沿保存比较结果；队首命中在下一周期返回，未命中进入独立未命中 FIFO。

DCache 顶层和控制器采用以下参数。顶层与控制器的参数集合相同，未设置只用于顶层的保留参数。

参数	默认值	有效范围	功能
CACHE_WAYS	1	1、2	设置 DCache 路数。
CACHE_ENTRIES	64	2 至 2048 的二次幂	设置全部路合计的缓存行数。
OUTSTANDING_DEPTH	2	1 至 4	设置读输入 FIFO、未命中 FIFO 和响应 FIFO 深度。
C_AXI_DATA_WIDTH	32	不小于 8 的二次幂	设置 AXI4-Lite 数据位宽和缓存行宽度。
C_AXI_ADDR_WIDTH	32	大于 CACHE_ADDR_WIDTH	设置 AXI4-Lite 地址位宽。
CACHE_ADDR_WIDTH	18	小于 C_AXI_ADDR_WIDTH	设置参与标签、组索引和字节位划分的有效地址宽度。
CACHE_BASE_ADDR	0	地址位宽允许范围	设置可缓存地址区域的基地址。

表 97 DCache 顶层与控制器参数

推荐配置将 `CACHE_ENTRIES` 设为 1024、`CACHE_WAYS` 设为 2、`OUTSTANDING_DEPTH` 设为 2。每一路保存 512 条缓存行，RV32 的 DCache 数据容量为 4 KiB，RV64 的 DCache 数据容量为 8 KiB；每条缓存行分别为 4 B 和 8 B。

DCache 顶层具有表 91 列出的全部时钟、复位、无效化和 `S_AXI/M_AXI` 五通道端口。DCache 控制器也具有表 91 的全部端口，并增加表 99 所列存储控制端口；两个表共同给出控制器的完整端口定义。DCache 模式下，表 91 中的所有通道均由控制器管理，不存在写通道直接连接。

写地址和写数据分别保存，允许 `AW` 和 `W` 以任意先后顺序握手，并允许已经保存的通道先向下游发出。只有 `AW` 和 `W` 均已保存、没有待读填充或写提交占用存储写端口时，控制器才启动写地址标签检查。普通读标签检查使用下降沿读取端口，写地址标签检查使用上升沿 `B` 端口，两者可以在同一周期进行；读填充或写提交执行时暂停新的标签检查，但输入 FIFO 仍可接收 `AR`，直到 FIFO 满。

`M_AXI_BREADY` 只在 `AW` 和 `W` 均已发出、写地址检查已经完成或在本周期完成、当前没有尚未返回上游的 `B` 响应，并且没有待执行的写提交时有效。下游 `B` 完成握手后，`BRESP` 写入独立响应寄存器并保持到上游完成 `B` 握手；错误响应同样返回上游，但不建立本地写提交。只有 `BRESP=OKAY`、地址可缓存、有效位初始化已经结束且当前没有无效化请求时，控制器才允许本地写提交。无效化先于 `B` 响应到达或发生在写提交等待期间时，标签和数据均不得修改。

下游 `R` 以 `OKAY` 返回可填充读请求时，控制器先把数据、标签、组索引和填充路登记到填充寄存器，下一周期再写标签和数据 `BRAM`。读填充与写提交同时等待存储写端口时，读填充优先，写提交状态保持到后续周期；读响应在 `R` 握手时即可直接返回或写入响应 FIFO，不等待本地填充完成。

DCache 的本地更新规则如下表所示。

访问情况	下游响应条件	标签与有效位处理	数据处理
读命中	本地完成	保持；两路配置将 LRU 指向另一条路	返回命中路数据。
读未命中	<code>RRESP=OKAY</code>	在无效路或 LRU 路写入标签并置有效	写入完整 <code>C_AXI_DATA_WIDTH</code> 位缓存行数据。
读错误	<code>RRESP!=OKAY</code>	不改变	不写数据，只向上游返回错误响应。
整字写命中	<code>BRESP=OKAY</code>	保持；两路配置更新 LRU	覆盖完整缓存行。

访问情况	下游响应条件	标签与有效位处理	数据处理
整字写未命中	BRESP=OKAY	分配无效路或 LRU 路并置有效	写入完整缓存行。
部分字节写命中	BRESP=OKAY	保持；两路配置更新 LRU	只更新 WSTRB 指定字节。
部分字节写未命中	BRESP=OKAY	不分配	不改变本地数据。
写错误	BRESP!=OKAY	不改变	不改变本地数据。

表 98 DCache 读写更新规则

DCache 控制器与存储阵列之间的专用接口如下表所示。标准 AXI4-Lite 端口与表 91 相同。

信号名	方向	位宽	有效条件与功能
cache_read_enable_o	控制器输出	1	固定为高，使 BRAM 在每个下降沿读取当前队首索引；控制器只在读请求进入标签检查后采用下一上升沿的读出值。
cache_read_index_o	控制器输出	INDEX_WIDTH	普通读请求的组索引。
cache_read_data_i	控制器输入	CACHE_WAYS*C_AXI_DATA_WIDTH	各路数据 BRAM 读出值。
cache_read_tags_i	控制器输入	CACHE_WAYS*TAG_WIDTH	各路普通读标签。
cache_read_valids_i	控制器输入	CACHE_WAYS	各路普通读有效位。
write_probe_enable_o	控制器输出	1	AW 与 W 已保存且标签端口可用时启动写地址检查。
write_probe_index_o	控制器输出	INDEX_WIDTH	写地址对应的组索引。
write_probe_tags_i	控制器输入	CACHE_WAYS*TAG_WIDTH	写地址检查读取的各路标签。
write_probe_valids_i	控制器输入	CACHE_WAYS	写地址检查读取的各路有效位。
tag_clear_enable_o	控制器输出	1	复位初始化或失效期间逐组清除有效位。
tag_clear_index_o	控制器输出	INDEX_WIDTH	当前清除组索引。
tag_write_enable_o	控制器输出	1	读填充或成功整字写分配时写标签。
tag_write_index_o	控制器输出	INDEX_WIDTH	标签写入组索引。
tag_write_tag_o	控制器输出	TAG_WIDTH	需要写入的新标签。

信号名	方向	位宽	有效条件与功能
tag_write_way_o	控制器输出	WAY_INDEX_WIDT H	标签写入路。
data_write_enable_o	控制器输出	1	读填充或成功写提交时写数据 BRAM。
data_write_index_o	控制器输出	INDEX_WIDTH	数据写入组索引。
data_write_way_o	控制器输出	WAY_INDEX_WIDT H	数据写入路。
data_write_data_o	控制器输出	C_AXI_DATA_WIDT H	填充数据或写通数据。
data_write_strb_o	控制器输出	C_AXI_DATA_WIDT H/8	完整填充为全 1，部分字节写采用上游WSTRB。

表 99 DCache 控制器内部端口

DCache 数据存储和标签有效位存储的参数如下表所示。两个模块按路生成独立存储体，SET_COUNT 表示每路组数。

模块	参数	默认值	功能
数据存储	CACHE_WAYS	1	设置并行数据存储路数。
数据存储	SET_COUNT	64	设置每路存储深度。
数据存储	C_AXI_DATA_WIDT H	32	设置每项数据宽度。
数据存储	INDEX_WIDTH	log2(SET_COUNT)	设置组索引宽度。
数据存储	WAY_INDEX_WIDT H	max(1,log2(CACHE_WAYS))	设置目标路编号宽度。
标签有效位存储	CACHE_WAYS	1	设置并行标签存储路数。
标签有效位存储	SET_COUNT	64	设置每路存储深度。
标签有效位存储	TAG_WIDTH	8	设置标签宽度。
标签有效位存储	INDEX_WIDTH	log2(SET_COUNT)	设置组索引宽度。
标签有效位存储	WAY_INDEX_WIDT H	max(1,log2(CACHE_WAYS))	设置目标路编号宽度。

表 100 DCache 存储单元参数

模块	端口名	方向	位宽	有效条件与功能
数据存储	clk	输入	1	下降沿同步读和上升沿同步写时钟。
数据存储	read_enable_i	输入	1	为高时在下降沿采样读索引。

模块	端口名	方向	位宽	有效条件与功能
数据存储	read_index_i	输入	INDEX_WIDTH	全部路共同采用的读组索引。
数据存储	read_data_o	输出	CACHE_WAYS* C_AXI_DATA_ WIDTH	各路读数据。
数据存储	write_enable_i	输入	1	读填充或成功写提交使能。
数据存储	write_index_i	输入	INDEX_WIDTH	数据写入组索引。
数据存储	write_way_i	输入	WAY_INDEX_W IDTH	数据写入目标路。
数据存储	write_data_i	输入	C_AXI_DATA_ WIDTH	填充数据或写通数据。
数据存储	write_strb_i	输入	C_AXI_DATA_ WIDTH/8	逐字节写使能；读填充时全部置1。
标签有效位存储	clk	输入	1	普通读使用下降沿，写地址检查和写入使用上升沿。
标签有效位存储	read_enable_i	输入	1	普通读标签检查使能。
标签有效位存储	read_bram_index_i	输入	INDEX_WIDTH	普通读组索引。
标签有效位存储	read_bram_tags_o	输出	CACHE_WAYS* TAG_WIDTH	普通读的各路标签。
标签有效位存储	read_bram_valids_o	输出	CACHE_WAYS	普通读的各路有效位。
标签有效位存储	write_probe_enable_i	输入	1	写地址标签检查使能。
标签有效位存储	write_probe_index_i	输入	INDEX_WIDTH	写地址对应的组索引。
标签有效位存储	write_probe_tags_o	输出	CACHE_WAYS* TAG_WIDTH	写地址检查的各路标签。
标签有效位存储	write_probe_valids_o	输出	CACHE_WAYS	写地址检查的各路有效位。
标签有效位存储	clear_enable_i	输入	1	逐组清除使能，优先于普通标签写入。

模块	端口名	方向	位宽	有效条件与功能
标签有效位存储	clear_index_i	输入	INDEX_WIDTH	当前清除组索引。
标签有效位存储	write_enable_i	输入	1	读填充或成功整字写分配的标签写使能。
标签有效位存储	write_index_i	输入	INDEX_WIDTH	标签写入组索引。
标签有效位存储	write_tag_i	输入	TAG_WIDTH	新标签。
标签有效位存储	write_way_i	输入	WAY_INDEX_WIDTH	标签写入目标路。

表 101 DCache 存储单元端口

ICache 与 DCache 的标签有效位存储都以同步标签存储单元作为每一路的基本存储体，其参数和端口如下表所示。A 端在下降沿读取，B 端和写入端使用上升沿；同一上升沿同时请求写入和 B 端读取时，写入优先，B 端读出寄存器保持原值。

类别	名称	方向或默认值	位宽或有效范围	功能
参数	SET_COUNT	64	2 至 2048 的二次幂	设置存储深度。
参数	DATA_WIDTH	8	正整数	设置每项标签与有效位合计宽度。
参数	INDEX_WIDTH	$\log_2(\text{SET_COUNT})$	正整数	设置地址宽度。
端口	clk	输入	1	同步读写时钟。
端口	read_enable_i	输入	1	A 端下降沿读使能。
端口	read_index_i	输入	INDEX_WIDTH	A 端读索引。
端口	read_data_o	输出	DATA_WIDTH	A 端登记后的读数据。
端口	read_b_enable_i	输入	1	B 端上升沿读使能。
端口	read_b_index_i	输入	INDEX_WIDTH	B 端读索引。
端口	read_b_data_o	输出	DATA_WIDTH	B 端登记后的读数据。
端口	write_enable_i	输入	1	上升沿写使能，优先于 B 端读取。
端口	write_index_i	输入	INDEX_WIDTH	写索引。
端口	write_data_i	输入	DATA_WIDTH	写数据。

表 102 同步标签存储单元参数与端口

DCache 标签 BRAM 的端口 A 在下降沿读取普通读请求，端口 B 在上升沿检查写地址或执行标签写入。标签写入优先于写地址检查，控制器仅在 AW 和 W 均已保存且标签写端口空闲时启动写地址检查。每路数据 BRAM 的深度为 CACHE_ENTRIES/CACHE_WAYS；推荐配置采用两路，每路保存 512 条缓存行，每条缓存行宽度为 C_AXI_DATA_WIDTH。读填充使用完整字节使能，部分字节写命中使用 WSTRB 作为逐字节写使能。

ICache 和 DCache 复位释放后都从第 0 组开始逐组清除有效位，完成最后一组清除后再额外屏蔽本地命中一个周期。清除期间的读请求仍可访问下游，未命中 FIFO 中的在途请求由 invalidated 状态禁止填充。数据 BRAM 和两路替换状态不复位；当某组没有有效行时，替换状态不会影响数据选择。两路 ICache 使用同步存储保存每组下一次替换路，并在下降沿读取；两路 DCache 为每组保存一位替换状态。读命中、读填充或成功本地写修改后，替换状态指向另一条路。

5.12.5 请求 FIFO、响应 FIFO 与顺序返回

未命中请求 FIFO 使用写指针接收新请求、发出指针控制下游 AR、返回指针匹配下游 R。发出指针和返回指针分开保存，因此深度为 2 时，两笔请求可以都已经完成 AR 握手并同时等待 R。失效请求会把 FIFO 内全部有效项标记为禁止填充，但不丢弃请求。

参数	默认值	功能
DEPTH	2	设置未命中请求项数。
AR_WIDTH	35	设置地址与访问属性合并字段宽度。
INDEX_WIDTH	6	设置 Cache 组索引宽度。
TAG_WIDTH	10	设置 Cache 标签宽度。
WAY_INDEX_WIDTH	1	设置填充路编号宽度。
RESPONSE_INDEX_WIDTH	1	设置响应 FIFO 位置编号宽度。

表 103 Cache 未命中请求 FIFO 参数

未命中 FIFO 端口	方向	位宽	功能
clk	输入	1	指针、计数和请求字段寄存时钟。
rst_n	输入	1	低电平有效异步复位，清空指针、计数和无效化状态。

未命中 FIFO 端口	方向	位宽	功能
invalidate_i	输入	1	将当前全部有效项标记为不得填充；同周期新入队项也保存该状态。
enqueue_valid_i	输入	1	新未命中请求有效。
enqueue_ready_o	输出	1	FIFO 未滿或同周期弹出队首时有效。
enqueue_ar_i	输入	AR_WIDTH	保存 AR 地址和访问属性。
enqueue_cacheable_i	输入	1	指示下游成功返回后是否允许填充。
enqueue_index_i	输入	INDEX_WIDTH	请求组索引。
enqueue_tag_i	输入	TAG_WIDTH	请求标签。
enqueue_way_i	输入	WAY_INDEX_WIDTH	预先选择的填充路。
enqueue_response_index_i	输入	RESPONSE_INDEX_WIDTH	对应响应 FIFO 位置编号。
empty_o	输出	1	FIFO 没有有效请求。
issue_ar_o	输出	AR_WIDTH	当前发往下游的 AR 字段。
issue_valid_o	输出	1	存在尚未发出的旧请求时有效；没有待发旧请求时，本周期新入队请求也可直接使其有效。
issue_ready_i	输入	1	下游 AR 已准备接收，握手后推进发出指针。
response_valid_o	输出	1	存在已经发出且等待返回的队首请求。
response_pop_i	输入	1	下游 R 完成处理后释放返回指针所指请求。
response_ar_o	输出	AR_WIDTH	返回队首的原始 AR 字段。
response_cacheable_o	输出	1	返回队首是否允许填充。
response_index_o	输出	INDEX_WIDTH	返回队首组索引。
response_tag_o	输出	TAG_WIDTH	返回队首标签。

未命中 FIFO 端口	方向	位宽	功能
response_way_o	输出	WAY_INDEX_WIDTH	返回队首填充路。
response_invalidated_o	输出	1	请求发出后是否发生失效；为高时禁止填充。
response_index_id_o	输出	RESPONSE_INDEX_WIDTH	返回队首对应的响应 FIFO 位置。

表 104 Cache 未命中请求 FIFO 端口

响应 FIFO 在读请求从输入暂存结构进入标签检查时分配位置，命中和未命中完成事件再按位置编号写入数据、响应状态和就绪位。读指针只查看队首位置；队首尚未就绪时，后面的就绪响应不能返回。

参数	默认值	功能
DEPTH	2	设置响应位置数量。
DATA_WIDTH	32	设置每项读数据宽度。
INDEX_WIDTH	$\max(1, \log_2(\text{DEPTH}))$	设置响应位置编号宽度。

表 105 Cache 响应 FIFO 参数

响应 FIFO 端口	方向	位宽	功能
clk	输入	1	响应位置、就绪位、指针和计数寄存时钟。
rst_n	输入	1	低电平有效异步复位，清空分配状态和就绪位。
allocate_valid_i	输入	1	一笔读请求进入标签检查，需要分配响应位置。
allocate_ready_o	输出	1	FIFO 有空位或同周期释放队首。
allocate_index_o	输出	INDEX_WIDTH	新分配的位置编号。
allocate_is_head_o	输出	1	新位置在分配后成为队首。
hit_write_valid_i	输入	1	命中完成事件有效。
hit_write_index_i	输入	INDEX_WIDTH	命中结果的响应位置编号。
hit_write_data_i	输入	DATA_WIDTH	命中数据。
hit_write_resp_i	输入	2	命中响应状态，正常为 OKAY。

响应 FIFO 端口	方向	位宽	功能
miss_write_valid_i	输入	1	未命中完成事件有效，优先于同周期命中写入。
miss_write_index_i	输入	INDEX_WIDTH	未命中结果的响应位置编号。
miss_write_data_i	输入	DATA_WIDTH	下游读数据。
miss_write_resp_i	输入	2	下游读响应状态。
head_valid_o	输出	1	队首已分配且完成数据已经写入。
head_allocated_o	输出	1	FIFO 至少存在一个已经分配的位置。
head_index_o	输出	INDEX_WIDTH	当前队首位置编号。
head_data_o	输出	DATA_WIDTH	当前队首数据。
head_resp_o	输出	2	当前队首响应状态。
head_pop_i	输入	1	上游完成 R 握手后释放队首。

表 106 Cache 响应 FIFO 端口

队首未命中直接返回需要同时满足：未命中 FIFO 存在已经发出的队首请求、该请求携带的响应位置编号等于响应 FIFO 读指针，并且队首位置尚未就绪。若上游可以接收数据，下游 R 直接完成上游握手；若上游暂停，数据写入该位置；若更早响应已经就绪，则先返回更早响应。该控制减少未命中返回后的等待周期，同时保持读响应顺序。

5.12.6 AXI4-Lite 乒乓缓存

乒乓缓存由通用双项通道缓存、可配置级数的通道流水、只读 AXI4-Lite 包装和读写 AXI4-Lite 包装组成。通用双项缓存的 DATA_WIDTH 参数决定通道字段宽度，FALL_THROUGH 决定空缓存时是否允许输入直接到达输出；通道流水的 STAGES 参数决定串联级数，STAGES=0 时直接连接。

模块	参数	默认值	有效范围与功能
双项通道缓存	DATA_WIDTH	1	正整数，设置通道合并字段宽度。
双项通道缓存	FALL_THROUGH	0	0、1，控制空缓存直通。
多级通道流水	DATA_WIDTH	1	正整数，设置通道合并字段宽度。
多级通道流水	STAGES	0	0、1、2，设置串联的双项缓存级数。

模块	参数	默认值	有效范围与功能
多级通道流水	FALL_THROUGH	0	0、1，传入每一级双项缓存。

表 107 乒乓缓存基本单元参数

双项通道缓存和多级通道流水采用相同的外部端口。STAGES=0 时，多级通道流水把输入和输出直接连接，不生成双项存储状态。

通用通道端口	方向	位宽	功能
clk	输入	1	缓存状态时钟。
rst_n	输入	1	低电平有效异步复位，清空读写指针和计数。
in_valid	输入	1	输入字段有效。
in_ready	输出	1	存在空位或同周期输出时允许接收。
in_data	输入	DATA_WIDTH	需要保存的 AXI 通道字段。
out_valid	输出	1	输出字段有效。
out_ready	输入	1	下游准备接收输出字段。
out_data	输出	DATA_WIDTH	当前输出字段，暂停时保持稳定。

表 108 乒乓缓存基本单元端口

只读包装分别对 AR 和 R 通道设置级数，其中 STAGES 控制 AR，RESPONSE_STAGES 控制 R，FALL_THROUGH 只控制 AR 空缓存直通。读写包装的 AW、W、B、AR 和 R 通道各自具有独立的通道流水和握手状态，分别保存本通道的完整字段，但五个通道共用 STAGES 级数。推荐配置中，M1 五个通道均为一级非直通。VALID 与字段一起逐级传递，READY 沿相反方向传播，任一通道暂停均不直接改变其它通道的有效状态。

模块	参数	默认值	有效范围与功能
只读包装	C_AXI_DATA_WIDTH	32	与系统数据总线一致，设置 RDATA 位宽。
只读包装	C_AXI_ADDR_WIDTH	32	与系统地址总线一致，设置 ARADDR 位宽。
只读包装	STAGES	0	0、1、2，设置 AR 通道级数。

模块	参数	默认值	有效范围与功能
只读包装	RESPONSE_STAGES	0	0、1、2，设置 R 通道级数；系统 M0 配置取 0。
只读包装	FALL_THROUGH	0	0、1，只控制 AR 通道空缓存直通。
读写包装	C_AXI_DATA_WIDTH	32	与系统数据总线一致，设置 WDATA、WSTRB 和 RDATA 位宽。
读写包装	C_AXI_ADDR_WIDTH	32	与系统地址总线一致，设置 AWADDR 和 ARADDR 位宽。
读写包装	STAGES	0	0、1、2，设置 AW、W、B、AR 和 R 五个通道共同采用的级数；推荐配置取 1。

表 109 AXI4-Lite 乒乓缓存包装参数

只读包装端口如下表所示。方向以包装模块为基准，S_AXI 面向 ICache，M_AXI 面向 AXI 互连。

端口名	方向	位宽	功能
clk	输入	1	AR 和 R 通道缓存时钟。
rst_n	输入	1	低电平有效异步复位，清空通道有效状态。
S_AXI_ARADDR	输入	C_AXI_ADDR_WIDTH	上游读地址。
S_AXI_ARPROT	输入	3	上游读访问属性。
S_AXI_ARVALID	输入	1	上游读地址有效。
S_AXI_ARREADY	输出	1	AR 通道可以接收输入时有效。
S_AXI_RDATA	输出	C_AXI_DATA_WIDTH	返回上游的读数据。
S_AXI_RRESP	输出	2	返回上游的读响应。
S_AXI_RVALID	输出	1	返回上游的读数据有效。
S_AXI_RREADY	输入	1	上游可以接收读数据时有效。

端口名	方向	位宽	功能
M_AXI_ARADDR	输出	C_AXI_ADDR_WIDT H	下游读地址。
M_AXI_ARPROT	输出	3	下游读访问属性。
M_AXI_ARVALID	输出	1	下游读地址有效。
M_AXI_ARREADY	输入	1	下游可以接收读地址时有效。
M_AXI_RDATA	输入	C_AXI_DATA_WIDT H	下游读数据。
M_AXI_RRESP	输入	2	下游读响应。
M_AXI_RVALID	输入	1	下游读数据有效。
M_AXI_RREADY	输出	1	R 通道可以接收下游数据时有效。

表 110 AXI4-Lite 只读乒乓缓存包装端口

读写包装端口如下表所示。AW、W、B、AR 和 R 通道各自使用独立通道流水，不共享有效状态。

端口名	方向	位宽	功能
clk	输入	1	五个 AXI4-Lite 通道的缓存时钟。
rst_n	输入	1	低电平有效异步复位，清空全部通道有效状态。
S_AXI_AWADDR	输入	C_AXI_ADDR_WIDT H	上游写地址。
S_AXI_AWPROT	输入	3	上游写访问属性。
S_AXI_AWVALID	输入	1	上游写地址有效。
S_AXI_AWREADY	输出	1	AW 通道可以接收输入时有效。
S_AXI_WDATA	输入	C_AXI_DATA_WIDT H	上游写数据。
S_AXI_WSTRB	输入	C_AXI_DATA_WIDT H/8	上游逐字节写使能。
S_AXI_WVALID	输入	1	上游写数据有效。
S_AXI_WREADY	输出	1	W 通道可以接收输入时有效。
S_AXI_BRESP	输出	2	返回上游的写响应。
S_AXI_BVALID	输出	1	返回上游的写响应有效。
S_AXI_BREADY	输入	1	上游可以接收写响应时有效。

端口名	方向	位宽	功能
S_AXI_ARADDR	输入	C_AXI_ADDR_WIDT H	上游读地址。
S_AXI_ARPROT	输入	3	上游读访问属性。
S_AXI_ARVALID	输入	1	上游读地址有效。
S_AXI_ARREADY	输出	1	AR 通道可以接收输入时有效。
S_AXI_RDATA	输出	C_AXI_DATA_WIDT H	返回上游的读数据。
S_AXI_RRESP	输出	2	返回上游的读响应。
S_AXI_RVALID	输出	1	返回上游的读数据有效。
S_AXI_RREADY	输入	1	上游可以接收读数据时有效。
M_AXI_AWADDR	输出	C_AXI_ADDR_WIDT H	下游写地址。
M_AXI_AWPROT	输出	3	下游写访问属性。
M_AXI_AWVALID	输出	1	下游写地址有效。
M_AXI_AWREADY	输入	1	下游可以接收写地址时有效。
M_AXI_WDATA	输出	C_AXI_DATA_WIDT H	下游写数据。
M_AXI_WSTRB	输出	C_AXI_DATA_WIDT H/8	下游逐字节写使能。
M_AXI_WVALID	输出	1	下游写数据有效。
M_AXI_WREADY	输入	1	下游可以接收写数据时有效。
M_AXI_BRESP	输入	2	下游写响应。
M_AXI_BVALID	输入	1	下游写响应有效。
M_AXI_BREADY	输出	1	B 通道可以接收下游响应时有效。
M_AXI_ARADDR	输出	C_AXI_ADDR_WIDT H	下游读地址。
M_AXI_ARPROT	输出	3	下游读访问属性。
M_AXI_ARVALID	输出	1	下游读地址有效。
M_AXI_ARREADY	输入	1	下游可以接收读地址时有效。
M_AXI_RDATA	输入	C_AXI_DATA_WIDT H	下游读数据。
M_AXI_RRESP	输入	2	下游读响应。
M_AXI_RVALID	输入	1	下游读数据有效。

端口名	方向	位宽	功能
M_AXI_RREADY	输出	1	R 通道可以接收下游数据时有效。

表 111 AXI4-Lite 读写乒乓缓存包装端口

每一级可保存两项，在满状态下允许同周期输出和输入，此时占用计数保持不变，连续吞吐为每周期一笔。非直通配置在无暂停时每级为相应传输方向增加一个周期；M0 的 AR 通道开启空缓存直通后，空状态不增加周期，只有发生下游暂停时才保存字段，M0 的 R 通道固定为零级。推荐的 M1 配置为 AW、W、B、AR 和 R 五个通道均使用一级非直通缓存。复位只清空有效状态和计数，保存字段不要求清零；下游暂停时，输出 VALID 和全部字段必须保持不变，直到完成握手。乒乓缓存不检查 RRESP、BRESP 或地址范围，也不改变 Cache 的请求编号和响应顺序。

6 分支预测参数化设计与对比

分支预测模块在同一套 RTL 中通过 ENABLE_BPU_ENHANCED、ENABLE_BPU_LEGACY 和 ENABLE_BPU_TAGE 条件生成。SoC 基础配置使用轻量 BPU，以满足 RV32IM 面积和时序约束；增强 BPU 用于控制流密集程序的性能配置；独立使用 core_top 时还可同时关闭两类预测器，由 BRU 在执行阶段处理全部控制转移。TAGE 是增强 BPU 内的可选方向预测结构，默认关闭。

6.1 参数化范围与配置传递

分支预测相关参数由 SoC 顶层传到 core_top 和 ifu。ifu 优先生成 enhanced_bpu，其次根据 ENABLE_BPU_LEGACY 生成 sbpu，两类开关均为 0 时生成无预测器分支。增强 BPU 关闭后不保留其预测表和预测状态寄存器；TAGE 关闭后只保留增强 BPU 的 BTB、GShare、循环预测器和 RAS 结构。

参数	所在层次	当前设计含义
ENABLE_BPU_ENHANCED	SoC/Core/IFU	打开增强版分支预测；为 1 时优先生成 enhanced_bpu。
ENABLE_BPU_LEGACY	Core/IFU	在增强 BPU 关闭时控制是否生成基础 sbpu；SoC 顶层默认在增强 BPU 关闭时将其置为 1。
ENABLE_BPU_TAGE	IFU/enhanced_bpu	在增强 BPU 内生成 TAGE-like 多历史表。该开关只有增强 BPU 打开时有效。

参数	所在层次	当前设计含义
BPU_BHT_ENTRIES	sbpu	基础 BPU 的 2 位方向计数器表项数，RTL 按 2 的幂规整。
ENABLE_BPU_LOOP_PRED	sbpu/enhanced_bpu	循环预测器开关。关闭时循环次数、标签和置信状态表项不应进入生成结构。
BPU_LOOP_ENTRIES	sbpu/enhanced_bpu	循环预测器表项数。
BPU_LOOP_TAG_WIDTH	sbpu/enhanced_bpu	循环预测器标签位宽。
BPU_LOOP_COUNT_WIDTH	sbpu/enhanced_bpu	循环迭代计数字段位宽。
ENABLE_BPU_CORR_PRED	sbpu	基础 BPU 的相关方向预测开关。
BPU_CORR_HISTORY_ENTRIES	sbpu	基础相关方向预测的局部历史表项数。
BPU_CORR_HISTORY_BITS	sbpu	基础相关方向预测的局部历史长度。
BPU_CORR_PHT_ENTRIES	sbpu	基础相关方向预测的 PHT 表项数。
BPU_JALR_ENTRIES	sbpu	基础 BPU 的 JALR 目标缓存表项数。
BPU_RAS_DEPTH	sbpu	基础 BPU 返回地址栈深度。
BPU_BTBTB_SETS	enhanced_bpu	增强 BPU 的 BTB 组数，保存标签、控制流类型和目标地址。
BPU_GSHARE_ENTRIES	enhanced_bpu	GShare PHT 表项数。
BPU_GHR_BITS	enhanced_bpu/dispatch/exu_bru	全局历史长度，同时决定预测快照字段位宽。
BPU_ENH_RAS_DEPTH	enhanced_bpu	增强 BPU 返回地址栈深度，与基础 BPU 的 BPU_RAS_DEPTH 分开传递。
BPU_TAGE_TABLES	enhanced_bpu	TAGE 表数量，RTL 内限制为 2 到 4 张表。
BPU_TAGE_ENTRIES	enhanced_bpu	每张 TAGE 表项数，综合时按 2 的幂规整。
BPU_TAGE_TAG_WIDTH	enhanced_bpu	TAGE 标签位宽，影响误命中率和比较器面积。
BPU_TAGE_HIST0	enhanced_bpu	第 0 张 TAGE 表的历史长度。
BPU_TAGE_HIST1	enhanced_bpu	第 1 张 TAGE 表的历史长度。

参数	所在层次	当前设计含义
BPU_TAGE_HIST2	enhanced_bpu	第 2 张 TAGE 表的历史长度。
BPU_TAGE_HIST3	enhanced_bpu	第 3 张 TAGE 表的历史长度。

表 112 BPU 相关参数含义表

6.2 基础轻量 BPU

基础 sbpu 面向面积优先配置。静态部分把后向条件分支以及前向 BNE、BGEU 作为倾向跳转类型，其余前向条件分支倾向不跳转；BHT 和相关方向预测器学习是否采用静态结果。BHT 和相关方向计数器初始为 2'b10，表示弱同意静态方向；相关方向选择计数器初始优先基础 BHT。直接 JAL 目标由返回指令时预先计算的 PC 相对目标给出；JALR 返回优先使用 RAS 栈顶，非返回类 JALR 可命中小型目标缓存；循环预测器使用少量状态改善固定次数循环的最后一次退出。

基础 BPU 的训练策略较保守。前端只输出预测方向、预测目标和少量标记；is_pred_branch_o 只标识被预测为跳转的条件分支，is_pred_jalr_o 只标识 RAS 或目标表命中并形成预测跳转的 JALR。固定 32 位指令配置中，direct_predict_o 表示被预测为跳转的条件分支或 JAL，jalr_predict_o 表示命中目标的 JALR；C 扩展配置中这两个快速控制输出固定为零，统一使用 branch_taken_o 和 branch_addr_o。BRU 在 EXU 阶段计算真实方向和真实目标后决定是否冲刷。若预测跳转但真实不跳，BRU 把 PC 修正到顺序地址；若预测不跳但真实跳转，BRU 把 PC 修正到真实目标；若方向正确但目标错误，BRU 仍触发取指地址修正并训练目标缓存。

RAS 只在直接 JAL 调用时压入返回地址，RV32 的 C.JAL 也属于压入事件；当前设计不把 JALR 调用作为 RAS 压入事件。BHT、循环表、相关历史表、相关模式历史表和 JALR 目标表的表项数至少为 2，并向上调整为 2 的幂；循环标签位宽和循环计数字段位宽至少为 1；相关历史长度至少为 1 且不超过相关模式历史表索引位宽；RAS 深度至少为 1，不要求为 2 的幂。循环表和 JALR 目标表初始无效。低电平复位抑制训练和 RAS 状态变化，但不清除运行期间已经学习的预测表内容。

6.3 增强 BPU 与可选 TAGE 设计

增强 enhanced_bpu 以 PC 查询预测表，不等待指令返回后再解析操作码。IFU 给出 pc_i 和 pc_valid_i 后，增强 BPU 并行查询 BTB、GShare PHT、循环预测器、RAS，并在 TAGE 打开时额外查询多张带标签历史表，输出

branch_taken、branch_addr、pred_taken、pred_target 和 pred_ghr。请求阶段的预测状态保存在增强 BPU 自身的两项数组中，并使用 ifu_axi_master 给出的地址 FIFO 写入和指令 FIFO 读出事件推进；随后随同一条指令通过 ifu_pipe、IDU 保留结构、Dispatch 和 EXU。

增强 BPU 的主要结构如下表 113 所示。

结构	设计实现	性能作用	面积/时序代价
BTB	按 PC 索引，保存有效位、标签、类型和目标。类型用于区分条件分支、JAL、CALL、RET、间接跳转。	让取指阶段提前获得目标地址，减少 JAL/JALR/RET 反复等待 EXU 修正取指地址。	标签比较和目标选择器进入 IFU PC 路径，表项过大会直接压低 WNS。
GShare	PC 索引与 GHR 异或得到 PHT 索引，计数器按真实跳转/不跳转饱和和更新。	对同一 PC 在不同历史上下文下的方向差异建模，提升普通条件分支预测。	GHR 快照需要随流水传递，PHT 读和 BTB 结果要在前端同拍合并。
循环预测器	保存循环 PC、迭代计数和置信状态。	小循环退出前后方向稳定，可用很小表项换到较高收益。	表项不能过深，否则收益可能被 IFU 表项选择器的面积与时序代价抵消。
RAS	CALL 写入返回地址，RET 预测时优先使用栈项。	函数返回目标通常由调用点决定，RAS 能避免 RET 被 BTB 冲突污染。	深度增大会增加指针、栈存储和冲刷恢复状态。
TAGE	生成 2 到 4 张不同历史长度的带标签表，每项包含标签、方向计数器和有用位。	对长历史相关条件分支更强，局部程序可提高方向准确率。	多表标签比较、提供者选择和分配更新面积大，默认关闭。

表 113 增强 BPU 结构表

TAGE 由 ENABLE_BPU_TAGE 条件生成。BPU_TAGE_TABLES 的有效范围限制为 2 到 4，避免过多预测表造成面积和时序代价失控。关闭 TAGE 时，提供者命中、TAGE 方向选择、有用位更新和分配逻辑都不生成，增强 BPU 退化为 BTB、GShare、循环预测器与 RAS 的组合。

6.3.1 增强 BPU 查询时序与条件分支恢复

增强分支预测单元中的分支目标表、方向预测表和带标签历史表各保存一份硬件状态，并使用独立的预测读端与训练读端访问同一存储阵列。BTB 使用一写

一异步读存储单元；BHT 和每张 TAGE 表使用一写两异步读存储单元，两个读端分别服务预测查询和训练查询。这些存储单元采用同步写、异步读，不设置复位、初始化或同地址读写前递。固定 32 位指令配置中，query_pc_q 和 query_ghr_q 保存下一次预测查询使用的程序计数器与全局历史；AXI4-Lite 读地址握手成功后，lookup_advance_i 允许查询状态前进。时钟沿之后，预测表异步读取这两个寄存器给出的地址和历史，并在下一个时钟沿前形成预测方向与目标地址。

- ◆ 时钟沿 N：取指单元写入下一程序计数器；预测单元保存查询地址和历史
- ◆ N 至 N+1：预测表读取该地址，形成预测方向和目标地址
- ◆ 时钟沿 N+1：取指单元根据预测或实际控制转移结果写入下一程序计数器

该结构在正常取指时支持每周期接受一次查询推进，同时减少预测表数量、训练写入扇出和程序计数器对预测表读地址的扇出。查询地址寄存器仅服务预测表读取，IFU 的程序计数器继续服务指令读地址。C 扩展开启时，增强 BPU 直接使用 pc_i 查询预测表，并以已确认控制流更新全局历史；下述条件分支专用恢复仅用于 C 扩展关闭的固定宽度配置。

条件分支的实际方向在分支比较结束后确定。发射级同时保存与当前预测方向相反时的取指地址和全局历史寄存器（GHR）值，无需为另一个方向再读取一份预测表。dispatch_pipe 在预测跳转时保存顺序地址，在预测不跳转时保存分支目标地址；恢复用 GHR 由预测历史左移后追加预测方向的反值形成。

信号	源模块	目标模块	位宽	定义
bpu_cond_recovery_pc_o	dispatch_pipe	ifu	INST_ADDR_WIDTH	条件分支方向错误时的实际下一取指地址。
bpu_cond_recovery_ghr_o	dispatch_pipe	ifu	BPU_GHR_BITS	按实际分支方向形成的 GHR 值。

表 114 条件分支恢复字段

exu_bru 在增强 BPU 开启、C 扩展关闭且当前指令为六种条件分支之一时，对方向预测错误给出两个互斥的输出信号。中断正在进入或分支目标地址不对齐时，专用信号保持无效。

信号	源模块	目标模块	位宽	触发条件	恢复地址
cond_taken_redirect_o	exu_bru	ifu	1	预测不跳转、实际跳转	分支目标地址。

信号	源模块	目标模块	位宽	触发条件	恢复地址
cond_rollback _redirect_o	exu_bru	ifu	1	预测跳转、 实际不跳转	顺序地址。

表 115 条件分支方向错误的恢复控制

IFU 的取指地址选择优先级为：中断入口、条件分支方向恢复、BRU 通用地址修正、分支预测、暂停保持和顺序取指。专用恢复信号有效时，IFU 在同一时钟沿将已保存的地址和 GHR 写入增强 BPU 的查询寄存器，不进入通用暂停过程。无条件跳转、JALR 目标错误、分支目标地址错误和取指屏障继续使用 BRU 通用地址修正。C 扩展开启时，顺序地址可能为当前地址加 2 或加 4，因此条件分支方向错误使用通用地址修正。

6.4 流水线信号传递与回写通路

增强 BPU 的正确性要求预测 PC、返回指令和后端真实结果保持对齐。取指端存在 AXI 未完成事务，请求发出和指令返回可能相隔多个周期；IDU 还有指令保留栈；Dispatch 可能因为 RAW/WAW 或执行单元忙状态保持等待槽。因此预测信息必须与指令本身采用相同的握手、暂停和冲刷规则。

阶段	预测相关流水信息	进入条件	保持/清除条件
IFU PC 选择	branch_taken、 branch_addr、BRU 恢 复、中断跳转	PC 可更新且无前端 暂停	中断优先于 BRU 恢 复，二者均优先于预 测跳转；暂停时 PC 保持
IFU AXI 主接口与 BPU 状态数组	AXI FIFO 保存取指 地址和直接目标； BPU 数组保存预测方 向、目标、类别和 GHR	AR 握手、地址 FIFO 写入或指令 FIFO 读 出事件	冲刷后返回的旧请求 不得重新置有效，各 状态数组与 FIFO 指 针同步推进
IDU/IRS	指令、PC、预测信 息、C 指令长度	IFU 返回有效指令且 IDU 可接收	保留栈满时反压；冲 刷清除错误路径
Dispatch	控制流类型、默认下 一 PC、预测信息	IDU 有效且 HDU 允 许发射	控制转移属于有序 类，不进入等待项； 冲刷时取消错误路径 状态
EXU/BRU	比较操作数、真实目 标、预测目标/GHR	Dispatch 发射 BJP 请求	BRU 在方向或目标不 一致时修正取指地 址，并输出训练信息
BPU 更新	更新 PC、目标、类 型、真实方向、GHR	BRU 更新有效且无中 断屏蔽	异常或中断路径不训 练错误控制流

表 116 分支预测流水线信息与控制状态表

BRU 以实际执行结果判断预测是否一致：条件分支由比较结果决定，JAL/JALR 在控制流请求有效时实际跳转；JALR 目标使用 $(rs1 + imm) \& \sim 1$ ；普通条件分支和 JAL 使用 Dispatch 已计算好的 $PC+imm$ 。方向不一致和目标不一致都会触发冲刷，但统计定义不同：方向命中率只关心跳转或不跳转，完整预测正确率要求方向和目标同时正确。

6.5 配置选择原则与设计结论

基础 BPU、增强 BPU 和 TAGE 采用互斥的条件生成关系。基础 BPU 保留小型方向表、循环预测器、相关方向预测、JALR 目标缓存和 RAS；增强 BPU 增加 BTB、GShare、增强 RAS 以及可选 TAGE。TAGE 只在增强 BPU 打开时生成，关闭时不保留表项、标签比较和训练状态。LSU L0 Cache、C/A/Zb 扩展以及非对齐访存同样通过独立参数选择，不改变 IFU、IDU、Dispatch、EXU 和 WBU 的公共接口。

配置项	关闭状态	打开状态	结构影响
基础 BPU	IFU 不生成基础预测表	生成小型 BHT、循环预测器、相关方向预测、JALR 目标缓存和 RAS	预测状态随取指请求和指令返回保持一致。
增强 BPU	不生成 BTB、GShare 和增强 RAS	按 PC 查询 BTB、GShare、RAS 及可选 TAGE	预测目标在指令返回前即可形成。
TAGE	不生成多历史表和标签比较	在增强 BPU 内生成 2 至 4 张带标签历史表	训练字段、表项和选择控制只在增强 BPU 内有效。
LSU L0 Cache	LSU 直接使用普通 AXI4-Lite 访存路径	Dispatch 保存标签/有效位影子表，EXU LSU 保存数据体	命中加载沿普通写回和旁路通路返回。
Zba/Zbb/Zbs/Zbc	IDU 不生成对应译码和控制字段	IDU、EXU 按扩展生成指令处理路径	扩展只改变相关指令类别，不改变主流水边界。

表 117 可选结构的配置选择原则

7 时序收敛优化设计

Alkaid 的时序难点集中在宽数据旁路、分支取指地址修正、同步总线存储器、可选预测表和外设中断控制等路径交汇的位置。本项目的时序优化原则是：为每条跨模块数据路径设置明确的寄存器边界，以 CPU 核和 SoC 顶层分别统计资源，并通过条件生成隔离关闭的可选结构。

7.1 关键路径来源分析

最典型的长路径有四类。

第一类是执行结果旁路到分支比较。若把 LSU 同步 RAM 返回数据和 ALU 结果拉回 Dispatch，再由 Dispatch 组合决定分支操作数，会形成“执行数据 -> HDU/Dispatch 选择 -> BRU 比较 -> 取指地址修正 -> IFU PC 使能/IDU 保留结构写使能”的长组合逻辑路径。当前四级和五级只在 EXU 内允许条件分支选择 ALU 或 LSU 结果，不允许选择 MUL 结果；三级关闭该局部旁路。

第二类是 IFU PC 更新路径。基础 BPU、增强 BPU、EXU 取指地址修正、CLINT 陷入、暂停/冲刷和 C 指令半字对齐都会影响下一 PC。PC 使能和 PC 数据端扇出高，任何把分支比较、保留栈写使能或 AXI 未完成事务状态直接接入 PC 更新的改写，都可能把最长路径转移到前端。

第三类是 Dispatch 与 BRU 地址生成路径。存储器访问地址以及 JAL 和条件分支目标需要加法器；三级和四级的非增强预测配置共享一套 Dispatch 加法器，五级的非增强预测配置使用两套独立加法器，增强 BPU 配置为当前指令和等待项预先形成候选地址。JALR 目标由 BRU 使用 rs1 与 I 型立即数独立计算，不使用 AGU 保存数据。

第四类是可选表项和参数化流水线数据。增强 BPU、TAGE、LSU L0 Cache、非对齐访存、C/A/Zb 扩展都会增加流水线数据位宽和选择器。如果开关关闭后只是输出常量但仍保留表项或宽寄存器，面积和时序都会受到影响。因此本项目要求使用条件生成块隔离结构，而不是仅在运行时屏蔽使能。

7.2 级间寄存器与组合逻辑路径拆分

流水线寄存器位置由 PIPELINE_STAGES 确定。ifu_pipe 在三种配置中都保存指令、PC 和预测信息；idu_id_pipe 只有五级配置采用；dispatch_pipe 在四级和五级配置中保存执行请求、操作数、commit id、存储器访问地址、直接控制转移目标和可选预测信息，三级配置则组合直通。WBU 输出寄存器在全部配置中保留。

四级和五级的宽数据旁路被限制在 EXU 局部暂存区内。ALU、LSU 和 MUL 完成后把结果写入本地暂存区，后续指令在 EXU 内通过局部选择器选择操作数。Dispatch 与 HDU 根据依赖 commit id 和按 commit id 展开的旁路

可选状态判断是否可发射，不把完整 32 位或 64 位数据拉回前一级。三级配置不生成局部旁路，依赖必须等待 WBU 完成。

存储器访问地址是例外，因为 LSU 发起 AXI4-Lite 请求前必须得到确定地址。Dispatch 内的 AGU 保存项处理地址基址依赖：前序 ALU 结果可直接写入保存项，前序 LSU 结果需要先使依赖指令进入等待项，再在结果到达后保存基址并唤醒。该机制只服务存储器访问 rs1，不用于 JALR、条件分支或普通运算操作数。

LSU L0 Cache 的时序处理也遵循同一原则。Dispatch 侧只保存标签/有效位影子表并判断命中，EXU LSU 侧保存数据体。命中信息由一位命中标志、路号和索引组成，并随发射进入 EXU；EXU 只在 LSU 内部选择缓存数据或 AXI 读返回数据。这样可以减少依赖加载的分支等待，但不会把缓存数据组合送到 Dispatch 分支比较路径。

各参数组合采用以下时序处理规则。

对象	处理方式	功能保持条件
主流水线	按三级、四级、五级配置选择寄存器边界	指令有效位、控制字段和 commit id 跨级对齐，异常处理行为保持不变。
三级数据冒险检查	依赖指令等待写回或完成通知，不将执行结果直接反馈到当前发射判断	执行结果不影响同周期发射决定。
四级条件分支	复用算术逻辑单元比较结果	比较结果与分支条件类型一一对应。
分支预测查询	使用预测单元内部查询地址寄存器与单份预测表	取指地址、恢复地址和全局历史在时钟边沿对齐。

表 118 参数化结构的时序处理规则

三级、四级和五级配置分别采用与目标频率对应的寄存器边界，Cache、乒乓缓存和分支预测结构则按各自参数独立生成，不在不同配置之间复用需要保存状态的内部结构。

7.3 面积优化与参数默认值策略

面积优化要求每个可选功能满足“关闭后不生成相关结构、打开后接口字段完整”的原则。RV32IM + Zicsr 基础配置关闭 ENABLE_MISALIGNED_DATA、ENABLE_LSU_HOT_CACHE、ENABLE_BPU_ENHANCED、ENABLE_BPU_TAGE 和可选指令扩展，只保留

必要执行路径。RV64、C、A、Zba/Zbb/Zbs、LSU L0 Cache、增强 BPU 和外设模块都作为参数化可选项。

对面积敏感的结构优先使用小状态机和局部寄存器。例如 DIV 顺序除法采用紧凑状态编码和试减法复用比较；MUL 避免默认恢复大组合快路径；WBU 对 ALU/MUL 使用小 FIFO 缓解同拍冲突，而不是给每个执行单元建立完整重排序缓冲；HDU 使用 4 个 commit id 槽位表示有限乱序窗口，而不是扩大成通用发射队列。

外设模块也通过参数生成。perip_top 中 Timer、SPI、I2C0、I2C1、UART0、UART1、GPIO0 和 GPIO1 均有独立 ENABLE_PERIP_* 开关。参数关闭时裁剪内部实例并提供固定 APB 返回和安全引脚状态，但不改变 SoC RTL 顶层端口集合。UART1、I2C、SPI 和 PWM 依赖 GPIO0 IOF；若 GPIO0 关闭，这些功能无法送到顶层 GPIO 引脚。

7.4 实现约束与层次边界

目标器件、目标时钟、同步存储器读出方式和 AXI4-Lite 接口时序共同决定流水线寄存器位置。三级、四级和五级配置使用独立的条件生成分支选择级间寄存器；可选功能关闭时不生成对应表项、比较器、数据选择器和流水字段，打开时则在原有模块边界内增加所需状态与控制信号。

CPU 核与 SoC 顶层保持稳定的层次边界。CPU 核只包含取指、译码、分发、执行、写回、寄存器和中断控制；SoC 顶层连接 AXI 互连、Cache、乒乓缓存、存储器和外设。该层次划分使处理器配置变化不会改变 M0/M1 主接口定义，也不会把外设选择逻辑引入 CPU 内部流水控制。

约束来源	影响对象	设计处理
目标时钟	IFU、IDU、Dispatch、EXU 与 WBU 级间路径	由 PIPELINE_STAGES 选择三级、四级或五级寄存器边界。
同步存储器	IFU、LSU、ICache 与 DCache	使用请求跟踪、响应 FIFO 和已登记的读出数据，不假设地址发出周期即可取得数据。
AXI4-Lite 接口	M0/M1 请求与响应通道	各通道独立握手，未完成事务状态只在接口模块和对应 FIFO 内保存。

约束来源	影响对象	设计处理
可选功能参数	BPU、TAGE、LSU L0 Cache、指令集扩展与非对齐访存	通过条件生成增加或删除完整硬件结构，不使用运行时固定使能代替结构裁剪。

表 119 实现约束对模块结构的影响

配置层次	主要参数	参数作用范围
SoC 顶层	存储容量、Cache、乒乓缓存和外设使能	决定 CPU 外部存储访问结构、外设实例和顶层引脚功能。
CPU 顶层	数据宽度、流水线级数、指令集扩展和分支预测选择	传入 IFU、IDU、Dispatch、EXU、HDU、WBU、GPR 和 CSR。
前端与分发	BPU 表项、TAGE 表项、等待项和流水寄存器选择	决定预测结构、指令保持方式和发射级寄存边界。
执行与写回	LSU L0 Cache、非对齐访存、乘除法结构和写回 FIFO 深度	决定执行单元内部状态、局部旁路和完成结果保存方式。
存储访问	Cache 容量、相联路数、未完成请求深度和乒乓缓存级数	决定 M0/M1 外部缓冲、标签数据阵列和响应顺序控制。

表 120 参数配置的层次传递关系

8 流水线冒险与控制设计

Alkaid 采用轻度乱序单发射与有限乱序完成。取指和译码保持程序顺序，Dispatch 每周期最多向 EXU 发射一条指令；三级和五级配置可在一项等待结构中保存被阻塞的非有序类指令，并允许无寄存器冲突、无顺序约束的当前指令先发射，四级配置则保持被阻塞的当前输入。ALU、LSU、MUL、DIV 和 CSR 的完成时间不同，较晚发射的短延迟指令可能先完成。设计使用 commit id 状态、可选等待项、四级和五级的 EXU 局部旁路以及 WBU 最新 commit id 过滤保证架构状态正确。

8.1 RAW/WAW/结构冒险分类

四级和五级 RAW 冒险通过两份 32 项最近分配表查询 rs1 与 rs2 对应的 commit id，并结合四个未完成槽位以及 ALU、LSU、MUL 旁路可选状态判断。五级在 ID 推进时保存依赖编号，四级直接使用当前组合译码地址查询。三级使用四项紧凑比较结构并关闭 EXU 局部旁路，依赖必须等待 WBU 完成。

写后写处理随执行类别和流水线级数变化。四级和五级对较早的 DIV、CSR、分支、跳转与 OTHER 类写入实施阻塞，允许 ALU、MUL 和加载的同 rd 指令重叠；WBU 为每个 GPR 保存最新 commit id，被较晚指令替代的结果只释放 HDU 槽位。三级对全部尚未完成的同 rd 写入实施阻塞，WBU 不需要最新 commit id 过滤。

结构冒险来自执行单元忙状态、LSU 未完成事务、WBU 仲裁和 IDU/Dispatch 保留队列。执行单元不可接收请求时，Dispatch 停止发射或把当前指令捕获到等待槽；WBU 未就绪时执行单元保持结果，不提前释放 HDU；IDU 指令保留栈满时 IFU 取指被反压，避免预测信息和指令错位。

冒险类型	触发条件	处理机制	正确性约束
ALU -> ALU/BRU RAW	四级或五级的当前指令依赖前序 ALU 结果	EXU 局部 ALU 暂存数据可用于 ALU 或条件分支	数据只在 EXU 操作数选择器使用；三级等待 WBU 完成
LSU -> ALU/BRU RAW	四级或五级的当前指令依赖加载结果	LSU 结果就绪后可用于 ALU 或条件分支	R 通道握手前不得解除依赖；三级等待 WBU 完成
MUL -> ALU/MUL/存储数据 RAW	四级或五级的当前指令依赖乘法结果	EXU 局部 MUL 暂存数据可用于 ALU、MUL 或存储数据	条件分支不能直接选择 MUL 结果
ALU/LSU -> 存储器访问地址	加载或存储基址依赖前序结果	ALU 结果可直接进入 AGU 保存项；LSU 依赖先进入等待项，结果到达后保存	只用于存储器访问基址，不用于 JALR
JALR 地址依赖	JALR 目标依赖寄存器	HDU 阻塞至 WBU 与 GPR 提供操作数，BRU 再计算目标	不使用 AGU 保存项或 EXU 局部旁路
WAW	多条未完成指令写同一 rd	四级和五级按类别阻塞并由 WBU 最新 commit id 过滤；三级全部阻塞	被替代结果仍释放 commit id，但不写 GPR
WBU 端口冲突	多执行单元同拍完成	按流水线配置采用固定优先级，并使用 ALU/MUL 小型暂存结构	暂存结果保留数据与 commit id，最终写入资格由对应配置的 WAW 机制决定

表 121 流水线冒险分类与处理机制表

8.2 单项等待槽状态机

三级和五级 Dispatch 使用一项等待结构，保存已经译码但源操作数暂时不可用的非有序类指令。保存字段包括译码控制、立即数、PC、原始指令、源寄存器数据、目的寄存器、写使能、源寄存器读使能、执行类别、依赖 commit id 和 C 指令长度标志，不保存分支预测状态。从等待项发射时预测输出为零；分支、跳转、CSR、系统、存储和非法指令不进入等待项。四级配置不启用该结构。

等待结构使三级和五级 Dispatch 具备受限越过发射能力。旧指令等待操作数时，当前 IDU 指令若与它没有 RAW、WAW 或顺序冲突，可以先进入执行单元。每周期仍只发射一条指令，发射次序只在一项等待范围内变化，完成次序由执行单元延迟决定。commit id 只有 4 项，四级和五级 WBU 通过每个 GPR 的最新 commit id 保护寄存器状态，三级则由 HDU 阻塞全部 WAW。该机制不提供任意深度重排序，也不允许错误路径指令修改架构状态。

等待槽的状态可以概括为四种。

状态	进入条件	输出行为	退出条件
空闲	三级或五级的 wait_valid=0	Dispatch 优先尝试发射当前 IDU 指令	非有序类指令因源操作数未就绪且允许保存
等待操作数	RAW 依赖尚未就绪	HDU 继续监控 WBU 完成和旁路暂存区有效信号	前序依赖指令完成或对应暂存数据可用
可发射	操作数就绪且执行单元可接收	等待槽指令优先于新指令进入 dispatch_pipe	发射后，已登记的退休事件在下一周期使 wait_valid 失效；若本拍同时捕获当前指令，则替换为新等待项并保持有效
冲刷	EXU 取指地址修正或 CLINT 陷入	清除错误路径等待项，并取消相应架构写入资格	冲刷完成后回到空闲

表 122 等待槽状态表

等待项存在时，非有序类当前指令可在无 RAW、WAW 和结构冲突时越过。五级配置还允许不读取等待项目的寄存器、且不写同一目的寄存器的前向条件分支在等待项未就绪时先发射；三级配置不允许该情况，四级配置不生成等待项。其它控制转移仍保持顺序。

等待结构与 HDU 分三步协作。捕获时保存 rs1 与 rs2 就绪状态、依赖 commit id 和允许的旁路来源；随后根据 WBU 完成、ALU/LSU/MUL 旁路可用和 AGU 保存项状态更新就绪条件；发射时与当前 IDU 指令共用 dispatch_logic 和 dispatch_pipe，不会跳过地址生成或异常检查。等待项不携带 BPU 预测信息，因此只适用于不改变控制流的指令。

该越过发射的禁止条件比普通 ALU RAW 更保守。除允许越过的五级前向条件分支外，若当前指令读取等待槽的 rd、写同一个 rd、访问 CSR/SYS、可能产生异常、需要按顺序执行存储，或改变 PC，则必须等待等待槽先发射或被冲刷清除。也就是说，等待槽的性能收益来自不相互依赖的短延迟指令先发射，而不是通过推测执行破坏程序顺序。

8.3 加载后分支瓶颈与 LSU L0 Cache

数据存储器通过 AXI4-Lite 同步返回后，加载结果不能在同一周期供下一条分支比较使用，依赖指令必须由 HDU 保持到结果有效。LSU L0 Cache 将标签检查前移到 Dispatch，将数据体保留在 EXU LSU 内部，在不改变 AXI4-Lite 接口的条件下缩短热点加载的返回路径。

LSU L0 Cache 的设计不是通用一致性缓存。Dispatch 侧保存标签/有效位影子表，用于判断当前加载地址是否命中；EXU LSU 侧保存数据体，用于命中时返回数据。只有可缓存的 DMEM 地址进入表项，存储、AMO、拆分/非对齐访问、外设、CLINT、PLIC 和 APB 访问都按保守规则失效或绕过。命中时，LSU 可在内部把加载命中数据作为返回结果送往 EXU 局部旁路和 WBU，随后的分支指令不必等待普通 AXI 读返回。

该结构只覆盖小工作集的加载访问，不改变普通 AXI4-Lite 访问、非对齐拆分和原子操作状态机。参数关闭时不生成影子标签、数据 RAM 和状态修改端口；参数打开时，LSU L0 Cache 的深度和路数由 LSU_HOT_CACHE_DEPTH、LSU_HOT_CACHE_WAYS 设置。

8.4 冲刷、异常、中断与接口兼容

流水线控制采用统一的有效、暂停和冲刷规则。stall_flag_o[0] 表示冲刷，stall_flag_o[1] 表示 IF 暂停，stall_flag_o[2] 表示 ID 暂停，stall_flag_o[3] 表示 Dispatch 暂停。Ctrl 以组合逻辑产生这些信号：IF 暂停来自指令保留结构，ID 暂停为 EXU 与 HDU 暂停的逻辑或，Dispatch 暂停只来自 EXU，冲刷为

有效 BRU 地址修正请求与 CLINT 请求的逻辑或。EXU 暂停会抑制 BRU 地址修正，HDU 暂停不会取消已经在 EXU 确认的控制转移。

异常和中断由 CLINT 陷入逻辑统一处理。同步异常包括非法指令、ecall、ebreak 和取指或访存地址不对齐，异步中断包括外部、定时器和软件中断。事件先保存在 S_INT_IDLE 的待处理有效位中，等待原子长指令锁定和存储事务结束，再依次写 mepc、mstatus、可选 mtval 和 mcause。int_assert_o 触发冲刷，int_jump_o 与 int_addr_o 控制 IFU 采用 mtvec 或 mepc；四级配置比三级和普通五级延后一拍发出跳转。

接口兼容性要求每个模块对有效、就绪、暂停、冲刷和提交释放的解释一致。IFU 返回指令必须携带同一请求的 PC 和预测信息；IDU 保留栈不能只保存指令而丢失 C 长度或 GHR；Dispatch 分配 commit id 后若被冲刷，HDU 必须回滚；EXU 若结果未被 WBU 接收，不能释放槽位；WBU 无论是否真正写 GPR，都必须对完成指令输出提交释放。这些接口不变量使参数化功能可以组合：打开 RV64、C、A、Zb、LSU L0 Cache 或增强 BPU 时，只增加对应流水线数据和功能路径，不改变其他模块对级间协议的基本假设。

RISC-V CPU 性能与验证报告

1 验证目标与范围

本章面向当前 CPU 设计，说明验证对象、验证环境、测试方法、测试用例和性能分析。验证覆盖三级、四级和五级流水线配置，以及 RV32、RV64、I、M、A、C、Zicsr、Zifencei、Zba、Zbb、Zbs 和 Zbc。模块级测试用于检查接口和状态控制，整核程序用于检查架构结果，覆盖率统计用于观察 RTL 与 feature 的执行范围。各项结论均基于相同配置下完成的功能检查、覆盖率统计和性能数据。

1.1 流水线配置

三个流水线配置分别完成 RTL 展开、编译、仿真和覆盖率采集，不共用编译数据库、测试数据库或整核程序镜像。下表在首次使用配置编号时定义其含义；后文的 P3、P4 和 P5 均指代此表所列流水线配置，而不表示未定义的配置档位。

配置编号	REGRESSION_STAGES	主流水线	主要验证重点
------	-------------------	------	--------

配置编号	REGRESSION_STAGES	主流水线	主要验证重点
P3	3	取指、译码-发射-执行、写回	合并执行级、紧凑型 HDU、单周期 RV32 MUL 和异常保持
P4	4	取指、译码-发射、执行、写回	组合译码发射、局部旁路、异常目标地址保持和写回处理
P5	5	取指、译码、发射、执行、写回	完整级间寄存器、局部旁路、地址生成和写回处理

表 123 流水线验证配置

1.2 验证范围

验证范围包括 CPU 核、AXI4-Lite 互连、AXI2APB 转换、IMEM、DMEM、SoC 顶层组合逻辑和自研通用 RTL。外部外设 IP 核不计入自研 RTL 覆盖率，但外设子系统顶层、地址选择和中断接入仍由整核程序及相应模块测试检查。Cache 与乒乓缓存的专项设计和性能分析不纳入本章的验证结论。

范围	主要对象	检查内容
CPU 前端	IFU、基础 BPU、增强 BPU、IDU、压缩指令处理	取指请求、返回匹配、预测、地址修正、指令流整理和译码
分发与执行	Dispatch、HDU、ALU、BRU、MUL、DIV、LSU、CSR 执行单元、WBU、GPR	数据冒险、发射、旁路、长延迟完成、写回和 x0 抑制
中断与系统控制	CLINT、PLIC、CSR、Ctrl	机器模式异常、中断、mret、CSR 写入、MMIO 和中断优先级
存储与总线	AXI 互连、AXI2APB、IMEM、DMEM、通用 RAM/FIFO	AXI4-Lite 五通道握手、目标选择、读写顺序、APB 等待和存储器响应
SoC 顶层	alkaid_soc_top、外设子系统顶层	时钟复位连接、地址空间、MMIO、程序镜像加载和外设中断接入

表 124 验证范围

1.3 验证结果判定方式

每项模块测试必须满足自检错误计数为零、断言无失败、规定的完成标志出现，并生成与该 Case 对应的日志和覆盖率数据库。整核程序必须满足程序内部检查为零失败、tohost 成功写入、系统测试平台输出 SYSTEM_CHECK_PASS、程序计数器活性检查未超时，并在适用程序中通过 Spike 参考模型的八个 32 位程序结果校验字比较。覆盖率结果只有在任务状态、源码摘要、覆盖率数据库和合并报告来自同一次执行时才可用于统计。

2 验证环境与方法

2.1 验证环境结构

验证环境由 Verilator 系统级功能仿真平台和 VCS（数字逻辑仿真工具）验证平台组成。Verilator 平台运行官方 riscv-tests 指令集测试，并直接运行 CoreMark、排序程序及其他 C 语言功能程序，用于检查完整 CPU、存储器和 MMIO 协同工作时的架构结果。VCS 平台承担系统级回归、模块级验证、riscv-dv 随机程序、Spike 参考模型对照、断言检查、波形分析和覆盖率统计。两类平台使用相同的 RTL、流水线参数、指令集配置、程序镜像格式和程序结束协议，因此前者的功能测试结果可以作为后续回归验证的输入依据。

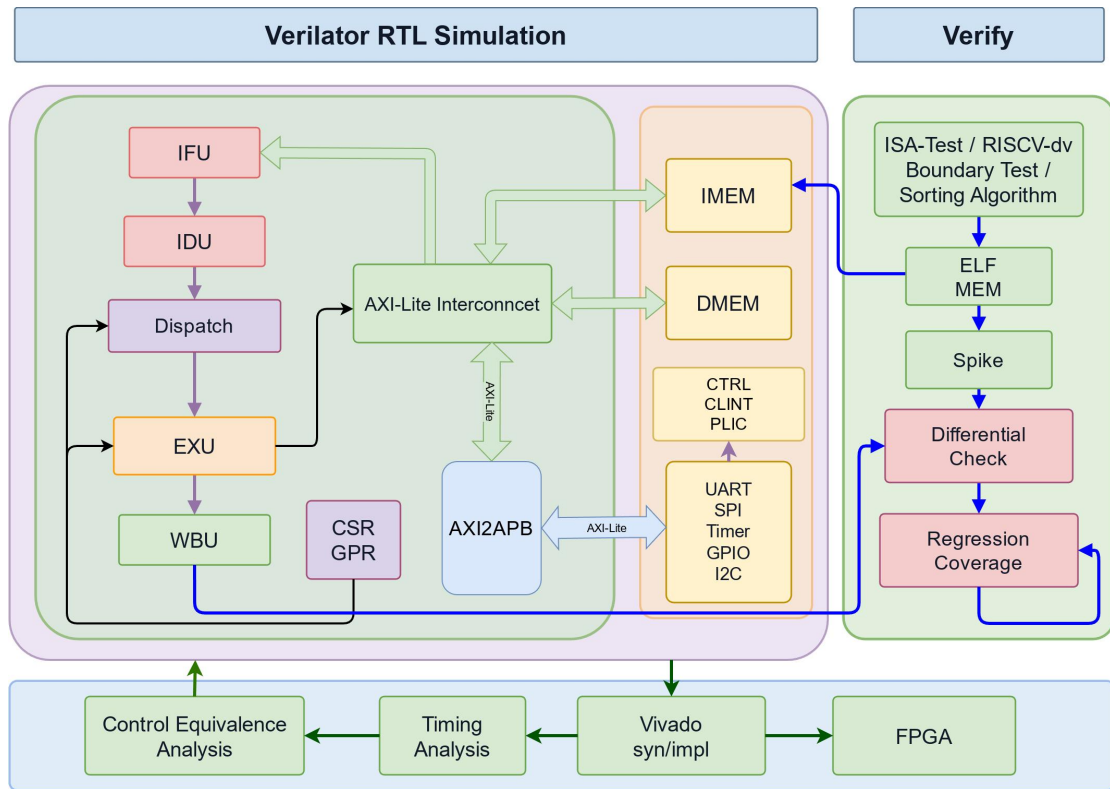


图 18 验证环境结构

图中左侧是测试程序、模块级自检测平台 and Case 配置。模块级测试平台直接驱动被测设计的端口事务，并在每个时钟沿检查请求、响应、复位、暂停和冲刷条件；协议类 Case 进一步检查 VALID/READY 交互、地址与数据保持、响应次序和错误响应。各模块 Case 使用独立的编译结果与覆盖率数据库，因此失败结果可以定位到具体模块、参数和测试步骤。

图中部是整核系统测试平台。该平台将 ELF（可执行与可链接格式）镜像装入指令紧耦合存储器（ITCM）和数据紧耦合存储器（DTCM），实例化完整 CPU、SoC 存储模型与 MMIO 模型，并持续观察 WBU 完成信息、GPR 写回、各执行单元有效信号、存储器写入和程序结束状态地址 tohost。IFU、IDU、Dispatch、ALU、MUL、DIV、LSU、CSR/CLINT 与 WBU 均有对应的整核 Feature 检查；当程序写入 tohost 后，平台还检查程序结果、程序计数器活性和断言状态。

图中右侧给出程序级参考比较和覆盖率处理。公共启动程序、C 程序、汇编程序及 riscv-dv 随机程序均编译为确定的 ELF；Spike 运行同一 ELF，RTL 运行由 RTL monitor（监视器）记录完成信息和程序结果区写入，比较程序据此生成结果报告。模块、整核和程序任务的 VCS 覆盖率数据库先按 P3、P4、P5 分别合并，再形成跨流水线统计结果。Verilator 覆盖率数据仅服务于 Verilator 测试，不与 VCS 覆盖率数据库合并。

2.1.1 仿真平台分工

Verilator 平台面向系统级功能验证，完整实例化 CPU、AXI4-Lite 存储器和 MMIO 模型。该平台运行官方 `riscv-tests` 指令集测试，并可直接运行 CoreMark、排序程序及其他 C 语言功能程序，检查指令执行、异常返回、存储器访问、程序结束状态及基本执行统计。其编译和运行速度适合在 RTL 或配置发生变化后检查多项程序的功能结果。

VCS 平台面向系统级回归分析和模块级验证。系统级任务运行定向程序、边界条件程序、排序程序及 `riscv-dv` 随机程序，并通过 RTL monitor、Spike 参考模型、断言和波形检查完成指令、寄存器写回、访存结果及异常处理；模块级任务直接施加端口事务，检查握手、状态保持、暂停、冲刷和异常分支。VCS 平台还为各流水线配置分别生成覆盖率数据库，并通过 URG（覆盖率合并与报告工具）完成覆盖率统计。

验证流程先在 Verilator 平台检查官方指令集测试和 C 语言功能程序，再由 VCS 平台针对 P3、P4、P5 配置执行更完整的系统级回归、模块级 Case 和覆盖率统计。两类平台分别保存日志和覆盖率数据，Verilator 的覆盖率数据不并入 VCS 覆盖率统计；程序内部检查结果、`tohost` 结束状态和程序结果校验字在两类平台中采用相同的检查方式。

2.2 Case 管理、构建与调度

Case 配置规定每项测试适用的流水线级数、编译参数、运行参数和检查器。调度程序将一项逻辑 Case 展开为 P3、P4、P5 的独立任务，并为每项任务建立单独的编译与运行环境。RTL、测试程序、Case 配置或运行参数发生变化时，相关任务重新编译和执行。

2.3 模块级自检测测试平台

模块级测试不依赖验证类库，而是从 DUT 的公开端口施加请求、响应、复位、暂停和冲刷条件。协议类测试逐次检查 VALID/READY 交互、地址和数据保持、响应顺序与错误响应；执行类测试检查操作码、操作数、commit id、异常和写回数据；状态类测试检查复位、冲刷、暂停、资源释放和恢复。每项 case 使用独立覆盖率数据库，因此失败可以定位到模块、参数与测试步骤。

2.3.1 RV32 ALU 运算与地址前递 Case

该 Case 将 ALU 作为独立组合执行单元检查，使用 RV32 数据宽度从公开接口逐项施加操作数、操作类型、结果选择、旁路选择和完成编号。每组输入在时钟上升沿之前建立，随后只保留一个请求周期；这样可以把一次运算的输入字段、结果字段和控制字段对应起来，避免把多个请求的结果混在同一观察窗口内。操作序列覆盖整数加法、减法、逻辑运算、移位、比较、LUI、AUIPC，以及当前配置打开的 Zba、Zbb 和 Zbs 操作。对需要地址生成的操作，还同时检查 ALU 结果是否能够作为 AGU 前递数据使用。

Case 的检查项目如下表所示。表中“实际操作数”指考虑 `alu_pass_op1_i` 和 `alu_pass_op2_i` 后送入运算逻辑的数据；旁路选择有效时，比较结果和算术结果都必须基于旁路数据，而不能继续使用原始寄存器读数。

检查项目	激励与观察信号	通过条件
运算结果	<code>alu_op1_i</code> 、 <code>alu_op2_i</code> 、 <code>alu_op_info_i</code> 、 <code>alu_result_sel_i</code> 、 <code>result_o</code>	<code>result_o</code> 与该操作类型的 RV32 参考结果一致，位宽截断和符号扩展符合指令定义。
操作数旁路	<code>alu_pass_op1_i</code> 、 <code>alu_pass_op2_i</code> 、 <code>alu_result_bypass_op1_i</code> 、 <code>alu_result_bypass_op2_i</code>	旁路选择置位时，结果和比较标志使用旁路数据；旁路选择清零时，使用原始操作数。
地址前递	<code>agu_forward_valid_o</code> 、 <code>agu_forward_result_o</code> 、 <code>agu_forward_arch_valid_o</code>	ADD、SUB、LUI、AUIPC 和 Zba 移位加法产生有效地址前递，其数据等于 <code>result_o</code> ；逻辑、普通移位和比较操作不产生地址前递。
比较结果	<code>compare_eq_o</code> 、 <code>compare_lt_signed_o</code> 、 <code>compare_lt_unsigned_o</code>	分别正确反映相等、有符号小于和无符号小于关系，且使用实际操作数。
完成信息	<code>commit_id_i</code> 、 <code>commit_id_o</code> 、 <code>reg_we_o</code> 、 <code>reg_waddr_o</code>	结果有效时 <code>commit_id_o</code> 与请求保持一致；写回使能和目的寄存器符合当前流水线配置。
控制与取消	<code>hazard_stall_i</code> 、 <code>int_assert_i</code> 、 <code>misaligned_fetch_i</code>	<code>hazard_stall_i</code> 只要求上游保持请求，不能改变组合结果； <code>int_assert_i</code> 抑制写回、旁路和地址前递， <code>misaligned_fetch_i</code> 取消架构写入资格并将目的寄存器地址置为 <code>x0</code> 。

图 18-1 展示了该 Case 的典型波形。顶部的 agu_forward_arch_valid_o 和 agu_forward_valid_o 只在允许地址前递的操作区间置位，agu_forward_result_o 与当前操作的 ALU 结果保持一致。中部的 alu_op1_i、alu_op2_i 和 alu_op_info_i 按请求逐组变化，完成编号 commit_id_o 按请求顺序变化；这三组信号共同用于确认操作类型、数据和完成信息没有错位。底部的三个比较输出分别显示相等、有符号比较和无符号比较结果，hazard_stall_i、int_assert_i 和 misaligned_fetch_i 在普通运算窗口保持无效，Case 的控制分支测试则在独立请求中逐一置位并检查抑制效果。

该波形对应 EXU 功能验证中 ALU 运算、比较和地址前递方向的检查内容；它把组合输出和控制输入放在同一时间轴上，便于定位操作选择、数据旁路或完成编号不一致的问题。

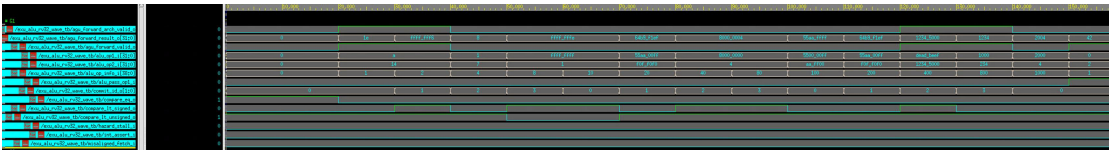


图 18-1 RV32 ALU 运算与地址前递模块级验证波形

2.3.2 RV64 MUL 迭代计算与结果保持 Case

该 Case 面向 RV64 MUL 模块级波形检查，直接检查 RV64 MUL 的输入接收、MULW 快速路径、分段迭代、高半乘积、结果保持和复位恢复。请求在 ready_o 有效时提交，operand_capture_i、valid_in、multiplicand_i、multiplier_i、op_i、reg_waddr_i 和 commit_id_i 在接收边沿前建立；请求接收后立即撤销输入，防止同一请求被重复采样。MULW 的低 32 位乘法使用快速路径，MUL、MULH、MULHSU 和 MULHU 使用 16 位片段逐项累加的迭代路径，状态依次经历空闲、迭代和结果保持阶段。操作数的正负组合覆盖正数乘正数、正数乘负数、负数乘正数和负数乘负数；MULW 另外检查低 32 位乘积的 RV64 符号扩展。

该 Case 的主要检查点如下表所示。

检查项目	激励与观察信号	通过条件
接收就绪	ready_o、fast_ready_o、slow_ready_o、valid_in	复位释放后接口进入就绪状态；只有在相应就绪信号有效时接收请求。

检查项目	激励与观察信号	通过条件
低位乘法与字操作	op_i、multiplicand_i、multiplier_i、result_o	MUL 的低 64 位结果和 MULW 的 32 位结果正确，MULW 输出按 RV64 规则进行符号扩展。
高半乘法	MULH、MULHSU、MULHU 请求及 result_o	乘积高 64 位的符号处理与有符号、混合符号或无符号定义一致。
结果附属信息	reg_waddr_i、reg_waddr_o、commit_id_i、commit_id_o	结果有效时目的寄存器地址和 commit id 与接收请求一致，不能被后续请求覆盖。
写回阻塞	ctrl_ready_i、valid_o、result_o、reg_waddr_o、commit_id_o	写回端连续两个周期不接收时，结果及其附属信息保持稳定；恢复接收后只完成一次释放。
复位中止	迭代期间拉低 rst_n，观察 valid_o 和 ready_o	正在计算的请求被取消，valid_o 清零，接口恢复就绪，不产生旧请求结果。
非法操作保护	op_i 不选择合法乘法类型	result_o 输出零，接口仍按正常结果流程释放，目的寄存器地址和 commit id 保持该请求信息。

图 18-2 给出了 RV64 MUL Case 的关键时序片段。operand_capture_i 在请求接收窗口置位，op_i、两个操作数和目的寄存器地址在同一窗口建立；随后 ready_o 反映计算单元是否能够接收新请求。高半乘法的迭代过程占用多个时钟周期，完成后 valid_o 置位并输出 result_o、reg_waddr_o 和 commit_id_o。图中 ctrl_ready_i 的变化用于观察写回阻塞：当接收端暂不接收时，结果有效保持，附属信息不改变；恢复接收后，valid_o 在一个完成周期后释放。快速通道的 fast_ready_o 与普通接收信号同时显示了 MULW 快速路径和其它迭代乘法在输入接收阶段的区别。

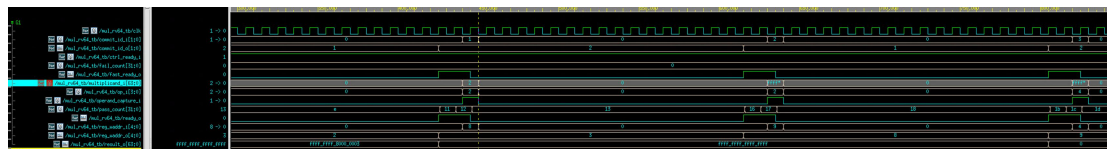


图 18-2 RV64 MUL 迭代计算与结果保持模块级验证波形

2.4 整核系统测试平台

整核测试平台加载 ITCM 与 DTCM 镜像，运行完整 CPU 和 SoC，并观察提交、GPR 写回、执行单元有效信号、存储器访问、tohost 和异常状态。IFU、IDU、Dispatch、ALU、MUL/DIV、LSU、CSR/CLINT 与 WBU 分别设置整核 Feature 检查，确保程序通过不是由未执行目标单元造成。Feature coverage group retire_cg 采样指令编码、目的寄存器区间和写回属性，stage_cg 采样前端、Dispatch 和执行单元执行状态组合，pipeline_cg 采样三级、四级和五级 RTL 展开配置。

2.5 程序完成与结果检查

公共启动程序完成栈初始化和 .bss 清零。程序结束时写入检查结果区、连续八个 32 位程序结果校验字（ELF 符号名为 program_signature）和 tohost。系统测试平台通过 MEM_WRITE_DBG 记录结果校验区的写入，Spike 仅保留相同地址范围的提交记录。程序内部失败数为零、tohost 成功、系统检查通过、程序计数器持续前进且八个程序结果校验字逐项相同，才将程序 Case 记为通过。

2.6 验证策略

2.7 回归执行流程

1. 调度程序读取 case 配置、随机程序配置、源码清单和流水线配置，生成逐项任务计划与输入摘要。
2. P3、P4 和 P5 在独立目录完成模块 DUT、整核基础配置以及适用 RV32/RV64 指令集配置的编译。
3. 模块测试、整核 Feature 检查、指令集自检、riscv-dv 程序、边界条件程序、汇编程序、排序程序和参数化访存程序按计划执行。
4. 程序运行器检查内部结果、tohost、系统日志和 Spike 程序结果校验字；模块运行器检查自检日志、断言和适用的覆盖率目标。
5. 每个流水线配置合并相互兼容的模块、系统和程序覆盖率数据库，形成单配置统计结果。

6. P3、P4 和 P5 的单配置统计结果合并为跨流水线统计，不使用排除文件。

7. 合并程序检查输入数据库、源码摘要、自研 RTL 层次、行覆盖率、模块定义行覆盖率和 feature coverage group；未覆盖 feature 回到相应 case 补充激励后重新执行。

3 指令集测试

3.1 测试平台与执行结果

本项目使用 RISC-V 项目提供的官方 riscv-tests 标准自检程序集验证指令集功能。该测试集按照 RV32 与 RV64 数据宽度组织，并进一步划分为基础整数指令、乘除指令、机器态功能、原子指令、压缩指令和位操作扩展。每项测试由定向汇编指令序列、预期结果比较和标准结束状态组成，能够把单条指令功能、相邻指令数据冒险、控制转移、访存和异常处理纳入同一项自检过程。

Verilator 系统级功能仿真平台将测试程序装入完整 CPU 和存储器模型。测试完成后，平台根据程序自检结果和结束状态给出通过、失败或未完成状态，并统计执行周期数、动态执行指令数（INSTS）和每周期执行的动态指令数（IPC）。只有程序到达标准结束状态且自检结果正确时才记为通过；超过运行限制或未形成唯一结束状态时记为未完成。

测试集合	官方测试组	适用数据宽度	主要检查内容
基础整数与乘除	用户态基础整数（UI）、用户态乘除（UM）	RV32、RV64	基础整数、乘除、条件分支、跳转、访存和程序结束
机器态与异常	机器态基础整数（MI）	RV32、RV64	CSR、异常入口、mret、非法指令和机器态控制
原子与压缩	用户态原子（UA）、用户态压缩（UC）	RV32、RV64	A 扩展原子访问、C 扩展压缩指令和指令流边界
位操作扩展	用户态 Zba、Zbb、Zbs、Zbc	RV32、RV64	Zba、Zbb、Zbs 和 Zbc 的算术、逻辑、单比特及无进位乘法操作

表 125 指令集测试集合

图 19 与图 23 分别给出 RV32、RV64 的基础整数和乘除指令测试结果，图 20 与图 24 给出机器态测试结果，图 21 与图 25 给出原子和压缩指令测试结果，图

22 与图 26 给出 Zba、Zbb、Zbc、Zbs 扩展测试结果。各图同时列出测试状态、执行周期数、动态执行指令数和 IPC，可用于核对功能结果及基本执行统计。图 20 中的断点测试以及图 24 中的断点与 CSR 测试显示为失败，其余图示项目显示为通过；这些结果反映该次 Verilator 系统级功能仿真的实际状态，失败项目需要结合 VCS 定向 Case 进一步分析。

PASS	Cycles: 553	Insts: 347	IPC: 0.6275	rv32ui-p-fence_i.log
PASS	Cycles: 393	Insts: 249	IPC: 0.6336	rv32ui-p-andi.log
PASS	Cycles: 472	Insts: 147	IPC: 0.3114	rv32um-p-rem.log
PASS	Cycles: 501	Insts: 299	IPC: 0.5968	rv32ui-p-lbu.log
PASS	Cycles: 501	Insts: 299	IPC: 0.5968	rv32ui-p-lb.log
PASS	Cycles: 196	Insts: 116	IPC: 0.5918	rv32ui-p-lui.log
PASS	Cycles: 708	Insts: 502	IPC: 0.7090	rv32ui-p-sltu.log
PASS	Cycles: 525	Insts: 342	IPC: 0.6514	rv32ui-p-beq.log
PASS	Cycles: 442	Insts: 288	IPC: 0.6516	rv32ui-p-addi.log
PASS	Cycles: 763	Insts: 538	IPC: 0.7051	rv32ui-p-xor.log
PASS	Cycles: 472	Insts: 147	IPC: 0.3114	rv32um-p-remu.log
PASS	Cycles: 786	Insts: 558	IPC: 0.7099	rv32ui-p-sra.log
PASS	Cycles: 477	Insts: 143	IPC: 0.2998	rv32um-p-divu.log
PASS	Cycles: 708	Insts: 502	IPC: 0.7090	rv32ui-p-slt.log
PASS	Cycles: 779	Insts: 552	IPC: 0.7086	rv32ui-p-srl.log
PASS	Cycles: 557	Insts: 337	IPC: 0.6050	rv32ui-p-bne.log
PASS	Cycles: 453	Insts: 288	IPC: 0.6358	rv32ui-p-slti.log
PASS	Cycles: 467	Insts: 302	IPC: 0.6467	rv32ui-p-srai.log
PASS	Cycles: 615	Insts: 377	IPC: 0.6130	rv32ui-p-bgeu.log
PASS	Cycles: 472	Insts: 147	IPC: 0.3114	rv32um-p-div.log
PASS	Cycles: 808	Insts: 558	IPC: 0.6906	rv32ui-p-sh.log
PASS	Cycles: 492	Insts: 324	IPC: 0.6585	rv32ui-p-lhu.log
PASS	Cycles: 511	Insts: 329	IPC: 0.6438	rv32ui-p-lw.log
PASS	Cycles: 397	Insts: 256	IPC: 0.6448	rv32ui-p-ori.log
PASS	Cycles: 834	Insts: 505	IPC: 0.6055	rv32um-p-mulhsu.log
PASS	Cycles: 177	Insts: 106	IPC: 0.5989	rv32ui-p-jal.log
PASS	Cycles: 582	Insts: 360	IPC: 0.6186	rv32ui-p-bge.log
PASS	Cycles: 462	Insts: 296	IPC: 0.6407	rv32ui-p-srli.log
PASS	Cycles: 377	Insts: 258	IPC: 0.6844	rv32ui-p-xori.log
PASS	Cycles: 436	Insts: 284	IPC: 0.6514	rv32ui-p-slli.log
PASS	Cycles: 834	Insts: 505	IPC: 0.6055	rv32um-p-mulhu.log
PASS	Cycles: 733	Insts: 508	IPC: 0.6930	rv32ui-p-add.log
PASS	Cycles: 773	Insts: 539	IPC: 0.6973	rv32ui-p-sll.log
PASS	Cycles: 703	Insts: 508	IPC: 0.7226	rv32ui-p-sub.log
PASS	Cycles: 183	Insts: 109	IPC: 0.5956	rv32ui-p-auipc.log
PASS	Cycles: 749	Insts: 526	IPC: 0.7023	rv32ui-p-st_ld.log
PASS	Cycles: 758	Insts: 531	IPC: 0.7005	rv32ui-p-and.log
PASS	Cycles: 749	Insts: 539	IPC: 0.7196	rv32ui-p-or.log
PASS	Cycles: 1657	Insts: 1006	IPC: 0.6071	rv32ui-p-ld_st.log
PASS	Cycles: 150	Insts: 92	IPC: 0.6133	rv32ui-p-simple.log
PASS	Cycles: 834	Insts: 505	IPC: 0.6055	rv32um-p-mulh.log
PASS	Cycles: 538	Insts: 367	IPC: 0.6822	rv32ui-p-bltu.log
PASS	Cycles: 525	Insts: 342	IPC: 0.6514	rv32ui-p-blb.log
PASS	Cycles: 772	Insts: 505	IPC: 0.6541	rv32ui-p-sb.log
PASS	Cycles: 277	Insts: 166	IPC: 0.5993	rv32ui-p-jalr.log
PASS	Cycles: 774	Insts: 505	IPC: 0.6525	rv32um-p-mul.log
PASS	Cycles: 453	Insts: 288	IPC: 0.6358	rv32ui-p-sltiu.log
PASS	Cycles: 517	Insts: 315	IPC: 0.6093	rv32ui-p-lh.log
PASS	Cycles: 816	Insts: 560	IPC: 0.6863	rv32ui-p-sw.log

图 19 RV32I 与 RV32M 指令集测试输出

PASS		Cycles: 184	Insts: 111	IPC: 0.6033		rv32mi-p-mcsr.log
PASS		Cycles: 326	Insts: 199	IPC: 0.6104		rv32mi-p-sw-misaligned.log
PASS		Cycles: 228	Insts: 131	IPC: 0.5746		rv32mi-p-lh-misaligned.log
PASS		Cycles: 291	Insts: 178	IPC: 0.6117		rv32mi-p-csr.log
PASS		Cycles: 204	Insts: 120	IPC: 0.5882		rv32mi-p-sbreak.log
PASS		Cycles: 272	Insts: 160	IPC: 0.5882		rv32mi-p-zicntr.log
PASS		Cycles: 218	Insts: 124	IPC: 0.5688		rv32mi-p-illegal.log
PASS		Cycles: 233	Insts: 139	IPC: 0.5966		rv32mi-p-sh-misaligned.log
PASS		Cycles: 168	Insts: 105	IPC: 0.6250		rv32mi-p-pmpaddr.log
PASS		Cycles: 465	Insts: 259	IPC: 0.5570		rv32mi-p-ma_fetch.log
PASS		Cycles: 518	Insts: 318	IPC: 0.6139		rv32mi-p-ma_addr.log
PASS		Cycles: 179	Insts: 108	IPC: 0.6034		rv32mi-p-scall.log
PASS		Cycles: 186	Insts: 25	IPC: 0.1344		rv32mi-p-instret_overflow.log
PASS		Cycles: 329	Insts: 195	IPC: 0.5927		rv32mi-p-lw-misaligned.log
FAIL		Cycles: 191	Insts: 117	IPC: 0.6126		rv32mi-p-breakpoint.log
PASS		Cycles: 206	Insts: 120	IPC: 0.5825		rv32mi-p-shamt.log

图 20 RV32 机器态指令集测试输出

PASS		Cycles: 197	Insts: 129	IPC: 0.6548		rv32ua-p-amoxor_w.log
PASS		Cycles: 222	Insts: 138	IPC: 0.6216		rv32ua-p-amomin_w.log
PASS		Cycles: 14610	Insts: 12455	IPC: 0.8525		rv32ua-p-lrsc.log
PASS		Cycles: 196	Insts: 126	IPC: 0.6429		rv32ua-p-amominu_w.log
PASS		Cycles: 196	Insts: 125	IPC: 0.6378		rv32ua-p-amoad_w.log
PASS		Cycles: 195	Insts: 125	IPC: 0.6410		rv32ua-p-amoor_w.log
PASS		Cycles: 415	Insts: 262	IPC: 0.6313		rv32uc-p-rvc.log
PASS		Cycles: 202	Insts: 127	IPC: 0.6287		rv32ua-p-amoad_w.log
PASS		Cycles: 222	Insts: 138	IPC: 0.6216		rv32ua-p-amomax_w.log
PASS		Cycles: 197	Insts: 126	IPC: 0.6396		rv32ua-p-amomaxu_w.log
PASS		Cycles: 196	Insts: 125	IPC: 0.6378		rv32ua-p-amoswap_w.log

图 21 RV32A 与 RV32C 指令集测试输出

PASS	Cycles: 758	Insts: 508	IPC: 0.6702	rv32uzbb-p-minu.log
PASS	Cycles: 415	Insts: 264	IPC: 0.6361	rv32uzbb-p-sext_b.log
PASS	Cycles: 406	Insts: 262	IPC: 0.6453	rv32uzbb-p-sext_h.log
PASS	Cycles: 835	Insts: 601	IPC: 0.7198	rv32uzbs-p-bset.log
PASS	Cycles: 733	Insts: 511	IPC: 0.6971	rv32uzba-p-sh2add.log
PASS	Cycles: 783	Insts: 503	IPC: 0.6424	rv32uzbc-p-clmul.log
PASS	Cycles: 819	Insts: 582	IPC: 0.7106	rv32uzbb-p-rol.log
PASS	Cycles: 480	Insts: 324	IPC: 0.6750	rv32uzbs-p-bclri.log
PASS	Cycles: 739	Insts: 515	IPC: 0.6969	rv32uzbb-p-max.log
PASS	Cycles: 807	Insts: 554	IPC: 0.6865	rv32uzbs-p-binv.log
PASS	Cycles: 415	Insts: 264	IPC: 0.6361	rv32uzbb-p-clz.log
PASS	Cycles: 418	Insts: 265	IPC: 0.6340	rv32uzbb-p-orc_b.log
PASS	Cycles: 781	Insts: 555	IPC: 0.7106	rv32uzbb-p-ror.log
PASS	Cycles: 769	Insts: 537	IPC: 0.6983	rv32uzbb-p-orn.log
PASS	Cycles: 783	Insts: 503	IPC: 0.6424	rv32uzbc-p-clmulh.log
PASS	Cycles: 736	Insts: 522	IPC: 0.7092	rv32uzbb-p-andn.log
PASS	Cycles: 415	Insts: 264	IPC: 0.6361	rv32uzbb-p-cpop.log
PASS	Cycles: 480	Insts: 324	IPC: 0.6750	rv32uzbs-p-bseti.log
PASS	Cycles: 413	Insts: 266	IPC: 0.6441	rv32uzbb-p-zext_h.log
PASS	Cycles: 725	Insts: 503	IPC: 0.6938	rv32uzbb-p-min.log
PASS	Cycles: 733	Insts: 511	IPC: 0.6971	rv32uzba-p-sh1add.log
PASS	Cycles: 733	Insts: 511	IPC: 0.6971	rv32uzba-p-sh3add.log
PASS	Cycles: 454	Insts: 289	IPC: 0.6366	rv32uzbb-p-rori.log
PASS	Cycles: 474	Insts: 306	IPC: 0.6456	rv32uzbs-p-bexti.log
PASS	Cycles: 763	Insts: 538	IPC: 0.7051	rv32uzbb-p-xnor.log
PASS	Cycles: 417	Insts: 272	IPC: 0.6523	rv32uzbb-p-rev8.log
PASS	Cycles: 466	Insts: 302	IPC: 0.6481	rv32uzbs-p-binvi.log
PASS	Cycles: 827	Insts: 574	IPC: 0.6941	rv32uzbs-p-bext.log
PASS	Cycles: 783	Insts: 503	IPC: 0.6424	rv32uzbc-p-clmulr.log
PASS	Cycles: 835	Insts: 601	IPC: 0.7198	rv32uzbs-p-bclr.log
PASS	Cycles: 415	Insts: 264	IPC: 0.6361	rv32uzbb-p-ctz.log
PASS	Cycles: 754	Insts: 508	IPC: 0.6737	rv32uzbb-p-maxu.log

图 22 RV32 Zba、Zbb、Zbc、Zbs 扩展测试输出

PASS	Cycles: 525	Insts: 351	IPC: 0.6686	rv64ui-p-sraiw.log
PASS	Cycles: 576	Insts: 356	IPC: 0.6181	rv64ui-p-bge.log
PASS	Cycles: 402	Insts: 268	IPC: 0.6667	rv64ui-p-andi.log
PASS	Cycles: 828	Insts: 592	IPC: 0.7150	rv64ui-p-and.log
PASS	Cycles: 523	Insts: 338	IPC: 0.6463	rv64ui-p-bltn.log
PASS	Cycles: 383	Insts: 259	IPC: 0.6762	rv64ui-p-xori.log
PASS	Cycles: 1045	Insts: 153	IPC: 0.1464	rv64um-p-remu.log
PASS	Cycles: 1343	Insts: 604	IPC: 0.4497	rv64um-p-mul.log
PASS	Cycles: 487	Insts: 317	IPC: 0.6509	rv64ui-p-slli.log
PASS	Cycles: 731	Insts: 509	IPC: 0.6963	rv64ui-p-addw.log
PASS	Cycles: 652	Insts: 143	IPC: 0.2193	rv64um-p-divw.log
PASS	Cycles: 679	Insts: 446	IPC: 0.6568	rv64um-p-mulw.log
PASS	Cycles: 1318	Insts: 624	IPC: 0.4734	rv64um-p-mulhu.log
PASS	Cycles: 2327	Insts: 1459	IPC: 0.6270	rv64ui-p-ldst.log
PASS	Cycles: 743	Insts: 523	IPC: 0.7039	rv64ui-p-sltn.log
PASS	Cycles: 523	Insts: 338	IPC: 0.6463	rv64ui-p-beq.log
PASS	Cycles: 197	Insts: 117	IPC: 0.5939	rv64ui-p-lui.log
PASS	Cycles: 601	Insts: 429	IPC: 0.7138	rv64ui-p-bltn.log
PASS	Cycles: 804	Insts: 554	IPC: 0.6891	rv64ui-p-sh.log
PASS	Cycles: 177	Insts: 111	IPC: 0.6271	rv64ui-p-auipc.log
PASS	Cycles: 492	Insts: 297	IPC: 0.6037	rv64ui-p-lb.log
PASS	Cycles: 505	Insts: 335	IPC: 0.6634	rv64ui-p-lw.log
PASS	Cycles: 860	Insts: 596	IPC: 0.6930	rv64ui-p-sraw.log
PASS	Cycles: 1042	Insts: 148	IPC: 0.1420	rv64um-p-div.log
PASS	Cycles: 821	Insts: 558	IPC: 0.6797	rv64ui-p-sw.log
PASS	Cycles: 499	Insts: 323	IPC: 0.6473	rv64ui-p-srli.log
PASS	Cycles: 506	Insts: 338	IPC: 0.6680	rv64ui-p-srliw.log
PASS	Cycles: 545	Insts: 347	IPC: 0.6367	rv64ui-p-fence_i.log
PASS	Cycles: 859	Insts: 584	IPC: 0.6799	rv64ui-p-slli.log
PASS	Cycles: 1049	Insts: 159	IPC: 0.1516	rv64um-p-divu.log
PASS	Cycles: 1053	Insts: 152	IPC: 0.1443	rv64um-p-rem.log
PASS	Cycles: 472	Insts: 302	IPC: 0.6398	rv64ui-p-srai.log
PASS	Cycles: 855	Insts: 593	IPC: 0.6936	rv64ui-p-srlw.log
PASS	Cycles: 492	Insts: 297	IPC: 0.6037	rv64ui-p-lbu.log
PASS	Cycles: 174	Insts: 107	IPC: 0.6149	rv64ui-p-jal.log
PASS	Cycles: 1029	Insts: 769	IPC: 0.7473	rv64ui-p-stld.log
PASS	Cycles: 673	Insts: 446	IPC: 0.6627	rv64ui-p-bgeu.log
PASS	Cycles: 457	Insts: 284	IPC: 0.6214	rv64ui-p-sltn.log
PASS	Cycles: 387	Insts: 261	IPC: 0.6744	rv64ui-p-ori.log
PASS	Cycles: 1257	Insts: 587	IPC: 0.4670	rv64um-p-mulhsu.log
PASS	Cycles: 550	Insts: 369	IPC: 0.6709	rv64ui-p-lwu.log
PASS	Cycles: 513	Insts: 330	IPC: 0.6433	rv64ui-p-lhu.log
PASS	Cycles: 755	Insts: 501	IPC: 0.6636	rv64ui-p-sb.log
PASS	Cycles: 942	Insts: 678	IPC: 0.7197	rv64ui-p-sd.log
PASS	Cycles: 652	Insts: 143	IPC: 0.2193	rv64um-p-remuw.log
PASS	Cycles: 732	Insts: 504	IPC: 0.6885	rv64ui-p-subw.log
PASS	Cycles: 829	Insts: 622	IPC: 0.7503	rv64ui-p-or.log
PASS	Cycles: 147	Insts: 93	IPC: 0.6327	rv64ui-p-simple.log
PASS	Cycles: 1257	Insts: 587	IPC: 0.4670	rv64um-p-mulh.log
PASS	Cycles: 285	Insts: 167	IPC: 0.5860	rv64ui-p-jalr.log
PASS	Cycles: 457	Insts: 284	IPC: 0.6214	rv64ui-p-sltn.log
PASS	Cycles: 817	Insts: 620	IPC: 0.7589	rv64ui-p-xor.log
PASS	Cycles: 702	Insts: 503	IPC: 0.7165	rv64ui-p-sltn.log
PASS	Cycles: 447	Insts: 294	IPC: 0.6577	rv64ui-p-addiw.log
PASS	Cycles: 449	Insts: 289	IPC: 0.6437	rv64ui-p-addi.log
PASS	Cycles: 668	Insts: 482	IPC: 0.7216	rv64ui-p-ld.log
PASS	Cycles: 656	Insts: 151	IPC: 0.2302	rv64um-p-divuw.log
PASS	Cycles: 562	Insts: 343	IPC: 0.6103	rv64ui-p-bne.log
PASS	Cycles: 519	Insts: 316	IPC: 0.6089	rv64ui-p-lh.log
PASS	Cycles: 666	Insts: 149	IPC: 0.2237	rv64um-p-remw.log
PASS	Cycles: 833	Insts: 584	IPC: 0.7011	rv64ui-p-sllw.log
PASS	Cycles: 755	Insts: 522	IPC: 0.6914	rv64ui-p-add.log
PASS	Cycles: 798	Insts: 556	IPC: 0.6967	rv64ui-p-sra.log
PASS	Cycles: 725	Insts: 508	IPC: 0.7007	rv64ui-p-sub.log
PASS	Cycles: 486	Insts: 324	IPC: 0.6667	rv64ui-p-slliw.log
PASS	Cycles: 838	Insts: 601	IPC: 0.7172	rv64ui-p-srl.log

图 23 RV64I 与 RV64M 指令集测试输出

PASS		Cycles: 216	Insts: 121	IPC: 0.5602		rv64mi-p-illegal.log
PASS		Cycles: 163	Insts: 16	IPC: 0.0982		rv64mi-p-instret_overflow.log
PASS		Cycles: 174	Insts: 106	IPC: 0.6092		rv64mi-p-scall.log
PASS		Cycles: 325	Insts: 196	IPC: 0.6031		rv64mi-p-sw-misaligned.log
PASS		Cycles: 1176	Insts: 723	IPC: 0.6148		rv64mi-p-ma_addr.log
PASS		Cycles: 226	Insts: 128	IPC: 0.5664		rv64mi-p-lh-misaligned.log
PASS		Cycles: 607	Insts: 388	IPC: 0.6392		rv64mi-p-ld-misaligned.log
PASS		Cycles: 182	Insts: 108	IPC: 0.5934		rv64mi-p-mcsr.log
PASS		Cycles: 166	Insts: 102	IPC: 0.6145		rv64mi-p-pmpaddr.log
PASS		Cycles: 601	Insts: 408	IPC: 0.6789		rv64mi-p-sd-misaligned.log
PASS		Cycles: 446	Insts: 240	IPC: 0.5381		rv64mi-p-ma_fetch.log
PASS		Cycles: 218	Insts: 125	IPC: 0.5734		rv64mi-p-zicntr.log
PASS		Cycles: 206	Insts: 117	IPC: 0.5680		rv64mi-p-sbreak.log
PASS		Cycles: 231	Insts: 136	IPC: 0.5887		rv64mi-p-sh-misaligned.log
PASS		Cycles: 331	Insts: 192	IPC: 0.5801		rv64mi-p-lw-misaligned.log
FAIL		Cycles: 204	Insts: 121	IPC: 0.5931		rv64mi-p-breakpoint.log
PASS		Cycles: 313	Insts: 185	IPC: 0.5911		rv64mi-p-csr.log

图 24 RV64 机器态指令集测试输出

PASS		Cycles: 230	Insts: 147	IPC: 0.6391		rv64ua-p-amomaxu_w.log
PASS		Cycles: 208	Insts: 128	IPC: 0.6154		rv64ua-p-amoand_w.log
PASS		Cycles: 230	Insts: 147	IPC: 0.6391		rv64ua-p-amomax_w.log
PASS		Cycles: 202	Insts: 129	IPC: 0.6386		rv64ua-p-amoaddd_w.log
PASS		Cycles: 232	Insts: 147	IPC: 0.6336		rv64ua-p-amomin_w.log
PASS		Cycles: 205	Insts: 126	IPC: 0.6146		rv64ua-p-amoxor_w.log
PASS		Cycles: 205	Insts: 128	IPC: 0.6244		rv64ua-p-amominu_d.log
PASS		Cycles: 204	Insts: 127	IPC: 0.6225		rv64ua-p-amoor_d.log
PASS		Cycles: 210	Insts: 129	IPC: 0.6143		rv64ua-p-amoand_d.log
PASS		Cycles: 205	Insts: 128	IPC: 0.6244		rv64ua-p-amomax_d.log
PASS		Cycles: 210	Insts: 129	IPC: 0.6143		rv64ua-p-amoswap_d.log
PASS		Cycles: 205	Insts: 130	IPC: 0.6341		rv64ua-p-amoxor_d.log
PASS		Cycles: 204	Insts: 127	IPC: 0.6225		rv64ua-p-amoor_w.log
PASS		Cycles: 14626	Insts: 12459	IPC: 0.8518		rv64ua-p-lrsc.log
PASS		Cycles: 205	Insts: 128	IPC: 0.6244		rv64ua-p-amomin_d.log
PASS		Cycles: 213	Insts: 132	IPC: 0.6197		rv64ua-p-amoaddd_d.log
PASS		Cycles: 230	Insts: 147	IPC: 0.6391		rv64ua-p-amominu_w.log
PASS		Cycles: 481	Insts: 307	IPC: 0.6383		rv64uc-p-rvc.log
PASS		Cycles: 205	Insts: 128	IPC: 0.6244		rv64ua-p-amomaxu_d.log
PASS		Cycles: 208	Insts: 128	IPC: 0.6154		rv64ua-p-amoswap_w.log

图 25 RV64A 与 RV64C 指令集测试输出

PASS	Cycles: 427	Insts: 278	IPC: 0.6511	rv64uzbb-p-clz.log
PASS	Cycles: 835	Insts: 511	IPC: 0.6120	rv64uzbc-p-clmulr.log
PASS	Cycles: 405	Insts: 266	IPC: 0.6568	rv64uzbb-p-ctzw.log
PASS	Cycles: 768	Insts: 548	IPC: 0.7135	rv64uzbb-p-rorw.log
PASS	Cycles: 936	Insts: 704	IPC: 0.7521	rv64uzbs-p-bclr.log
PASS	Cycles: 887	Insts: 642	IPC: 0.7238	rv64uzbs-p-bext.log
PASS	Cycles: 744	Insts: 533	IPC: 0.7164	rv64uzba-p-sh3add_uw.log
PASS	Cycles: 816	Insts: 598	IPC: 0.7328	rv64uzbb-p-andn.log
PASS	Cycles: 729	Insts: 508	IPC: 0.6968	rv64uzbb-p-max.log
PASS	Cycles: 427	Insts: 278	IPC: 0.6511	rv64uzbb-p-ctz.log
PASS	Cycles: 816	Insts: 597	IPC: 0.7316	rv64uzbb-p-rol.log
PASS	Cycles: 420	Insts: 265	IPC: 0.6310	rv64uzbb-p-clzw.log
PASS	Cycles: 475	Insts: 320	IPC: 0.6737	rv64uzbb-p-rev8.log
PASS	Cycles: 430	Insts: 281	IPC: 0.6535	rv64uzbb-p-sext_h.log
PASS	Cycles: 838	Insts: 588	IPC: 0.7017	rv64uzbb-p-rolw.log
PASS	Cycles: 757	Insts: 524	IPC: 0.6922	rv64uzba-p-sh3add.log
PASS	Cycles: 822	Insts: 614	IPC: 0.7470	rv64uzbb-p-xnor.log
PASS	Cycles: 444	Insts: 302	IPC: 0.6802	rv64uzbb-p-orc_b.log
PASS	Cycles: 744	Insts: 533	IPC: 0.7164	rv64uzba-p-sh1add_uw.log
PASS	Cycles: 825	Insts: 615	IPC: 0.7455	rv64uzbb-p-orn.log
PASS	Cycles: 525	Insts: 375	IPC: 0.7143	rv64uzbs-p-bseti.log
PASS	Cycles: 825	Insts: 513	IPC: 0.6218	rv64uzbc-p-clmulh.log
PASS	Cycles: 870	Insts: 611	IPC: 0.7023	rv64uzbs-p-binw.log
PASS	Cycles: 951	Insts: 709	IPC: 0.7455	rv64uzbs-p-bset.log
PASS	Cycles: 757	Insts: 524	IPC: 0.6922	rv64uzba-p-sh1add.log
PASS	Cycles: 757	Insts: 524	IPC: 0.6922	rv64uzba-p-sh2add.log
PASS	Cycles: 427	Insts: 278	IPC: 0.6511	rv64uzbb-p-cpop.log
PASS	Cycles: 420	Insts: 265	IPC: 0.6310	rv64uzbb-p-cpopw.log
PASS	Cycles: 815	Insts: 505	IPC: 0.6196	rv64uzbc-p-clmul.log
PASS	Cycles: 503	Insts: 332	IPC: 0.6600	rv64uzbb-p-rori.log
PASS	Cycles: 499	Insts: 324	IPC: 0.6493	rv64uzbs-p-binvi.log
PASS	Cycles: 755	Insts: 526	IPC: 0.6967	rv64uzbb-p-minu.log
PASS	Cycles: 850	Insts: 616	IPC: 0.7247	rv64uzbb-p-ror.log
PASS	Cycles: 744	Insts: 533	IPC: 0.7164	rv64uzba-p-sh2add_uw.log
PASS	Cycles: 748	Insts: 507	IPC: 0.6778	rv64uzbb-p-min.log
PASS	Cycles: 498	Insts: 326	IPC: 0.6546	rv64uzba-p-slli_uw.log
PASS	Cycles: 486	Insts: 337	IPC: 0.6934	rv64uzbs-p-bexti.log
PASS	Cycles: 770	Insts: 526	IPC: 0.6831	rv64uzba-p-add_uw.log
PASS	Cycles: 521	Insts: 362	IPC: 0.6948	rv64uzbs-p-bclri.log
PASS	Cycles: 416	Insts: 285	IPC: 0.6851	rv64uzbb-p-zext_h.log
PASS	Cycles: 427	Insts: 278	IPC: 0.6511	rv64uzbb-p-sext_b.log
PASS	Cycles: 766	Insts: 537	IPC: 0.7010	rv64uzbb-p-maxu.log
PASS	Cycles: 447	Insts: 290	IPC: 0.6488	rv64uzbb-p-roriw.log

图 26 RV64 Zba、Zbb、Zbc、Zbs 扩展测试输出

3.2 指令集自检程序的组织方式

除官方 riscv-tests 外，VCS 平台还运行针对当前 CPU 指令集配置编写的补充自检程序。各程序按照目标 ISA、ABI（应用程序二进制接口）和流水线配置分别编译，并在 P3、P4 和 P5 上独立执行。RV32 与 RV64 使用各自的程序镜像，不混用不同数据宽度的 ELF（可执行与可链接格式）。程序内部逐项比较指令结果并写入程序结果校验字，随后由系统测试平台和 Spike 参考模型完成第二层检查。

3.3 指令集 Feature List

Feature ID	指令集范围	Feature 描述	对应 Case
ISA-RV32A-01	RV32A	LR.W 与 SC.W 的保留状态和条件写入	程序 Case: rv32a_selfcheck
ISA-RV32A-02	RV32A	32 位 AMO 的旧值返回	程序 Case: rv32a_selfcheck
ISA-RV32C-01	RV32C	压缩算术指令执行	程序 Case: rv32c_selfcheck
ISA-RV32C-02	RV32C	压缩控制转移指令执行	程序 Case: rv32c_selfcheck
ISA-RV32C-03	RV32C	非法压缩编码异常	程序 Case: rv32c_selfcheck
ISA-RV64I-01	RV64I	64 位整数算术指令执行	程序 Case: rv64i_zicsr_zifencei_selfcheck
ISA-RV64I-02	RV64I	32 位字操作的符号扩展	程序 Case: rv64i_zicsr_zifencei_selfcheck
ISA-RV64I-03	RV64I	双字访存结果	程序 Case: rv64i_zicsr_zifencei_selfcheck
ISA-RV64I-04	Zicsr	CSR 指令读写结果	程序 Case: rv64i_zicsr_zifencei_selfcheck
ISA-RV64I-05	Zifencei	fence.i 后的取指结果	程序 Case: rv64i_zicsr_zifencei_selfcheck
ISA-RV64M-01	RV64M	64 位乘法结果	程序 Case: rv64m_selfcheck
ISA-RV64M-02	RV64M	高半乘积结果	程序 Case: rv64m_selfcheck
ISA-RV64M-03	RV64M	除法结果	程序 Case: rv64m_selfcheck
ISA-RV64M-04	RV64M	余数指令结果	程序 Case: rv64m_selfcheck
ISA-RV64A-01	RV64A	32 位与 64 位 LR/SC	程序 Case: rv64a_selfcheck
ISA-RV64A-02	RV64A	32 位与 64 位 AMO 读改写	程序 Case: rv64a_selfcheck

Feature ID	指令集范围	Feature 描述	对应 Case
ISA-RV64C-01	RV64C	压缩字操作	程序 Case: rv64c_selfcheck
ISA-RV64C-02	RV64C	压缩双字栈访问	程序 Case: rv64c_selfcheck
ISA-RV64C-03	RV64C	压缩控制转移指令	程序 Case: rv64c_selfcheck
ISA-RV64ZBA-01	RV64 Zba	地址生成位操作结果	程序 Case: rv64_bitmanip_selfcheck
ISA-RV64ZBB-01	RV64 Zbb	基础位操作结果	程序 Case: rv64_bitmanip_selfcheck
ISA-RV64ZBS-01	RV64 Zbs	单比特操作结果	程序 Case: rv64_bitmanip_selfcheck
ISA-RV64ZBC-01	RV64 Zbc	无进位乘法结果	程序 Case: rv64_bitmanip_selfcheck

表 126 指令集 Feature List

3.4 指令集 Case 清单

Case ID	字长与编译选项	测试重点	结果检查
程序 Case: rv32a_selfcheck	RV32, rv32imac_zba_zbb_zbs _zbc_zicsr_zifencei, ilp32	LR/SC 成功与失败、 所有 RV32 AMO	程序内部结果、 tohost、程序结果校验 字和 Spike 对照
程序 Case: rv32c_selfcheck	RV32, rv32imac_zba_zbb_zbs _zbc_zicsr_zifencei, ilp32	压缩算术、栈访问、 分支、跳转和非法编 码	程序内部结果、 tohost、程序结果校验 字和 Spike 对照
程序 Case: rv64i_zicsr_zifencei_se lfcheck	RV64, rv64imac_zba_zbb_zbs _zbc_zicsr_zifencei, lp64	RV64I、字操作、双 字访存、CSR、fence.i	程序内部结果、 tohost、程序结果校验 字和 Spike 对照
程序 Case: rv64m_selfcheck	RV64, rv64imac_zba_zbb_zbs _zbc_zicsr_zifencei, lp64	乘除、余数、高半乘 积、字乘除	程序内部结果、 tohost、程序结果校验 字和 Spike 对照
程序 Case: rv64a_selfcheck	RV64, rv64imac_zba_zbb_zbs _zbc_zicsr_zifencei, lp64	原子读改写、LR/SC 和双字访问	程序内部结果、 tohost、程序结果校验 字和 Spike 对照

Case ID	字长与编译选项	测试重点	结果检查
程序 Case: rv64c_selfcheck	RV64, rv64imac_zba_zbb_zbs _zbc_zicsr_zifencei, lp64	压缩字操作、栈访问、控制转移	程序内部结果、tohost、程序结果校验字和 Spike 对照
程序 Case: rv64_bitmanip_selfcheck	RV64, rv64imac_zba_zbb_zbs _zbc_zicsr_zifencei, lp64	Zba、Zbb、Zbs、Zbc 指令结果与边界位	程序内部结果、tohost、程序结果校验字和 Spike 对照

表 127 指令集 Case 清单

3.5 汇编指令序列补充

自检程序集之外，汇编程序以短序列锁定相邻指令、地址低位、异常入口和长延迟完成的相对位置。sw_forwarding_alu、sw_forwarding_load、sw_forwarding_mul_div 与 sw_forwarding_csr 分别检查 ALU、加载、乘除与 CSR 结果被紧随指令使用的情况；sw_unsigned_branch_jalr 检查无符号条件分支和 JALR 地址最低位清零；sw_fence_stall、sw_fence_div_repro 和 sw_div_fence_independent 检查 fence.i 与长延迟执行交叠；非自然对齐访问程序集检查地址跨越、读回、符号扩展和异常路径。

4 riscv-dv 类别随机程序与 Spike 对照

4.1 riscv-dv 的使用方式

RISC-V 指令验证生成框架（riscv-dv）提供了按指令类别组织的随机测试模型。本工程从 riscv-dv 中选择当前 CPU 支持的测试类别，并按照测试名称、指令生成类别、随机种子、指令数量、ISA、ABI 和已启用扩展生成汇编程序，避免随机程序包含当前 CPU 未实现的指令、异常入口或特权状态。

该集成不直接调用 riscv-dv 的通用 run.py 或 SystemVerilog 生成器，而是以 riscv-dv 的类别划分为基础，由本工程生成程序。这样可以保留算术、跳转、循环、访存、异常和扩展指令的随机组合，同时使启动程序、异常处理、物理存储器布局与 tohost 完成协议保持确定。固定种子、生成参数和程序文本均保存于任务目录，失败 Case 可以在相同流水线配置下再次执行。

4.2 随机程序类别与规模

当前回归包含 21 类 RV32 程序和 21 类 RV64 程序。完整计划中每一类别使用两个固定随机种子；每个流水线配置执行 84 个随机程序和 8 个整核 Feature 检查。基础算术和混合程序最多生成 4000 条主体指令，跳转、循环和混合控制流程序通常生成 3000 条，压缩与位操作程序生成 2400 条，异常、中断、fence.i 和 CSR 程序使用较短但控制更集中的指令序列。

字长	类别	程序名称	主要检查对象
RV32	基础整数与控制流	rv32_arithmetic_basic、rv32_rand_instr、rv32_jump_stress、rv32_loop、rv32_rand_jump	RV32I/M、算术、分支、JAL/JALR、循环和写回
RV32	访存与系统	rv32_memory_stress、rv32_no_fence、rv32_illegal_instr、rv32_ebreak、rv32_csr	load/store、fence.i、异常、CSR 与返回
RV32	扩展	rv32_compressed、rv32_amo、rv32_bitmanip、rv32_invalid_csr、rv32_unaligned_load_store、rv32_full_interrupt	C、A、Zba/Zbb/Zbs/Zbc、非法 CSR、非自然对齐和中断
RV64	基础整数与控制流	rv64_arithmetic_basic、rv64_rand_instr、rv64_jump_stress、rv64_loop、rv64_rand_jump	RV64I/M、字操作、控制转移和写回
RV64	访存与系统	rv64_load_store、rv64_no_fence、rv64_illegal_instr、rv64_ebreak、rv64_csr	双字访存、fence.i、异常、CSR 与返回
RV64	扩展	rv64_compressed、rv64_amo、rv64_bitmanip、rv64_invalid_csr、rv64_unaligned_load_store、rv64_full_interrupt	C、A、Zba/Zbb/Zbs/Zbc、非法 CSR、非自然对齐和中断

4.3 生成、编译与运行

每个随机程序采用固定种子、ISA、ABI、指令数量与 feature 集合。生成后，程序与运行时支持和链接脚本编译为静态 ELF。运行器从 ELF 符号表读取 tohost 与 program_signature 的地址，再依次完成 Spike 执行、程序镜像拆分、整核 RTL 执行和结果比较。Spike 输出提交事件；RTL 运行使用相同 ISA、ABI、程序镜像和流水线配置，并向 monitor 提供程序结果校验区的起止地址。失败程序按照相同的 ISA、ABI、种子和流水线配置重新执行。

4.4 RTL monitor 与完成信息采集

整核 RTL monitor 由完成信息观察和存储器写入观察两部分组成。monitor 在 Dispatch 的 wb_alloc_valid_o、wb_alloc_commit_id_o、issue_dec_pc 和 issue_inst 观察点接收已接受的分配信息，并在为需要写回的指令分配 commit id 时，将该指令的 PC 和指令编码保存到 commit_probe_fifo[commit id]。monitor 连接 WBU（写回单元）的 commit_valid、commit_id、reg_we、reg_waddr 和 reg_wdata 观察信号；在 commit_valid 有效的时钟沿，WBU 输出 commit id、通用寄存器（GPR）写使能、目的寄存器号和写回数据，monitor 以该 commit id 取回先前保存的 PC 与指令编码，从而在不同流水线级数和不同执行延迟下恢复同一条指令的观察信息。启用 +commit_dbg 时，平台输出这些字段，用于查看完成事件。

完成信息监视器还建立三类断言：有效完成事件的 PC、指令编码和写回字段不得为未知值；当未启用 C 扩展时，PC 必须按四字节对齐，启用 C 扩展时 PC 的最低位必须为零；GPR 写回不得写入 x0。该组检查针对 WBU 可见事件，不以完成事件的出现次序替代程序结果检查。

存储器写入 monitor 观察数据主接口的写地址通道（AW）和写数据通道（W）握手。由于 AXI4-Lite 的写地址和写数据可在不同周期完成握手，monitor 分别保存先到达的地址或数据，待两者齐备后组成一次完整写入。只有地址落在程序结果校验区时才输出 MEM_WRITE_DBG；写入 tohost 的成功值后，系统测试平台确认程序结束。该处理避免把地址与数据不同周期到达误作两次访问，也使 RTL 日志与 Spike 日志使用同一组结果校验地址。

4.5 Spike 参考模型对照与时序一致性

Spike 是顺序参考模型，使用与 RTL 完全相同的 ELF、ISA、特权模式和初始程序数据。Spike 以提交日志记录程序执行，RTL monitor 以 MEM_WRITE_DBG 记录程序结果校验区的写入；结果比较程序从两侧日志中按地址提取连续八个 32 位程序结果校验字。在 RTL 日志中，比较程序只接受四个连续字节有效的写入，并根据字节使能从 RV64 数据总线的正确位置提取对应的 32 位结果校验字。

两侧比较的时序边界是程序完成，而不是某个流水线级的同一时钟沿。P3、P4 和 P5 的级间寄存器位置不同，MUL、DIV 和 LSU 的完成消息也可能在不同周期抵达 WBU；相互独立的指令可能以不同于发射次序的顺序完成。commit id 使 RTL monitor 能够确认完成事件所属的指令，主比较则以确定的结果校验地址和 tohost 成功写入为准，不要求 Spike 与 RTL 在同一周期或相同完成次序产生事件。因此，流水线暂停、地址修正、长延迟执行和有限乱序完成只改变 RTL 的完成时刻，不会改变对程序架构结果的比较。

启用 +commit_dbg 时，比较程序还将 Spike 与 RTL 的 GPR 写回事件按 PC、指令编码、目的寄存器和写回数据组成不区分先后次序的多重集合，用于辅助定位。该统计不决定 Case 的通过状态；通过条件仍为程序内部检查、tohost、系统测试平台检查和八个程序结果校验字全部一致。程序结果校验字可发现算术、控制转移、访存回读、异常返回和程序完成状态的不一致，但不覆盖全部 GPR、CSR 和存储器内容，因此模块断言、程序内部检查、整核 Feature 检查和覆盖率统计仍需同时使用。

4.6 代表性随机程序

rv32_rand_instr 生成算术、逻辑、移位、比较、分支和 load/store 的长指令序列，适合检查译码、Dispatch、HDU、旁路和 WBU 是否持续协同工作。rv32_jump_stress 与 rv64_rand_jump 增加短距离条件分支、JAL、JALR、循环和随机目标地址，检查前端预测与地址修正后的取指持续性。rv32_unaligned_load_store 与 rv64_unaligned_load_store 使用低位为 1、2、3 的地址，分别检查已实现的非自然对齐访问或异常路径。rv32_bitmanip 与 rv64_bitmanip 使用零、全一、最高位、交替位和固定模式检查 Zba、Zbb、Zbs、Zbc。rv32_illegal_instr、rv64_invalid_csr、rv64_ebreak 和中断类别在处理程序

中读取 mcause、mepc 和计数器，修正返回地址，再执行后续指令，以检查异常入口、CSR 可见性和 mret 返回。

5 边界条件 Case 设计

5.1 设计原则

边界条件测试针对数值端点、地址跨越、相邻指令数据冒险、多个执行单元近邻完成、异常与访存交叠、暂停与恢复等状态组织。每条测试在触发条件前后设置独立检查点，覆盖零间隔、一个指令间隔、多个指令间隔、32 位与 64 位边界、页首页末、队列接近满、长延迟完成与异常返回等位置。C 程序使用统一的边界检查定义保存检查编号、实际值、期望值、失败数和程序结果校验字；汇编程序使用统一宏写入明确失败状态；访存和排序程序在有效数据两侧设置保护字。

5.2 数据冒险检测方法

冒险或控制条件	激励构造	检测方式
读后写数据冒险	ALU、MUL、DIV、CSR 或 load 的结果被下一条指令立即读取	检查使用结果的寄存器值、程序结果校验字和 GPR 写回数据，旧值被误用会触发程序内部错误
写后写数据冒险	两条相邻指令写同一目的寄存器，或不同执行单元在相邻周期完成	检查最终寄存器值、commit id 释放和 WBU 选择顺序
load-use	load 后零间隔执行算术、条件分支、store 或地址计算	检查算术结果、分支方向、写入数据和最终回读值
store/load 顺序	不同宽度 store、由长延迟结果形成的 store 数据、紧随其后的 load	检查字节写使能、存储器最终值、load 回读和保护字
控制转移与冲刷	taken/not-taken 分支、JAL/JALR、异常和 mret 后紧跟具有写入副作用的指令	检查目标地址结果，并确认未执行路径的寄存器写入和 store 未发生
长延迟完成	DIV、MUL、CSR、load 和 ALU 在相邻时间范围内完成	检查每项结果、commit id、WBU 接收和普通指令继续执行
异常与访存	ecall、非法指令、EBREAK 或无效 CSR 靠近 store、load、DIV 和分支	检查 mcause、mepc、异常前已接受的操作、被清除的操作和返回后的新操作

表 129 数据冒险与控制条件检测

5.3 流水线边界条件

流水线测试先固定一条指令的输入、执行单元、完成事件和写回寄存器，再将相关指令放在零间隔、一个指令间隔和多个指令间隔的位置。相同程序在 P3、P4 和 P5 执行，检查流水线级数只改变处理时序而不改变架构结果。若第一条指令仍在 MUL、DIV、CSR 或 LSU 中，相关指令必须等待正确结果；若分支、异常或冲刷取消相关指令，该指令不得产生 GPR 写入或 store。

hazard_edges 只改变相邻指令间隔，用于定位 RAW 与 WAW 控制；completion_concurrency_edges 让 ALU、MUL、DIV、load 和 CSR 在近邻周期完成，检查 WBU 与 commit id 状态；pipeline_drain_edges 在长延迟运算和存储访问后执行连续普通指令，检查流水线排空后再次发射；x0_load_commit_id 在 load、ALU 和 MUL 操作中重复写 x0，并在其后执行非零目的寄存器的相关指令，检查 x0 不接收数据且不会占用错误的 commit id 状态。

流水线边界条件设置四个相互独立的观察点：提交接口的操作码、目的寄存器和写回数据；stage_cg 的 IF、Dispatch、ALU、MUL、DIV 和 LSU 执行状态；程序结果区与八个程序结果校验字；程序计数器活性检查。任一观察点不满足预期，Case 即失败，因此可区分结果错误、写回次序错误、目标单元未执行和程序停止等不同问题。

5.4 访存边界条件

访存测试按访问宽度、地址低位、响应等待和前后指令关系组织。memory_edges 在同一字内依次执行 byte、halfword、word store，再以有符号和无符号 load 回读，检查各字节写使能、符号扩展和零扩展。dependency_memory_edges 与 sw_misaligned_forwarding_mix 让 ALU、MUL、load 和 CSR 的结果立即作为 store 地址或数据，再以不同访问宽度回读。地址覆盖 32 字节边界、64 字节边界、页末和页初，并在数据两侧检查保护字，避免只检查目标数据而漏检越界写。

dcm_response_default 与 dcm_response_registered 对比 DTCM 直接响应和增加一级数据寄存器后的整核行为。每个参数化访存 case 都检查 tohost、程序计数器活性、仿真超时、load/store 回读和延迟读响应不会覆盖已完成的后

续 store。该组测试只描述普通 DTCM 响应路径，不作为 Cache 或乒乓缓存专项结论。

5.5 异常与恢复边界条件

异常测试在执行前配置 mtvec，在 ecall、非法指令、EBREAK、无效 CSR 或访存操作附近触发异常。处理程序读取 mcause，读取并增加 mepc，写回 mepc 后执行 mret。主程序检查异常计数、异常前已经完成的 store、被清除路径上的 store，以及返回后的新 load/store。该序列同时检查异常原因、异常地址、已经接受操作的保存、错误路径清除和返回后的流水线恢复。

trap_lsu_token_edges 先执行 store 再触发 ecall，检查较早 store 保留；随后在分支错误路径放置 store，检查保护位置仍为零；最后在 DIV 结果写入附近触发异常，检查长延迟结果与较早 store 均不丢失。sw_fpu_fs_off_illegal 在未启用浮点功能的配置中执行固定浮点编码，处理程序读取 mcause 和 mepc 并返回，主程序检查后续整数结果。该用例把异常触发与异常字段可被处理程序读取分开检查，覆盖 P4/P5 异常 CSR 写入、取指地址保持和 mret。

5.6 边界条件 Case 清单

类别	C 程序	汇编程序	主要检查内容
算术与执行	alu_edges、 early_alu_edges、 muldiv_edges	sw_mul_div_edge_resu lts	有符号与无符号端 点、乘除零值、溢 出、长延迟结果
分支与前端	branch_predictor_press ure、pipeline_edges	sw_control_sequence、 sw_unsigned_branch_j alr、 sw_jalr_lui_bypass	条件分支、 JAL/JALR、地址修正 和冲刷
数据冒险与完成次序	completion_broadcast_ edges、 completion_concurrenc y_edges、 hazard_edges、 writeback_pressure	sw_dependency_bound ary、 sw_load_interlock、 sw_forwarding_alu、 sw_forwarding_load、 sw_forwarding_mul_di v、sw_forwarding_csr	RAW、WAW、执行 单元近邻完成、WBU 选择和流水线排空

类别	C 程序	汇编程序	主要检查内容
CSR、异常与返回	csr_counter_edges、 csr_trap_edges、 ebreak_trap_edges、 exception_stress、 trap_pipeline_edges	sw_fpu_fs_off_illegal	CSR 读写、计数器、 非法指令、 EBREAK、mepc 与 mret
存储器访问	memory_edges、 mmio_dtcmm_order_edges、 trap_lsu_token_edges	sw_address_boundary、 sw_data_boundary、 sw_subword_readback_matrix、 sw_misaligned_boundaries、 sw_misaligned_overlap	子字访问、地址端 点、MMIO 与 DMEM 次序、非自然 对齐访问和异常
位操作扩展	bitmanip_pressure	sw_forwarding_bitmanip、 sw_bitmanip_address_data、 sw_bitmanip_immediate	Zba、Zbb、Zbs、Zbc 的端点位、移位量和 相关使用

表 130 边界条件 Case 清单

5.7 排序程序与编译组合

排序程序集包含 bubble、heap、insertion、merge、quick、radix、selection 和 shell 八种算法。每种算法在 O0、O1、O2、O3、Os、Og、O2_noinline 和 O3_app_unroll 八种编译配置下运行，三个流水线配置共形成 192 个执行任务。编译选项改变 load/store、分支和循环组织方式，使同一算法能够检查不同指令序列下的处理结果一致性。

每个排序程序测试长度 0、1、2、3、31、32、33、99、100、101，并使用升序、降序、全相等、周期重复、单个极值、最小值与最大值交替、三个固定随机种子和固定 100 元素样本。检查不只判断输出是否非降序：程序先由独立参考实现给出期望数组，再逐元素比较，同时计算三种多重集合摘要检查元素丢失或重复，并检查工作区两侧各四个保护字。

以 bubble_sort 为例，长度 0 和 1 检查空循环与单元素返回，长度 31/32/33 检查循环计数在分块长度附近是否少执行或多执行，长度 99/100/101 检查工作区上界和交换次数，降序与全相等输入检查交换条件与终止条件。heap、merge、quick、radix 和 shell 分别补充父子索引、长度不等的合并、重复值分区、符号位和位段处理、不同步长跨区访问等算法特有边界。

6 Feature List

本章的 Feature List 按模块拆分，每个模块单独使用一张表。每行只定义一个能够独立检查的 Feature，并且只指定一个主要验证 Case；同一功能在不同流水线级数或编译配置下执行时，沿用同一个逻辑 Case 名称，由任务配置确定具体实例。若一个模块包含多种互不依赖的功能或边界条件，则分别定义 Feature ID，避免在同一项 Feature 中并列多个检查目标。

6.1 CPU 前端、分发与执行 Feature List

6.1.1 core_top Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-CORE-01	core_top	P3、P4、P5 流水线级数选择	程序 Case: pipeline_edges
F-CORE-02	core_top	流水线暂停时的级间状态保持	程序 Case: pipeline_drain_edges
F-CORE-03	core_top	流水线冲刷时的无效指令清除	程序 Case: trap_pipeline_edges
F-CORE-04	core_top	完成接口向写回级传递结果	系统 Case: alkaid_wbu_test
F-CORE-05	core_top	RV32 配置下的参数传递	程序 Case: rv32c_selfcheck
F-CORE-06	core_top	RV64 配置下的参数传递	程序 Case: rv64i_zicsr_zifencei_selfcheck

表 131-1 core_top Feature List

6.1.2 IFU Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-IFU-01	ifu_ifetch	程序计数器顺序推进	系统 Case: alkaid_ifu_test
F-IFU-02	ifu_axi_master	AXI4-Lite 取指请求握手	模块 Case: ifu_axi_master
F-IFU-03	ifu_axi_master	取指响应与请求地址一致	模块 Case: ifu_enhanced_recovery
F-IFU-04	ifu_pipe	暂停时保持取指级状态	系统 Case: alkaid_ifu_test

Feature ID	RTL 模块	Feature 描述	对应 Case
F-IFU-05	recovery_interface	分支修正后的取指恢复	模块 Case: recovery_interface
F-IFU-06	recovery_interface	异常冲刷后的请求恢复	模块 Case: ifu_enhanced_recovery
F-IFU-07	ifu_c_stream	压缩指令半字选择	模块 Case: ifu_c_stream
F-IFU-08	rv_c_decompressor	RV32 压缩指令解压缩	模块 Case: alkaid_rvc_test
F-IFU-09	rv_c_decompressor	RV64 压缩指令解压缩	程序 Case: rv64c_selfcheck
F-IFU-10	rv_c_decompressor	非法压缩编码识别	程序 Case: rv32c_selfcheck

表 131-2 IFU Feature List

6.1.3 BPU Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-BPU-01	sbpu	基础 BHT 条件分支方向预测	模块 Case: sbpu
F-BPU-02	sbpu	返回地址栈的压入与弹出	程序 Case: branch_predictor_pressure
F-BPU-03	sbpu	基础目标地址预测	系统 Case: rv32_jump_stress
F-BPU-04	enhanced_bpu	BTB 目标地址查询	模块 Case: enhanced_bpu
F-BPU-05	enhanced_bpu	GShare 条件分支方向预测	模块 Case: enhanced_bpu
F-BPU-06	enhanced_bpu	循环分支预测	程序 Case: branch_predictor_pressure
F-BPU-07	enhanced_bpu	JALR 目标地址预测	系统 Case: rv64_rand_jump
F-BPU-08	enhanced_bpu	TAGE 条件分支方向预测	模块 Case: enhanced_bpu
F-BPU-09	enhanced_bpu	预测状态训练	模块 Case: enhanced_bpu
F-BPU-10	enhanced_bpu	冲刷后的历史状态恢复	模块 Case: ifu_enhanced_recovery

表 131-3 BPU Feature List

6.1.4 IDU Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-IDU-01	idu_decode	操作码译码	系统 Case: alkaid_idu_test
F-IDU-02	idu_decode	立即数生成	系统 Case: rv32_rand_instr
F-IDU-03	idu_decode	源寄存器地址译码	系统 Case: rv64_rand_instr
F-IDU-04	idu_decode	目的寄存器写使能译码	系统 Case: alkaid_idu_test
F-IDU-05	idu_decode	CSR 指令译码	程序 Case: rv64i_zicsr_zifencei_selfcheck
F-IDU-06	idu_decode	非法指令异常识别	系统 Case: rv64_illegal_instr
F-IDU-07	inst_reserve_stack	未发射指令保留	程序 Case: pipeline_edges
F-IDU-08	inst_reserve_stack	压缩指令长度保持	模块 Case: ifu_c_stream
F-IDU-09	idu_id_pipe	暂停时保持译码结果	系统 Case: alkaid_idu_test
F-IDU-10	idu_id_pipe	冲刷时清除译码有效状态	程序 Case: trap_pipeline_edges

表 131-4 IDU Feature List

6.1.5 Dispatch Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-DSP-01	dispatch	单发射指令选择	模块 Case: dispatch
F-DSP-02	dispatch_logic	执行单元请求生成	系统 Case: alkaid_dispatch_test
F-DSP-03	dispatch_pipe	暂停时保持源操作数	模块 Case: dispatch_pipe
F-DSP-04	dispatch_logic	异常指令发射抑制	模块 Case: dispatch
F-DSP-05	agu	访存有效地址计算	程序 Case: sw_address_boundary
F-DSP-06	dispatch_logic	访存数据宽度传递	程序 Case: sw_data_boundary
F-DSP-07	dispatch_pipe	发射流水寄存器保持	模块 Case: dispatch_pipe

Feature ID	RTL 模块	Feature 描述	对应 Case
F-DSP-08	dispatch_hot_cache	LSU L0 Cache 命中等待	模块 Case: dispatch_hot_cache
F-DSP-09	dispatch_hot_cache	LSU L0 Cache 关闭时旁路	系统 Case: hot_cache_pingpong_stage1

表 131-5 Dispatch Feature List

6.1.6 HDU Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-HDU-01	hdu	RAW 数据冒险检测	程序 Case: hazard_edges
F-HDU-02	hdu	WAW 数据冒险检测	程序 Case: sw_dependency_boundary
F-HDU-03	hdu	执行单元占用检测	模块 Case: hdu
F-HDU-04	hdu	加载结果旁路选择	程序 Case: sw_load_interlock
F-HDU-05	hdu	冲刷时撤销等待状态	模块 Case: hdu
F-HDU-06	hdu	写回完成后的资源释放	程序 Case: pipeline_drain_edges
F-HDU-07	hdu	x0 写入不建立等待状态	程序 Case: x0_load_commit_id
F-HDU-08	hdu	长延迟指令等待状态保持	程序 Case: completion_concurrency_edges

表 131-6 HDU Feature List

6.1.7 EXU Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-EXU-01	exu	单发射执行单元选择	模块 Case: exu_feature
F-EXU-02	exu	执行结果送入 WBU	系统 Case: alkaid_alu_test
F-EXU-03	exu	暂停时保持 LSU 状态	模块 Case: exu_lsu_state_recovery
F-EXU-04	exu	长延迟执行状态保持	程序 Case: pipeline_drain_edges

Feature ID	RTL 模块	Feature 描述	对应 Case
F-EXU-05	exu	冲刷时取消未完成操作	程序 Case: trap_pipeline_edges
F-EXU-06	exu	多执行单元近邻完成	程序 Case: completion_concurrency_edges
F-EXU-07	exu	结果有效信号持续到写回接收	程序 Case: writeback_pressure

表 131-7 EXU Feature List

6.1.8 ALU Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-ALU-01	exu_alu	整数加减法结果	程序 Case: alu_edges
F-ALU-02	exu_alu	按位逻辑运算结果	模块 Case: exu_feature
F-ALU-03	exu_alu	逻辑与算术移位结果	程序 Case: alu_edges
F-ALU-04	exu_alu	有符号与无符号比较结果	程序 Case: alu_edges
F-ALU-05	exu_alu	Zba 地址生成位操作	程序 Case: rv64_bitmanip_selfcheck
F-ALU-06	exu_alu_bitcount32	Zbb 位计数结果	程序 Case: bitmanip_pressure
F-ALU-07	exu_alu	Zbs 单比特操作	程序 Case: sw_bitmanip_immediate

表 131-8 ALU Feature List

6.1.9 BRU Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-BRU-01	exu_bru	条件分支方向判断	程序 Case: branch_predictor_pressure
F-BRU-02	exu_bru	JAL 目标地址计算	程序 Case: sw_control_sequence
F-BRU-03	exu_bru	JALR 目标地址最低位清零	程序 Case: sw_unsigned_branch_jalr

Feature ID	RTL 模块	Feature 描述	对应 Case
F-BRU-04	exu_bru	JALR 源操作数旁路	程序 Case: sw_jalr_lui_bypass
F-BRU-05	exu_bru	预测错误后的地址修正	程序 Case: branch_predictor_pressure
F-BRU-06	exu_bru	fence.i 触发取指冲刷	程序 Case: sw_fence_stall
F-BRU-07	exu_bru	控制转移错误路径清除	模块 Case: exu_feature

表 131-9 BRU Feature List

6.1.10 MUL Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-MUL-01	mul	RV32 低位乘积	模块 Case: mul_rv32_single
F-MUL-02	mul	RV64 低位乘积	模块 Case: mul_rv64
F-MUL-03	mul	RV64 高半乘积	模块 Case: mul_rv64_segmented
F-MUL-04	mul	有符号与无符号操作数组合	程序 Case: muldiv_edges
F-MUL-05	mul	面积优先分段计算	模块 Case: mul_area_recovery
F-MUL-06	exu_mul	Zbc 无进位乘法低位结果	模块 Case: exu_zbc_p3
F-MUL-07	exu_mul	Zbc 无进位乘法高位结果	程序 Case: rv64_bitmanip_selfcheck
F-MUL-08	exu_mul	乘法结果被相邻指令读取	程序 Case: sw_forwarding_bitmanip
F-MUL-09	mul	RV64 MULW 快速低位路径与符号扩展	模块 Case: mul_rv64
F-MUL-10	mul	RV64 高半乘法迭代结果	模块 Case: mul_rv64
F-MUL-11	mul	RV64 写回阻塞时结果保持	模块 Case: mul_rv64
F-MUL-12	mul	RV64 复位中止后的状态恢复	模块 Case: mul_rv64

表 131-10 MUL Feature List

6.1.11 DIV Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-DIV-01	div	有符号除法商	程序 Case: muldiv_edges
F-DIV-02	div	无符号除法商	程序 Case: rv64m_selfcheck
F-DIV-03	div	余数指令结果	程序 Case: rv64m_selfcheck
F-DIV-04	div	除数为零时的规定结果	程序 Case: muldiv_edges
F-DIV-05	div	最小负数除以负一时的规定结果	程序 Case: muldiv_edges
F-DIV-06	div	RV64 商位分步计算的中间状态保持	程序 Case: completion_concurrency_edges
F-DIV-07	exu_div	除法结果持续到写回接收	程序 Case: pipeline_drain_edges

表 131-11 DIV Feature List

6.1.12 LSU Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-LSU-01	exu_lsu	byte、halfword、word 加载宽度选择	程序 Case: sw_subword_readback_matrix
F-LSU-02	exu_lsu	加载结果符号扩展	程序 Case: memory_edges
F-LSU-03	exu_lsu	存储字节写使能生成	模块 Case: exu_lsu_state_recovery
F-LSU-04	exu_lsu	响应等待期间保持访存状态	模块 Case: exu_lsu_state_recovery
F-LSU-05	exu_lsu	LR/SC 保留状态控制	程序 Case: rv32a_selfcheck
F-LSU-06	exu_lsu	AMO 读改写次序	程序 Case: rv64a_selfcheck
F-LSU-07	exu_lsu	AXI4-Lite 写地址与写数据独立握手	模块 Case: exu_feature
F-LSU-08	exu_lsu	异常发生时隔离未接受的访存操作	程序 Case: trap_lsu_token_edges

Feature ID	RTL 模块	Feature 描述	对应 Case
F-LSU-09	exu_lsu	非自然对齐访问拆分	程序 Case: sw_misaligned_boundaries
F-LSU-10	exu_lsu	跨边界访存结果合并	程序 Case: sw_misaligned_overlap
F-LSU-11	exu_lsu	MMIO 与 DMEM 访问区分	程序 Case: mmio_dtcmm_order_edges
F-LSU-12	exu_lsu	DTCM 延迟响应接收	系统 Case: dtcm_response_registered

表 131-12 LSU Feature List

6.1.13 CSR Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-CSR-01	exu_csr_unit	机器模式 CSR 读写	系统 Case: rv32_csr
F-CSR-02	csr	周期与指令计数器读取	程序 Case: csr_counter_edges
F-CSR-03	exu_csr_unit	CSR 写后读前递	模块 Case: csr_forward
F-CSR-04	exu_csr_unit	非法 CSR 地址异常	系统 Case: rv64_invalid_csr
F-CSR-05	csr	异常原因写入 mcause	程序 Case: csr_trap_edges
F-CSR-06	csr	异常地址写入 mepc	程序 Case: ebreak_trap_edges
F-CSR-07	csr	异常附加信息写入 mtval	程序 Case: sw_fpu_fs_off_illegal
F-CSR-08	csr	mret 恢复机器模式状态	程序 Case: exception_stress

表 131-13 CSR Feature List

6.1.14 WBU Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-WBU-01	wbu	多执行单元完成结果选择	模块 Case: reg_wb_fifo
F-WBU-02	reg_wb_fifo	写回等待期间保持结果	程序 Case: writeback_pressure

Feature ID	RTL 模块	Feature 描述	对应 Case
F-WBU-03	wbu	近邻完成结果的写回次序	程序 Case: completion_concurrency_edges
F-WBU-04	wbu	完成后释放 commit id	程序 Case: pipeline_drain_edges
F-WBU-05	wbu	冲刷时取消无效完成项	模块 Case: hdu
F-WBU-06	wbu	x0 写回抑制	程序 Case: x0_load_commit_id
F-WBU-07	wbu	无 GPR 写回指令的完成释放	系统 Case: alkaid_wbu_test

表 131-14 WBU Feature List

6.1.15 GPR Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-GPR-01	gpr	双读口独立读取	模块 Case: alkaid_gpr_test
F-GPR-02	gpr	单写口寄存器更新	模块 Case: alkaid_gpr_test
F-GPR-03	gpr	x0 恒为零	程序 Case: x0_load_commit_id
F-GPR-04	gpr	同周期写后读前递	模块 Case: alkaid_gpr_test
F-GPR-05	gpr	原始读数据送入旁路选择	模块 Case: reg_wb_fifo

表 131-15 GPR Feature List

6.2 中断、总线、存储器与 SoC Feature List

6.2.1 CLINT Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-CLINT-01	clint_swi	msip 的 MMIO 读写	模块 Case: clint_mmio
F-CLINT-02	clint	mtime 计数递增	程序 Case: csr_counter_edges
F-CLINT-03	clint	mtimecmp 比较产生定时器中断	系统 Case: rv64_full_interrupt

Feature ID	RTL 模块	Feature 描述	对应 Case
F-CLINT-04	clint	软件中断进入异常处理	系统 Case: rv32_full_interrupt
F-CLINT-05	clint	中断等待期间保持待处理状态	程序 Case: csr_trap_edges
F-CLINT-06	clint	中断响应时触发流水线冲刷	程序 Case: exception_stress
F-CLINT-07	clint	mret 后恢复正常取指	程序 Case: csr_trap_edges

表 132-1 CLINT Feature List

6.2.2 PLIC Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-PLIC-01	plic	中断源使能寄存器读写	模块 Case: plic_mmio
F-PLIC-02	plic_priority_arbiter	多中断源优先级选择	模块 Case: alkaid_plic_arbiter_test
F-PLIC-03	plic_priority_arbiter	最高优先级中断编号输出	模块 Case: alkaid_plic_arbiter_test
F-PLIC-04	plic	优先级寄存器 MMIO 访问	模块 Case: plic_mmio
F-PLIC-05	plic_top	处理函数参数输出	模块 Case: plic_mmio
F-PLIC-06	plic_top	外部中断送入 CPU	系统 Case: rv64_full_interrupt

表 132-2 PLIC Feature List

6.2.3 AXI 互连 Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-AXI-01	axi_interconnect	M0 与 M1 同时请求时的仲裁	模块 Case: axi_interconnect
F-AXI-02	axi_interconnect	IMEM 读响应返回正确主机	模块 Case: axi_interconnect
F-AXI-03	axi_interconnect	目标地址译码	模块 Case: axi_interconnect
F-AXI-04	axi_interconnect	未完成读事务计数	模块 Case: axi_interconnect_p2

Feature ID	RTL 模块	Feature 描述	对应 Case
F-AXI-05	axi_interconnect	M1 零级缓冲时的访问次序	模块 Case: axi_interconnect_m1_p0_order
F-AXI-06	axi_interconnect	M1 一级缓冲时的访问次序	模块 Case: axi_interconnect_m1_p1_order
F-AXI-07	axi_interconnect	M1 两级缓冲时的访问次序	模块 Case: axi_interconnect_m1_p2_order
F-AXI-08	axi_interconnect	MMIO 与 DTCM 连续访问次序	程序 Case: mmio_dtcmm_order_edges

表 132-3 AXI 互连 Feature List

6.2.4 AXI2APB Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-AXI2APB-01	axi2apb	AXI4-Lite 地址转换为 APB 地址	模块 Case: axi2apb
F-AXI2APB-02	axi2apb	APB 目标选择信号生成	模块 Case: axi2apb
F-AXI2APB-03	axi2apb	APB 等待状态保持	模块 Case: axi2apb
F-AXI2APB-04	axi2apb	APB 读数据返回 AXI4-Lite	模块 Case: axi2apb
F-AXI2APB-05	axi2apb	APB 写完成响应返回 AXI4-Lite	模块 Case: axi2apb
F-AXI2APB-06	axi2apb	APB 错误响应转换	模块 Case: axi2apb

表 132-4 AXI2APB Feature List

6.2.5 gnrl_ram Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-RAM-01	gnrl_ram_axi	AXI4-Lite RAM 读取	模块 Case: gnrl_ram_axi
F-RAM-02	gnrl_ram_axi	字节写使能控制	模块 Case: gnrl_ram_axi
F-RAM-03	gnrl_ram_axi	响应等待期间保持读数据	模块 Case: gnrl_ram_axi
F-RAM-04	gnrl_ram_axi	初始化数据可被 CPU 读取	系统 Case: rv32_memory_stress

表 132-5 gnrl_ram Feature List

6.2.6 owned_leaf Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-OWNED-01	owned_leaf	FIFO 空满状态	模块 Case: owned_leaf
F-OWNED-02	owned_leaf	计数器上界与回绕	模块 Case: perf_probe
F-OWNED-03	owned_leaf	时钟门控使能控制	模块 Case: owned_leaf
F-OWNED-04	owned_leaf	伪双端口存储单元并行读写	模块 Case: owned_leaf

表 132-6 owned_leaf Feature List

6.2.7 pdram_axi_controller Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-PDRAM-01	pdram_axi_controller	外部 RAM 地址输出	模块 Case: pdram_axi
F-PDRAM-02	pdram_axi_controller	外部 RAM 读取控制	模块 Case: pdram_axi
F-PDRAM-03	pdram_axi_controller	外部 RAM 写入控制	模块 Case: pdram_axi
F-PDRAM-04	pdram_axi_controller	等待状态下保持请求	系统 Case: dtdcm_response_registered
F-PDRAM-05	pdram_axi_controller	读取数据返回 AXI4-Lite	模块 Case: pdram_axi

表 132-7 pdram_axi_controller Feature List

6.2.8 simple_axi_ram Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-SRAM-01	simple_axi_ram	读地址通道握手	模块 Case: simple_axi_ram
F-SRAM-02	simple_axi_ram	写地址通道握手	模块 Case: simple_axi_ram
F-SRAM-03	simple_axi_ram	写数据通道握手	模块 Case: simple_axi_ram

Feature ID	RTL 模块	Feature 描述	对应 Case
F-SRAM-04	simple_axi_ram	直接读响应	系统 Case: dttm_response_default
F-SRAM-05	simple_axi_ram	寄存读响应	系统 Case: dttm_response_registered

表 132-8 simple_axi_ram Feature List

6.2.9 SoC 顶层 Feature List

Feature ID	RTL 模块	Feature 描述	对应 Case
F-SOC-01	alkaid_soc_top	时钟与复位接入	SoC 基准 Case
F-SOC-02	alkaid_soc_top	CPU 数据接口接入外设子系统	模块 Case: jyd_perip
F-SOC-03	alkaid_soc_top	MMIO 地址空间选择	程序 Case: mmio_dttm_order_edges
F-SOC-04	alkaid_soc_top	程序镜像装入存储器	SoC 基准 Case
F-SOC-05	alkaid_soc_top	外设中断接入 CPU	系统 Case: rv64_full_interrupt

表 132-9 SoC 顶层 Feature List

6.3 指令集 Feature List

Feature ID	指令集范围	Feature 描述	对应 Case
F-ISA-01	RV32I	基础整数算术指令执行	系统 Case: rv32_arithmetic_basic
F-ISA-02	RV32I	条件分支与跳转指令执行	系统 Case: rv32_jump_stress
F-ISA-03	RV32I	基础加载与存储指令执行	系统 Case: rv32_memory_stress
F-ISA-04	RV32M	整数乘除指令执行	程序 Case: muldiv_edges
F-ISA-05	RV32A	32 位原子指令执行	程序 Case: rv32a_selfcheck
F-ISA-06	RV32C	32 位压缩指令执行	程序 Case: rv32c_selfcheck
F-ISA-07	RV32 Zicsr	CSR 指令执行	系统 Case: rv32_csr
F-ISA-08	RV32 Zifencei	fence.i 指令执行	程序 Case: sw_fence_stall

Feature ID	指令集范围	Feature 描述	对应 Case
F-ISA-09	RV32 Zba	地址生成位操作指令执行	系统 Case: rv32_bitmanip
F-ISA-10	RV32 Zbb	基础位操作指令执行	系统 Case: rv32_bitmanip
F-ISA-11	RV32 Zbs	单比特指令执行	程序 Case: sw_bitmanip_immediate
F-ISA-12	RV32 Zbc	无进位乘法指令执行	程序 Case: sw_forwarding_bitmanip
F-ISA-13	RV64I	64 位基础整数指令执行	程序 Case: rv64i_zicsr_zifencei_selfcheck
F-ISA-14	RV64M	64 位乘除指令执行	程序 Case: rv64m_selfcheck
F-ISA-15	RV64A	64 位原子指令执行	程序 Case: rv64a_selfcheck
F-ISA-16	RV64C	64 位压缩指令执行	程序 Case: rv64c_selfcheck
F-ISA-17	RV64 Zicsr	64 位配置下的 CSR 指令执行	系统 Case: rv64_csr
F-ISA-18	RV64 Zifencei	64 位配置下的 fence.i 指令执行	程序 Case: rv64i_zicsr_zifencei_selfcheck
F-ISA-19	RV64 Zba	64 位地址生成位操作指令执行	程序 Case: rv64_bitmanip_selfcheck
F-ISA-20	RV64 Zbb	64 位基础位操作指令执行	程序 Case: rv64_bitmanip_selfcheck
F-ISA-21	RV64 Zbs	64 位单比特指令执行	程序 Case: rv64_bitmanip_selfcheck
F-ISA-22	RV64 Zbc	64 位无进位乘法指令执行	程序 Case: rv64_bitmanip_selfcheck

表 133 指令集 Feature List 与字长检查矩阵

7 测试用例清单与覆盖率统计

7.1 模块级 Case 清单

Case 组	Case ID	检查内容
基础单元	模块 Case: alkaid_gpr_test、 模块 Case: alkaid_rvc_test、 模块 Case: alkaid_plic_arbiter_test、模块 Case: csr_forward	GPR 双读口与 x0、RV32C 解压缩、PLIC 优先级、CSR 写后读前递
RV64 MUL 波形案例	模块 Case: mul_rv64	RV64 MUL/MULW/MULH/MULHS U/MULHU、结果保持、复位 中止和非法操作保护
MUL 参数	模块 Case: mul_rv64、模块 Case: mul_rv64_segmented、 模块 Case: mul_area_recovery、模块 Case: mul_rv32_single	RV64 迭代与分段乘法、 RV32 面积优先与单周期乘法
EXU、RVC 与计数	模块 Case: rvc_rv64、模块 Case: exu_feature、模块 Case: exu_lsu_state_recovery、模块 Case: perf_probe	压缩编码、执行扩展、LSU 复位恢复、MUL/LSU 事件计 数
单项诊断	模块 Case: exu_zbc_p3、模 块 Case: ifu_enhanced_recovery_verilato r	P3 Zbc 完成编号、增强取指 恢复逐周期检查；不进入批量 覆盖率任务
调度、相关与写回	模块 Case: reg_wb_fifo、模 块 Case: dispatch_pipe、模块 Case: dispatch、模块 Case: dispatch_hot_cache、模块 Case: hdu	FIFO、源操作数保持、执行选 择、LSU L0 Cache 等待和数 据冒险状态
取指与预测	模块 Case: ifu_axi_master、 模块 Case: sbpu、模块 Case: enhanced_bpu、模块 Case: ifu_c_stream、模块 Case: ifu_enhanced_recovery、模块 Case: recovery_interface	AXI 取指、基础/增强预测、 压缩流和恢复接口

Case 组	Case ID	检查内容
AXI/APB 与 Cache	模块 Case: axi2apb、模块 Case: axi_buffer_cache、模块 Case: axi_buffer_cache_overlap	APB 等待与错误响应、Cache 命中/缺失/替换和重叠读缺失
AXI 互连参数	模块 Case: axi_interconnect、模块 Case: axi_interconnect_p2、模块 Case: axi_interconnect_m1_p0_order、模块 Case: axi_interconnect_m1_p1_order、模块 Case: axi_interconnect_m1_p2_order	默认与两级缓冲、M1 零至两级时的 DMEM/APB 读写次序
中断与外设	模块 Case: clint_mmio、模块 Case: plic_mmio、模块 Case: jyd_perip	CLINT、PLIC 和自研 APB 外设寄存器访问
存储与通用 RTL	模块 Case: gnrl_ram_axi、模块 Case: owned_leaf、模块 Case: pdram_axi、模块 Case: simple_axi_ram	AXI RAM、FIFO、计数器、时钟门控、伪双端口 RAM 和简化 AXI RAM

表 134 模块级 Case 清单

模块 Case: exu_zbc_p3 和 模块 Case: ifu_enhanced_recovery_verilator 用于单项诊断，不计入批量覆盖率任务。正式模块 case 按 P3、P4、P5 展开运行，以覆盖流水线切换产生的 RTL 结构差异。

7.2 整核 Feature 与程序 Case 清单

整核 Feature 检查包含 alkaid_ifu_test、alkaid_idu_test、alkaid_dispatch_test、alkaid_alu_test、alkaid_muldiv_test、alkaid_lsu_test、alkaid_csr_clint_test 和 alkaid_wbu_test。每项检查要求对应模块在程序执行期间出现预期的有效信号，并与程序结果检查共同使用。程序集合的规模和用途如下表所示。

程序集合	逻辑规模	P3/P4/P5 任务数	主要检查
指令集定向自检	7 个程序	21	程序内部结果、tohost、Spike 八个程序结果校验字
固定 O2 边界条件程序	22 个程序	66	算术、控制转移、数据冒险、CSR、异常和访存

程序集合	逻辑规模	P3/P4/P5 任务数	主要检查
汇编程序	39 个程序	117	基础指令、M/Zb 扩展、FENCE、异常、数据冒险和非自然对齐
排序编译组合	8 算法 × 8 配置	192	91 组内部检查、tohost、Spike 八个程序结果校验字
边界条件编译组合	21 程序 × 8 配置	504	编译选项改变后的完整执行结果
补充 C 程序	4 个程序	12	ALU/分支、CSR、访存数据冒险和位操作
x0 commit id 程序	1 个程序，仅 P4/P5	2	x0 抑制、commit id 释放和相关结果

表 135 整核程序 Case 清单

7.3 访存参数 Case 清单

访存参数测试包含十项 Case。前九项在 P3、P4、P5 上执行，LSU L0 Cache 组合仅在 P5 上执行，因此共形成 28 项任务。每项任务使用同一组访存边界程序，并检查 DTCM 响应延迟、AXI4-Lite 乒乓缓存级数和 LSU L0 Cache 参数改变后，程序结果、存储顺序及程序计数器活性保持一致。

Case ID	DTCM 读数据寄存	M0 乒乓缓存级数	M1 乒乓缓存级数	LSU L0 Cache
dcm_response_default	0	0	0	关闭
dcm_response_registered	1	0	0	关闭
m0_pingpong_stage1	0	1	0	关闭
m0_pingpong_stage2	0	2	0	关闭
m1_pingpong_stage1	0	0	1	关闭
m1_pingpong_stage2	0	0	2	关闭
pingpong_m0s1_m1s2	0	1	2	关闭
pingpong_m0s2_m1s1	0	2	1	关闭

Case ID	DTCM 读数据寄存	M0 乒乓缓存级数	M1 乒乓缓存级数	LSU L0 Cache
dtcm_registered_pingpong_stage2	1	2	2	关闭
hot_cache_pingpong_stage1（仅 P5）	0	1	1	深度 64、1 way

表 136 访存参数 Case 清单

7.4 回归计划

Make 目标	任务组成	任务数
run_regression	111 模块 + 28 普通访存参数 + 276 系统 + 914 程序	1329
run_regression_cov	完整回归 + 3 SoC 覆盖率参考任务 + 7 合并任务	1339
run_regression_cov_min	111 模块 + 28 普通访存参数 + 150 系统 + 218 程序 + 3 SoC 覆盖率参考任务 + 7 合并任务	517

表 137 回归计划

精简覆盖率回归保留全部正式模块 Case、全部普通访存参数 Case、42 类随机程序的第一个种子、八项整核 Feature 检查、固定 O2 边界条件程序、汇编程序、指令集自检和补充 C 程序。它只减少重复随机种子与排序、边界条件的多编译配置组合，不改变已启用 Feature 的覆盖率统计范围。

7.5 覆盖率统计方法

VCS 编译与仿真启用行、条件、分支、状态机、翻转和断言覆盖率；feature coverage 来自整核测试平台和部分模块测试平台中的 covergroup。Verilator 覆盖率数据与 VCS 覆盖率数据库分别处理，不相互合并。P3、P4、P5 的单配置结果合并时不使用排除文件；单配置统计只允许对固定参数关闭且无法在该配置中展开的 RTL 对象使用合法排除记录。feature coverage group 不使用排除记录。

7.6 验证结果说明

模块、整核和程序 Case 分别给出通过条件与覆盖率统计方法。完整回归及跨流水线覆盖率合并完成前，不以单项 Case 或局部覆盖率百分比代表全设

计验证结果。性能数据与验证状态分别在本章的性能分析和回归统计中说明，二者不相互替代。

8 验证结果

8.1 回归执行状态

完整功能回归包含 1299 项任务，包括 111 项模块 Case、28 项访存参数 Case、276 项系统任务和 884 项程序任务；完整覆盖率回归在此基础上增加 3 项 SoC 参考任务和 7 项合并任务，共 1309 项。精简覆盖率回归包含 523 项任务，按 P3、P4、P5 分别展开为 173、174 和 175 项。

完整覆盖率回归实际通过 1309/1309 项，包含三个流水线配置的模块、系统、程序、SoC 参考和合并任务；回归末项为跨配置合并任务，结果为 PASS。图 26-1 给出回归结束阶段的终端结果。已完成的访存参数测试为 P3 9/9、P4 9/9、P5 10/10，共 28/28 项通过；该结果说明访存压力程序在对应参数组合下完成，并作为完整回归的一部分接受统一状态检查。

```
[1291/1309] PASS p5/program/boundary/03_app_unroll/lsu_queue_token_edges
[1292/1309] PASS p5/program/boundary/03_app_unroll/memory_edges
[1293/1309] PASS p5/program/boundary/03_app_unroll/mmio_dtc_order_edges
[1294/1309] PASS p5/program/boundary/03_app_unroll/muldiv_edges
[1295/1309] PASS p5/program/boundary/03_app_unroll/pipeline_drain_edges
[1296/1309] PASS p5/program/boundary/03_app_unroll/pipeline_edges
[1297/1309] PASS p5/program/boundary/03_app_unroll/producer_token_edges
[1298/1309] PASS p5/program/boundary/03_app_unroll/trap_lsu_token_edges
[1299/1309] PASS p5/program/boundary/03_app_unroll/trap_pipeline_edges
[1300/1309] PASS p5/program/boundary/extra/alu_branch_edges
[1301/1309] PASS p5/program/boundary/extra/csr_edges
[1302/1309] PASS p5/program/boundary/extra/dependency_memory_edges
[1303/1309] PASS p5/program/boundary/extra/load_commit_id
[1304/1309] PASS p5/soc/reference
[1305/1309] PASS p5/merge/soc
[1306/1309] PASS p5/merge/multi
[1307/1309] PASS merge/all
[1308/1309] PASS merge/all
[1309/1309] PASS merge/all
PASS: simulated coverage regression completed; tasks=1309/1309; log=/media/proj_tmp/alkaid_simulator/build/verif/regression_cov_scroll_check/run.log; state=/media/proj_tmp/alkaid_simulator/build/verif/regression_cov_scroll_check/state.json
```

图 26-1 完整覆盖率回归 1309/1309 全部通过

8.2 代码覆盖率统计

完整覆盖率回归已经通过，P3、P4、P5 的代码覆盖率统计如下。raw 表示未应用 profile 级排除项的统计，after-waive 表示仅应用固定参数无法展开的合法排除项后的统计。表中同时列出普通 RTL line 和 Module-definition line，功能组统计不在本表范围内。

配置	状态	SCORE	line	condition	toggle	FSM	branch	assertion	Module-definition line
P3	raw	86.72%	96.69%	78.73%	61.81%	98.44%	84.65%	100.00%	96.27%
P3	after-waive	87.56%	100.00%	78.80%	61.81%	98.44%	86.32%	100.00%	100.00%
P4	raw	87.53%	97.33%	79.15%	65.40%	97.98%	85.35%	100.00%	97.06%
P4	after-waive	88.24%	100.00%	79.21%	65.40%	97.98%	86.86%	100.00%	100.00%
P5	raw	88.62%	98.24%	81.68%	66.99%	97.98%	86.86%	100.00%	98.05%
P5	after-waive	89.13%	100.00%	81.72%	66.99%	97.98%	88.08%	100.00%	100.00%

表 137-1 P3、P4、P5 覆盖率统计

P3、P4、P5 的 after-waive 统计均达到 line 100% 和 Module-definition line 100%。相对于 raw 统计，SCORE 分别提高 0.84、0.71 和 0.51 个百分点，condition 和 branch 有小幅提高，toggle、FSM 和 assertion 保持不变。排除项只用于固定参数关闭且无法在该配置中展开的 RTL 对象，不用于隐藏可通过测试执行的代码或功能组。

图 26-2 至图 26-7 给出 P3、P4、P5 两种统计状态下的覆盖率层次截图。截图中的顶层 alkaid_soc_top 指标与表 137-1 一致，绿色和红色条形分别表示已覆盖和未覆盖部分；after-waive 截图中的 line 指标均达到 100%。

Name	Score	Line	Condition	Toggle	FSM	Branch	Assert
alkaid_soc_top	86.72%	96.69%	78.73%	61.81%	98.44%	84.65%	100.00%
u_axi2apb	85.80%	100.00%	81.25%	76.60%	75.00%	96.15%	
u_axi_interconnect	86.28%	100.00%	86.87%	73.41%		84.83%	
u_clint	82.08%	95.52%	86.23%	64.09%	73.91%	90.65%	
u_core	86.79%	94.12%	78.74%	60.94%	99.69%	87.23%	100.00%
u_dmem	88.61%	100.00%	71.23%	86.14%		97.06%	
u_imem	86.28%	100.00%	64.38%	83.69%		97.06%	
u_perip_top	38.31%		50.00%	14.92%		50.00%	
u_plic	75.50%	100.00%	73.11%	41.47%	80.00%	82.94%	

图 26-2 P3 raw 覆盖率层次统计

Name	Score	Line	Condition	Toggle	FSM	Branch	Assert
alkaid_soc_top	87.56%	100.00%	78.80%	61.81%	98.44%	86.32%	100.00%
u_axi2apb	85.80%	100.00%	81.25%	76.60%	75.00%	96.15%	
u_axi_interconnect	86.28%	100.00%	86.87%	73.41%		84.83%	
u_clint	84.00%	100.00%	87.80%	64.09%	73.91%	94.17%	
u_core	88.24%	100.00%	78.78%	60.94%	99.69%	90.00%	100.00%
u_dmem	88.61%	100.00%	71.23%	86.14%		97.06%	
u_imem	86.28%	100.00%	64.38%	83.69%		97.06%	
u_perip_top	38.31%		50.00%	14.92%		50.00%	
u_plic	75.50%	100.00%	73.11%	41.47%	80.00%	82.94%	

图 26-3 P3 after-waive 覆盖率层次统计

me	Score	Line	Condition	Toggle	FSM	Branch	Assert
alkaid_soc_top	87.53%	97.33%	79.15%	65.40%	97.98%	85.35%	100.00%
u_axi2apb	85.80%	100.00%	81.25%	76.60%	75.00%	96.15%	
u_axi_interconnect	87.40%	100.00%	86.55%	78.24%		84.83%	
u_clint	83.38%	95.96%	86.96%	66.61%	73.91%	93.46%	
u_core	87.87%	95.76%	79.33%	65.02%	99.19%	87.94%	100.00%
u_dmem	88.61%	100.00%	71.23%	86.14%		97.06%	
u_imem	85.26%	100.00%	60.27%	83.69%		97.06%	
u_perip_top	38.31%		50.00%	14.92%		50.00%	
u_plic	75.50%	100.00%	73.11%	41.47%	80.00%	82.94%	

图 26-4 P4 raw 覆盖率层次统计

Name	Score	Line	Condition	Toggle	FSM	Branch	Assert
alkaid_soc_top	88.24%	100.00%	79.21%	65.40%	97.98%	86.86%	100.00%
u_axi2apb	85.80%	100.00%	81.25%	76.60%	75.00%	96.15%	
u_axi_interconnect	87.40%	100.00%	86.55%	78.24%		84.83%	
u_clint	85.02%	100.00%	88.40%	66.61%	73.91%	96.15%	
u_core	88.99%	100.00%	79.36%	65.02%	99.19%	90.39%	100.00%
u_dmem	88.61%	100.00%	71.23%	86.14%		97.06%	
u_imem	85.26%	100.00%	60.27%	83.69%		97.06%	
u_perip_top	38.31%		50.00%	14.92%		50.00%	
u_plic	75.50%	100.00%	73.11%	41.47%	80.00%	82.94%	

图 26-5 P4 after-waive 覆盖率层次统计

Hierarchy												Modules	Groups	Asserts	Statistics	Tests			
*																			
Name												Score	Line	Condition	Toggle	FSM	Branch	Assert	
alkaid_soc_top												88.62%	98.24%	81.68%	66.99%	97.98%	86.86%	100.00%	
u_axi2apb												85.80%	100.00%	81.25%	76.60%	75.00%	96.15%		
u_axi_interconnect												87.40%	100.00%	86.55%	78.24%		84.83%		
u_clint												83.38%	95.96%	86.96%	66.61%	73.91%	93.46%		
u_core												89.45%	97.51%	82.56%	67.12%	99.19%	90.29%	100.00%	
u_dmem												88.61%	100.00%	71.23%	86.14%		97.06%		
u_imem												85.26%	100.00%	60.27%	83.69%		97.06%		
u_perip_top												38.31%		50.00%	14.92%		50.00%		
u_plic												75.50%	100.00%	73.11%	41.47%	80.00%	82.94%		

图 26-6 P5 raw 覆盖率层次统计

Hierarchy											Modules	Groups	Asserts	Statistics	Tests					
*																				
Name											Score	Line	Condition	Toggle	FSM	Branch	Assert			
alkaid_soc_top											89.13%	100.00%	81.72%	66.99%	97.98%	88.08%	100.00%			
u_axi2apb											85.80%	100.00%	81.25%	76.60%	75.00%	96.15%				
u_axi_interconnect											87.40%	100.00%	86.55%	78.24%		84.83%				
u_clint											85.02%	100.00%	88.40%	66.61%	73.91%	96.15%				
u_core											90.18%	100.00%	82.56%	67.12%	99.19%	92.20%	100.00%			
u_dmem											88.61%	100.00%	71.23%	86.14%		97.06%				
u_imem											85.26%	100.00%	60.27%	83.69%		97.06%				
u_perip_top											38.31%		50.00%	14.92%		50.00%				
u_plic											75.50%	100.00%	73.11%	41.47%	80.00%	82.94%				

图 26-7 P5 after-waive 覆盖率层次统计

8.3 覆盖率结果说明

完整回归任务、三个流水线配置的系统级合并任务和跨流水线合并任务均已通过。表 137-1 给出当前 P3、P4、P5 的正式代码覆盖率结果；其中 after-waive 的 line 和 Module-definition line 均达到 100%，SCORE、condition、branch、toggle、FSM 和 assertion 同时列出，便于区分整体得分与各项代码指

标。功能组统计应与相同任务批次的覆盖率数据库同时核对，本报告不以代码覆盖率数值替代功能组结果。

9 性能分析

9.1 配置定义与统计条件

性能分析使用表 123 定义的 P3、P4 和 P5。默认配置关闭 Zba、Zbb、Zbs、Zbc、非自然对齐访问、LSU L0 Cache、CSR 读预译码、增强 BPU 和 TAGE；性能配置打开这些功能。CoreMark 统一在 100 MHz 仿真时钟下运行，工作集为 CoreMark 2K performance run，ITERATIONS=1。Iterations/MHz 等于 CoreMark 的 Iterations/Sec 除以 100。FPGA 器件、Vivado 版本、实现约束和上板结果统一放在本报告末尾的第 10 章。

RV32 使用 -march=rv32im_zicsr、-mabi=ilp32 与 -mtune=alkaid；RV64 使用 -march=rv64im_zicsr、-mabi=lp64、-fno-web 与 -ftree-dse。RV32 与 RV64 使用各自的 ABI 和编译选项，因此两者数值差异反映两套完整构建配置，不单独归因于数据宽度。

9.2 RV32 统一时钟条件下的 CoreMark 对比

CoreMark 采用 100 MHz 仿真时钟、2K performance run 和 ITERATIONS=1。动态执行指令数（INSTS）和每周期执行的动态指令数（IPC）由运行时性能计数器给出；NA 表示该配置未产生对应统计值，因此不以缺失指标推定性能错误。

配置	统计时钟	总节拍数	总时间(s)	动态执行指令数(INSTS)	IPC	Iterations/Sec	Iterations/MHz	分支预测准确率	CRC
P3 默认	100 MHz	287139	0.002869	293605	0.8362	348.461195	3.48461195	92.3977%	e714/1fd7/8e3a/e714
P3 性能	100 MHz	231023	0.002309	228436	0.9100	433.065410	4.33065410	92.5179%	e714/1fd7/8e3a/e714
P4 默认	100 MHz	297080	0.002969	293610	0.8114	336.745689	3.36745689	92.3977%	e714/1fd7/8e3a/e714
P4 性能	100 MHz	239743	0.002396	228432	0.8767	417.334401	4.17334401	92.5200%	e714/1fd7/8e3a/e714

配置	统计时钟	总节拍数	总时间 (s)	动态执行指令数 (INSTS)	IPC	Iterations/Sec	Iterations/MHz	分支预测准确率	CRC
P5 默认	100 MHz	300948	0.003008	293377	0.8004	332.446808	3.32446808	92.3028%	e714/1fd7/8e3a/e714
P5 性能	100 MHz	243451	0.002432	228432	0.8629	411.184210	4.11184210	92.5284%	e714/1fd7/8e3a/e714

表 138 RV32 统一 100 MHz CoreMark 对比

性能配置相对同级默认配置的 Iterations/MHz 分别提高 24.28%、23.93% 和 23.68%。新汇总同时给出了三组性能配置的动态执行指令数和 IPC：P3、P4、P5 性能配置的 INSTS 分别比同级默认配置减少约 22.2%、22.2% 和 22.1%，IPC 分别提高约 8.8%、8.0% 和 7.8%。指令数减少与性能配置启用 Zba、Zbb、Zbs、Zbc 后采用对应指令集选项有关，IPC 提升则反映分支预测、LSU L0 Cache 和流水线控制共同减少了等待周期。统一 100 MHz 条件下，P3 性能配置的 Iterations/MHz 最高；按各自 FPGA 目标频率线性估算时，P5 性能配置具有最高的每秒迭代数。P3、P4、P5 性能配置每千 LUT 的 Iterations/MHz 分别为 0.753550、0.614903 和 0.508640，表明性能配置以较多逻辑资源换取更短的 CoreMark 执行时间。

9.3 RV64 CoreMark 与 RV32 对比

配置	运行状态	总节拍数	总时间 (s)	动态执行指令数 (INSTS)	IPC	Iterations/Sec	Iterations/MHz	分支预测准确率	CRC	相对同配置 RV32
P3 默认	通过	367149	0.003671	365820	0.8319	272.402370	2.72402370	92.7453%	e714/1fd7/8e3a/e714	-21.83%
P3 性能	通过	271388	0.002713	261111	0.8909	368.514150	3.68514150	91.8768%	e714/1fd7/8e3a/e714	-14.91%

配置	运行状态	总节拍数	总时间 (s)	动态执行指令数 (INST S)	IPC	Iterations/Sec	Iterations/MHz	分支预测准确率	CRC	相对同配置 RV32
P4 默认	通过	371373	0.003711	366012	0.8229	269.396551	2.69396551	92.7492%	e714/1fd7/8e3a/e714	-20.00%
P4 性能	通过	278828	0.002787	261111	0.8661	358.700642	3.58700642	91.8768%	e714/1fd7/8e3a/e714	-14.05%
P5 默认	通过	374888	0.003747	365920	0.8140	266.820355	2.66820355	92.7570%	e714/1fd7/8e3a/e714	-19.74%
P5 性能	通过	283265	0.002831	261111	0.8523	353.187160	3.53187160	91.8729%	e714/1fd7/8e3a/e714	-14.10%

表 139 RV64 统一 100 MHz CoreMark 结果及 RV32 对比

RV64 默认配置的动态执行指令数为 365820、366012 和 365920，分别比对应 RV32 默认配置高 24.60%、24.66% 和 24.73%；三组 IPC 分别为 0.8319、0.8229 和 0.8140，与对应 RV32 配置的 0.8362、0.8114 和 0.8004 接近。因而，RV64 默认配置的 CoreMark 执行时间增加，主要原因是同一工作集在 LP64 ABI 下生成并动态执行了更多指令，而不是 IPC 明显下降或仅由 XLEN 增加造成。RV64 的 P4 和 P5 性能配置相对同级默认配置分别提高 33.15% 和 32.37%，但其迭代率仍比对应 RV32 构建结果低约 14%。CoreMark 汇总报告中的 P3 性能配置仅包含总节拍数和分支预测准确率，其余字段为 NA，因此不用于 RV64 性能配置间比较。

9.4 Cache 与乒乓缓存性能对比

Cache 与 AXI4-Lite 乒乓缓存的结构、参数及接口定义见第 2.5 节和第 5.12 节；专项命中率、资源和 CoreMark 组合数据不列入本节性能统计表。

9.5 Cache 资源、命中率与时序分析

Cache 的资源、命中率和时序统计应在统一参数、统一程序镜像和统一 FPGA 实现条件下采集，本节不以局部运行结果替代完整统计。

9.6 RTL structure 与时序优化辅助分析

rtl-structure（RTL 结构分析）是用于辅助时序优化的源级结构检查工具。它在 RTL 修改后、综合实现前，按照指定顶层模块、参数、宏定义、include 路径和 file list 展开设计，建立可排序的组合逻辑结构视图，帮助定位逻辑深度较大、控制条件过多、扇出较高或跨模块组合逻辑未被寄存器截断的路径。该工具用于缩小时序优化的检查范围和比较 RTL 修改前后的结构变化，不替代 FPGA 综合、布局布线和静态时序分析（STA）。

工具的输入必须与待比较的实现结果来自同一 RTL 快照和同一配置。首先由 Verilator 生成保留模块边界、always 块和信号连接信息的 elaboration tree JSON，同时保留源文件表，以便回到 RTL 声明处检查存储器属性。结构分析器随后从赋值右端表达式、if/case 控制条件、模块端口连接和过程敏感依赖中建立有向图，并输出结构 JSON、最差结构路径列表和人可读的路径报告。完成同一配置的 Vivado 综合后，还可以把去重的时序路径报告作为独立输入，生成结构路径与 Vivado 端点族的对照结果。

结构图中的节点表示信号、寄存器 D 输入、寄存器使能或复位控制端以及模块边界端口；有向边表示信号在赋值、条件控制或模块连接中的依赖关系。寄存器 D 和寄存器控制端被作为真正的时序端点，普通模块输出和子模块输入只保留在结构统计中，不与寄存器端点混合排序。分析器同时检查组合环和自反馈；发现组合环时，路径深度不作为正常优化指标使用，而是先作为结构问题报告。

逻辑深度采用保守且可审查的源级估算。算术、比较、动态数组选择和条件选择按照运算复杂度计入深度，固定的位选、拼接和直接连线不额外增加逻辑级数；长的 AND、OR、XOR 归约按平衡树估算，避免把表达式的书写顺序直接当作 FPGA 实现级数。分析器同时保留路径上的运算符深度、控制条件深度、信号序列、源代码位置和扇出信息，用于比较不同 RTL 方案的结构变化，而不是把估算值当作器件延迟。

同步存储宏需要单独处理。只有在动态数组选择位于时序赋值右端、源 RTL 紧邻声明带有 ram_style 为 block 或 ultra 的属性、并且该属性能够通过

源文件表核实时，才将该读操作作为同步存储宏边界。此时宏读对 **logical depth** 的贡献为零，但报告继续保存原始数组选择深度、存储宏类型、实例层次、条目数、位宽、地址位宽、读取寄存器和声明位置。异步 RAM、LUTRAM、没有属性的寄存器数组以及组合读仍按普通动态选择计算，不能仅凭模块名或数组名跳过深度统计。

报告同时给出两种深度：**logical depth** 表示确认同步存储宏边界后的结构深度，**raw depth** 保留数组选择等原始组合成本。顶层层次展开后的寄存器端点最大深度用于观察跨模块结构，模块局部最大深度用于定位具体 RTL；两者与 **raw depth** 的差异可以说明优化来自存储宏边界、模块寄存器边界还是普通组合逻辑变化。当前 P5 基线的层次端点最大深度为 10，模块 **logical depth** 为 9，**raw depth** 为 15，组合环数为零；这些数值用于结构比较，不等价于 Vivado 的 **logic levels** 或布线延迟。

路径报告按照深度、端点类型、运算符深度、起点层次、终点层次、信号序列和源代码位置排序。总线位号和 **generate** 展开编号只在结构一致时归并，普通实例层次保持不变，避免把不同模块的同名信号合并。去重后保留每个结构路径的排名、**logical/raw depth**、运算符深度、完整起点和终点层次、信号序列、位置及重复数量，既便于人工检查，也能作为后续对照的机器输入。

Vivado 对照分为端点族对照和完整起点至终点对照。端点族对照只确认结构报告和 Vivado 报告是否指向同一组目的寄存器，适合检查结构分析是否覆盖了重点寄存器；完整对照还要求起点和终点的层次后缀一致，才可用于比较路径排名。两者不能混用：同一目的寄存器并不说明组合路径相同，端点族对照也不能推导 WNS、TNS 或布线延迟。对照输入必须使用相同 RTL 快照、参数、顶层模块、时钟约束和综合配置；无法满足时，结果只能作为未校准的结构观察。

该工具在时序优化中的使用顺序是：先用结构报告筛选最大 **logical/raw depth**、控制条件深度、扇出和跨模块路径，再回到 RTL 判断是否需要增加流水线寄存器、拆分条件选择、减少宽选择器、调整模块边界或改变存储器读写组织，最后使用同一配置重新综合和实现，以 Vivado 的 WNS、TNS、WHS、THS 和实际路径报告确认改变是否有效。结构分析提供早期方向和回归告警，最终时序结论仍以实现阶段的静态时序分析为准。

分析阶段	处理内容	主要输出	使用目的
输入准备	固定顶层、参数、宏定义、 include 路径、 file list 和 RTL 快照	展开文件清单、源文件表	保证不同候选使用相同设计边界

分析阶段	处理内容	主要输出	使用目的
结构展开	保留模块、always 块、信号连接和源位置的 elaboration tree	tree JSON、展开日志	为结构图建立提供层次信息
结构分析	建立依赖图，计算 logical/raw depth、控制条件深度、扇出并检查组合环	structure JSON	定位可能影响时序的 RTL 结构
路径整理	按端点类型排序并按结构归并总线位族	timing paths Markdown、端点目录	形成稳定、可复查的候选路径列表
Vivado 对照	将同一配置的去重时序报告与结构端点族和完整路径比较	correlation JSON、对照报告	判断结构报告的观察范围，不替代 STA

9.7 性能分析结论

P3、P4 和 P5 分别体现单位频率执行效率、100 MHz 平衡点和更高目标频率的取舍。RV32 的三种性能配置以及 RV64 的 P4、P5 性能配置在统一 100 MHz CoreMark 中均减少总节拍数，但 CPU 核 LUT 明显增加。P5 适合以较高 FPGA 目标频率优先的配置，P3 适合资源受限且重视单位频率执行效率的配置，P4 位于两者之间。性能数据不替代功能验证和覆盖率统计；P3 RV64 性能配置需要形成完整统计后再纳入性能比较。

10 FPGA 实现与上板结果

10.1 实现条件与六种配置

FPGA 实现统一使用 Xilinx Artix-7 xc7a200tfbg484-3、Vivado 2024.2.1、RT-Thread Nano 以及初始化的 ITCM 和 DTCM。P3、P4、P5 的目标时钟分别为 50 MHz、100 MHz 和 150 MHz，每个流水线级数各设置默认配置和性能配置，共六种实现结果。性能配置打开 Zba、Zbb、Zbs、Zbc、非自然对齐访问、LSU L0 Cache、CSR 读预译码、增强 BPU 和 TAGE；ICache 与 DCache 在这六种矩阵配置中均关闭。资源统计以 u_core 层次为主，时序同时报告综合和实现结果。

最差建立时序裕量（WNS）、总建立时序裕量（TNS）、最差保持时序裕量（WHS）和总保持时序裕量（THS）均以 ns 为单位。下表给出六档完整

wrapper 综合和 post-route 实现结果；CPU 核 LUT 给出综合到实现的变化，FF 和 DSP 为实现结果。

配置	目标频率	综合 WNS / TNS / WHS / THS (ns)	实现 WNS / TNS / WHS / THS (ns)	CPU 核 LUT (综合 →实现)	CPU 核 FF	CPU 核 DSP	实现状态
P3 默认	50 MHz	+3.268 / 0 / +0.029 / 0	+0.353 / 0 / +0.075 / 0	3336 → 3159	1677	4	实现满足 目标
P3 性能	50 MHz	+2.248 / 0 / +0.035 / 0	+1.080 / 0 / +0.060 / 0	5927 → 5735	2576	4	实现满足 目标
P4 默认	100 MHz	-0.178 / - 0.487 / +0.029 / 0	+0.091 / 0 / +0.074 / 0	3686 → 3495	2372	2	实现满足 目标
P4 性能	100 MHz	-0.272 / - 0.272 / +0.029 / 0	+0.239 / 0 / +0.069 / 0	6936 → 6713	3358	5	实现满足 目标
P5 默认	150 MHz	-0.444 / - 2.151 / +0.029 / 0	+0.040 / 0 / +0.075 / 0	4217 → 4270	2840	2	实现满足 目标
P5 性能	150 MHz	-0.316 / - 24.624 / +0.029 / 0	+0.007 / 0 / +0.080 / 0	8019 → 8084	3844	5	实现满足 目标

表 140 六种 FPGA 实现结果

10.2 资源占用与 CoreMark 上板截图占位

下表为六种配置各自的两类插图位置。资源占用截图应展示 Vivado 完整 wrapper 的资源汇总和 u_core 层次统计；CoreMark 上板截图应展示对应位流下载后的串口输出、迭代次数、运行时间、Iterations/Sec、IPC 和 CRC。插图补入后，截图中的配置参数必须与表 140 的目标频率、ISA 开关和性能配置保持一致。

Name	Slice LUTs (134600)	Slice Registers (269200)	F7 Muxes (67300)	F8 Muxes (33650)	Slice Slices (33650)	LUT as Logic (134600)	LUT as Memory (46200)	Block RAM Block (365)	DSPs (740)	Bonded I/O (285)	BUFGCTRL (32)	MMCM2_ADV (10)
✖ N_alkaid_soc_t1_wrapper	5408	4580	154	14	2444	5113	295	48	4	36	2	1
✖ [0]_alkaid_soc_t1_0 (alkaid_soc_t1_0)	5408	4580	154	14	2444	5113	295	48	4	0	2	1
✖ [0]_alkaid_soc_0 (alkaid_soc_t1_alkaid_soc_0_0)	5393	4547	154	14	2431	5099	294	48	4	0	0	0
✖ [0]_inst (alkaid_soc_t1_alkaid_soc_0_0_alkaid_soc_0_0_0)	5393	4547	154	14	2431	5099	294	48	4	0	0	0
✖ [0]_u_axi2apb (alkaid_soc_t1_alkaid_soc_0_0_0_axi2apb)	89	104	0	0	61	89	0	0	0	0	0	0
> [0]_u_axi_interconnect (alkaid_soc_t1_alkaid_soc_0_0_0_axi_interconnect)	230	219	0	0	153	230	0	0	0	0	0	0
> [0]_u_clint (alkaid_soc_t1_alkaid_soc_0_0_clint)	196	409	0	0	176	196	0	0	0	0	0	0
> [0]_u_core (alkaid_soc_t1_alkaid_soc_0_0_core)	3159	1677	4	0	1091	2937	222	0	4	0	0	0
> [0]_u_dmem (alkaid_soc_t1_alkaid_soc_0_0_dmem)	120	29	0	0	42	86	34	16	0	0	0	0
> [0]_u_imem (alkaid_soc_t1_alkaid_soc_0_0_imem)	164	29	0	0	72	130	34	32	0	0	0	0
> [0]_u_perip_top (alkaid_soc_t1_alkaid_soc_0_0_perip_top)	413	924	82	14	403	413	0	0	0	0	0	0
> [0]_u_plic (alkaid_soc_t1_alkaid_soc_0_0_plic_top)	1031	1156	68	0	656	1027	4	0	0	0	0	0
> [0]_clk_wiz_0 (alkaid_soc_t1_clk_wiz_0_0)	0	0	0	0	0	0	0	0	0	0	2	1
> [0]_proc_sys_reset_0 (alkaid_soc_t1_proc_sys_reset_0)	15	33	0	0	13	14	1	0	0	0	0	0

图 27 P3 默认 Vivado 资源汇总

```

[INFO] Benchmark finished. Processing results...
2K performance run parameters for coremark.
CoreMark Size : 666
Total ticks : 574278000
Total time (secs) : 5.738000
Iterations/Sec : 348.461195
ERROR! Must execute for at least 10 secs for a valid result!
Iterations : 2060
Compiler version : GCC16.1.0
Compiler flags : -O3 -mtune=alcaid -ffunction-sections -fdata-sections -fno-common -funroll-loops -finline-functions --param max-inline-insns-auto=20 -falign-functions=4 -falign-
Memory location : STATIC
seedcrc : 0xe9f5
[INFO] Outputting CRC and results. Please check below...
[0]crlct : 0xe714
[0]crrcmatrix : 0x1d7
[0]crrcstate : 0x8e3a
[0]crrcfinal : 0xe714
Errors detected
[INFO] Attached. You may now check all results above

```

图 28 P3 默认 CoreMark 上板输出

	Name	Slice LUTs (134600)	Slice Registers (269200)	F7 Muxes (67300)	F8 Muxes (33650)	Slice (33650)	LUT as Logic (134600)	LUT as Memory (46200)	Block RAM Tile (365)	DSPs (740)	Bonded IOB (285)	BUFGCTRL (32)	MMCME2_ADV (10)
✓	naikaid_soc_t1_wrapper	8018	5471	230	18	3392	7279	739	48	4	36	2	1
✓	naikaid_soc_t1_j (aikaid_soc_t1)	8018	5471	230	18	3392	7279	739	48	4	0	2	1
✓	naikaid_soc_0 (aikaid_soc_t1_a)	8003	5438	230	18	3381	7265	738	48	4	0	0	0
✓	inst (aikaid_soc_t1_aikaid_2)	8003	5438	230	18	3381	7265	738	48	4	0	0	0
	u_axi2apb (aikaid_soc_t1_a)	80	104	0	0	83	80	0	0	0	0	0	0
>	u_axi_interconnect (aikaid_soc_t1_a)	242	214	32	0	160	242	0	0	0	0	0	0
>	u_clint (aikaid_soc_t1_a)	254	407	0	0	196	254	0	0	0	0	0	0
>	u_core (aikaid_soc_t1_a)	5735	2576	93	12	2035	5069	666	0	4	0	0	0
>	u_dmem (aikaid_soc_t1_a)	117	29	0	0	55	83	34	16	0	0	0	0
>	u_imem (aikaid_soc_t1_a)	114	29	0	0	44	80	34	32	0	0	0	0
>	u_perip_top (aikaid_soc_t1_a)	439	924	46	6	431	439	0	0	0	0	0	0
>	u_plic (aikaid_soc_t1_a)	1029	1155	59	0	683	1025	4	0	0	0	0	0
>	clk_wiz_0 (aikaid_soc_t1_clk_wiz_0)	0	0	0	0	0	0	0	0	0	0	2	1
>	proc_sys_reset_0 (aikaid_soc_t1_proc_sys_reset_0)	15	33	0	0	11	14	1	0	0	0	0	0

图 29 P3 性能 Vivado 资源汇总

```
[INFO] Benchmark finished. Processing results...
2K performance run parameters for coremark.
CoreMark Size      : 666
Total ticks        : 462046000
Total time (secs)  : 4.618000
Iterations/Sec     : 433.065410
ERROR! Must execute for at least 10 secs for a valid result!
Iterations         : 2000
Compiler version   : GCC16.1.0
Compiler flags     : -O3 -mtune=alkaid -ffunction-sections -fdata-sections -fno-common -funroll-loops -finline-functions --param max-inline-insns-auto=2
Memory location    : STATIC
seedcrc           : 0xe9f5
[INFO] Outputting CRC and results. Please check below...
[0]crc1list       : 0xe714
[0]crcmatrix      : 0x1d7
[0]crcstate       : 0x8e3a
[0]crcfinal       : 0xe714
Errors detected
[INFO] Program finished. You may now check all results above.
```

图 30 P3 性能 CoreMark 上板输出

	Name	Slice LUTs (134600)	Slice Registers (269200)	F7 Muxes (67300)	F8 Muxes (33650)	Slice (33650)	LUT as Logic (134600)	LUT as Memory (46200)	Block RAM Tile (365)	DSPs (740)	Bonded JOB (285)	BUGCTRL (32)	MMCME2_ADV (10)
✓	alkaid_soc_t1_wrapper	5764	5276	186	14	2732	5367	397	48	2	36	2	1
✓	alkaid_soc_t1_j (alkaid_soc_t1)	5764	5276	186	14	2732	5367	397	48	2	0	2	1
✓	alkaid_soc_0 (alkaid_soc_t1_a)	5749	5243	186	14	2723	5353	396	48	2	0	0	0
✓	inst (alkaid_soc_t1_alkaid_s)	5745	5243	186	14	2722	5353	392	48	2	0	0	0
	u_axi2apb (alkaid_soc_t1_a)	89	104	0	0	61	89	0	0	0	0	0	0
>	u_axi_interconnect (alkaid_soc_t1_a)	227	219	0	0	145	227	0	0	0	0	0	0
>	u_clint (alkaid_soc_t1_alkaid_s)	216	410	0	0	169	216	0	0	0	0	0	0
>	u_core (alkaid_soc_t1_alkaid_s)	3495	2372	36	0	1309	3175	320	0	2	0	0	0
>	u_dmem (alkaid_soc_t1_alkaid_s)	121	29	0	0	53	87	34	16	0	0	0	0
>	u_imem (alkaid_soc_t1_alkaid_s)	163	29	0	0	76	129	34	32	0	0	0	0
>	u_perip_top (alkaid_soc_t1_alkaid_s)	413	924	82	14	425	413	0	0	0	0	0	0
>	u_plic (alkaid_soc_t1_alkaid_s)	1030	1156	68	0	733	1026	4	0	0	0	0	0
>	clk_wiz_0 (alkaid_soc_t1_clk_wiz_0)	0	0	0	0	0	0	0	0	0	0	2	1
>	proc_sys_reset_0 (alkaid_soc_t1_proc_sys_reset_0)	15	33	0	0	9	14	1	0	0	0	0	0

图 31 P4 默认 Vivado 资源汇总

```
[INFO] Benchmark finished. Processing results...
2x performance run parameters for coremark.
CoresPerSize : 666
Total ticks : 594150000
Total time (secs): 5.930000
Iterations/Sec : 336.745689
ERROR! Must execute for at least 10 secs for a valid result!
Iterations : 2000
Compiler version : GCC16.1.0
Compiler options: -O3 -fcommon -fdata-sections -ffunction-sections -fno-common -funroll-loops -finline-functions -fparam-max-inline-insns-auto=20 -falign-functions=4 -falign-jumps=4 -falign-loops=4 -fno-strict-aliasing
Memory location : $MTIA71
Memory location : 0xe09f5
swedrc : 0xe09f5
[INFO] outputting OK and results. Please check below...
0)JrcList : 0xe714
0)JrcMatrix : 0x1f07
0)JrcState : 0x0a3a
0)JrcFinal : 0xe714
Errors detected
[INFO] Program finished. You may now check all results above.
```

图 32 P4 默认 CoreMark 上板输出

Name	Slice LUTs (134600)	Slice Registers (269200)	F7 Muxes (67300)	F8 Muxes (33650)	Slice (33650)	LUT as Logic (134600)	LUT as Memory (46200)	Block RAM Tile (365)	DSPs (740)	Bonded IOB (285)	BUFGCTRL (32)	MMCME2_ADV (10)
✖ N alkaid_soc_t1_wrapper	9222	6259	354	66	3761	8267	955	48	5	36	2	1
✖ I alkaid_soc_t1_j(alkaid_soc_t1)	9222	6259	354	66	3761	8267	955	48	5	0	2	1
✖ I alkaid_soc_0(alkaid_soc_t1_a)	9207	6226	354	66	3750	8253	954	48	5	0	0	0
✖ I inst(alkaid_soc_t1_alkaid_j)	9015	6226	338	66	3702	8253	762	48	5	0	0	0
I u_axi2apb(alkaid_soc_t1_axi2apb)	89	104	0	0	76	89	0	0	0	0	0	0
> I u_axi_interconnect(alkaid_soc_t1_axi2apb_axi_interconnect)	231	219	0	0	153	231	0	0	0	0	0	0
> I u_clint(alkaid_soc_t1_axi2apb_axi_interconnect_clint)	283	409	0	0	199	283	0	0	0	0	0	0
> I u_core(alkaid_soc_t1_axi2apb_axi_interconnect_axi2apb_axi2apb_core)	6713	3358	197	52	2395	6023	690	0	5	0	0	0
> I u_dmem(alkaid_soc_t1_axi2apb_axi_interconnect_axi2apb_axi2apb_core_dmem)	121	29	0	0	48	87	34	16	0	0	0	0
> I u_imem(alkaid_soc_t1_axi2apb_axi_interconnect_axi2apb_axi2apb_core_dmem_imem)	137	29	0	0	59	103	34	32	0	0	0	0
> I u_perip_top(alkaid_soc_t1_axi2apb_axi_interconnect_axi2apb_axi2apb_core_dmem_imem_perip_top)	413	924	82	14	439	413	0	0	0	0	0	0
> I u_plc(alkaid_soc_t1_axi2apb_axi_interconnect_axi2apb_axi2apb_core_dmem_imem_perip_top_plc)	1032	1154	59	0	620	1028	4	0	0	0	0	0
> I clk_wiz_0(alkaid_soc_t1_clk_wiz_0)	0	0	0	0	0	0	0	0	0	0	2	1
> I proc_sys_reset_0(alkaid_soc_t1_clk_wiz_0_proc_sys_reset_0)	15	33	0	0	11	14	1	0	0	0	0	0

图 33 P4 性能 Vivado 资源汇总

```
[INFO] Benchmark finished. Processing results...
2K performance run parameters for coremark.
CoreMark Size      : 666
Total ticks        : 479486000
Total time (secs)  : 4.792000
Iterations/Sec     : 417.334401
ERROR! Must execute for at least 10 secs for a valid result!
Iterations         : 2000
Compiler version   : GCC16.1.0
Compiler flags     : -O3 -mtune=alkaid -ffunction-sections -fdata-sections -fno-common -funroll-loops -finline-functions --param max-inline-insns-auto=20 -falign-functions=16
Memory location    : STATIC
seedccr           : 0xe9f5
seedcrr           : 0xe9f5
[INFO] Outputting CRC and results. Please check below...
[0]crrclist       : 0xe714
[0]crrcmatrix     : 0x1d7
[0]crrcstate      : 0x8e3a
[0]crrcfinal      : 0xe714
```

图 34 P4 性能 CoreMark 上板输出

Name	Slice LUTs (134600)	Slice Registers (269200)	F7 Muxes (67300)	F8 Muxes (33650)	Slice (33650)	LUT as Logic (134600)	LUT as Memory (46200)	Block RAM Tile (365)	DSPs (740)	Bonded IOB (285)	BUFGCTRL (32)	MMCM2_ADV (10)
alkaid_soc_t1_wrapper	6570	5744	188	14	2899	6173	397	48	2	36	2	1
alkaid_soc_t1_j (alkaid_soc_t1)	6570	5744	188	14	2899	6173	397	48	2	0	2	1
alkaid_soc_0 (alkaid_soc_t1_0)	6555	5711	188	14	2888	6159	396	48	2	0	0	0
inst (alkaid_soc_t1_0_alkaid_soc_top)	6551	5711	188	14	2887	6159	392	48	2	0	0	0
u_axi2apb (alkaid_soc_t1_0_axi2apb)	92	104	0	0	78	92	0	0	0	0	0	0
u_axi_interconnect (alkaid_soc_t1_0_axi_interconnect)	240	219	0	0	171	240	0	0	0	0	0	0
u_clint (alkaid_soc_t1_0_clint)	197	410	0	0	173	197	0	0	0	0	0	0
u_core (alkaid_soc_t1_0_core)	4270	2840	38	0	1496	3950	320	0	2	0	0	0
u_dmem (alkaid_soc_t1_0_dmem)	122	29	0	0	57	88	34	16	0	0	0	0
u_jmem (alkaid_soc_t1_0_jmem)	181	29	0	0	100	147	34	32	0	0	0	0
u_perip_top (alkaid_soc_t1_0_perip_top)	414	924	82	14	399	414	0	0	0	0	0	0
u_plic (alkaid_soc_t1_0_plic)	1041	1156	68	0	770	1037	4	0	0	0	0	0
clk_wiz_0 (alkaid_soc_t1_clk_wiz_0)	0	0	0	0	0	0	0	0	0	0	2	1
proc_sys_reset_0 (alkaid_soc_t1_proc_sys_reset_0)	15	33	0	0	11	14	1	0	0	0	0	0

图 35 P5 默认 Vivado 资源汇总

```

[INFO] Benchmark finished. Processing results...
2K performance run parameters for coremark.
CoreMark Size : 666
Total ticks : 601896000
Total time (secs): 6.016000
Iterations/Sec : 332.446808
ERROR! Must execute for at least 10 secs for a valid result!
Iterations : 2000
Compiler version : GCC16.1.0
Compiler flags : -O3 -mtune=alkaid -ffunction-sections -fdata-sections -fno-common -funroll-loops -finline-functions --param max-
Memory location : STATIC
seedcrc : 0xe9f5
[INFO] Outputting CRC and results. Please check below...
[0]crclist : 0xe714
[0]crcmatrix : 0x1fd7
[0]crcstate : 0x8e3a
[0]crcfinal : 0xe714
Errors detected
[INFO] Program finished. You may now check all results above.

```

图 36 P5 默认 CoreMark 上板输出

Name	Slice LUTs (134600)	Slice Registers (269200)	F7 Muxes (67300)	F8 Muxes (33650)	Slice (33650)	LUT as Logic (134600)	LUT as Memory (46200)	Block RAM Tile (365)	DSPs (740)	Bonded IOB (285)	BUFGCTRL (32)	MMCM2_ADV (10)
alkaid_soc_t1_wrapper	10555	6745	385	78	3891	9540	1015	48	5	36	2	1
alkaid_soc_t1_j (alkaid_soc_t1)	10555	6745	385	78	3891	9540	1015	48	5	0	2	1
alkaid_soc_0 (alkaid_soc_t1_0_alkaid_soc_0)	10540	6712	385	78	3881	9526	1014	48	5	0	0	0
inst (alkaid_soc_t1_0_alkaid_soc_0_alkaid_soc_top)	10348	6712	369	78	3834	9526	822	48	5	0	0	0
u_axi2apb (alkaid_soc_t1_0_alkaid_soc_0_axi2apb)	93	104	0	0	77	93	0	0	0	0	0	0
u_axi_interconnect (alkaid_soc_t1_0_alkaid_soc_0_axi_interconnect)	234	219	0	0	174	234	0	0	0	0	0	0
u_clint (alkaid_soc_t1_0_alkaid_soc_0_clint)	202	408	0	0	181	202	0	0	0	0	0	0
u_core (alkaid_soc_t1_0_alkaid_soc_0_core_top)	8084	3844	228	64	2532	7334	750	0	5	0	0	0
u_dmem (alkaid_soc_t1_0_alkaid_soc_0_dmem)	122	29	0	0	62	88	34	16	0	0	0	0
u_jmem (alkaid_soc_t1_0_alkaid_soc_0_jmem)	156	29	0	0	85	122	34	32	0	0	0	0
u_perip_top (alkaid_soc_t1_0_alkaid_soc_0_perip_top)	414	924	82	14	388	414	0	0	0	0	0	0
u_plic (alkaid_soc_t1_0_alkaid_soc_0_plic_top)	1042	1155	59	0	686	1038	4	0	0	0	0	0
clk_wiz_0 (alkaid_soc_t1_clk_wiz_0)	0	0	0	0	0	0	0	0	0	0	2	1
proc_sys_reset_0 (alkaid_soc_t1_proc_sys_reset_0)	15	33	0	0	10	14	1	0	0	0	0	0

图 37 P5 性能 Vivado 资源汇总

```

TIMED BRANCH SNAPSHOT: total=101724000 cond=94840000 pred=39790000 fallback=3000000 false_pos=3000000 false_neg=4080000
[INFO] Benchmark finished. Processing results...
2K performance run parameters for coremark.
CoreMark Size : 666
Total ticks : 486902000
Total time (secs): 4.864000
Iterations/Sec : 411.184210
ERROR! Must execute for at least 10 secs for a valid result!
Iterations : 2000
Compiler version : GCC16.1.0
Compiler flags : -O3 -mtune=alkaid -ffunction-sections -fdata-sections -fno-common -funroll-loops -finline-functions
Memory location : STATIC
seedcrc : 0xe9f5
[INFO] Outputting CRC and results. Please check below...
[0]crclist : 0xe714
[0]crcmatrix : 0x1fd7
[0]crcstate : 0x8e3a
[0]crcfinal : 0xe714
Errors detected

```

图 38 P5 性能 CoreMark 上板输出

10.3 实现结果说明

六种配置的 post-route 实现结果均具有正 WNS、零 TNS、正 WHS 和零 THS，说明在各自目标频率下完成了时序收敛。性能配置的 CPU 核实现 LUT 相对同级默认配置分别增加约 81.54%、92.07% 和 89.32%，资源增长主要来自增强 BPU、TAGE、LSU L0 Cache、局部旁路和扩展指令执行控制。P3 性能配置保留最大的频率裕量，P4 性能配置在 100 MHz 下取得 +0.239 ns 的实现 WNS，P5 性能配置面向 150 MHz 目标。P5 性能配置的 post-route WNS 为 +0.007 ns，满足目标频率；其综合 WNS 为 -0.316 ns，说明综合阶段仍存在建立时序裕量不足。

RISC-V CPU 软件设计介绍

1 软件系统结构

Alkaid 软件系统由裸机板级支持包（BSP）、RISC-V GCC 构建配置、RT-Thread Nano 移植、外设驱动和 THP 应用组成。BSP 建立复位入口、运行栈、内存区段、机器模式陷入入口和存储器编址输入输出（MMIO）访问接口；RT-Thread Nano 在此基础上接入线程调度、系统节拍、软件中断、外部中断、堆区、UART 控制台和 MSH 命令行；THP 应用通过 I2C 访问环境传感器与 OLED 显示模块。

1.1 软件组成与硬件接口

软件层次	主要组成	与硬件的关系
启动与运行时	复位入口、链接配置、异常入口、基础字符输出和内存支持	设置 gp、sp 和 mtvec，建立 C 运行环境，并通过 UART 输出字符。
机器模式支持	核心本地中断控制器（CLINT）驱动、平台级中断控制器（PLIC）驱动和控制状态寄存器访问	配置软件中断、定时器中断、外部中断使能及处理函数。
APB 外设驱动	UART、GPIO、SPI、I2C、Timer/PWM 和引脚复用	通过 M1 AXI4-Lite、AXI 互连和 AXI2APB 访问外设寄存器。
编译与链接	-march、应用二进制接口（ABI）、代码模型、链接区段和存储镜像生成	保证软件指令集、寄存器宽度和 IMEM/DMEM 地址与硬件配置一致。
RT-Thread Nano	libcpu、板级初始化、线程、时钟节拍、堆区、组件初始化和 MSH	使用 CLINT 定时器推进内核节拍，以软件中断完成调度请求，以 UART 作为控制台。
THP 应用	命令处理、I2C 访问、传感器适配和 OLED 显示	使用 GPIO0[18:19] 或 I2C0 访问外部器件，并通过 UART 输出状态。

表 142 软件系统组成

软件访问 CLINT、PLIC 和 APB 外设时使用普通加载与存储指令。寄存器对象采用 volatile 访问，防止编译器删除、合并或改变具有硬件副作用的读写。外设驱动不直接处理 AXI4-Lite 或 APB 握手，握手和响应由 LSU、AXI 互连及 AXI2APB 完成。

1.2 地址空间与构建配置

地址范围	目标	软件用途
0x0200_0000 - 0x0200_FFFF	CLINT	访问 msip、mtimecmp 和 mtime。
0x0C00_0000 - 0x0C00_FFFF	PLIC	配置中断使能、优先级、处理函数和处理参数。
0x8000_0000 - 0x8001_FFFF	IMEM	保存复位入口、程序代码、异常入口和初始化表。
0x8010_0000 - 0x8010_FFFF	DMEM	保存只读数据、可写数据、.bss、堆和栈。
0x8400_0000 - 0x8400_0FFF	Timer/PWM	配置四组定时器、比较输出和事件。
0x8400_1000 - 0x8400_1FFF	SPI0	配置串行时钟、片选、发送、接收和中断。
0x8400_2000 - 0x8400_2FFF	I2C0	配置分频、控制、命令、发送和接收。
0x8400_3000 - 0x8400_3FFF	I2C1	提供第二组 I2C 控制器寄存器。
0x8400_4000 - 0x8400_4FFF	UART0	提供独立引脚的调试控制台。
0x8400_5000 - 0x8400_5FFF	UART1	提供经 GPIO0 复用的第二组串口。
0x8400_6000 - 0x8400_6FFF	GPIO0	配置 32 位输入、输出、方向、中断和 IOFCFG。
0x8400_7000 - 0x8400_7FFF	GPIO1	配置第二组 32 位通用输入输出。

表 143 软件可访问地址空间

构建系统依据 XLEN 和扩展使能项生成 `-march`。基础形式为 `rv32im_zicsr` 或 `rv64im_zicsr`，并按硬件配置加入 `A`、`C`、`Zba`、`Zbb`、`Zbc` 和 `Zbs`。RV32 使用 `ilp32 ABI`，RV64 使用 `lp64 ABI`，代码模型采用 `medany`。链接完成后，程序镜像按 IMEM 和 DMEM 地址范围拆分，分别用于两个存储器的初始化。

2 裸机 BSP 设计

裸机 BSP 负责从复位到应用入口的全部基础软件工作，包括寄存器初始化、内存区段初始化、陷入入口设置、CLINT/PLIC 驱动、外设访问、字符输出和基础运行库接口。RT-Thread Nano 复用同一套 CLINT、PLIC、UART、GPIO 和链接区段定义，避免操作系统配置与裸机配置使用不同的硬件地址。

2.1 启动与链接

2.1.1 复位启动流程

复位后，CPU 从 IMEM 基地址开始执行。启动程序首先关闭 gp 初始化附近的链接松弛，以链接符号设置全局指针，再以 _sp 设置栈指针。仿真配置可清除软件结果寄存器，随后启动程序按 32 位存储粒度清零 .bss，设置 mtvec，并依据构建配置进入裸机 main 或 RT-Thread entry。应用入口返回后，CPU 停留在保持循环中。

```
_start:
    .option push
    .option norelax
    la      gp, __global_pointer$
    .option pop
    la      sp, _sp

    la      a0, __bss_start
    la      a1, _end
bss_clear_loop:
    bgeu    a0, a1, bss_clear_end
    sw      zero, 0(a0)
    addi    a0, a0, 4
    j       bss_clear_loop
bss_clear_end:
    la      a0, trap_entry
    csrw    mtvec, a0
    call    SystemInit
    call    main
```

代码示例 1 裸机复位入口的主要初始化顺序

2.1.2 内存区段组织

链接配置定义 128 KiB 的 IMEM 区域和 64 KiB 的 DMEM 区域。 .init、.text、异常入口、构造函数表、RT-Thread 初始化函数表和 MSH 符号表放置在 IMEM； .rodata、.data、小数据段和 .bss 放置在 DMEM；堆和栈作为不装入镜像的区域保留在 DMEM。默认堆大小为 16 KiB，默认栈大小为 16 KiB，栈顶符号同时供裸机入口和 RT-Thread CPU 端口使用。

区段	存储区域	主要内容	初始化方式
----	------	------	-------

区段	存储区域	主要内容	初始化方式
.init、.text	IMEM	复位入口、程序代码、异常入口和初始化函数表	由 IMEM 镜像装入。
.rodata、.data、.sdata	DMEM	常量、已初始化全局变量和小数据	由 DMEM 镜像装入。
.bss	DMEM	未初始化全局变量和静态变量	启动程序清零。
.heap	DMEM	动态内存区域	由裸机分配器或 RT-Thread 堆管理器初始化。
.stack	DMEM	主栈和初始线程上下文	启动程序设置 sp，运行期间向低地址增长。

表 144 软件内存区段

2.2 陷入入口与上下文

2.2.1 异常与中断分派

陷入入口读取 mcause，以原因号选择处理函数。原因号 3、7 和 11 在同步异常与异步中断中分别具有不同含义，入口先检查 mcause 最高位，再区分断点与机器软件中断、存储访问错误与机器定时器中断、机器模式环境调用与机器外部中断。

mcause 原因号	同步异常	异步中断
3	断点异常	机器软件中断。
7	存储或 AMO 访问错误	机器定时器中断。
11	机器模式环境调用	机器外部中断。

表 145 共用原因号的分派规则

普通异常处理函数采用弱符号定义，应用可以提供同名实现。没有专用实现时，默认处理函数停留在对应错误位置，便于观察异常原因。外设事件不直接使用异常原因号 16 至 26 进入 CPU；机器外部中断先进入统一入口，再由 PLIC 给出的处理函数和参数完成第二级分派。

2.2.2 上下文保存与返回

裸机入口保存 ra、t0-t6、a0-a7 和 mepc。这些寄存器覆盖调用 C 处理函数时可能被修改的调用者保存寄存器。处理函数结束后，入口恢复 mepc 和通

用寄存器，最后执行 `mret`。该现场用于异常处理和外设中断分派，不等同于 RT-Thread 的完整线程现场；线程切换还需要保存线程栈指针、`mstatus`、返回位置以及配置允许时的浮点寄存器。

默认裸机处理过程不主动重新打开全局中断，因此处理函数按非嵌套方式执行。设备处理函数必须在返回前清除外设自身的中断挂起状态，否则 PLIC 仍会报告同一中断源。

2.3 CLINT 驱动

CLINT 通过 MMIO 提供 `msip`、64 位 `mtimecmp` 和 64 位 `mtime`。`msip` 产生机器软件中断；`mtime` 按系统时钟递增；当 `mtime` 大于或等于 `mtimecmp` 时产生机器定时器中断。驱动以 64 位接口表示时间值，使裸机延时、RT-Thread 节拍和基准程序使用一致的计时单位。

2.3.1 软件中断

软件中断由 `msip[0]` 控制。写 1 发起请求，写 0 清除请求。RT-Thread 将机器软件中断用于调度请求，因此处理函数进入后必须先清除 `msip`，防止 `mret` 后立即再次进入。

```
static inline void CLINT_SetSWIRQ(void)
{
    CLINT->MSIP |= 1U;
}

static inline void CLINT_ClearSWIRQ(void)
{
    CLINT->MSIP &= ~1U;
}
```

代码示例 2 CLINT 软件中断设置与清除

2.3.2 定时器中断

`SysTick_Config()` 将 `mtime` 清零，写入首个 `mtimecmp`，并打开机器定时器中断。周期性中断处理函数读取当前 `mtime`，把节拍周期加到当前值后写入新的 `mtimecmp`。若 64 位加法发生回绕，则同时重建 `mtime` 和 `mtimecmp` 的起点。

定时器处理顺序固定为先调用 `SysTick_Reload()` 写入下一比较值，再执行上层节拍处理。RT-Thread Nano 的定时器处理函数随后调用 `rt_tick_increase()`。先重装比较值可以避免处理函数运行期间 `mtime` 持续满足旧比较条件而造成连续进入。

2.4 PLIC 驱动

PLIC 管理 11 个外设中断源。软件寄存器包括中断使能、每个中断源的 8 位优先级、处理函数表、处理参数表，以及当前仲裁结果 `mvec` 和 `marg`。本设计不使用通用 PLIC 的 `claim/complete` 寄存器形式；硬件直接给出选中处理函数和参数，软件调用处理函数后由设备驱动清除外设挂起状态。

2.4.1 中断使能与优先级

`int_en` 的每一位控制一个中断源是否参加仲裁，优先级数值越大，多个有效中断同时出现时越先被选择。优先级 0 表示最低优先级，不等同于禁止；软件关闭中断源时必须清除对应 `int_en` 位。PLIC 初始化过程关闭全部中断源，把优先级写为 0，并为每项安装默认处理函数。

```
void plic_set_priority(uint8_t irq_id, uint8_t priority)
{
    if (irq_id < PLIC_NUM_SOURCES)
    {
        PLIC->int_pri[irq_id] = priority;
    }
}

void plic_enable_irq(uint8_t irq_id)
{
    if (irq_id < PLIC_NUM_SOURCES)
    {
        PLIC->int_en |= 1U << irq_id;
    }
}
```

代码示例 3 PLIC 优先级与使能控制

2.4.2 外部中断分派

设备驱动通过 `plic_set_handler()` 写入处理函数和参数。外设产生中断后，PLIC 在已使能请求中选择最高优先级项，并把相应表项送到 `mvec` 和 `marg`。CPU 进入机器外部中断入口后调用 `plic_dispatch()`；该函数读取 `mvec` 与 `marg`，检查函数指针非零后执行处理函数。

```
void plic_dispatch(void)
{
    void (*handler)(void *);
    void *arg;

    handler = (void (*)(void *))(uintptr_t)PLIC->mvec;
    arg = (void *)(uintptr_t)PLIC->marg;
    if (handler != 0)
    {
        handler(arg);
    }
}
```

代码示例 4 PLIC 外部中断分派

2.5 APB 外设与引脚复用

UART、GPIO、SPI、I2C 和 Timer/PWM 驱动以结构体描述寄存器偏移，并通过易失性指针访问对应 APB 地址。UART0 使用独立顶层引脚，主要承担启动信息和 MSH 控制台；UART1、I2C0、I2C1、SPI0 和 PWM 与 GPIO0 共享引脚，需要先配置 GPIO0.IOFCFG。

外设功能	GPIO0 引脚	软件选择接口	引脚方向
PWM	GPIO0[0:15]	<code>pinmux_select_pwm()</code>	16 路输出。
UART1	GPIO0[16]、 GPIO0[17]	<code>pinmux_select_uart1()</code>	RX 输入，TX 输出。
I2C0	GPIO0[18]、 GPIO0[19]	<code>pinmux_select_i2c0()</code>	SCL、SDA 双向开漏。
I2C1	GPIO0[20]、 GPIO0[21]	<code>pinmux_select_i2c1()</code>	SCL、SDA 双向开漏。
SPI0	GPIO0[22:30]	<code>pinmux_select_spi0()</code>	SCK 和片选输出， DQ 按传输方向控制。
普通 GPIO0	GPIO0[0:31]	<code>pinmux_select_gpio0()</code>	由 PADDR 控制输入或输出。

表 146 GPIO0 引脚复用的软件接口

IOFCFG 位为 0 时，引脚由 GPIO0 的 PADIN、PADOUT 和 PADDIR 控制；位为 1 时，引脚连接对应外设功能。GPIO0[31] 没有专用功能，始终作为普通 GPIO 使用。外设寄存器可访问并不表示引脚已经切换，应用必须在访问 UART1、I2C、SPI 或 PWM 前完成相应选择。

2.6 运行时支持

BSP 提供字符输出、延时、平台时钟、基础内存接口和程序结束控制。printf 宏使用轻量字符格式化函数，最终由 UART 轮询发送字符；裸机与 RT-Thread Nano 采用相同的 UART 寄存器定义。标准库构建关闭默认启动文件，由本设计的复位入口和链接区段提供运行环境。

堆区由 `_heap_start` 和 `_heap_end` 确定，RT-Thread Nano 使用 `_end` 与 `_heap_end` 初始化系统堆。栈位于 DMEM 高地址区域，裸机只有一个主栈，RT-Thread 的每个线程还具有独立线程栈。应用增大静态数据、线程数量或线程栈时，必须同时检查 64 KiB DMEM 内的 `.data`、`.bss`、堆和栈是否互相侵占。

3 编译器配置与指令调度

3.1 指令集与 ABI

`-march` 由硬件指令集参数生成，`-mabi` 由 XLEN 决定。Zbc 与 Zba、Zbb、Zbs 一样独立加入指令集字符串；软件只有在硬件打开对应扩展时才能使用这些指令。`-mtune` 只影响指令选择代价和调度顺序，不会替代 `-march` 对可用指令的约束。

构建属性	RV32	RV64
基础指令集	rv32im_zicsr	rv64im_zicsr
可选扩展	A、C、Zba、Zbb、Zbc、Zbs	A、C、Zba、Zbb、Zbc、Zbs
ABI	ilp32	lp64
代码模型	medany	medany
默认 Alkaid 调度参数	<code>-mtune=alkaid</code>	默认不附加，避免未确认的 RV64 调度描述影响构建

表 147 编译器目标配置

3.2 Alkaid 调度选项

RT-Thread Nano、CoreMark 和 Dhrystone 的 RV32 构建默认附加 -mtune=alkaid，RV64 构建默认不附加该选项。-mtune 使用编译器内置的 Alkaid 代价选择与调度策略，但不改变 -march、ABI 或可用寄存器集合。项目构建只依赖工具链对该选项的支持，不在软件报告中假定工具链内部采用某个固定资源表。

Alkaid 是单发射 CPU，加载、MUL、DIV 和 AXI4-Lite 访问具有不同的完成时间。编译器可以依据调度选项调整没有数据依赖关系的指令顺序，但 CPU 仍必须由 HDU、局部旁路、执行单元保持状态和 WBU 正确处理全部 RAW、WAW、结构冒险、冲刷与异常。调度结果不能作为硬件正确性的前提。

编译选项	选择方式	作用范围
-march	由 XLEN 与指令扩展开关生成	确定允许生成的 RISC-V 指令。
-mabi	RV32 选择 ilp32，RV64 选择 lp64	确定参数传递、返回值和数据类型布局。
-mcmmodel=medany	两种位宽统一采用	允许代码和静态数据使用中等任意地址代码模型。
-mtune=alkaid	RV32 默认启用，RV64 默认关闭	影响指令代价选择和调度顺序，不启用新的指令。
-ffunction-sections -fdata-sections	裸机与 RT-Thread Nano 构建启用	把函数和数据分入独立区段，配合链接删除未使用内容。

表 148 主要编译选项

3.3 构建参数与硬件一致性

软件和硬件必须使用相同的 XLEN、A、C、Zba、Zbb、Zbc 与 Zbs 配置。扩展关闭时，软件构建不得在 -march 中加入对应名称；扩展打开时，启动代码、库和应用使用同一指令集字符串。Zbc 只表示无进位乘法指令扩展，不改变 ABI，也不作为 CPU 微架构配置名称。

三级、四级和五级流水线具有相同的 RISC-V 架构结果，因此不改变 -march 或 ABI。流水线级数、Cache、乒乓缓存和分支预测只改变执行时间及内部控制，软件镜像仍通过统一的 IMEM、DMEM 和 MMIO 地址访问系统。硬件参数发生变化后，应重新生成软件镜像，防止沿用与当前指令扩展或存储容量不一致的构建产物。

4 RT-Thread Nano 移植

RT-Thread Nano 是面向资源受限系统的实时操作系统（RTOS）。Alkaid 移植包括 libcpu、线程上下文、机器模式陷入入口、CLINT 系统节拍、PLIC 外部中断、UART 控制台、堆区、组件初始化和 MSH 命令行。

4.1 libcpu 与线程上下文

libcpu 提供 `rt_hw_stack_init()`、`rt_hw_context_switch_to()`、`rt_hw_context_switch()` 和 `rt_hw_context_switch_interrupt()`。新线程栈帧把 `ra` 设置为线程退出函数，把 `a0` 设置为入口参数，把 `mepc` 设置为线程入口，并设置 `mstatus.MPP` 与 `mstatus.MPIE`，使线程通过机器模式返回开始运行。

普通线程切换保存原线程栈指针并装入目标线程栈指针；中断中的线程切换先保存待切换线程信息，在统一中断退出位置恢复目标线程上下文。启用浮点单元（FPU）配置时，栈帧和切换入口增加浮点寄存器状态；CPU 未配置浮点扩展时不生成这部分软件指令。

4.2 板级初始化与中断接入

板级入口按照表 149 完成初始化。各步骤只有在相应 RT-Thread 配置项打开时才执行，UART、组件和堆区均保留条件编译边界。

顺序	初始化动作	主要结果
1	<code>rt_hw_interrupt_init()</code>	建立 CPU 中断处理表。
2	<code>rt_hw_ticksetup()</code>	设置 CLINT 比较值并安装机器定时器处理函数。
3	安装机器软件中断处理函数	调度软件中断进入后清除 <code>msip</code> 。
4	<code>plic_init()</code> 并安装机器外部中断分派函数	关闭未配置中断源，建立外部中断处理入口。
5	初始化 GPIO0 默认输出状态	使板级输出在应用启动前具有确定值。
6	初始化 UART0 控制台	设置 115200 波特率、一个停止位和无奇偶校验。
7	初始化板级组件	调用导出的驱动和应用初始化函数。
8	初始化系统堆	使用 <code>_end</code> 到 <code>_heap_end</code> 的地址范围。

表 149 RT-Thread Nano 板级初始化顺序

系统节拍周期为 `SOC_TIMER_FREQ/RT_TICK_PER_SECOND`。定时器中断先设置下一次 `mtimecmp`，再调用 `rt_tick_increase()`。软件中断处理函数清除 `msip`；机器外部中断处理函数调用 `PLIC` 分派接口。设备处理函数若修改 `RT-Thread` 内核对象，应遵守内核的中断进入与退出规则。

4.3 UART 控制台

Nano 配置使用 `UART0` 轮询控制台。输出函数逐字符写入 `UART`；遇到换行符时先发送回车符，再发送换行符。输入函数检查接收就绪位，有字符时读取接收寄存器，无字符时短暂延时并返回。该路径不依赖接收中断，适合最小系统的 `MSH` 交互。

完整 `RT-Thread` 配置还提供串口设备驱动，可把 `UART0` 和 `UART1` 注册为读写设备，并通过 `PLIC` 接收事件通知。两种配置共用 `UART` 初始化和寄存器访问接口，但 Nano 控制台不把完整串口设备框架作为启动条件。

4.4 链接脚本

链接脚本同时规定程序区段的存储位置、运行时边界符号和 `RT-Thread` 需要保留的注册表区段。`.init`、`.text`、异常入口、初始化函数表和 `MSH` 注册表位于 `IMEM`；`.rodata`、`.data`、`.sdata` 和 `.bss` 位于 `DMEM`；堆区和栈区使用 `DMEM` 中不装入镜像的空间。启动程序使用 `__bss_start`、`_end` 和 `_sp`，`RT-Thread` 使用 `_heap_start` 与 `_heap_end` 建立动态内存区。

链接脚本为 `FSymTab`、`VSymTab`、`rti_fn` 和 `RTMSymTab` 保留独立区段，并通过 `KEEP` 防止链接器删除没有普通函数引用的注册项。`MSH` 命令导出宏把命令名称、函数地址和帮助信息放入 `FSymTab`，系统启动后根据起止符号遍历该区段，因此应用不需要在 `main` 中逐项注册命令。堆区默认保留 16 KiB，栈大小由 `__stack_size` 决定，默认值为 16 KiB；调整应用静态数据、线程数量或线程栈时，必须同时检查 `DMEM` 中各区域的边界。

```
. = ALIGN(8);
__fsymtab_start = .;
KEEP(*(FSymTab))
__fsymtab_end = .;

. = ALIGN(8);
__vsymtab_start = .;
```

```
KEEP(*(VSymTab))
__vsymtab_end = .;

. = ALIGN(8);
__rt_init_start = .;
KEEP(*(SORT(.rti_fn*)))
__rt_init_end = .;

. = ALIGN(8);
__rtmsymtab_start = .;
KEEP(*(RTMSymTab))
__rtmsymtab_end = .;
```

代码示例 5 链接脚本保留 MSH 命令与组件初始化区段

THP 应用的 `thp` 命令和系统内置命令采用相同的区段保留机制；组件初始化函数则通过 `rti_fn` 区段在板级初始化阶段按顺序调用。

5 THP 温湿度气压与 OLED 应用

5.1 应用结构

THP 应用由 MSH 命令、采样调度、I2C 访问、传感器适配和 OLED 显示五个部分组成。一次刷新由统一的采样函数完成：先获得信号量，再依次探测并读取在线传感器，生成一份样本快照，随后把同一份快照送到 UART 和 OLED，最后释放信号量。后台线程和交互命令共用这条刷新路径，避免两个执行上下文同时操作 I2C 总线或覆盖显示缓冲区。

I2C 访问层对上提供设备探测、连续读写、寄存器读写和 OLED 控制字节传输；传感器适配层负责初始化、原始数据读取、校验和物理量换算；显示层把有效样本排版为 128×64 像素页面。上层命令不直接操作 GPIO 或 I2C 控制器寄存器，因此更换 I2C 后端不会改变传感器和显示代码。

5.2 传感器探测与数据处理

传感器	I2C 地址	数据内容	处理方式
AHT30	0x38	温度、湿度	发起测量命令，等待完成后读取原始值并换算。

传感器	I2C 地址	数据内容	处理方式
HDC3020	0x44	温度、湿度	执行软复位和测量命令，读取数据并计算物理量。
BME280	0x76 或 0x77	温度、湿度、气压	读取芯片标识与校准系数，按校准参数补偿原始数据。
BME680	0x76 或 0x77	温度、湿度、气压	读取校准系数和测量数据；气体测量原始字段已读取，气体电阻补偿输出暂不启用。

表 150 THP 支持的环境传感器

应用先按地址读取设备标识，再执行对应初始化和采样流程；同一轮采样可以读取多个在线传感器。HDC3020 检查两组数据校验值，AHT30 等待测量完成后再读取数据，BME280 和 BME680 使用器件校准参数换算温度、湿度和气压。BME680 的气体测量原始字段会被读取，但当前样本结构中的 `gas_valid` 保持为 0，因此终端和 OLED 不显示未经补偿的气体电阻。某个设备无应答或校验失败时，该设备样本保持无效，其它设备继续读取；`thp reset` 清除设备状态，下一轮刷新重新探测。

5.3 I2C 访问

默认配置使用 GPIO 软件 I2C，GPIO0[18] 为 SCL，GPIO0[19] 为 SDA。驱动把输出低电平设置为 GPIO 输出并写 0，把高电平设置为输入状态并由外部上拉电阻释放总线；读取 SCL、SDA 电平用于应答和时钟延展检查。

硬件 I2C 配置先调用 `pinmux_select_i2c0()`，再依据 `CPU_FREQ_HZ` 和目标 100 kHz 频率计算分频值。控制器按 START、WRITE、READ、ACK 和 STOP 命令完成传输，驱动同时使用系统节拍超时和最大循环次数限制等待时间。发生无应答、仲裁丢失或超时时，驱动发送 STOP；控制器仍处于忙状态时执行复位，使 MSH 可以继续响应命令。

两种后端对上层提供相同的设备探测、连续写、连续读、先写寄存器地址再读、单字节寄存器写入和带控制字节的数据流写入接口。传感器与 OLED 只调用这些公共接口，不依赖 GPIO 时序或 APB I2C0 寄存器细节。

5.4 OLED 显示

OLED 使用地址 0x3c 的 SSD1306，分辨率为 128×64。初始化设置显示时钟、

复用比、显示偏移、寻址方式、段方向和电荷泵；帧缓冲区按 8 页组织，每页保存 128 字节。刷新时先发送页地址和列地址，再连续发送该页的 128 字节数据，八页全部写完后形成一幅完整画面。

页面第 0 页显示 THP MONITOR 和当前来源，来源为 MSH 单次读取时显示 MSH，后台线程刷新时显示 AUTO；其余页面按在线传感器显示温度、湿度和气压。每次刷新先清空帧缓冲区，再根据有效样本重新排版，因此传感器离线或字段失效时不会留下上一轮数值。

图 39 展示了 FPGA 板上的实际连接：传感器模块通过 I2C 接入，SSD1306 OLED 显示当前四类传感器的温度、湿度和气压，标题栏中的 AUTO 表示画面由后台刷新线程更新。照片同时说明了 GPIO0 引脚复用、I2C 总线和显示模块之间的板级连接关系。

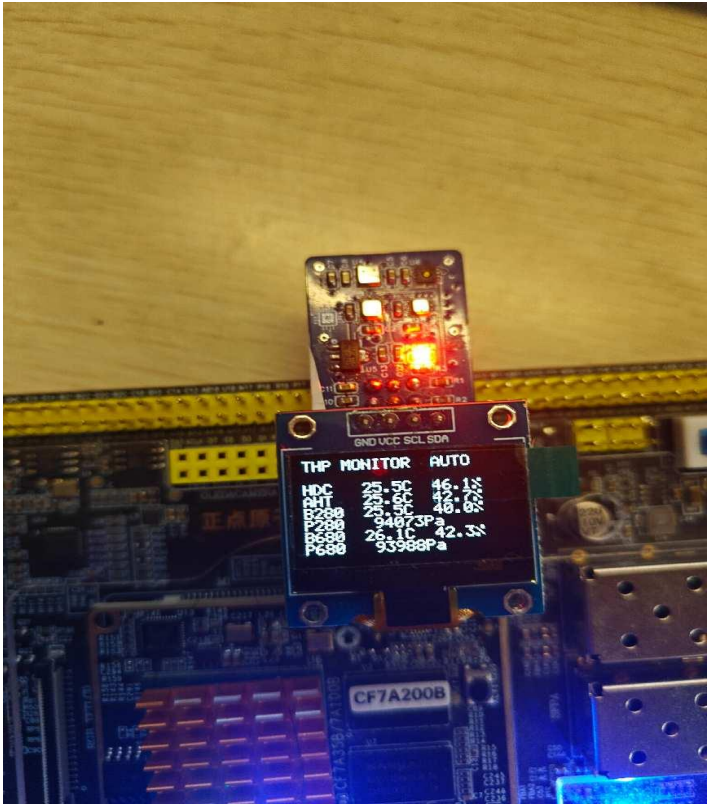


图 39 THP 应用板级运行结果

5.5 MSH 命令与后台刷新

MSH 命令	功能
thp、thp once 或 thp read	读取全部在线传感器，在 UART 输出结果并刷新 OLED。
thp <seconds>	以指定秒数启动或修改后台刷新周期。
thp auto <seconds>	以指定秒数启动或修改后台刷新周期。

MSH 命令	功能
thp interval <seconds>	与 thp auto <seconds> 相同。
thp off、thp stop 或 thp disable	停止后台刷新，并在 OLED 显示停止状态。
thp status	显示在线传感器、刷新状态、周期和最近一次样本。
thp reset	清除传感器识别状态和最近一次样本。

表 151 THP MSH 命令

后台线程以 1 至 3600 秒为合法周期，参数 0 用于停止刷新。线程和交互命令使用同一个二值信号量保护传感器读取与 OLED 写入，保证一轮 I2C 事务和整屏更新连续完成。后台等待按不超过 100 ms 的小步长分段执行并检查停止标志，停止命令无需等待完整周期结束。

图 40 给出了 MSH 控制台的典型交互。系统启动后先显示 RT-Thread Nano 提示符，输入 thp 会依次打印 HDC3020、AHT30、BME280 和 BME680 的有效样本；输入 thp status 会显示在线设备、后台刷新状态、周期和最近一次样本。该输出与图 39 中的 OLED 画面来自同一轮采样快照，能够同时确认命令解析、I2C 读取、数据换算和显示刷新均已执行。

```

[thp] OLED background refresh interval=1 s
msh >
\ | /
- RT -   Thread Operating System
/ | \   3.1.5 build Jul  7 2026
2006 - 2020 Copyright by rt-thread team
msh >thp h
usage:
  thp [read|once]      read once, print result and refresh OLED
  thp <seconds>        refresh OLED periodically in background
  thp auto <seconds>   refresh OLED periodically in background
  thp off/stop         stop background OLED refresh
  thp status|reset
msh >thp status
[thp] sensor=none auto=off interval=0 s
msh >thp
[thp] HDC3020 temp=25.97 C humidity=46.90 %RH
[thp] AHT30 temp=26.21 C humidity=42.64 %RH
[thp] BME280 temp=25.92 C humidity=41.28 %RH pressure=94072 Pa
[thp] BME680 temp=26.63 C humidity=42.15 %RH pressure=93992 Pa
msh >thp status
[thp] sensor=HDC3020/AHT30/BME280/BME680 auto=off interval=0 s
[thp:last] HDC3020 temp=25.97 C humidity=46.90 %RH
[thp:last] AHT30 temp=26.21 C humidity=42.64 %RH
[thp:last] BME280 temp=25.92 C humidity=41.28 %RH pressure=94072 Pa
[thp:last] BME680 temp=26.63 C humidity=42.15 %RH pressure=93992 Pa
msh >

```

图 40 THP 命令输出与 OLED 刷新信息